

Julia

В качестве ВТОРОГО ЯЗЫКА



Эрик Энгхейм

 MANNING

 ДМК
ИЗДАТЕЛЬСТВО

Язык Julia, изначально разработанный для высокопроизводительной обработки данных, стал языком программирования общего назначения, в котором идеально сочетаются простота, гибкость и производительность. Он предлагает удобные для разработчиков функциональные возможности: сборка мусора, динамическая типизация, гибкий подход к конкурентности и распределенным вычислениям.

Эта книга знакомит читателя с языком Julia и демонстрирует все его аспекты — от базового синтаксиса до продвинутого функционала — на примере ряда проектов. Описываются применение базовых типов и структур данных при симуляции пуска ракеты, использование словарей для интерпретации римских цифр, множественная диспетчеризация Julia в симулированном сражении между рыцарями и лучниками.

Попутно вы сравните объектно ориентированный и функциональный стили программирования — язык Julia поддерживает оба!

В числе рассматриваемых тем:

- типы данных: числа, строковые литералы, массивы и словари;
- немедленный отклик с помощью интерактивной среды языка Julia (REPL);
- упрощение взаимодействий между частями исходного кода благодаря множественной диспетчеризации;
- обмен исходным кодом с использованием модулей и пакетов.

Издание подойдет читателям, уже знакомым с каким-либо языком программирования, например Python, JavaScript или C#.

Эрик Энхейм — писатель, автор видеокурсов, разработчик программного обеспечения. Выступает с докладами на конференциях.

«Данная книга побудила меня начать использовать Julia для серьезных проектов. Это основательный обзор базовых функциональных возможностей языка, который не повторяет уже известные вам основы программирования».

*Джонатан Оуэнс,
GE Global Research*

«Все, что нужно, чтобы освоить язык Julia».

*Аманда Деблер,
Schaeffler Technologies*

«Язык Julia для бывалых программистов».

Кевин Чунг, Carleton University

«Идеально для тех, кто хочет изучить язык Julia, но не знает, с чего начать».

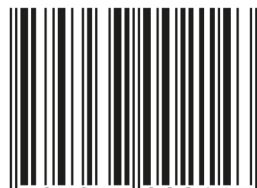
*Симона Сгуацца,
Университет прикладных наук
и искусств Южной Швейцарии*

Интернет-магазин: www.dmkpress.com

Оптовая продажа: КТК «Галактика»
books@alians-kniga.ru



ISBN 978-5-93700-214-3



9 785937 002143 >

Эрик Энгхейм

Julia в качестве второго языка

Julia as a Second Language

GENERAL PURPOSE PROGRAMMING
WITH A TASTE OF DATA SCIENCE

ERIK ENGHEIM



MANNING
Shelter Island

Julia в качестве второго языка

ОБЩЕЦЕЛЕВОЕ ПРОГРАММИРОВАНИЕ
С ПРИВКУСОМ НАУКИ О ДАННЫХ

ЭРИК ЭНГХЕЙМ



Москва, 2023

УДК 004.04
ББК 32.372
Э62

Энгхейм Э.

Э62 Julia в качестве второго языка / пер. с англ. А. В. Логунова. – М.: ДМК Пресс, 2023. – 446 с.: ил.

ISBN 978-5-93700-214-3

В языке Julia идеально сочетаются простота, гибкость и производительность. Книга позволяет освоить этот язык с нуля, демонстрируя все его аспекты – от базового синтаксиса до продвинутых функциональных возможностей – на примере ряда проектов. Описываются применение базовых типов и структур данных при симуляции пуска ракеты, использование словарей для интерпретации римских цифр, множественная диспетчеризация Julia в симулированном сражении между рыцарями и лучниками. Попутно сравниваются объектно ориентированный и функциональный стили программирования (язык Julia поддерживает оба).

Издание подойдет читателям, уже знакомым с каким-либо языком программирования, например Python, JavaScript или C#.

УДК 004.04
ББК 32.372

Copyright © DMK Press 2023. Authorized translation of the English edition © 2023 Manning Publications. This translation is published and sold by permission of Manning Publications, the owner of all rights to publish and sell the same.

Все права защищены. Любая часть этой книги не может быть воспроизведена в какой бы то ни было форме и какими бы то ни было средствами без письменного разрешения владельцев авторских прав.

ISBN 978-1-6172-9971-1 (англ.)
ISBN 978-5-93700-214-3 (рус.)

© Manning Publications, 2023
© Перевод, оформление, издание, ДМК Пресс, 2023

*Посвящается моим бабушке и дедушке,
которые приобрели мне мой первый компьютер – Amiga 1000*

Оглавление

Часть I ОСНОВЫ	24
1 ■ Почему именно Julia?.....	25
2 ■ Julia в качестве калькулятора	42
3 ■ Поток управления	63
4 ■ Julia в качестве электронной таблицы.....	92
5 ■ Работа с текстом.....	122
6 ■ Хранение данных в словарях	144
Часть II ТИПЫ	161
7 ■ Понимание типов.....	162
8 ■ Сборка ракеты	185
9 ■ Конвертация и приведение типов	208
10 ■ Представление неизвестных значений	227
Часть III КОЛЛЕКЦИИ	239
11 ■ Работа со строковыми литералами.....	240
12 ■ Понимание коллекций Julia	273
13 ■ Работа с множествами	302
14 ■ Работа с векторами и матрицами.....	322
Часть IV ТЕХНОЛОГИЯ РАЗРАБОТКИ ПРОГРАММНОГО ОБЕСПЕЧЕНИЯ	336
15 ■ Функциональное программирование на языке Julia	337
16 ■ Организация и модуляризация исходного кода	363
Часть V УГЛУБЛЕННОЕ ИЗУЧЕНИЕ	387
17 ■ Ввод и вывод.....	388
18 ■ Определение параметрических типов	401

Содержание

Оглавление	6
Предисловие	15
Признательности	16
Об этой книге	17
Об авторе	22
Об иллюстрации на обложке	23
 Часть I ОСНОВЫ	 24
 1 Почему именно Julia?	 25
1.1 Что из себя представляет язык Julia?	26
1.1.1 Плюсы и минусы статически и динамически типизированных языков	27
1.2 Julia сочетает в себе элегантность, продуктивность и производительность	28
1.3 Для чего был создан язык Julia	29
1.3.1 Ученые нуждаются в интерактивном программировании, которое предлагается динамически типизированными языками	30
1.3.2 Разработчикам в других областях тоже нужна интерактивность, предлагаемая динамически типизированным языком	32
1.4 Более высокая производительность языка Julia решает проблему двух языков	33
1.5 Julia для всех	34
1.6 Что можно разрабатывать на языке Julia?	36
1.6.1 Julia в науке	36
1.6.2 Использование Julia в ненаучных целях	37
1.7 Где язык Julia менее идеален	37
1.8 Что вы узнаете из этой книги	39
Резюме	40
 2 Julia в качестве калькулятора	 42
2.1 Командная строка Julia	43
2.2 Использование констант и переменных	44
2.2.1 Присваивание и привязывание значений к переменным	48

2.2.2	Использование переменной <code>ans</code>	49
2.2.3	Что такое литеральный коэффициент?	50
2.3	Разные типы чисел и их длина в битах	51
2.3.1	Написание чисел с использованием разных числовых форматов	52
2.4	Числа с плавающей точкой	54
2.4.1	Выполнение операций на целых числах и числах с плавающей точкой	55
2.5	Определение функций	56
2.5.1	Хранение определений функций в файле	57
2.5.2	Работа с функциями в интерактивной среде <code>REPL</code>	58
2.5.3	Функции повсюду	59
2.5.4	Функции для работы с числами	60
2.6	Как использовать числа на практике	62
	Резюме	62

3	Поток управления	63
3.1	Навигация и тригонометрия	64
3.2	Булевы выражения	66
3.2.1	Составные инструкции	68
3.3	Циклы	69
3.3.1	Блок-схема	71
3.3.2	Составление математической таблицы синусной функции	72
3.3.3	Объекты-диапазоны	74
3.3.4	Циклы <code>for</code>	75
3.4	Многострочные функции	76
3.4.1	Реализация тригонометрической синусной функции	77
3.5	Реализация факториала	79
3.6	Факториал с использованием рекурсии	80
3.7	Инструкции <code>if</code>	80
3.7.1	Инструкции <code>if-else</code>	81
3.7.2	Подчиненный компонент <code>elseif</code>	82
3.8	Исключения для обработки ошибок	83
3.9	Поток управления в отличие от потока данных	85
3.10	Подсчет кроликов	86
3.10.1	Базовый случай	89
3.10.2	Итерации по сравнению с рекурсией	89
3.10.3	Возвращать или не возвращать	90
	Резюме	91

4	Julia в качестве электронной таблицы	92
4.1	Анализ продажи пиццы	93
4.2	Различные типы массивов	94
4.3	Выполнение операций на массивах	96
4.4	Работа с модулем <code>Statistics</code>	98

4.5	Доступ к элементам	100
4.6	Создание массивов	103
4.7	Отображение значений в массиве	105
4.8	Знакомство с буквенными символами и строковыми литералами	109
4.9	Хранение данных о пиццах в кортежах	112
4.10	Фильтрация пицц на основе предикатов	116
4.10.1	Комбинирование функций более высокого порядка	117
4.11	Отображение и редукция массива	118
4.11.1	Таблица синусов с использованием <i>map</i> и <i>reduce</i>	119
4.12	Подсчет совпадений с использованием булевых массивов	120
	Резюме	121

5 Работа с текстом

5.1	Создание хорошо отформатированной таблицы продажи пицц	123
5.1.1	Функции <i>print</i> , <i>println</i> и <i>printstyled</i>	124
5.1.2	Печать нескольких элементов	128
5.1.3	Печать нескольких пицц	129
5.1.4	Выравнивание с помощью функций <i>lpad</i> и <i>rpadd</i>	130
5.1.5	Добавление линий	131
5.2	Распечатка тригонометрической таблицы	132
5.3	Чтение и запись продажи пицц в CSV-файлы	134
5.3.1	Запись продаж пицц в файл	136
5.3.2	Чтение продаж пицц из файла	137
5.4	Взаимодействие с пользователем	140
	Резюме	142

6 Хранение данных в словарях

6.1	Синтаксический разбор римских цифр	145
6.2	Использование типа <i>Dict</i>	147
6.3	Прокручивание буквенных символов в цикле	149
6.4	Перечисление значений и индексов	150
6.5	Объяснение процесса конвертации	151
6.6	Использование словарей	152
6.6.1	Создание словарей	152
6.6.2	Доступ к элементам	154
6.7	Зачем использовать словарь?	155
6.8	Использование именованных кортежей в качестве словарей	158
6.8.1	Когда применять именованный кортеж?	159
6.8.2	Связывание всего воедино	160
	Резюме	160

Часть II **ТИПЫ**161

7	Понимание типов	162
7.1	Создание составных типов из примитивных.....	163
7.2	Обследование иерархий типов	166
7.3	Создание симулятора сражений.....	170
7.3.1	Определение типов воинов.....	170
7.3.2	Добавление воинам поведения	172
7.3.3	Использование множественной диспетчеризации для вызова методов.....	175
7.4	Как Julia выбирает метод, который следует вызвать.....	179
7.4.1	Сравнение множественной диспетчеризации Julia с объектно ориентированными языками	181
7.4.2	Чем множественная диспетчеризация отличается от перегрузки функций?	183
	Резюме	184
8	Сборка ракеты	185
8.1	Сборка простой ракеты в исходном коде.....	186
8.2	Поддержание инвариантов в исходном коде.....	191
8.3	Создание объектов с помощью функций-конструкторов.....	192
8.4	Различия между внешними и внутренними конструкторами.....	194
8.5	Моделирование ракетных двигателей и полезной нагрузки	195
8.6	Сборка простой ракеты	198
8.7	Создание ракеты с несколькими ступенями и двигателями	199
8.8	Запуск ракеты в космос	204
	Резюме	207
9	Конвертация и приведение типов	208
9.1	Обследование системы приведения чисел	209
9.2	Понимание конвертации чисел.....	212
9.3	Определение конкретно-прикладных единиц измерения углов	215
9.3.1	Определение конструкторов углов	217
9.3.2	Определение арифметических операций на углах	218
9.3.3	Определение функций доступа для извлечения градусов, минут и секунд	219
9.3.4	Отображение углов DMS на экране	220
9.3.5	Определение конвертаций типов.....	221
9.3.6	Создание красивых литералов углов	222
9.3.7	Приведения типов	223
	Резюме	225

10	Представление неизвестных значений	227
10.1	Объект <code>nothing</code>	228
10.2	Применение объекта <code>nothing</code> в структурах данных	229
10.2.1	Что такое параметрический тип?	230
10.2.2	Использование типов-объединений для завершения товарного поезда	232
10.3	Отсутствующие значения	234
10.4	Не число	235
10.5	Неопределенные данные	237
10.6	Сведение всего воедино	237
	Резюме	238

Часть III КОЛЛЕКЦИИ 239

11	Работа со строковыми литералами	240
11.1	UTF-8 и Юникод	241
11.1.1	Понимание взаимосвязи между кодовыми точками и кодовыми единицами	242
11.2	Операции на строковых литералах	248
11.2.1	Конвертирование верблюжьего регистра в змеиный регистр	251
11.2.2	Конвертирование между числами и строковыми литералами	253
11.2.3	Интерполяция и конкатенация строковых литералов	254
11.2.4	Форматирование с использованием функции <code>sprintf</code>	256
11.3	Использование строковой интерполяции для генерации исходного кода	258
11.4	Работа с нестандартными строковыми литералами	261
11.4.1	Строковые литералы <code>DateFormat</code>	263
11.4.2	Неформатированные строковые литералы	265
11.4.3	Использование регулярных выражений для сопоставления текста	265
11.4.4	Создание больших целых чисел с помощью <code>BigInt</code>	269
11.4.5	MIME-типы	270
	Резюме	271

12	Понимание коллекций Julia	273
12.1	Определение интерфейсов	274
12.2	Пример интерфейса топливного бака	277
12.3	Интерфейсы по соглашению	281
12.4	Реализация итеративной обработки кластера двигателей	282
12.4.1	Наделение кластеров итерируемостью	284
12.5	Реализация итеративной обработки ступеней ракеты	287
12.5.1	Добавление поддержки функций <code>map</code> и <code>collect</code>	289
12.6	Сравнение связанных списков и массивов	292

12.6.1	Добавление и удаление элементов.....	294
12.7	Полезность конкретно-прикладных типов	300
	Резюме	301

13 Работа с множествами

13.1	Какие задачи можно решать с помощью множеств?	303
13.2	Что такое множество?	304
13.2.1	Сравнение свойств множеств и массивов	305
13.3	Как использовать операции на множествах.....	309
13.4	Как использовать множества в исходном коде.....	312
13.5	Поиск товаров с помощью операций на множествах	313
13.5.1	Определение и использование перечислений.....	315
13.5.2	Создание тестовых данных для выполнения запросов.....	316
13.5.3	Поиск винтов	316
13.5.4	Размещение объектов-винтов в множествах.....	317
13.5.5	Поиск винтов с помощью словарей.....	318
13.6	Поиск в инструменте отслеживания дефектов с помощью множеств	319
13.7	Реляционные базы данных и множества	320
	Резюме	321

14 Работа с векторами и матрицами

14.1	Векторы и матрицы в математике.....	323
14.2	Конструирование матрицы из строк и столбцов	323
14.3	Размер, длина и норма массива	325
14.4	Вертикальная и горизонтальная нарезки массива.....	326
14.5	Комбинирование матриц и векторов.....	331
14.6	Создание матриц.....	334
	Резюме	335

Часть IV ТЕХНОЛОГИЯ РАЗРАБОТКИ ПРОГРАММНОГО ОБЕСПЕЧЕНИЯ.....336

15 Функциональное программирование на языке Julia.....337

15.1	Чем функциональное программирование отличается от объектно ориентированного?	338
15.2	Как и зачем учиться мыслить функционально.....	339
15.3	Избегание глубокой вложенности вызовов с помощью выстраивания функций в цепочки	340
15.3.1	Понимание анонимных функций и замыканий	341
15.3.2	Использование конвейерного оператора >	343
15.3.3	Удобное генерирование новых функций с помощью частичного применения.....	344

15.4	Реализация шифра Цезаря и подстановочного шифра	345
15.4.1	Реализация шифра Цезаря	347
15.4.2	Реализация подстановочных шифров	349
15.5	Разработка службы, не зависящей от алгоритма шифрования	352
15.6	Разработка службы шифрования с использованием объектно ориентированного программирования	353
15.7	Разработка службы шифрования с использованием функционального программирования	357
15.7.1	Определение функционального шифра Цезаря	357
15.7.2	Определение подстановочного шифра с функциональным колоритом	359
15.7.3	Реализация функциональной службы хранения паролей	360
	Резюме	362

16 Организация и модуляризация исходного кода

16.1	Настройка рабочей среды	364
16.1.1	Использование пакета в интерактивной среде REPL языка Julia	368
16.1.2	Как модули соотносятся с пакетами	370
16.2	Создание собственного пакета и модуля	371
16.2.1	Генерирование пакета	371
16.2.2	Добавление исходного кода в пакет	374
16.3	Модификация и разработка пакета	376
16.4	Устранение распространенных заблуждений о модулях	381
16.5	Тестирование пакета	382
	Резюме	386

Часть V УГЛУБЛЕННОЕ ИЗУЧЕНИЕ

17 Ввод и вывод

17.1	Знакомство с системой ввода-вывода языка Julia	389
17.2	Чтение данных из процесса	391
17.3	Чтение из сокета и запись в сокет	392
17.4	Синтаксический разбор CSV-файла	394
17.4.1	Загрузка данных ракетного двигателя	396
17.4.2	Сохранение данных ракетных двигателей	399
	Резюме	400

18 Определение параметрических типов

18.1	Определение параметрических методов	402
18.2	Определение параметрических типов	405
18.3	Выгоды от параметрических типов для типобезопасности	407

18.4	Выгоды в производительности за счет параметрических типов	410
18.5	Выгоды от параметрических типов для памяти.....	412
	Резюме	413
	<i>Дополнение А. Установка и конфигурирование среды Julia</i>	<i>414</i>
	<i>Дополнение В. Числа.....</i>	<i>422</i>
	<i>Предметный указатель</i>	<i>430</i>

Предисловие

Я начал программировать еще подростком, изучая забавные книги с комиксами о волшебниках и черепахах. Я читал журналы, в которых мне показывали, как писать свои собственные простые игры или создавать на экране глупые эффекты. Мне было забавно.

Но когда я поступил в университет, в моих книгах стали обсуждаться банковские счета, балансы, отделы продаж, работники и работодатели. Я стал задумываться, а не будет ли моя жизнь программиста означать надевание серого костюма и написание исходного кода для систем расчета заработной платы. О ужас!

По крайней мере, половина моей группы страстно ненавидела программирование. И я не мог их винить. Почему книги по программированию должны быть такими скучными, функциональными и осмысленными?

Где ощущение приключений и чувство юмора? Юмор и веселье недооценены. Кого волнует, что книга безрассудна и содержит глупые шутки, если она побуждает вас учиться и получать удовольствие от учебы?

Это одна из причин, по которой я написал эту книгу. Я хотел, чтобы читатель получал удовольствие от изучения программирования – не от шуток, а от работы с интересными и забавными примерами программирования.

Обещаю, примеров моделирования отдела продаж не будет. Вместо этого мы будем делать такие вещи, как имитация запуска ракет, притворяться, что Цезарь отправляет секретное сообщение своим полководцам, используя древнеримские методы шифрования, и многое другое.

Вторая важная причина, по которой я захотел написать эту книгу, заключается в том, что люди продолжают меня спрашивать: «Разве язык Julia не предназначен только для науки и ученых?» Язык Julia добился в этой области больших успехов, поэтому сообщество Julia сегодня полно умных людей, которые работают над сложными задачами, такими как разработка новых лекарств и моделирование распространения инфекционных заболеваний, изменение климата и экономика.

Нет и еще раз нет, для того чтобы использовать Julia, вовсе не нужно быть гением или ученым. Julia – замечательный язык общецелевого программирования для всех! Я не ученый, и мне нравится использовать его уже более 9 лет. Вы обнаружите, что с Julia вы сможете решать задачи быстрее и элегантнее, чем раньше. И вишенкой на торте является то, что вычислительно-емкий исходный код будет исполняться невероятно быстро.

Признательности

Эта книга пережила несколько воплощений. В какой-то момент она была в самиздате. Позже меня свел случай с издательством Manning Publications, и мы договорились работать над изданием моей книги. В то время я не осознавал, в какую работу втягиваюсь. Я думал, что внесу лишь незначительные изменения в существующую книгу, но, посмотрев на полученные мною отзывы, понял, что придется вносить много правок.

Иногда хотелось сдаться. Однако, несмотря на трудности, считаю, что обширная система, созданная издательством Manning в помощь нам, авторам, помогла мне сделать книгу значительно лучше. За это я должен поблагодарить Николь Баттерфилд, которая организовала подписание контракта с издательством Manning. У меня было два редактора: Лесли Трайтес на ранней стадии книги и Марина Майклс, которая благодаря своему значительному опыту и твердой руке помогла мне выйти на финишную прямую. Хотел бы выразить благодарность Милану Курчичу, моему редактору-консультанту по техническому развитию, который очень помог своими отзывами в доведении понятности материала до уровня моей целевой аудитории. Мой редактор Кристиан Берк был для меня, не носителя английского языка, неоценим, исправляя любые нечеткие конструкции и грамматику.

Кроме того, хотел бы поблагодарить рецензентов, которые уделили время чтению моей рукописи на разных этапах ее разработки и предоставили бесценные отзывы: Алана Лентона, Аманде Деблер, Энди Робинсона, Криса Бейли, Дэниелу Кенни, Даррина Бишопа, Эли Майост, Эмануэлу Пиччинелли, Ганеша Сваминатана, Герта Ван Лаэтхема, Джеффа Барто, Иво Балберта, Джереми Чена, Джона Зётебир, Джонатана Оуэнса, Йорта Роденбурга, Катю Паткин, Кевина Чунга, Кшиштофа Енджеевски, Луиса Луангкесорна, Марка Томаса, Мора Уайлдера, Майка Барана, Никоса Канакариса, Нинослава Черкеса, Орландо Алехо Мендес Моралеса, Патрика Ригана, Пола Силистьяну, Пола Вербеке, Самвида Мистри, Симона Сгуацца, Стива Грей-Уилсона, Тимоти Володзко и Томаса Хеймана.

Особая благодарность Маурицио Томази, техническому корректору, за его последний тщательный просмотр исходного кода незадолго до того, как книга была запущена в производство. Наконец, спасибо создателям языка Julia. Вы создали язык программирования будущего, который, уверен, изменит компьютерную индустрию. Возможно, это покажется преувеличением, но я искренне верю, что Julia станет важной вехой в эволюции языков программирования.

Об этой книге

Книга «Julia в качестве второго языка» представляет собой введение в язык программирования Julia для разработчиков программного обеспечения. Она не только охватывает синтаксис и семантику языка, но и обучает читателя думать и работать как разработчик Julia, уделяя особое внимание интерактивному программированию в среде, основанной на цикле чтения–вычисления–печати (REPL).

Кому стоит прочесть эту книгу?

Книга «Julia в качестве второго языка» написана для разработчиков, интересующихся языком программирования Julia, но не обязательно имеющих научную или математическую подготовку. Книга также является хорошей отправной точкой для всех, кто хочет изучить науку о данных или научные вычисления, поскольку язык Julia очень хорошо подходит для такой работы. Однако это не исключает других применений. Данная книга будет полезна любому разработчику, который хотел бы программировать на современном высокопроизводительном языке, повышающем его продуктивность.

Как эта книга устроена

Книга состоит из пяти частей, включающих 18 глав.

Часть I охватывает основы языка.

- Глава 1 объясняет особенности языка Julia, причину его создания и преимущества использования данного языка программирования.
- Глава 2 посвящена работе с числами на языке Julia. В ней показываются способы применения его интерактивной среды REPL в качестве очень сложного калькулятора.
- Глава 3 объясняет инструкции потока управления, такие как инструкции `if`, циклы `while` и `for`, путем реализации тригонометрической функции и вычисления чисел Фибоначчи.
- Глава 4 объясняет способы работы с коллекциями чисел, используя тип `Array`. Читатели рассмотрят пример с данными о продажах пиццы.
- Глава 5 посвящена работе с текстом. В этой главе вы познакомитесь с созданием демонстраций красиво отформатированных данных о продажах пиццы с использованием цвета, а также с чтением данных о пицце из файлов и записью их в файлы.

- Глава 6 посвящена реализации программы конвертации римских цифр в десятичный формат, используя словарный тип.

Часть II посвящена более подробному описанию системы типов языка Julia.

- Глава 7 излагает иерархию типов Julia и способы определения своих собственных составных типов. Это одна из самых важных глав, потому что в ней также объясняется одна из самых важных и уникальных функциональностей языка Julia – множественная диспетчеризация.
- Глава 8 демонстрирует пример исходного кода имитации ракеты, который мы будем использовать в нескольких последующих главах. Эта глава посвящена определению типов для различных частей ракеты.
- Глава 9 посвящена более подробному рассмотрению числовой конвертации и приведению типов в Julia путем создания примера исходного кода, работающего с разными единицами измерения градусов. Эта глава помогает закрепить понимание системы множественной диспетчеризации Julia.
- Глава 10 посвящена способам представления несуществующих, отсутствующих или неопределенных объектов на языке Julia.

В части III вновь рассматриваются типы коллекций, такие как массивы, словари и строковые литералы, которые были рассмотрены в части I, но на этот раз во всех подробностях.

- Глава 11 посвящена более подробному рассмотрению строковых литералов, в том числе использованию Юникода и UTF-8 в Julia, а также их влиянию на применение вами строковых литералов.
- Глава 12 посвящена объяснению общих для всех коллекций Julia черт, таких как прокручивание элементов в цикле, а также разработка своих собственных коллекций.
- Глава 13 демонстрирует несколько примеров исходного кода с использованием множеств и операций на множествах, чтобы показать организацию и поиск данных во многих типах приложений.
- Глава 14 иллюстрирует приемы работы с массивами разных размерностей, такими как векторы и матрицы, и их комбинирования.

Часть IV посвящена методам организации исходного кода на разных уровнях, включая модуляризацию на уровне функций вплоть до пакетов, файлов и каталогов.

- Глава 15 посвящена более подробному рассмотрению применений функций в Julia с акцентом на отличия функционального программирования от объектно ориентированного.
- Глава 16 посвящена организации исходного кода в модули, использованию сторонних пакетов и созданию своих собствен-

ных пакетов для распространения исходного кода среди других разработчиков.

Часть V посвящена углубленному изложению деталей, которые трудно объяснить без предыдущих глав, которые заложили основы.

- Глава 17 основана на главе 5. Вы подробно узнаете о системе ввода-вывода Julia, читая и записывая ракетные двигатели в файлы, сокет и конвейеры в формате CSV.
- Глава 18 объясняет приемы определения параметрического типа данных и причину, по которой параметрические типы выгодны для производительности, потребления памяти и правильности исходного кода.

Об исходном коде

Эта книга содержит много примеров исходного кода, как в пронумерованных листингах, так и в виде обычного текста. В обоих случаях исходный код отформатирован вот таким шрифтом фиксированной ширины.

Во многих случаях изначальный исходный код был переформатирован; мы добавили разрывы строк и переработали отступы, чтобы они соответствовали доступному пространству книжной страницы. В редких случаях даже этого было недостаточно, и листинги содержали маркеры продолжения строки (↵). Кроме того, комментарии в исходном коде часто из листингов удалялись, когда исходный код описывался в тексте. Многие листинги сопровождаются аннотациями к исходному коду, выделяя важные концепции.

Большая часть исходного кода пишется в интерактивной среде REPL¹ (работающей в цикле чтения–вычисления–печати) языка Julia либо в оболочке Unix. В этих случаях вы видите приглашение `julia>`, `shell>`, `help?>` или `$`. При наборе текста исходного кода их вставлять не нужно. Однако при вставке примеров исходного кода в окно терминала приглашение обычно отфильтровывается самим языком Julia.

Исходный код, предназначенный для записи в файл, обычно не отображается с приглашением. Однако если хотите, можете вставлять этот исходный код в интерактивную среду REPL.

Исполняемые фрагменты исходного кода можно получить из liveBook-версии (онлайновой версии) этой книги по адресу <https://livebook.manning.com/book/julia-as-a-second-language>. Полный исходный код примеров книги доступен для скачивания с веб-сайта издательства Manning по адресу <https://www.manning.com/books/julia-as-a-second-language> и с GitHub по адресу <https://github.com/ordovician/code-samples-julia-second-language>.

Для выполнения примеров исходного кода этой книги рекомендуется версия Julia 1.7 или выше.

¹ Англ. read-evaluate-print-loop. – Прим. перев.

Дискуссионный форум liveBook

Покупка книги «Julia в качестве второго языка» включает в себя бесплатный доступ к онлайн-платформе чтения книг издательства Manning под названием liveBook. Используя эксклюзивные возможности обсуждения на liveBook, можно прикреплять комментарии к книге глобально либо к определенным разделам или абзацам. Там можно легко делать заметки для себя, задавать технические вопросы и отвечать на них, а также получать помощь от автора и других пользователей. В целях получения доступа к форуму перейдите по ссылке <https://livebook.manning.com/book/julia-as-a-second-language/discussion>. Подробнее о форумах издательства Manning и правилах поведения также можете узнать по адресу <https://livebook.manning.com/discussion>.

Издательство Manning видит свою обязанность перед читателями в том, чтобы предоставлять место, где может происходить содержательный диалог между отдельными читателями и между читателями и автором. Это обязательство не требует от автора какого-то конкретного объема участия, чей вклад в форум остается добровольным (и неоплачиваемым). Мы предлагаем вам попробовать задать автору несколько сложных вопросов, чтобы поддержать его интерес! Форум и архивы предыдущих обсуждений будут доступны на веб-сайте издателя на протяжении всего времени, пока книга находится в печати.

Другие онлайн-ресурсы

Нужна дополнительная помощь? Язык Julia имеет активное рабочее пространство/сообщество Slack с более чем 10 000 участников, со многими из которых вы можете общаться в реальном времени. Информация о регистрации находится на странице <https://julialang.org/slack>.

- Julia Discourse (<https://discourse.julialang.org>) – это форумная платформа, где можно задать вопросы, связанные с Julia.
- На странице сообщества Julia по адресу <https://julialang.org/community> есть информация о каналах YouTube, предстоящих мероприятиях Julia, репозиториях GitHub и каналах Twitter.
- Официальная документация по языку и стандартной библиотеке Julia находится по адресу <https://docs.julialang.org/en/v1/>.

Отзывы и пожелания

Мы всегда рады отзывам наших читателей. Расскажите нам, что вы думаете об этой книге, – что понравилось или, может быть, не понравилось. Отзывы важны для нас, чтобы выпускать книги, которые будут для вас максимально полезны.

Вы можете написать отзыв на нашем сайте www.dmkpress.com, зайдя на страницу книги и оставив комментарий в разделе «Отзывы и рецензии». Также можно послать письмо главному редактору по адресу dmkpress@gmail.com; при этом укажите название книги в теме письма.

Если вы являетесь экспертом в какой-либо области и заинтересованы в написании новой книги, заполните форму на нашем сайте по адресу http://dmkpress.com/authors/publish_book/ или напишите в издательство по адресу dmkpress@gmail.com.

Список опечаток

Хотя мы приняли все возможные меры для того, чтобы обеспечить высокое качество наших текстов, ошибки все равно случаются. Если вы найдете ошибку в одной из наших книг, мы будем очень благодарны, если вы сообщите о ней главному редактору по адресу dmkpress@gmail.com. Сделав это, вы избавите других читателей от недопонимания и поможете нам улучшить последующие издания этой книги.

Нарушение авторских прав

Пиратство в интернете по-прежнему остается насущной проблемой. Издательства «ДМК Пресс» и Manning Publications очень серьезно относятся к вопросам защиты авторских прав и лицензирования. Если вы столкнетесь в интернете с незаконной публикацией какой-либо из наших книг, пожалуйста, пришлите нам ссылку на интернет-ресурс, чтобы мы могли применить санкции.

Ссылку на подозрительные материалы можно прислать по адресу электронной почты dmkpress@gmail.com.

Мы высоко ценим любую помощь по защите наших авторов, благодаря которой мы можем предоставлять вам качественные материалы.

Об авторе

Эрик Энгхейм – писатель, докладчик на конференциях, автор видеокурсов и разработчик программного обеспечения. Большую часть своей карьеры он посвятил разработке программного обеспечения 3D-моделирования и симулирования резервуаров в норвежской газовой и нефтяной промышленности. Эрик также несколько лет работал разработчиком iOS и Android. Эрик программирует на Julia, а также пишет и снимает видео о данном языке с 2013 года.

Об иллюстрации на обложке

Рисунок на обложке книги «Julia в качестве второго языка» – это «Paysanne Anglaise», или «английская крестьянка», взятая из сборника Жака Грассе де Сен-Совера, опубликованного в 1788 году. Каждая иллюстрация прекрасно нарисована и раскрашена вручную.

В те дни было легко определять место жительства людей, их род занятий и положение в обществе просто по их одежде. Издательство Manning отмечает изобретательность и инициативу вычислительного бизнеса книжными обложками, основанными на богатом разнообразии региональной культуры многовековой давности, оживленными фотографиями из таких коллекций, как эта.

Часть I

Основы

Эти главы охватывают весь язык Julia на базовом уровне. В последующих главах затронутые в этой части темы будут расширены. Вы научитесь работать с числами, массивами, инструкциями `if`, циклами `for`, строковыми литералами, базовым вводом-выводом, а также узнаете о хранении и извлечении данных из словарей. В последующих частях книги эти темы будут представлены подробнее.

Почему именно Julia?



Эта глава охватывает следующие ниже темы:

- тип задач, решаемых с помощью Julia;
- преимущества быстрого динамически типизированного языка;
- как Julia повышает продуктивность программиста.

У вас есть возможность выбирать из сотен языков программирования, многие из которых намного популярнее языка Julia. Так почему же именно Julia?

Хотите писать исходный код быстрее, чем раньше? А что скажете насчет создания систем с меньшим числом строк исходного кода, которое требуется в обычной ситуации? Наверняка такая продуктивность будет достигнута за счет плачевной производительности и большого потребления памяти. Отнюдь. На самом деле язык Julia предпочтителен для климатических моделей следующего поколения, которые имеют экстремальные требования по производительности и памяти.

Знаю, что такие похвалы, несомненно, создадут впечатление неудачной рекламной уловки продавца подержанных автомобилей, но нельзя отрицать, что Julia во многих отношениях является революционным языком программирования. Возможно, вы спросите: «Раз уж Julia – такой замечательный язык, то почему им не пользуются

все? Почему так много людей до сих пор используют язык программирования C?» Тут важны знакомство, пакеты, библиотеки и сообщество разработчиков. Перенос специфического программного обеспечения, разработанного в крупных организациях, не делается под влиянием момента.

Многих из вас, кто читает эту книгу, возможно, не интересует язык более эффективного и продуктивного программирования. Вместо этого вас прежде всего интересует, а что, собственно говоря, на нем можно разрабатывать. На этот вопрос есть простой ответ: все, что угодно. Julia – это язык общецелевого программирования.

По-видимому, такой ответ не будет удовлетворительным. В принципе, на JavaScript тоже можно разрабатывать все, что угодно. Тем не менее вы знаете, что JavaScript доминирует в разработке клиентских частей веб-приложений. На Lua тоже можно писать что угодно, но он в основном используется как скриптовый язык для компьютерных игр. При чтении этой книги, возможно, вас в первую очередь будет интересовать, какую именно работу Julia может для вас выполнять.

В настоящее время сообщество Julia наиболее активно занимается научными вычислениями, наукой о данных и машинным обучением. Но изучение языка Julia – это еще и ставка на будущее. Язык с такими мощными возможностями не останется в узкой нише. Если вы будете читать дальше, то вам станут понятнее особенности языка Julia и причины, по которым он обладает таким потенциалом. Я также затрону области, в которых Julia не идеален.

1.1 Что из себя представляет язык Julia?

Julia – это язык общецелевого и многоплатформенного программирования, который характерен следующим:

- пригодностью для численного анализа и вычислительной науки;
- динамической типизацией;
- высокой производительностью и компиляцией во время выполнения программы;
- автоматическим управлением памятью (сборкой мусора);
- компонуемостью.

Это немало, и некоторые из приведенных выше свойств выглядят противоречиво. Так как же языку Julia удастся быть общецелевым и при этом приспособленным для численного программирования? Он предназначен для общих задач, потому что, как и Python, язык Julia можно использовать практически для чего угодно. Он ориентирован на числовую обработку, потому что, как и MATLAB, хорошо подходит для численного программирования. Но он не ограничен численным программированием; он неплох и для других целей. Под *компонуемостью* подразумевается легкость, с которой Julia позволяет выражать многие архитектурные шаблоны объектно ориенти-

рованного и функционального программирования, способствуя ре-использованию исходного кода.

1.1.1 Плюсы и минусы статически и динамически типизированных языков

Давайте сосредоточимся на одном аспекте языка Julia: на его динамической типизируемости. Обычно языки программирования делятся на две большие категории:

- динамически типизированные;
- статически типизированные.

В статических языках типы есть у выражений; в динамических языках типы есть у значений.

– Стефан Карпински (Stefan Karpinski),
создатель языка Julia

Примерами статически типизированных языков являются C/C++, C#, Java, Swift, Go, Rust, Pascal и Fortran. В статически типизированном языке проверки типов выполняются по всему исходному коду перед выполнением программы.

Примерами динамически типизированных языков являются Python, Perl, Ruby, JavaScript, MATLAB и LISP. Динамически типизированные языки выполняют проверку типов во время работы программы. К сожалению, динамически типизированные языки работают очень медленно.

В динамических языках к значениям, таким как числа, буквенные символы и строковые литералы, прикреплены теги, указывающие тип, к которому они относятся. Эти теги позволяют программам, написанным на динамически типизированном языке, проверять правильность типов во время их исполнения.

Язык Julia необычен тем, что он является динамически типизированным и высокопроизводительным языком. Для многих наличие этих двух свойств выглядит как противоречие. Подобная уникальная черта языка Julia стала возможной благодаря тому, что данный язык был специально разработан под JIT-компиляцию¹ и в нем для всех вызовов функций используется особая функциональность, именуемая *множественной диспетчеризацией*². В таких языках, как C/C++ и Fortran, используется AOT-компиляция². Компилятор транслирует всю программу в машинный код перед ее исполнением. В других языках, таких как Python, Ruby и Basic, используется интерпретатор. В интерпретируемых языках каждая строка исходного кода чита-

¹ Англ. just-in-time (JIT) compilation; син. компиляция во время выполнения программы. – Прим. перев.

² Англ. ahead-of-time (AOT) compilation; син. компиляция перед выполнением программы. – Прим. перев.

ется и интерпретируется во время выполнения программы и затем инструкции исполняются. Теперь, когда у вас есть представление об особенностях языка Julia, можно приступить к обсуждению его привлекательности.

Конструктивное исполнение языка и JIT-компиляция

В принципе, язык программирования отделен от метода, используемого для работы с ним. И тем не менее вы обнаружите, что я говорю о Julia как о JIT-компилируемом языке, а о Fortran как о AOT-компилируемом языке. Строго говоря, это не совсем точно. Например, Julia может работать и посредством интерпретатора. Однако большинство языков были разработаны под конкретную форму исполнения. Julia был разработан под JIT-компиляцию.

1.2 Julia сочетает в себе элегантность, продуктивность и производительность

Хотя производительность является одним из ключевых преимуществ Julia, в 2013 году мое внимание привлекли его продуманность, мощность и простота в использовании. У меня была программа, которую я переписал на нескольких языках, чтобы сравнить выразительность, простоту в использовании и продуктивность программирования на каждом языке. На языке Julia мне удалось написать самый элегантный, компактный и легко читаемый вариант этого исходного кода. С тех пор я испробовал много других языков программирования, но так и не приблизился к тому, чего добился на языке Julia. Вот несколько однострочников, иллюстрирующих выразительность данного языка.

Листинг 1.1 Однострочники на языке Julia

```
filter(!isempty, readlines(filename)) # удалить пустые строки
filter(endswith(".png"), readdir())   # получить PNG-файлы
findall(==(4), [4, 8, 4, 2, 5, 1])    # найти все индексы числа 4
```

Занимаясь программированием с 1990-х годов, у меня были периоды, когда я чувствовал, что ну все, с меня программирования достаточно; язык Julia помог мне вернуть радость от программирования. Частично причина заключалась в том, что, освоив Julia, чувствуешь, что в твоём инструментарии появился язык, который работает как член твоего коллектива, а не против тебя. Думаю, что многие из нас имели опыт работы над задачей в ситуации, когда у вас было хорошее представление о методе ее решения, но используемый язык портил все дело. Ограничения языка вынуждают добавлять одну программную уловку за другой. На языке Julia я могу разрабатывать программное обеспечение так, как хочу, без языковых препон.

Еще один аспект, повышающий продуктивность и удовольствие, – поставляемая в комплекте обширная стандартная библиотека. Вы беретесь за дело с места в карьер и можете делать многое, не рыская по всему интернету в поисках какой-нибудь библиотеки, которая делает то, что вы хотите. Julia поможет вам, независимо от ваших потребностей, будь то линейная алгебра, статистика, HTTP, манипулирование строковыми литералами или же работа с разными форматами даты. А если нужной возможности в стандартной библиотеке нет, то в Julia есть тесно интегрированный менеджер пакетов, который упрощает добавление сторонних библиотек. Программирование на языке Julia почти заставляет вас чувствовать себя виноватым либо избалованным, потому что есть возможность разрабатывать богатые и элегантные абстракции без ущерба для целей по производительности.

Еще одним существенным преимуществом Julia является то, что данному языку легко обучаться. Эта простота помогает Julia с течением времени расширять сообщество разработчиков. В целях понимания причины легкой усваиваемости языка Julia давайте возьмем всем известную программу *Привет, мир*, написанную на Julia:

```
print("Hello world")
```

При ее выполнении приведенный выше исходный код выводит на экран текст *Hello world*. Несмотря на ее тривиальность, для того чтобы сделать нечто такое же простое, во многих языках требуется немало сложных строительных лесов. Ниже приведена программа на Java, которая делает то же самое:

```
public class Main {  
    public static void main(String[] args) {  
        System.out.print("Hello world");  
    }  
}
```

Приведенный выше фрагмент исходного кода одновременно выливает на новичка гораздо большее, нередко ошеломляющее, число концепций. Язык Julia легче в освоении, потому что он позволяет сосредотачиваться на одной концепции за раз. Например, можно научиться писать функцию, даже не видя определения типа. Благодаря огромному числу функциональностей, доступных прямо «из коробки», для написания полезного исходного кода даже не нужно уметь импортировать внешние библиотеки.

1.3 Для чего был создан язык Julia

В целях более глубокого ознакомления с тем, что Julia дает вам как разработчику, прежде всего необходимо лучше понять причину соз-

дания языка Julia. Создатели языка программирования Julia хотели решить то, что они назвали *проблемой двух языков*.

Эта проблема связана с тем, что многие программы пишутся с использованием двух разных языков программирования, каждый из которых имеет разные характеристики. В научной сфере часто предпочтение отдается динамическим языкам машинного обучения и анализа данных. Однако эти языки обычно не дают достаточно хорошей производительности. Поэтому программные решения нередко приходится переписывать на высокопроизводительных статически типизированных языках. Но в чем причина существования этого предпочтения? Почему бы не писать весь исходный код на традиционном высокопроизводительном статически типизированном языке?

1.3.1 Ученые нуждаются в интерактивном программировании, которое предлагается динамически типизированными языками

Ученые начали писать программное обеспечение, в том числе крупномасштабные метеорологические симуляции, на Fortran¹ и с использованием нейронных сетей² на C или C++³. Эти языки обеспечивают производительность, необходимую для решения указанных крупномасштабных задач. Однако эти языки имеют свою цену. Они тяготеют к жесткости, многословности и малой выразительности – все это снижает продуктивность программиста.

Однако фундаментальная проблема заключается в том, что эти языки не подходят для интерактивного программирования. Что я имею в виду? Интерактивное программирование – это возможность писать исходный код и получать немедленный отклик.

Интерактивное программирование имеет большое значение в науке о данных и машинном обучении. В типичном процессе анализа данных разработчик загружает большие объемы данных в интерактивную среду программирования и затем их обследует. После этого он выполняет различные виды анализа данных. Эти виды анализа могут предусматривать поиск среднего и максимального значений или построение гистограмм. Результаты первичного анализа сообщают программисту о характере последующих шагов.

На рис. 1.1 этот процесс показан на динамически типизированном языке. Все начинается с выполнения исходного кода загрузки

¹ Fortran (Formula Translation) – это старый язык научных вычислений.

² Нейронные сети – это своего рода алгоритм, симулирующий работу человеческого мозга.

³ C и C++ – это родственные и широко используемые статически типизированные языки системного программирования.

ки данных, которые затем можно обследовать. Однако при этом не обязательно проходить весь этот процесс после изменения исходного кода. Есть возможность изменять исходный код и сразу же видеть изменения. Перезагрузка огромных объемов данных не требуется.

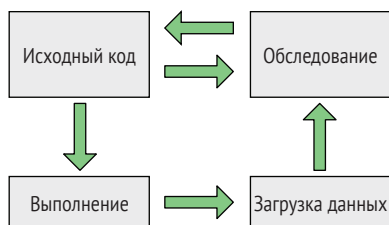


Рис. 1.1 В динамически типизированных языках можно переключаться между программированием и обследованием. Большие наборы данных не нужно перезагружать в память

Давайте сравним этот опыт с использованием статически типизированного языка, к примеру такого, как Fortran, C/C++ или Java¹. Разработчик должен писать исходный код загрузки данных и обследовать их, ничего не зная о том, как выглядят данные. Затем ему приходится ждать до тех пор, пока программа не сделает следующее:

- 1 скомпилируется;
- 2 запустится, а затем загрузит большой объем данных.

В этот момент разработчик видит график данных и статистические величины, которые дают ему информацию, необходимую для выбора следующего вида анализа. Но выбор следующего вида анализа потребовал бы повторения всего цикла заново. На каждой итерации должен перезагружаться большой блок данных. За счет этого чрезвычайно замедляется каждая итерация, замедляя весь процесс анализа в целом. Это статический, неинтерактивный способ программирования (рис. 1.2).

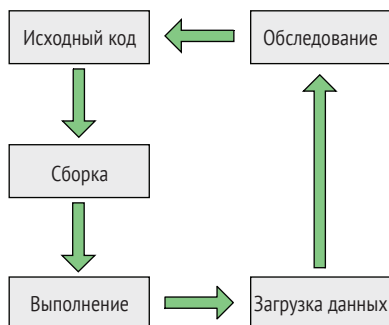


Рис. 1.2 Статически типизированные языки требуют повторения всего цикла

¹ Java используется во многих веб-серверных программах и в телефонах на базе Android.

1.3.2 Разработчикам в других областях тоже нужна интерактивность, предлагаемая динамически типизированным языком

Эта проблема не уникальна для ученых; разработчики игр уже давно столкнулись с той же проблемой. Игровые движки обычно пишутся на таких языках, как C или C++, которые компилируются в быстрый машинный код. Эта часть программного обеспечения нередко выполняет понятные и четко определенные действия, такие как рисование объектов на экране и проверка на столкновение объектов друг с другом.

Как и у аналитика данных, у разработчика игр есть много исходного кода, для удовлетворительной работы которого требуются многочисленные итерации. В частности, при разработке хорошего игрового процесса требуется много экспериментов и итераций. Нужно настраивать и изменять исходный код поведения персонажей в игре. Нужно многократно экспериментировать с композицией карты или уровнем, чтобы все работало, как надо. По этой причине почти во всех игровых движках используется второй язык, который позволяет изменять исходный код на лету. Часто это такие языки, как Lua¹, JavaScript и Python².

На этих языках исходный код игровых персонажей и карт можно изменять без перекомпиляции и перезагрузки карт, уровней и персонажей. Благодаря этому появляется возможность экспериментировать с игрой, делать паузы, вносить изменения в исходный код и сразу же продолжать с новыми изменениями.

Профессионалы машинного обучения сталкиваются с аналогичными проблемами. Они разрабатывают предсказательные модели, такие как нейронные сети, в которые они подают большие объемы данных, чтобы тренировать модели. Нередко здесь требуется столько же искусства, сколько и науки. Для обеспечения результативной работы моделей необходимо экспериментировать. Если приходится перезагружать тренировочные данные при каждом изменении модели, то процесс разработки замедляется. По этой причине в научном сообществе стали очень популярными динамически типизированные языки, такие как Python, R и MATLAB.

Однако, поскольку эти языки не очень быстрые, их используют в паре с такими языками, как Fortran и C/C++, чтобы выжимать нужную производительность. Нейронная сеть, созданная с помощью TensorFlow³ или PyTorch⁴, состоит из компонентов, написанных на

¹ Изначально Lua создавался как конфигурационный язык, но сегодня он в основном используется для написания игр.

² Сегодня Python – это один из самых популярных языков науки о данных и машинного обучения.

³ TensorFlow – это популярная библиотека машинного обучения и платформа на Python.

⁴ PyTorch – это популярная платформа машинного обучения на Python.

C/C++. Python используется для организации и соединения этих компонентов. Таким образом, во время выполнения эти компоненты можно реорганизовывать с помощью Python, не перезагружая всю программу.

Климатические и макроэкономические модели в процессе разработки, как правило, сначала разрабатываются на динамическом языке и тестируются на малом наборе данных. По завершении модели многие организации нанимают разработчиков на C/C++ или Fortran, чтобы те переписывали вычислительное решение на высокопроизводительном языке. Как следствие появляется лишний шаг, усложняющий процессы разработки и увеличивающий затраты.

1.4 *Более высокая производительность языка Julia решает проблему двух языков*

Язык Julia был создан решить проблему необходимости использования двух языков. Он позволяет сочетать гибкость динамически типизированного языка с производительностью статически типизированного языка. Вот почему приобрела популярность следующая ниже поговорка:

Julia ходит как Python и бежит как C.

– Популярная поговорка в сообществе Julia

Используя язык Julia, разработчики во многих областях могут писать исходный код с той же продуктивностью, что и на таких языках, как Python, Ruby, R и MATLAB. По этой причине язык Julia оказал глубокое влияние на отрасль. В июльском выпуске журнала Nature за 2019 год было взято несколько интервью с различными учеными об использовании ими языка Julia.

Например, благодаря переносу вычислительных моделей с R на Julia Мельбурнский университет добился 800-кратного улучшения. Джейн Херриман (Jane Herriman) из Калифорнийского технологического института материаловедения сообщает, что с тех пор, как она переписала свой Python'овский исходный код на Julia, его работа ускорилась в десять раз.

За час можно сделать то, на что в противном случае ушли бы недели или даже месяцы.

– Майкл Штумпф (Michael Stumpf)

В 2019 году на Международной конференции по суперкомпьютерам (SC19) один из создателей языка Julia Алан Эдельман (Alan Edelman) рассказал, как группа из Массачусетского технологического института (MIT) переписала часть своей климатической Fortran'овской

модели на языке Julia. Они заранее определили, что для них будет терпимо трехкратное замедление работы исходного кода. По их мнению, это было бы приемлемым компромиссом при получении доступа к языку высокого уровня с более высокой продуктивностью программирования. Вместо этого за счет переключения на Julia они получили 3-кратный прирост скорости.

Это всего лишь несколько историй из большого числа прецедентов, когда язык Julia революционизировал научные вычисления и высокопроизводительные вычисления в целом. Избегая проблемы двух языков, ученые получают возможность работать намного быстрее, чем раньше.

1.5 Julia для всех

Возможно, эти истории создают ложное впечатление о том, что Julia – это язык для умников в белых лабораторных халатах. Ничего подобного. Оказывается, многие черты, которые делают язык Julia отличным языком для ученых, также делают его отличным языком для всех остальных. Julia предлагает:

- мощные средства модуляризации и реиспользования исходного кода;
- строгую систему типов, помогающую выявлять дефекты в исходном коде во время выполнения;
- изощренную систему сокращения повторяющегося стереотипного исходного кода (метапрограммирование¹);
- богатую и гибкую систему типов, позволяющую моделировать широкий спектр задач;
- хорошо оснащенную стандартную библиотеку и различные сторонние библиотеки для решения различных задач;
- отличные средства обработки строковых литералов. Эта способность обычно является ключевым преимуществом любого многофункционального языка программирования, и это то, что изначально сделало популярными такие языки, как Perl, Python и Ruby;
- простое взаимодействие с большим числом других языков программирования и инструментов.

Хотя важное преимущество языка Julia относится к устранению им проблемы двух языков, это вовсе не означает, что отпадает потребность в интерфейсе с существующим исходным кодом, написанным на Fortran, C или C++. Смысл решения проблемы двух языков состоит в предотвращении необходимости писать исходный код на Fortran или C всякий раз, когда программист сталкивается с проблемой про-

¹ Метапрограммирование – это исходный код для написания исходного кода. Данная продвинутая концепция в книге не рассматривается.

изводительности. На всем протяжении можно оставаться в рамках языка Julia.

Однако если кто-то уже решил вашу задачу на другом языке, то, пожалуй, переписывание этого решения с чистого листа на языке Julia не будет иметь смысла. Python, R, C, C++ и Fortran имеют большие пакеты, которые создавались в течение многих лет, и сообщество Julia не может заменить их за одну ночь. В целях обеспечения продуктивной работы разработчикам Julia нужно уметь пользоваться существующими программными решениями.

В долгосрочной перспективе перенос унаследованного программного обеспечения на язык Julia даст очевидное преимущество. Поддержка старых библиотек Fortran часто требует от разработчиков гораздо больше усилий, чем поддержка библиотеки Julia.

Наибольшее преимущество, вероятно, заключается в предоставляемой языком Julia комбинаторной мощности. Определенные типы задач требуют создания больших монолитных библиотек. Julia, напротив, исключительно хорошо подходит для разработки малых библиотек, которые можно легко комбинировать, чтобы соответствовать функциональности, предлагаемой большими монолитными библиотеками на других языках. Приведу пример.

Машинное обучение, будучи животрепещущей темой, приводит в действие технологии беспилотного вождения автомобилей, распознавания лиц, распознавания голоса и многие другие инновационные технологии. Самыми известными пакетами машинного обучения являются PyTorch и TensorFlow. Эти пакеты представляют собой огромные монолиты, поддерживаемые большими коллективами разработчиков и сопровождаемых. Между ними нет никакого обмена исходным кодом. В Julia есть огромное число библиотек машинного обучения, таких как Knet, Flux (см. <https://fluxml.ai>) и Mocha (см. <http://mng.bz/epxG>). По сравнению с ними эти библиотеки – крошечные. Почему? Потому что возможностей PyTorch и TensorFlow можно достичь путем комбинирования нескольких малых библиотек Julia. Объяснение механизма и причин, по которым это работает, является сложной темой, требующей гораздо более глубоких знаний о языке Julia и о том, как работают нейросетевые библиотеки.

Наличие большого числа малых библиотек является преимуществом для общих приложений. Любой, кто разрабатывает программное обеспечение, выиграет от возможности реиспользовать существующие части программного обеспечения множеством новых способов, без необходимости изобретать велосипед. При использовании традиционных языков программирования часто приходится реализовывать одну и ту же функциональность. Например, библиотеки TensorFlow и PyTorch имеют много повторяющейся функциональности. Julia избегает дублирования, добавляя в библиотеки гораздо больше функциональных возможностей, общих между многими библиотеками машинного обучения. По мере прочтения глав этой книги вам будет становиться все яснее, как Julia удастся

это делать и почему во многих других языках эта возможность труднореализуема.

1.6 Что можно разрабатывать на языке Julia?

В принципе, на языке Julia можно разрабатывать все, что угодно. Однако у каждого языка есть экосистема пакетов и сообщество, которое подталкивает к одним типам разработки, а не к другим. Julia ничем не отличается.

1.6.1 Julia в науке

Язык Julia имеет сильное присутствие в науках. Данный язык используется, например, в:

- вычислительной биологии;
- статистике;
- машинном обучении;
- обработке изображений;
- вычислительном исчислении;
- физике.

Но язык Julia охватывает гораздо больше областей. Например, он используется в торговле энергией. Федеральная резервная система США применяет его для разработки сложных макроэкономических моделей. Лауреат Нобелевской премии Томас Дж. Сарджент (Thomas J. Sargent) основал платформу QuantEcon, которая продвигает педагогику в области количественной экономики с использованием как Julia, так и Python. Он горячий сторонник языка Julia, поскольку большие задачи в макроэкономике трудно решаемы с помощью других языков программирования. В интервью Лукасу Бивальду (Lukas Biewald) Питер Норвиг (Peter Norvig), работающий в Google известный исследователь искусственного интеллекта (ИИ), высказался, что, по его мнению, мир машинного обучения только выиграет от перехода на Julia.

Я был бы счастливее, если бы язык Julia стал основным языком ИИ.

– Питер Норвиг,
автор книги «Искусственный интеллект: современный подход»¹

Медико-биологические науки – еще одна очевидная область применения языка Julia. К 2025 году ежегодно будет собираться от 2 до 40 экзабайт данных о геноме человека. Большинство мейнстримных программ не смогут обрабатывать данные в таком масштабе. Для этого понадобится высокопроизводительный язык, такой как Julia,

¹ Artificial Intelligence, A Modern Approach. – Прим. перев.

который будет способен работать с различными форматами на различном оборудовании с максимально возможной производительностью.

На момент написания этой книги COVID-19 по-прежнему остается серьезной проблемой в мире. Пакет Julia Pathogen используется для моделирования инфекционных заболеваний и применяется исследователями COVID-19.

1.6.2 Использование Julia в ненаучных целях

А как насчет его ненаучного использования? В языке Julia также есть целый ряд пакетов для других интересных областей:

- Genie – полнофункциональный веб-фреймворк на основе архитектурного шаблона MVC;
- Blink – создание приложений с графическим пользовательским интерфейсом Electron на языке Julia;
- GTK – создание приложений с графическим пользовательским интерфейсом Julia с использованием популярного инструментария Linux GUI GTK;
- QML – создание кросс-платформенных графических интерфейсов с использованием языка разметки QML, используемого в инструментарии Qt GUI;
- GameZero – разработка игр для начинающих;
- Luxor – рисование векторных изображений;
- Miletus – составление финансовых контрактов;
- TerminalMenus – внедрение интерактивных меню в терминале;
- Gumbo – синтаксический разбор HTML-страниц;
- Cascadia – библиотека селекторов CSS для просмотра веб-страниц и извлечения полезной информации из веб-страниц;
- QRCode – создание изображений QR-кодов, широко применяемых в рекламе для отображения машиночитаемых URL-адресов.

Как видите, в Julia есть пакеты для общецелевого программирования.

1.7 Где язык Julia менее идеален

В принципе, Julia можно использовать практически для чего угодно, но поскольку данный язык молод, выбор библиотек не во всех областях одинаково обширен. Например, выбор пакетов для веб-разработки ограничен. Создать мобильное приложение на Julia пока что не получится. Он также не подходит для малых краткосрочных скриптов – таких, которые часто пишутся на Bash, Python или Ruby. Эти ограничения связаны с тем, что Julia компилируется на основе технологии JIT.

Это означает, что программы на Julia запускаются медленнее, чем, например, программы на Python или Bash, но начинают работать намного быстрее после того, как JIT-компилятор конвертирует критически важные части исходного кода в машинный код. Сообщество Julia постоянно прилагает усилия, чтобы минимизировать эту проблему, и существует громадное число способов ее решения. Решения предусматривают более оптимальное кеширование предыдущих JIT-компиляций и более избирательное отношение к тому, что следует JIT-компилировать.

Julia также неидеален для реально-временных систем. В реально-временной системе программное обеспечение должно реагировать на происходящее в реальном мире. Например, это можно сравнить с метеорологическим симулятором. В метеорологическом симуляторе не имеют значения события, происходящие в мире за пределами компьютера, на котором выполняется симуляция.

Однако если ваша программа должна обрабатывать данные, поступающие от измерительного прибора каждую миллисекунду, то внезапные сбои или задержки просто недопустимы. В противном случае есть риск потери важных данных измерений. Язык Julia укомплектован сборщиком мусора. Это означает, что данные, которые в вашей программе больше не используются, автоматически рециркулируются для других целей. Процесс определения планируемой к рециркуляции памяти обычно приводит к краткострочным случайным задержкам и заминкам в выполнении программы.

Эту проблему невозможно переоценить. Робототехнические комплексы, требующие реально-временного поведения, уже создаются на Julia. Исследователи из Массачусетского технологического института просимулировали реально-временное управление роботом-гуманоидом Boston Dynamics Atlas, удерживающим равновесие на плоской поверхности. Цель симуляции состояла в том, чтобы доказать, что язык Julia можно использовать для онлайн-управления роботами путем регулировки выделения и высвобождения памяти.

Julia плохо подходит для встраиваемых систем с ограниченной памятью. Причина в том, что Julia добивается высокой производительности за счет создания узкоспециализированных версий одного и того же исходного кода. Следовательно, потребление памяти для самого исходного кода Julia будет выше, чем, скажем, для исходного кода C, C++ или Python.

Наконец, точно так же, как Python, Ruby и другие динамические языки, Julia не подходит для типичного системного программирования, такого как создание систем баз данных или ядер операционных систем. Эти задачи, как правило, требуют детального контроля за использованием ресурсов, чего Julia не предлагает. Julia – это язык высокого уровня, ориентированный на простоту использования, а значит, многие детали использования ресурсов абстрагированы.

1.8 Что вы узнаете из этой книги

Если вы уже программируете на другом языке, эта книга для вас. Каждый язык программирования имеет уникальный набор функциональностей, инструментов и сообществ. В этой книге я сосредоточусь на уникальных характеристиках языка Julia, а также на инструментах и сообществе, созданном вокруг Julia, включая следующие неотъемлемые аспекты:

- 1 интерактивное программирование с использованием программы командной строки, работающей в цикле чтения–вычисления–печати (REPL)¹;
- 2 научно и математикоориентированные примеры исходного кода;
- 3 уникальная функциональность множественной диспетчеризации и система типов Julia;
- 4 функциональное программирование и его сравнение с объектно ориентированным программированием;
- 5 пакетно ориентированная разработка вместо разработки, ориентированной на приложения.

Разработка на основе интерактивной среды REPL означает возможность запускать инструмент командной строки Julia и начинать набирать выражения Julia, которые вычисляются при нажатии клавиши **Enter**:

```
julia> reverse("abc")  
"cba"
```

```
julia> 3+5  
8
```

На протяжении большей части книги я придерживаюсь именно такого подхода; возможно, указанный подход будет малознаком читателям, пришедшим из таких языков, как C/C++, Java и C#, но в сообществе Julia этот стиль разработки нередко является предпочтительным. Интерактивная среда REPL используется для экспериментирования, тестирования и отладки.

Поскольку Julia широко применяется в науке о данных, машинном обучении, математике и естественных науках, в этой книге используется много примеров, ориентированных на науку и математику, таких как вычисление значений синуса или симуляция запуска ракеты, а не на разработку веб-сайта или системы инвентаризации. В целом математика в этой книге излагается на уровне средней школы.

В этой книге вы найдете подробное описание системы множественной диспетчеризации и системы типов Julia. Указанные системы важны тем, что именно благодаря им Julia достигает высокой произ-

¹ REPL (англ. read-evaluate-print loop) – это интерактивная программа командной строки языка программирования.

водительности. Так как многие новички программирования на языке Julia в этих системах путаются, я остановлюсь на них несколько подробнее.

Поскольку в индустрии программного обеспечения по-прежнему доминируют языки объектно ориентированного программирования, переход к более функциональному стилю программирования Julia, возможно, будет дезориентировать. Поэтому я посвятил место демонстрации того, как одни и те же задачи можно решать в функциональном и объектно ориентированном стиле. На протяжении всей книги используются многие предпочтительные практики функционального программирования.

Работая с этой книгой, вы не увидите большого числа готовых приложений, то есть таких, которые запускаются при нажатии по значку приложения. Вы также не увидите созданных на языке Julia инструментов командной строки, которые можно выполнять из консоли. Этот подход будет новым, например, для разработчиков на Ruby и Python, которые очень привыкли создавать программное обеспечение в виде инструментов командной строки.

В отличие от этого подхода, сообщество разработчиков на языке Julia очень ориентировано на пакеты. Они рекомендуют создавать пакеты поверх автономных приложений, так как ими легче делиться с другими и реиспользовать. Это предпочтение отражено в цепочке инструментов Julia и менеджере пакетов. Julia вовсе не мешает создавать приложения, но эта книга поможет вам принять образ мыслей, ориентированный в первую очередь на пакеты. Сначала создать пакет, а затем превращать его в приложение.

Пакетно ориентированное мышление заметно в том, как доставляются инструменты Julia. Работа с менеджером пакетов и отладчиком сводится к загрузке определенных пакетов в интерактивную среду Julia и исполнению команд в ней, а не в оболочке. Этот подход к работе, возможно, будет знаком пользователям MATLAB и R. В этих двух языках основное внимание обычно уделяется пакетам, а не приложениям.

Пользующийся языком Julia типичный статистик, ученый или аналитик данных может загружать предпочитаемые пакеты в свою среду Julia и выполнять команды Julia, не кликая по какому-либо приложению, созданному на языке Julia. Интерактивная среда REPL обычно является неотъемлемой частью большинства рабочих процессов языка Julia.

Резюме

- Статическая типизация упрощает создание высокопроизводительных языков программирования и отлавливание ошибок типов до выполнения программы.

- Динамическая типизация позволяет создавать языки интерактивного программирования. В этом преимущество программирования, требующего быстрой итеративной обработки.
- Разработка научного исходного кода нередко требует возможности легко экспериментировать с большими наборами данных. А это требует возможностей интерактивного программирования, предлагаемого динамически типизированными языками.
- Научный исходный код часто требует высокой производительности, обеспечить которую динамически типизированные языки обычно не способны.
- Julia решает проблему двух языков, предлагая высокопроизводительный динамически типизированный язык. Эта способность помогает внедрять Julia в областях, требующих высокой производительности, таких как климатическое моделирование, астрономия и макроэкономические симуляции.
- Julia не ограничивается наукой: он также является отличным языком общецелевого программирования.

Julia в качестве калькулятора

Эта глава охватывает следующие ниже темы:

- работа с целыми числами, числами с плавающей точкой¹ и дробями;
- использование переменных для хранения длинных чисел;
- создание реиспользуемых вычислений путем определения функций;
- самые базовые типы Julia.

Даже если вы никогда не перестанете использовать Julia в качестве основного языка, вы все равно будете ценить его как замену настольному калькулятору. Julia может даже выступать в качестве продвинутого графического калькулятора, который обычно используется в старших классах средней школы (рис. 2.1). В качестве бонуса его можно использовать совершенно бесплатно.

Вспомните, что прежде чем начать бегать, вы сперва научились ходить, и как раз обследование чисел является отличным способом познакомиться с ключевыми концепциями Julia. Поскольку язык

¹ В переводе оставлена форма записи чисел, принятая в оригинале книги, где для отделения дробной части принято использовать точку и для отделения групп по три числа многозначного числа принято использовать запятую. – Прим. перев.

Julia не ограничивается только общецелевым программированием, а в первую очередь специально предназначен для числовых вычислений, манипулирование числами в Julia играет уникальную роль.

В этой главе вы рассмотрите те аспекты языка Julia, которые позволяют выполнять то же самое, что принято делать на калькуляторе. Конечно же, вы возразите, что не собираетесь использовать Julia в качестве калькулятора, однако это станет способом формирования фундамента для вашего понимания более сложных тем.

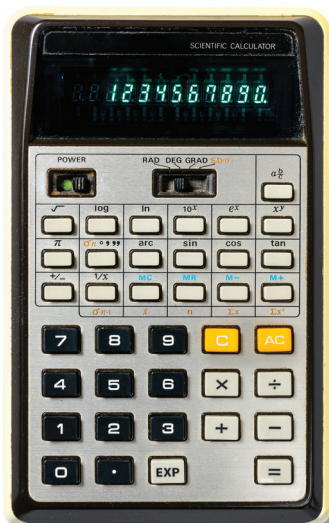


Рис. 2.1 Научный ретрокалькулятор. Можно ли заменить портативный калькулятор на интерактивную среду REPL языка Julia?

2.1 Командная строка Julia

Если вы установили и сконфигурировали Julia правильно (см. дополнение А), то можете набрать `julia` в строке терминала, чтобы запустить интерактивную среду REPL языка Julia. Указанная программа командной строки читает вводимые данные подобно калькулятору и печатает результат после нажатия клавиши **Enter**. Интерактивная среда REPL – это место, где можно тестировать свой исходный код, заглядывать в документацию и устанавливать стороннее программное обеспечение.

В этой главе вы сосредоточитесь на вычислении математических выражений в интерактивной среде REPL. В следующем ниже примере исходного кода показано, как запускать командную строку Julia из терминального (консольного) приложения. После запуска наберите $2 + 3$ и нажмите **Enter**. Julia вычислит это выражение и выведет результат 5:

```
$ julia
```

```

      _ _ _      _ _ _      _ _ _      _ _ _      _ _ _      _ _ _
     ( )  _ _ _  ( )  _ _ _  ( )  _ _ _  ( )  _ _ _  ( )  _ _ _
    _ _ _  _ _ _  _ _ _  _ _ _  _ _ _  _ _ _  _ _ _  _ _ _
   _ _ _  _ _ _  _ _ _  _ _ _  _ _ _  _ _ _  _ _ _  _ _ _
  _ _ _  _ _ _  _ _ _  _ _ _  _ _ _  _ _ _  _ _ _  _ _ _
 _ _ _  _ _ _  _ _ _  _ _ _  _ _ _  _ _ _  _ _ _  _ _ _
|_|/

Documentation: https://docs.julialang.org
Type "?" for help, "]" for Pkg help.
Version 1.6.0 (2021-03-24)
Official https://julialang.org/ release

julia> 2 + 3
5
```

Помимо сложения двух однозначных чисел, можно выполнять гораздо более сложные операции. В следующем ниже примере выполняется несколько очень распространенных математических операций, включая получение логарифма, синуса и квадратного корня. `rand` – это математическая функция, которая возвращает случайное число от 0 до 1:

```

julia> 4 * 5
20

julia> (1.5 + 3.5)*4 ①
20.0

julia> 2^3 ②
8

julia> log(2.71828^4) ③
3.999997309389128

julia> sin(3.1415926/2) ④
0.9999999999999997

julia> sqrt(9) ⑤
3.0

julia> rand() ⑥
0.14765146459147327
```

- ① Сложные выражения можно заключать в круглые скобки.
- ② Показатели степени; возведение двойки в степень три.
- ③ Натуральный логарифм числа 2.71, также именуемого числом Эйлера e .
- ④ Получение синуса $\pi/2$ радиан.
- ⑤ Квадратный корень из 9.
- ⑥ Генерирование случайного числа от 0 до 1.

2.2 *Использование констант и переменных*

Можно попробовать запомнить все цифры в таких числах, как 3.1415926... (π) или 2.71828... (число Эйлера e), однако это утоми-

тельно. На самом деле это невозможно, так как оба числа являются так называемыми иррациональными числами, то есть они имеют бесконечное число цифр. Поэтому гораздо лучше дать каждому числу имя, или, если быть точнее, *идентификатор*.

ВАЖНО *Переменные и константы задают области памяти, в которых хранятся значения (данные). Думайте о памяти как о длинном списке пронумерованных почтовых ящиков, каждый из которых содержит значение. Для того чтобы не запоминать номер содержащего значение почтового ящика, надо прикрепить к нему именованный идентификатор. Значение переменной можно изменять после ее создания, но значение константы остается неизменным.*

В Julia идентификаторы можно использовать для именования констант, переменных, типов и функций. `pi` (π), `golden` (φ) и число Эйлера `e` – это идентификаторы, используемые для обозначения числовых констант. И константы, и переменные упрощают запоминание длинных и сложных чисел:

```
julia> using Base.MathConstants ❶  
julia> pi  
 $\pi$  = 3.1415926535897...
```

```
julia> e ❷  
e = 2.7182818284590...
```

```
julia> golden ❸  
 $\varphi$  = 1.6180339887498...
```

```
julia> catalan ❹  
catalan = 0.9159655941772...
```

- ❶ Общематематические константы в Julia можно делать доступными. Константа `pi` всегда доступна, как и остальные.
- ❷ Число Эйлера широко используется с логарифмами.
- ❸ Золотое сечение часто используется в искусстве из эстетических соображений и проявляется в природе, например в спиральном расположении листьев.
- ❹ Постоянная Каталана.

С этими константами становится удобнее писать математические выражения. Кроме того, результаты станут более точными, потому что написать `e` или `pi` с достаточным числом цифр просто невозможно. Это иррациональные числа с бесконечным числом цифр. Математики на самом деле не знают, является ли число Каталана иррациональным или нет, но в Julia оно моделируется как иррациональное число:

```
julia> log(e^4)  
4.0
```

```
julia> sin(pi/2)  
1.0
```

Однако вы не ограничены использованием встроенных констант. В Julia с помощью ключевого слова `const` можно определять свои собственные константы и использовать их в вычислениях вместо числовых литералов¹:

```
julia> const foo = 3481
3481

julia> const bar = 200
200

julia> foo + bar
3681

julia> const qux = 8
8

julia> qux + 2
10
```

Возможно, вам будут интересны забавно звучащие имена `foo`, `bar` и `qux`. Эти бессмысленные слова обычно используются в примерах исходного кода, чтобы сообщать читателю, что в данном случае можно выбирать *любое* слово. Они отличаются от зарезервированных слов, таких как `if`, `while`, `const` и `function`; использовать их в качестве имен переменных не разрешено.

При написании идентификаторов Julia можно перемешивать и сочетать регистры. `fooBaR` и `F00bAr` одинаково допустимы. Но язык Julia чувствителен к регистру, поэтому эти идентификаторы будут трактоваться как *разные*.

СОВЕТ Идентификаторы Julia чувствительны к регистру. Идентификаторы `foo`, `Foo` и `F00` не будут считаться одинаковыми. Сегодня это стандартная практика в большинстве современных языков программирования.

При этом можно добавлять цифры, при условии что они не стоят в начале слова. Таким образом, идентификатор `f00bAr` допустим, а `100bAr` – нет. Вы должны уже привыкнуть к аналогичным правилам в других языках программирования.

Язык Julia необычен тем, что в нем часто используются греческие буквы, такие как π , θ , α и Δ . Причина в том, что математические формулы обычно пишутся греческими буквами. Когда математическое уравнение реализовано в исходном коде, его становится легче читать, если оно похоже на уравнение.

В целях обеспечения такой возможности создатели языка Julia внедрили в подключаемые модули интерактивной среды REPL и ре-

¹ Числовой литерал состоит из цифр от 0 до 9; это неименованная переменная.

доктора Julia специальные возможности, упрощающие написание греческих букв и других символов Юникода. Например, в интерактивной среде REPL надо набрать обратную косую черту, затем имя нужного знака и потом нажать клавишу **Tab**:

```
julia> \pi
```

После нажатия клавиши табуляции эти знаки превращаются в

```
julia> π
```

Ниже приводится сводка из нескольких популярных греческих букв и символов Юникода, которые можно использовать в своем исходном коде, с некоторыми комментариями о том, что они обычно означают.

Символ	Заполнение по Tab	Применение
π	\pi	Уравнение окружности
θ	\theta	Угол
Δ	\Delta	Разница или изменение чего-либо
e	\euler	Число Эйлера (важно в логарифмах)
√	\sqrt	Квадратный корень числа
φ	\varphi	Золотое сечение

Чем переменные отличаются от констант? Создание переменных очень похоже, за исключением того, что не используется ключевое слово `const`:

```
julia> y = 7
7
```

```
julia> z = 3
3
```

```
julia> y + z + 2
12
```

Так в чем же разница? Приведенное ниже взаимодействие в интерактивной среде REPL демонстрирует разницу между переменными и константами:

```
julia> const x = 9
9
```

```
julia> y = 7
7
```

```
julia> x = 12
WARNING: redefinition of constant x. This may fail, cause incorrect
answers, or produce other errors.
12
```

```
julia> y = 8  
8
```

В этом примере мы сделали x константой, а y – переменной. Обратите внимание, как Julia предупреждает о том, что мы пытаемся изменить значение константы. Хотя на самом деле это, возможно, и сработает, Julia не дает никаких гарантий, поэтому Julia вас предупреждает. Никогда не основывайте свой исходный код на неопределенном поведении.

ПОЛЕЗНЫЕ ГОРЯЧИЕ КЛАВИШИ Используйте **Ctrl+D**, чтобы выйти из Julia. **Ctrl+C** прерывает исполнение зацикленного исходного кода. Очистка экрана терминала зависит от операционной системы. Для очистки терминала в Mac следует использовать **Command+K**, а в Linux следует использовать **Ctrl+L**.

Перезапустите Julia, чтобы сделать все идентификаторы снова доступными.

2.2.1 Присваивание и привязывание значений к переменным

В Julia оператор `=` используется для присваивания значения переменной. Сравнение двух выражений на эквивалентность осуществляется с помощью двойного знака равенства `==`. Однако если быть точным, то, что Julia делает, является не присваиванием, а *привязыванием*. В целях более глубокого понимания механизма работы привязывания я представлю пример исходного кода, в котором переменная x сначала привязывается к значению 2. Затем она повторно привязывается к значению $x + 3$:

```
julia> x = 2      ❶  
2
```

```
julia> x = x + 3  ❷  
5
```

- ❶ Задать первоначальное значение x равным 0.
- ❷ Увеличить x на два.

Если бы этот пример исходного кода был написан на таком языке, как C/C++, Fortran или Pascal, то для хранения переменной x система выделила бы ячейку памяти. При каждом присваивании нового значения переменной x хранящееся в этой ячейке памяти число будет изменяться.

В случае с привязыванием все работает иначе. Каждое вычисление нужно трактовать как создание числа, которое помещается в другую ячейку памяти. Привязывание предусматривает перемещение самой метки x в новую ячейку памяти. Переменная перемещается в результат, а не результат перемещается в переменную. Рисунок 2.2

иллюстрирует пошаговое объяснение и поможет понять принцип работы привязывания.

- 1 Значение 2 сохраняется в ячейке памяти номер 2. Метка x прикрепляется к этому значению, что эквивалентно первоначальной инструкции присваивания $x = 2$.
- 2 Julia начинает вычислять $x + 3$ в выражении $x = x + 3$. Результат этого вычисления сохраняется в ячейке памяти под номером 4.
- 3 По завершении вычисления инструкции $x = x + 3$ Julia перемещает метку x в ячейку памяти 4.

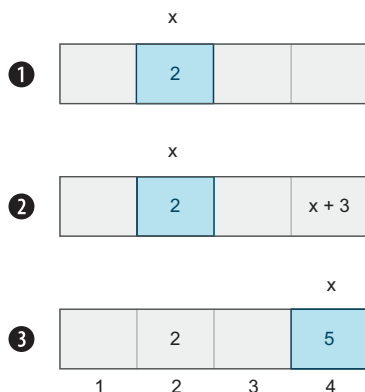


Рис. 2.2 Привязывание переменной x к разным ячейкам памяти

Но почему Julia и другие динамически типизированные языки выполняют привязывание вместо присваивания? На таком языке, как C/C++, можно писать инструкции, подобные следующим ниже.

Листинг 2.1 Присваивание в C/C++

```
int x = 4;
x = 5;

char ch = 'A';
ch = 'B';
```

Это работает, потому что компилятор обеспечивает, чтобы у программиста не было возможности поместить значение, которое не может поместиться в ячейку памяти, отведенную для переменной x или переменной ch . В динамически типизированном языке переменной x можно присваивать любое значение, поэтому она не может иметь predetermined размер в predetermined месте в памяти.

2.2.2 Использование переменной *ans*

В Julia есть специальная переменная, которая существует только при интерактивном использовании Julia и называется *ans* (от англ.

answer, то есть «ответ»). Интерактивная среда REPL присваивает ей значение последнего вычисленного выражения. Выражения в обычных программах ей не присваиваются.

Многие люди будут знакомы с подобной переменной, если они использовали продвинутые калькуляторы. Переменная `ans` всегда содержит результат последнего вычисления. Такое поведение выгодно в практическом плане, так как позволяет легко использовать результат последнего вычисления в следующем ниже вычислении:

```
julia> 3 + 2  
5
```

```
julia> ans*4  
20
```

```
julia> ans  
20
```

```
julia> ans - 8  
12
```

2.2.3 Что такое литеральный коэффициент?

Как известно из программы средней школы, выражения $3 \times x + 2 \times y$ записываются как $3x + 2y$. Julia позволяет писать умножение таким же образом. Экземпляры такой записи называются *литеральными коэффициентами*¹ как своего рода аббревиатура умножения числового литерала на константу или переменную:

```
julia> x = 3  
3
```

```
julia> 2x  
6
```

```
julia> 2*(3+2)  
10
```

```
julia> 2(3+2)  
10
```

Литеральные коэффициенты работают только для фактических числовых литералов. Например, π , e и ϕ не являются числовыми литералами. Можно написать 2π , но не $\pi 2$, потому что последнее подразумевало бы идентификатор.

Существует тонкая разница между использованием литеральных коэффициентов и выполнением умножения. Попробуйте разобраться в следующем ниже примере:

¹ Англ. literal coefficient. – Прим. перев.

```
julia> x = 5
5

julia> 1/2x
0.1

julia> 1/2*x
2.5
```

Что здесь происходит? $1/2x$ интерпретируется как $1/(2*x)$. Литеральные коэффициенты имеют более высокий приоритет, чем деление.

2.3 Разные типы чисел и их длина в битах

Julia имеет разные типы чисел, такие как целые числа со знаком, целые числа без знака и числа с плавающей точкой с разной длиной в битах. Если эти понятия вам незнакомы, то советую почитать о разных типах чисел в дополнении В.

Давайте сосредоточимся на специфических для Julia особенностях. В Julia целые числа со знаком называются `Int8`, `Int16`, `Int32`, `Int64` и `Int128`. Суффикс числа указывает на длину числа в битах. Имена типов целых чисел без знака формируются путем префиксации буквой `U`, давая типы `UInt8`, `UInt16`, `UInt32`, `UInt64` и `UInt128`.

Во время исполнения исходного кода нередко возникает потребность узнать минимальное и максимальное значения определенного целочисленного типа. Для получения минимального и максимального значений используются функции `typemin` и `typemax`. Например, вызов `typemin(Int8)` возвращает `-128`, потому что 8-битовое целое число не может представлять значение меньше `-128`. Вызов `typemax(Int8)` вернет `127`:

```
julia> typemax(Int8)
127

julia> typemin(Int8)
-128
```

Длина в битах числового литерала Julia по умолчанию равна длине 64-битового целого числа со знаком. В этом можно легко убедиться, применив функцию `typeof`, которая возвращает тип входного аргумента:

```
julia> typeof(1)
Int64
```

Тогда как формировать числа с другой длиной в битах? Если вы хотите создать 8-битовое число со знаком, то надо написать `Int8(x)`, где `x` – это число, которое нужно превратить в 8-битовое число. Это

работает для любого типа чисел. Естественно, если для длины в битах попытаться ввести слишком большое число, то будет выдано сообщение об ошибке:

```
julia> y = Int8(42)
42

julia> typeof(y)
Int8

julia> typeof(Int16(4))
Int16

julia> UInt8(256)
ERROR: InexactError: trunc(UInt8, 256)
```

Следует знать, что, в отличие от других популярных динамически типизируемых языков, таких как Python, Ruby и R, в Julia отсутствует автоматический выбор достаточно большого для хранения результата арифметической операции типа чисел. В Julia при сложении двух значений `Int8` результатом всегда будет значение `Int8`.

Если бы в других динамических языках результат был слишком большим, чтобы его можно было представить в виде 8-битового целого числа, то он был бы обновлен до `Int16`. В Julia вместо этого будет получено переполнение. Если концепция целочисленного переполнения вам незнакома, то обратитесь к прилагаемому дополнению В.

Иногда для хранения значения даже тип `Int128` недостаточно велик. В таких случаях используется тип `BigInt`, который может содержать целое число произвольного размера. За эту гибкость приходится платить более высоким потреблением памяти и более низкой производительностью, поэтому `BigInt` следует использовать только тогда, когда это действительно необходимо.

2.3.1 *Написание чисел с использованием разных числовых форматов*

Механизм записи числа и механизм хранения числа в памяти – это две разные вещи. Числа `0b1101`, `0x0d` и `13` хранятся в памяти компьютера в виде все тех же двоичных чисел. По умолчанию Julia показывает все числа со знаком в десятичном формате, а числа без знака, такие как `UInt8`, – в шестнадцатеричном формате:

```
julia> Int(0b1101)
13

julia> Int(0x0d)
13

julia> UInt8(13)
0x0d
```

Шестнадцатеричные числа популярны в низкоуровневом бит-ориентированном программировании. Это связано с тем, что четыре бита можно представлять ровно одним шестнадцатеричным числом. Восьмеричные числа также популярны, потому что для представления одного восьмеричного числа можно использовать ровно три бита.

Шестнадцатеричные и восьмеричные числа

Десятичные числа создаются путем комбинирования цифр от 0 до 9. Восьмеричные числа создаются путем комбинирования цифр от 0 до 7. Так, число 8 в восьмеричной системе счисления будет записано как 10.

С шестнадцатеричными числами возникает проблема, поскольку цифры в шестнадцатеричной системе счисления должны охватывать значения от 1 до 15; однако символы есть только для цифр от 0 до 9. Решение заключалось в использовании букв для цифр после 9; так, значение 10 представляется как A, 11 представляется как B и т. д. F означает 15. Наибольшее значение, которое может содержать 8-битовое целое число без знака, равно 0xff, что в десятичной форме соответствует числу 255.

Для написания восьмеричного числа следует использовать префикс 0o – для этого хорошее понимание не требуется. Главное, чтобы вы осознали тот факт, что существуют разные способы представления одного и того же числа. Это важно понимать во избежание путаницы при экспериментировании с целыми числами без знака в этой главе, поскольку Julia по умолчанию выводит их в шестнадцатеричной форме:

```
julia> 0o5 == 0b101
true

julia> 0o6 == 0b110
true

julia> 0o7 == 0b111
true

julia> 0o10 == 0b1000
true

julia> Int(0o10)
8

julia> Int(0o23)
19

julia> 2 * 8 + 3
19
```

2.4 Числа с плавающей точкой

Как и с целыми числами, существуют числа с плавающей точкой разной длины в битах. В Julia их размер по умолчанию равен 64 битам, то есть каждое число с плавающей точкой занимает 8 байт памяти. Используя больше бит, можно не только представлять большие числа, но и представлять числа с большей глубиной точности, или прецизионностью¹. Однако прецизионность не всегда важна. Для научных расчетов прецизионность имеет важное значение, но в обычных расчетах (например, для компьютерной графики) она играет меньшую роль. Один пиксел из миллионов, который немного отличается по позиции или цвету, не имеет большого значения. Наиболее распространенные типы с плавающей точкой, `Float64` и `Float32`, можно записывать как числовые литералы:

```
julia> 42.98      ❶  
42.98
```

```
julia> typeof(ans)  
Float64
```

```
julia> 34.23f0    ❷  
34.23f0
```

```
julia> typeof(ans)  
Float32
```

- ❶ 64-битовый числовой литерал с плавающей точкой. Это можно проверить функцией `typeof`.
- ❷ 32-битовое число с плавающей точкой.

Обратите внимание на использование суффикса `f0`, чтобы сделать число 32-битовым числом с плавающей точкой. Почему нет просто `f`, как в Java и C/C++? Это связано с особенностью литеральных коэффициентов. Взглянув на следующий ниже сеанс REPL, вы сможете понять происходящее:

```
julia> 0.5f  
ERROR: UndefVarError: f not defined
```

```
julia> f = 2  
2
```

```
julia> 0.5f  
1.0
```

```
julia> 2f  
4
```

¹ Англ. precision. – Прим. перев.

Если попытаться написать 32-битовое число с плавающей точкой, как в Java или C/C++, то Julia посчитает, что вы пытаетесь умножить число на переменную *f*. В первом случае это не удастся, потому что переменная *f* еще не была определена. Во втором случае это сработает, потому что *f* определена.

Как насчет других значений с плавающей точкой, таких как 16-битовые значения? В этих случаях необходимо выполнить конвертацию в явной форме:

```
julia> x = Float16(3.5) ①
Float16(3.5)

julia> typeof(x)
Float16

julia> z = Float16(4) + 5f0
9.0f0

julia> typeof(z) ②
Float32
```

① Конвертирование 64-битового значения с плавающей точкой в 16-битовое.

② Смешивание чисел разной длины в битах побуждает Julia выбирать для сохранения результата самый большой тип.

2.4.1 *Выполнение операций на целых числах и числах с плавающей точкой*

Хотя на числах с плавающей точкой и целых числах можно выполнять много одинаковых операций, эти операции не всегда дают одинаковые результаты. И есть операции, которые работают только на определенных типах чисел.

Например, оператор деления `\` выдает на выходе числа с плавающей точкой. Это не всегда то, что нужно. При работе с целыми числами вместо этого часто требуется частное и остаток. Это достигается с помощью функций `div` и `rem`:

```
julia> 4/2 ①
2.0

julia> 5/2 ①
2.5

julia> 5.0/2.0 ①
2.5

julia> div(5,2) ②
2

julia> rem(5,2) ③
1
```

```
julia> 5%2      ❸
1
```

- ❶ Обычный оператор деления дает результат с плавающей точкой.
- ❷ Целочисленное деление, которое дает целочисленный результат.
- ❸ Остаток также можно получить с помощью оператора %.

2.5 Определение функций

Вы уже познакомились с некоторыми функциями, такими как `sin`, `cos` и `√`. Эти функции можно найти в обычном калькуляторе; они принимают число на входе и возвращают число на выходе. Но какая фундаментальная идея функций? И второе: насколько одинаковы функции в математике и в Julia? Детали различаются, но концептуально они одинаковы. Функции имеют ноль или более входных значений, именуемых *аргументами*. Функции можно трактовать как фрагменты исходного кода, которые возвращают или вычисляют значение. Рассмотрим объем сферы:

$$V = \frac{4\pi r^3}{3}.$$

Насколько хорошо вы помните его вычисление? Мне вот часто приходится сверяться со справочником. На языке Julia это вычисление можно выполнить, написав следующий ниже исходный код:

```
julia> r = 4.5      ❶
4.5
```

```
julia> V = 4*pi*r^3/3      ❷
381.7035074111598
```

- ❶ Радиус сферы.
- ❷ Сохранение объема сферы в переменной V.

Переменные и константы облегчают запоминание длинных и сложных чисел. Во многих отношениях функции можно трактовать как расширение этой идеи. Они позволяют запоминать сложные расчеты. Вместо запоминания чисел, которые следует умножать и делить, нужно только запомнить имя функции:

```
julia> sphere_volume(r) = 4*pi*r^3/3      ❶
sphere_volume (generic function with 1 method)
```

```
julia> sphere_volume(2)      ❷
33.510321638291124
```

```
julia> sphere_volume(4)      ❸
268.082573106329
```

```
julia> sphere_volume(1)
4.1887902047863905
```

- ❶ Определить функцию `sphere_volume`, которая принимает один аргумент `r` со значением радиуса сферы.
- ❷ Применить ранее определенную функцию сферы, чтобы вычислить объем сферы с радиусом 2.
- ❸ Объем сферы с радиусом 4.

Обратите внимание, что, в отличие от переменной, при определении функции указывается один или несколько аргументов. Аргументы – это переменные, которые можно изменять при каждом вычислении. Например, при вычислении объема сферы мы хотим, чтобы значение π всякий раз было одинаковым; следовательно, π не является аргументом функции. Однако радиус аргументом является, потому что нас интересует вычисление объема сфер разных радиусов:

```
foo(x, y, z) = 2x + 4y - z
```

В приведенном выше фрагменте исходного кода показано простое определение функции. Это функция с именем `foo`, принимающая три разных аргумента с именами `x`, `y` и `z`. При этом можно иметь лишь несколько аргументов либо много. Правила их именования остаются такими же, как и для любого идентификатора Julia.

2.5.1 Хранение определений функций в файле

Писать определение каждой функции, которую вы хотите использовать в интерактивной среде REPL, при каждом перезапуске Julia было бы нецелесообразно. Вместо этого определения функций можно хранить в отдельных файлах исходного кода.

КОММЕНТАРИИ К ИСХОДНОМУ КОДУ В исходном коде можно писать комментарии, чтобы напоминать себе о выполняемой различными его частями работе. Комментарии начинаются со знака решетки: `#`. Все, что находится после него, компилятор Julia игнорирует.

Позже, когда потребуются содержащиеся в этом файле функции, его можно будет загружать в интерактивную среду REPL. Давайте я продемонстрирую это на примере. Мы создадим файл с именем `volumes.jl`. Внутри него сохраним функции вычисления объема сферы, цилиндра и конуса.

Листинг 2.2 Файл исходного кода `volumes.jl`

```
# Вычисления объема
sphere_volume(r)      = 4π*r^3/3
cylinder_volume(r, h) = π*r^2*h
cone_value(r, h)      = π*r^2*h/3
```



```
findfirst findmax findmin findnext
```

- ❶ Дважды нажать клавишу Tab, чтобы просмотреть полный список функций, начинающихся со слова find.

На веб-странице языка программирования Julia также находится руководство, в котором приведен полный список всех функций, встроенных в стандартную библиотеку Julia. К данному руководству можно обратиться по адресу docs.julialang.org/en/v1.

2.5.3 Функции повсюду

В языке Julia функции занимают центральное место. На самом деле даже обычные математические операторы определяются в Julia как функции. Приведу несколько примеров:

```
julia> 5 + 3      ❶
8
```

```
julia> +(5, 3)    ❷
8
```

```
julia> 8 - 2      ❶
6
```

```
julia> -(8, 2)    ❷
6
```

```
julia> +(2, 3, 5)  ❸
10
```

- ❶ Казалось бы, используется оператор, но на самом деле это вызов функции, написанный в инфиксной форме.
- ❷ Это операторы, вызываемые как обычные функции. Такой вызов называется вызовом в префиксной форме.
- ❸ Преимущество использования оператора + в качестве функции заключается в возможности использовать более двух аргументов (например, его можно использовать для сложения нескольких значений).

Для описания расстановки идентификаторов используются следующие термины: префиксная, инфиксная и суффиксная формы. Таким образом, $+(3, 4)$ будет считаться *префиксной* формой, а $3 + 4$ будет эквивалентной *инфиксной* формой.

Как узнать, можно ли использовать функцию в инфиксной форме? Все просто: имя функции должно быть символом. Например, функцию с именем foo нельзя использовать в инфиксной форме. Давайте создадим функцию, чтобы продемонстрировать принцип:

```
julia> x(a, b) = a^2 + b^2
x (generic function with 1 method)
```

```
julia> x(4, 10)
116
```

```
julia> 4 × 10
116
julia> 2 × 3
13
```

Здесь была создана функция с именем `×` (пишется `\times`), и поскольку она принимает два аргумента и является символом, ее можно использовать в инфиксной форме. Самый простой способ получить символ – написать LaTeX-подобную аббревиатуру и нажать клавишу **Tab** (например, `\Delta` для Δ). Символьное автозавершение будет работать в интерактивной среде REPL и во многих редакторах исходного кода Julia. Поддерживаемые в языке Julia LaTeX-подобные аббревиатуры находятся в официальной документации Julia по адресу <https://docs.julialang.org/en/v1/manual/unicode-input/>.

2.5.4 Функции для работы с числами

Вы уже видели несколько функций, которые можно использовать для работы с числами, но в Julia их большая коллекция. Я хотел бы показать несколько наиболее полезных, в частности для работы с целыми числами и числами с плавающей точкой. Ранее я показал некоторые операции на целых числах и числах с плавающей точкой. Однако теперь вы знаете, что операции – это просто функции. Все приведенные ниже варианты эквивалентны:

```
julia> 9 % 6
3
julia> %(9, 6)
3
julia> rem(9, 6)
3
```

В Julia действует принцип не ставить программиста перед необходимостью использовать только специальные символы Юникода. Такие операции, как целочисленное деление, также можно выполнять с помощью простых функций:

```
julia> 9÷4
2
julia> ÷(9, 4)
2
julia> div(9, 4)
2
```

На самом деле если нажать клавишу `?` и написать `div`, то вы получите ответ встроенной справочной системы, показывающей, что `div` имеет два имени¹:

¹ Вычисляет частное от x/y , усеченное до целого числа. – Прим. перев.

```
help?> div(9, 4)
div(x, y)
÷(x, y)
```

The quotient from Euclidean division. Computes x/y , truncated to an integer.

Очень полезно уметь округлять числа разными способами. Для этой цели в Julia есть функции `floor`, `ceil` и `round`:

```
julia> floor(3.2) ❶
3.0
```

```
julia> floor(3.6) ❶
3.0
```

```
julia> ceil(3.2) ❷
4.0
```

```
julia> ceil(3.8) ❷
4.0
```

```
julia> round(3.2) ❸
3.0
```

```
julia> round(3.5) ❸
4.0
```

- ❶ Всегда округляет вниз.
- ❷ Всегда округляет вверх.
- ❸ Округляет до ближайшего целого числа.

Но если мы округляем до целых чисел, то, вероятно, нужны и целочисленные типы:

```
julia> Int(round(3.5)) ❶
4
```

```
julia> round{Int64}(3.5) ❷
4
```

```
julia> round{Int8}(3.5)
4
```

```
julia> typeof(ans)
Int8
```

- ❶ Это нативный подход к получению целого числа.
- ❷ Это предлагаемый и наиболее эффективный подход: в качестве первого аргумента передается нужный тип, который желательно получить на выходе.

2.6 Как использовать числа на практике

Многие описанные здесь детали не являются чем-то, о чем нужно думать в обычном программировании. Я не хочу, чтобы вы забывали свою память слишком большим объемом ненужных деталей. Ключевым здесь является понимание принципа работы чисел в Julia. Он не уникален только для Julia, но, возможно, будет незнаком разработчикам из других динамических языков, таких как Python, R или Ruby.

В целях облегчения себе задачи нужно соблюдать несколько простых правил:

- 1 следует использовать размеры целых чисел и чисел с плавающей точкой, используемые по умолчанию. Рассматривать возможность применения меньших или больших чисел необходимо только тогда, когда этого требует производительность или особенность вашей задачи;
- 2 следует предпочитать целые числа со знаком целым числам без знака. Используя числа без знака, очень легко ошибиться. В целях облегчения себе задачи в большинстве случаев следует придерживаться чисел со знаком.

Резюме

- Julia поддерживает различные типы чисел, но два наиболее важных типа – это целые числа и числа с плавающей точкой.
- В отличие от чисел, используемых математиками, числа, используемые в программировании, имеют определенную длину в битах. Она задает максимальную и минимальную величины значений, которые можно будет хранить. Например, 8-битовое целое число со знаком не может хранить числа меньше –128 или больше 127.
- Переменные дают имена числам. Функции дают имена вычислениям и зависят от нуля или более аргументов.
- Присваивание значения переменной означает добавление идентификатора к этому значению. Новое присваивание другого значения той же самой переменной означает перенос стикера со старого на новое значение. Эта операция называется привязыванием.
- Два числа, которые выглядят в памяти одинаково, могут выглядеть по-разному на экране, потому что они относятся к разным типам.
- Числа можно вводить в разных форматах: двоичном, десятичном и шестнадцатеричном. Если вы не занимаетесь системным программированием, то десятичный формат обычно является предпочтительным.

3

Поток управления

Эта глава охватывает следующие ниже темы:

- использование булевых значений в условиях;
- многократное выполнение одного и того же исходного кода с использованием циклов `while` и `for`;
- выбор исполняемого исходного кода с помощью инструкции `if`;
- прокручивание числовых диапазонов в цикле;
- определение функций, охватывающих несколько строк кода;
- реализация потока управления с использованием рекурсии и циклов.

Наличие потока управления отличает компьютер от простого калькулятора. Калькуляторы предназначены для вычисления отдельных выражений. С другой стороны, компьютеры могут повторять одни и те же вычисления многократно с разными входными данными без вмешательства человека. Компьютеры могут выбирать одно вычисление и игнорировать другое в зависимости от истинности или ложности условия.

В этой главе вы обследуете примеры исходного кода, ориентированные на создание математических таблиц. Вы узнаете о таблицах

тригонометрических функций, так как такие таблицы хорошо известны и имеют историческое значение. Позже вы займетесь исполнением исходного кода по условию, который будет отслеживать рост кроликов с помощью метода, разработанного 800 лет назад итальянским математиком Фибоначчи.

3.1 Навигация и тригонометрия

В эпоху парусного каботажного судоходства использование математических таблиц получило более широкое распространение, и возникла необходимость в разработке способов автоматизации расчетов этих таблиц (навигация основана на вычислении углов и сторон треугольников; рис. 3.1). Указанные расчеты предусматривали вычисление тригонометрических функций, таких как синус, косинус и тангенс.

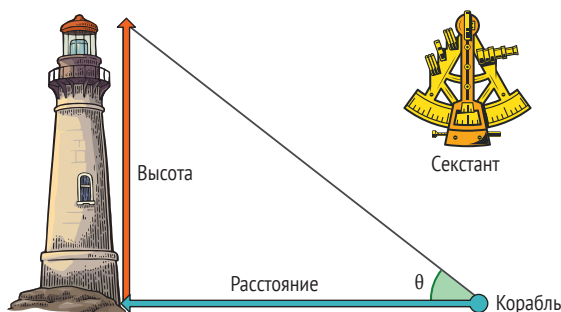


Рис. 3.1 Для измерения угла θ между горизонтом и вершиной маяка капитан корабля обычно использует секстант

Морские карты содержат высоты различных маяков. Таким образом, если нужно узнать свое положение в море, находясь недалеко от берега, то можно измерить угол между горизонтом и верхушкой маяка известной высоты. Благодаря этому можно получать расстояние до маяка. Однако все эти расчеты требуют вычисления тригонометрических функций, таких как синус и косинус, а у морских капитанов начала XVIII века не было карманных калькуляторов (табл. 3.1).

Таблица 3.1 Простая тригонометрическая таблица

угол	0°	30°	45°	60°	90°
sin	0.000	0.500	0.707	0.866	1.000

Вместо этого они использовали математические таблицы. Большие печатные таблицы с подробным описанием значений синуса, косинуса и тангенса для разных углов были обычными инструментами мореплавателей того времени. Давайте освежим ваши школь-

ные познания в математике, если вы давно не использовали тригонометрические функции. Глядя на треугольник на рис. 3.2, синус, косинус и тангенс определяются следующим образом:

$$\sin(\theta) = \frac{b}{h}; \quad \cos(\theta) = \frac{a}{h}; \quad \operatorname{tg}(\theta) = \frac{b}{a}.$$

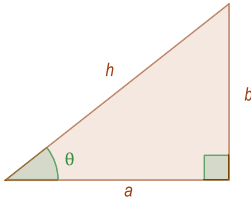


Рис. 3.2 Прямоугольный треугольник со сторонами длины a , b и h . Самая длинная сторона h называется гипотенузой и имеет угол θ

Так, например, синус угла равен длине противолежащей стороны, деленной на наибольшую сторону (гипотенузу) в прямоугольном треугольнике (один угол равен 90°). Сегодня для вычисления этих тригонометрических функций применяется калькулятор. Но что, если бы вы жили до 1972 года¹? Как бы вы выполняли эти расчеты вручную? На самом деле не существует канонического способа вычисления синуса и косинуса. Вместо этого существуют различные методы *аппроксимации*. Популярный метод вычисления синуса называется *рядом Тейлора*:

$$\sin(x) = \frac{x}{1!} - \frac{x^3}{3!} + \frac{x^5}{5!} - \dots$$

Этот ряд можно записать в более компактной и обобщенной форме как

$$\sin(x) = \sum_{i=0}^n (-1)^i \frac{x^{2i+1}}{(2i+1)!}.$$

Но математические таблицы не ограничиваются тригонометрическими функциями. Таблицы полезны для многих других применений, чтобы сокращать объем необходимых ручных вычислений.

В 1819 году это побудило Чарльза Бэббиджа приступить к созданию массивной механической вычислительной машины под названием «Разностная машина» (рис. 3.3). Она могла вычислять значения нескольких таблиц, многократно повторяя одни и те же вычисления. Говоря современным языком программирования, таблицы создавались с использованием циклов. Циклы основаны на вычислении (выполнении) одного и того же исходного кода не-

¹ В 1972 году компания Hewlett-Packard выпустила первый калькулятор под названием HP-35 с функциями синуса и косинуса.

сколько раз до тех пор, пока соблюдается заданное условие. Во всех языках программирования для определения условий используются булевы выражения. Мы пойдем по стопам Бэббиджа, создав такие циклы на языке Julia.

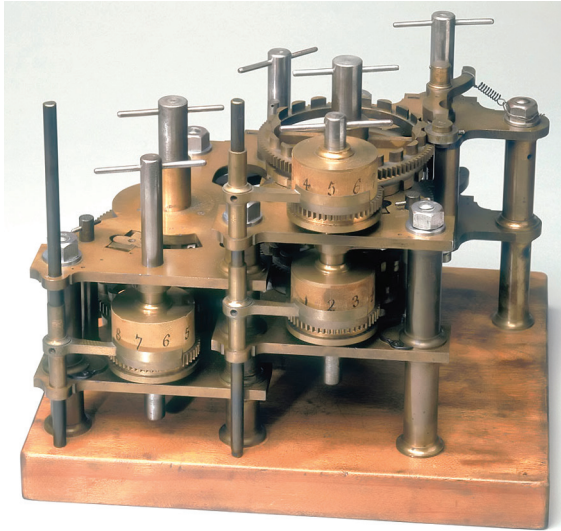


Рис. 3.3 Часть разностной машины Чарльза Бэббиджа, предшественницы аналитической машины – первого универсального механического компьютера

Наша цель – реализовать тригонометрические функции на Julia с помощью ряда Тейлора. Прежде чем это станет возможным, необходимо развить свое понимание булевых выражений, которые позволят разобраться в циклах `while`, `for` и инструкции `if`. Вы разовьете это понимание с помощью серии небольших примеров исходного кода, распечатывая и складывая числа и конвертируя градусы в радианы.

3.2 Булевы выражения

Одна из первых тем, о которых вы узнаете в начальной школе, – это логические выражения, которые в информатике называются булевыми. По иронии судьбы, большинство студентов никогда не практикуются в их использовании. Никто не говорит о том, что они являются одной из самых важных частей любого языка программирования. Вы уже встречали числовые выражения, такие как $3 + 5$; в результате их вычисления получается число. Булевы выражения, напротив, в результате вычисления дают истину (`true`) либо ложь (`false`). Их легче понять на нескольких практических примерах:

```
julia> 3 > 5      ❶
false
```

```
julia> 8 > 3      ❷
true
```

```
julia> 8 == 5 + 3  ❸
true
```

```
julia> 3 == 5      ❸
false
```

```
julia> 3 ≤ 3       ❹
true
```

```
julia> 3 ≤= 3      ❹
true
```

❶ 3 больше 5?

❷ 8 больше 3?

❸ Проверить равенство значений. Это не оператор присваивания.

❹ Меньше или равно.

В приведенном выше примере используется Юникод-версия оператора *меньше или равно*. Некоторые булевы операторы имеют варианты в кодировке Юникод. В следующей ниже таблице показано написание некоторых из них в интерактивной среде REPL:

Символ	Заполнение по Tab	Описание
≤	<code>\leq</code>	меньше или равно <code><=</code>
≥	<code>\geq</code>	больше или равно <code>>=</code>
≠	<code>\ne</code>	не равно <code>!=</code>
≈	<code>\approx</code>	приблизительно равно <code>isapprox(x, y)</code>

Булевы выражения возвращают булевы значения, которых всего два: `true` и `false`. Помните, как я сказал, что внутри компьютера есть только числа? Булевы значения ничем не отличаются:

```
julia> typeof(7 > 3)  ❶
Bool
```

```
julia> typeof(false)
Bool
```

```
julia> reinterpret(UInt8, false)  ❷
0x00
```

```
julia> reinterpret(UInt8, true)   ❸
0x01
```

❶ Булевы выражения производят значения типа `Bool`.

❷ Ложное значение хранится как 0.

❸ Истинное значение хранится как 1.

В отличие от многих других языков программирования, Julia разрешает выполнять арифметические действия над булевыми значениями. В арифметике булевы значения трактуются как 0 либо 1:

```
julia> true + true
2

julia> 3true + true
4

julia> true + false + true
2

julia> false + false
0
```

Для ясности лучше всего избегать использования булевых значений в качестве чисел. Однако бывают случаи, когда это очень удобно. Разработчики на языке Julia часто используют эту возможность, чтобы подсчитывать число истинных случаев. В главе 4 вы увидите пример.

3.2.1 Составные инструкции

Булевы выражения можно комбинировать с помощью операторов `||` и `&&`. Они выполняют так называемые логические операции *ИЛИ* (OR) и *И* (AND). Так, имея переменную `x`, можно, например, выяснить, меньше ли она числа 4 или больше числа 10:

```
julia> x = 3
3

julia> x < 4 || x > 10
true

julia> x = 5
5

julia> x < 4 || x > 10
false
```

В качестве альтернативы можно выяснить, больше ли `x` числа 4 и меньше числа 10:

```
julia> x > 4 && x < 10
true

julia> x = 12
12

julia> x > 4 && x < 10
false
```

Далее вы будете использовать булевы выражения для определения условий многократного повторения одного и того же исходного кода.

3.3 Циклы

Простейшей циклической конструкцией любого языка программирования является цикл `while`. Он позволяет повторять один и тот же исходный код снова и снова до тех пор, пока булево условие остается истинным, как показано в следующем ниже листинге.

Листинг 3.1 Простой цикл `while`

```
i = 0
while i < 5
    i = i + 1
end
```

Весь исходный код между ключевыми словами `while` и `end` повторяется многократно до тех пор, пока соблюдается условие `i < 5`. Этот исходный код можно скопировать и вставить в интерактивную среду REPL, но вы не увидите никакого результата. Почему? Потому что выражение `while end` ничего не вычисляет. Возможно, это выглядит немного абстрактно, поэтому давайте я приведу пример: `3 + 4` – это выражение, результатом которого является 7. Значение такого рода выражения можно сохранять в переменных:

```
julia> x = 3 + 4
7

julia> y = while false end ❶

julia> typeof(y) ❷
Nothing
```

- ❶ Бессмысленный цикл `while`, который сразу же завершается.
- ❷ Интерактивная среда REPL ничего не печатает.

Пример цикла `while` иллюстрирует несколько разных аспектов. В результате вычисления самого цикла получается значение, аналогичное вычислению выражения `3 + 4`. Это значение сохраняется в переменной `y`. Однако увидеть это значение в интерактивной среде REPL невозможно, потому что оно имеет тип `Nothing`.

Также обратите внимание, что цикл `while` вполне можно размещать в одной строке. Пробелы в Julia не так важны, как в Python; в Python нужно помнить об отступах инструкций, принадлежащих циклу. С другой стороны, в Julia пробелы действительно играют роль. Возьмем следующие ниже три присваивания:

```
x = 4
y = 8
z = 3
```

Если нужно, чтобы они были в одной строке, то их нужно разделить точкой с запятой:

```
x = 4; y = 8; z = 3
```

Возможно, вас интересует, почему я сохранил значение выражения `while` в переменной. Я сделал это исключительно в учебных целях, чтобы вы знали, что в Julia почти все является выражением, в результате вычисления которого получается значение. Даже присваивание дает значение (см. листинг 3.2). Хотя, возможно, такая особенность будет звучать как теоретическая диковинка, не представляющая для вас никакого интереса, она имеет много практических последствий. Она побуждает интерактивную среду REPL показывать значение, которое вы дали переменной в инструкции присваивания. Позже в этой главе, когда будут излагаться инструкции `if`, вы также увидите выгоды от трактовки всех частей в качестве выражения.

Листинг 3.2 Результатом присваивания является значение

```
julia> z = 3 + 2
5
```

ПРИМЕЧАНИЕ С технической точки зрения было бы неправильно называть присваивание инструкцией, потому что в Julia все является выражением. Однако для многих выражений в этой книге я буду использовать слово «инструкция». Причина в том, что это упрощает различие между присваиваниями и потоком управления, например инструкциями `if` и циклами `while`, а также другими математическими выражениями.

Интерактивная среда REPL всегда будет показывать значение внешнего выражения, а не внутреннего выражения. Например, если вы вычисляете `1 + (3+2)`, то никогда не увидите напечатанным число 5, потому что это значение подвыражения `3+2`. Точно так же вы не увидите `i = i + 1` внутри цикла. Для того чтобы увидеть значение переменной `i` в каждой итерации, нужно сообщить Julia в явной форме, чтобы значение было выведено на консоль. Это делается с помощью функций `print` и `println`:

```
julia> i = 0
0

julia> while i < 5
    i = i + 1
    print(i)
```

```

        end
12345

julia> i = 0
0

julia> while i < 5
        i = i + 1
        println(i)
    end
1
2
3
4
5

```

Из этих примеров вы, вероятно, увидите разницу. `println` – это сокращение от `print line`, то есть напечатать строку. Данная инструкция печатает переменную в отдельной строке. `print` и `println` можно использовать для вывода значений в явной форме вне циклов, но это требуется редко:

```

julia> print(3 + 4)
7

julia> print(i)
5

```

3.3.1 Блок-схема

Текст не очень хорошо дает визуализировать поток управления программой. Вы должны знать семантику. По этой причине большую популярность получили блок-схемы, изображающие поток программы с использованием прямоугольников и стрелок (рис. 3.4).



Рис. 3.4 Стандартные блоки блок-схемы

Раньше студентов учили составлять свои программы в виде блок-схем, а затем писать исходный код. Популярность объектно ориентированного программирования привела к тому, что блок-схемы вышли из употребления, поскольку с помощью них невозможно моделировать взаимосвязи между объектами. Тем не менее блок-схемы по-прежнему очень полезны при обучении потоку управления в программе. Если вы незнакомы с циклами, то блок-схемы помогут понять принцип их работы (рис. 3.5).

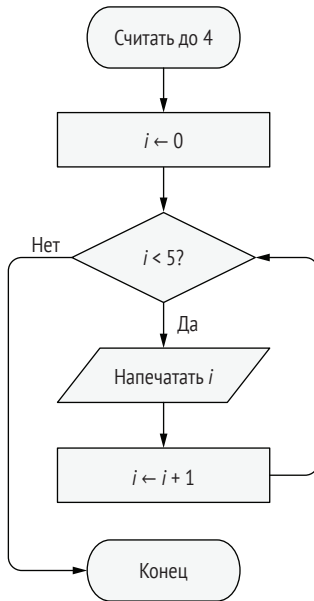


Рис. 3.5 Визуализация управления потоком в цикле `while`

Квадратные прямоугольники представляют выполняемые действия, а ромбовидные прямоугольники – решения, в которых поток управления разветвляется в разные стороны. Если условие $i < 5$? является истинным, то поток проследует за стрелкой, отмеченной словом «Да». В противном случае поток управления проследовал бы за стрелкой «Нет».

3.3.2 Составление математической таблицы синусной функции

Теперь у вас есть все базовые строительные блоки, для того чтобы повторить то, что делала разностная машина Чарльза Бэббиджа: расчет математических таблиц. Для простоты давайте начнем с распечатки углов, как показано ниже.

Листинг 3.3 Цикл, выводящий углы с шагом 15

```

angle = 0
while angle <= 90
  println(angle)
  angle = angle + 15
end
  
```

Этот фрагмент исходного кода можно скопировать и вставить в интерактивную среду REPL, в результате чего будет получен вот такой результат:

```

0
15
  
```

30
45
60
75
90

Прежде чем вычислять синус этих углов, необходимо конвертировать их в радианы, так как синусная, косинусная и тангенсная функции обычно работают не с градусами от 0° до 360° , а с радианами от 0 до 2π . Иллюстрации на рис. 3.6 показывают определение 1 радиана. Если дана окружность с радиусом r и по периметру окружности нарисовать дугу s длины r , то кусок пирога будет иметь угол, равный 1 радиану.



Рис. 3.6 Взаимосвязь между радианом и радиусом окружности

Длина окружности C и длина дуги s окружности определяются следующим образом:

$$C = 2\pi r; \quad \theta = \frac{s}{2\pi r}.$$

Из этих формул можно вывести функцию `deg2rad` конвертирования градусов в радианы.

Листинг 3.4 Конвертирование градусов в радианы

```
deg2rad(θ) = (θ/360)*2π
```

На самом деле писать эту функцию вовсе не нужно, потому что Julia уже поставляется с ней в своей стандартной библиотеке. С помощью этой функции можно видоизменить исходный код листинга 3.3 и создать небольшую программу, которая создает таблицу синусных значений, как показано ниже.

Листинг 3.5 Циклическая печать таблицы синусов

```
angle = 0
while angle <= 90
    rad = deg2rad(angle)
    x = sin(rad)
```



```
println(x)
angle = angle + 15
end
```

При выполнении приведенного выше фрагмента исходного кода мы получим следующий ниже результат:

```
0.0
0.25881904510252074
0.49999999999999994
0.7071067811865475
0.8660254037844386
0.9659258262890683
1.0
```

3.3.3 Объекты-диапазоны

При чтении обычного исходного кода Julia вы обнаружите, что подход к выполнению циклов на основе условий на самом деле не является обычным. Вместо него циклы, как правило, определяются с использованием объектов-диапазонов; диапазоны создаются с помощью оператора `:`. С диапазонами можно выполнять ряд действий, к примеру проверять присутствие конкретного значения в заданном диапазоне. В приведенном ниже примере, перед тем как запрашивать, не находится ли конкретное число в заданном диапазоне, мы получаем первую и последнюю части диапазона:

```
julia> r = 2:4          ❶
2:4

julia> first(r)         ❷
2

julia> last(r)          ❸
4

julia> in(1, r)         ❹
false

julia> in(3, r)         ❺
true

julia> 3 in r           ❻
true
```

- ❶ Сконструировать объект-диапазон и сохранить его в переменной `r`. Или, правильнее, привязать метку `r` к объекту-диапазону.
- ❷ Получить начало диапазона.
- ❸ Получить конец диапазона.
- ❹ Проверить, не находится ли `1` в диапазоне от `2` до `4`. Поскольку `1` находится за пределами диапазона, то в результате вычисления этого выражения будет получено булево значение `false`.

- 5 Проверить, не находится ли 3 в пределах диапазона. Да, находится.
- 6 Обычно символы Юникода можно использовать только в инфиксной форме, но это исключение. 3 in r эквивалентно in(3, r).

3.3.4 Циклы for

Объекты-диапазоны обычно используются в циклах for, но прежде чем показать пример цикла for, давайте я вам покажу, как использовать диапазон в цикле while. Указанный цикл повторяется до тех пор, пока условие `i in 0:4` остается истинным, как показано ниже.

Листинг 3.6 Цикл while с использованием диапазона

```
i = 0
while i in 0:4
    println(i)
    i = i + 1
end
```

Это настолько распространенная и полезная идиома, что были придуманы циклы for, которые позволяют убирать излишний стереотипный исходный код. Следующий ниже фрагмент исходного кода по поведению эквивалентен.

Листинг 3.7 Цикл for над диапазоном

```
for i in 0:4
    println(i)
end
```

А что насчет случаев, когда требуется увеличивать значение на 1 на каждой итерации? Когда мы вычисляли углы, мы делали это с шагом 15. Можно ли это сделать с помощью цикла for? Без проблем; как показано в следующем ниже листинге, диапазоны позволяют определять размер шага.

Листинг 3.8 Цикл for с пошаговым диапазоном

```
for angle in 0:15:90
    println(angle)
end
```

При выполнении приведенного выше фрагмента исходного кода мы получим следующий ниже результат:

```
0
15
30
45
60
75
90
```

Объекты, которые можно использовать в цикле `for`, такие как объекты-диапазоны, называются *итерируемыми объектами*. В Julia есть самые разные итерируемые объекты, и мы рассмотрим их в последующих главах.

3.4 Многострочные функции

Применявшиеся до сего времени функции определялись в одной строке, и такой подход весьма ограничивает возможности. Для решения более сложных задач требуется несколько строк кода. Как это делать? Для этого нужен способ отмечать начало и конец исходного кода, который должен включаться в функцию. Циклы `for` и `while` уже могут навести вас на мысль о том, как это делается. Многострочная функция начинается с ключевого слова `function` и заканчивается ключевым словом `end`. В следующем ниже примере исходного кода используется цикл, внутри которого выводятся значения синуса для всех углов от 0 до `max_angle` с приращением угла.

Листинг 3.9 Исходный код создания хранящейся в функции таблицы синусов

```
function print_sin_table(increment, max_angle)
    angle = 0
    while angle <= max_angle
        rad = deg2rad(angle)
        x = sin(rad)
        println(x)
        angle = angle + increment
    end
end
```

Обратите внимание, как за счет использования функциональных аргументов `increment` и `max_angle` вместо жестко заданных углов в 15 и 90 градусов в листинге 3.9 видоизменен предыдущий исходный код из листинга 3.5. Благодаря этому пользователи смогут легко изменять производимую таблицу. Например, посредством вызова `print_sin_table(1, 90)` пользователи смогут создавать значения синуса с шагом в 1 градус.

Так как же это связано с разностной машиной, созданной Чарльзом Бэббиджем? Бэббиджевский эквивалент функции `println` выводил числа не на экран компьютера, а на своего рода принтер. Разностную машину нужно было подключать к другой машине, которая печатала числа на металлических пластинах. Затем эти пластины использовались для печати числовых таблиц в книгах. Производимые числа также можно было бы отправлять на другие устройства, но это будет рассмотрено в последующих главах, посвященных вводу и выводу.

3.4.1 Реализация тригонометрической синусной функции

Теперь, когда вы научились использовать циклы и многострочные функции, у вас есть все строительные блоки, необходимые для разработки своей собственной функции `sin`, то есть вы можете воспроизвести то, что делает калькулятор. Снова взгляните на ряд Тейлора, используемый для вычисления функции $\sin(x)$:

$$\sin(x) = \frac{x}{1!} - \frac{x^3}{3!} + \frac{x^5}{5!} - \dots$$

Эту функцию можно записать в виде суммирования:

$$\sin(x) = \sum_{i=0}^{\infty} (-1)^i \frac{x^{2i+1}}{(2i+1)!}$$

Я не буду доказывать математически, как прийти к этому определению; здесь интересует демонстрация того, как для решения подобного рода задач могут использоваться компьютеры. Ручные вычисления стали настоящей проблемой в начале XIX века, когда возросло значение математики и естественных наук.

Если вы незнакомы с математическими обозначениями, то давайте я продемонстрирую принцип работы оператора $\sum$ в исходном коде. Начнем с простого случая:

$$f(n) = \sum_{x=1}^n 2x + 1.$$

Нижняя и верхняя половины символа $\sum$ в сущности определяют диапазон. Вы заявляете, что будете прокручивать переменную `x` от 1 до `n`. Оператор $\sum$ можно трактовать как выполнение цикла; он выполняет итерации по диапазону и суммирует значения выражения в этом диапазоне. Такое поведение можно симитировать с помощью цикла `for` следующим образом.

Листинг 3.10 Как работает оператор суммы

```
function f(n)
    total = 0
    for x in 1:n
        total += 2x + 1
    end
    total
end
```

- ❶ Служит для хранения общей суммы.
- ❷ Сокращенная форма для `total = total + 2x + 1`.
- ❸ Возвращаемое значение.

Функция возвращает значение своего последнего выражения. Во многих других языках это называется *возвращаемым значением*, и последнее выражение можно было бы записать так:

```
return total
```

Это допустимо и в Julia, но используется только тогда, когда нужно выйти из функции досрочно. В противном случае инструкция `return` в функциях Julia обычно опускается. Следующий ниже листинг должен помочь понять, как использовать ряд Тейлора для реализации синусной функции.

Листинг 3.11 Синус, реализованный с использованием ряда Тейлора

```
function sine(x)
    n = 5
    total = 0
    for i in 0:n
        total += (-1)^i*x^(2i+1)/factorial(2i + 1)
    end
    total
end
```

Размещение этой функции в отдельном файле (например, `trig.jl`) вместе с другими реализованными тригонометрическими функциями является хорошим упражнением для читателя. Реализуйте также косинус и тангенс; их определение на основе ряда Тейлора можно отыскать в интернете. Затем можно загрузить этот файл в Julia и сравнить со встроенной функцией `sin`:

```
julia> sine(0.5)
0.4794255386041834
```

```
julia> sin(0.5)
0.479425538604203
```

```
julia> sine(1.0)
0.841470984648068
```

```
julia> sin(1.0)
0.8414709848078965
```

Хорошо видно, что результаты очень похожи, несмотря на то что итерации выполняются только до $n = 5$. Точное определение подразумевает, что $n = \infty$, что непрактично реализовывать в исходном коде. Попробуйте использовать разные значения n , чтобы постараться получить такой же точный результат, как и у встроенной функции `sin`.

3.5 Реализация факториала

В нашей собственной функции `sine` используется встроенная функция `factorial`. Факториал числа n означает перемножение каждого числа от 1 до n . Так, факториал пяти будет $5 \times 4 \times 3 \times 2 \times 1$. Как его реализовать самостоятельно? К решению этой задачи есть много подходов. В данном разделе мы рассмотрим несколько из них:

- 1 с использованием встроенной функции `prod`;
- 2 с использованием цикла `while` для выполнения многократных умножений;
- 3 посредством многократного умножения путем комбинирования рекурсии с инструкцией `if`.

Функция `prod` может умножать все числа в диапазоне:

```
julia> fac(n) = prod(1:n) ❶
```

```
julia> fac(5) ❷  
120
```

```
julia> factorial(5) ❷  
120
```

- ❶ Определить свою собственную функцию факториала с именем `fac`, реализованную с помощью `prod`.
- ❷ Убедиться, что `fac` и `factorial` дают одинаковый результат.

Позэкспериментируйте с обеими функциями, чтобы в итоге получить одинаковый результат. Как и раньше, это делается с использованием цикла.

Листинг 3.12 Факториал, реализованный с использованием цикла

```
function fac(n)  
    prod = 1  
    while n >= 1  
        prod *= n  
        n -= 1  
    end  
    prod  
end
```

На каждой итерации цикла значение n уменьшается на 1 до тех пор, пока условие $n \geq 1$ больше не будет соблюдаться, после чего выполняется выход с произведением всех чисел в диапазоне от n до 1.

3.6 Факториал с использованием рекурсии

Существует еще один способ циклической обработки без использования циклов `while` и `for`, именуемый *рекурсией*. Взгляните на следующий ниже исходный код.

Листинг 3.13 Испорченная функция факториала с использованием рекурсии

```
fac(n) = n*fac(n-1)
```

Попробуйте ее выполнить. Она не совсем работает. Вы получите следующее сообщение об ошибке:

```
ERROR: StackOverflowError:  
Stacktrace:  
 [1] fac(n::Int64) (repeats 79984 times)
```

Это связано с тем, что функция `fac` не перестает вызывать саму себя. Точнее, в моем примере она вызывает `fac` 79 984 раза, то есть до тех пор, пока не завершится аварийно из-за нехватки памяти. В результате будет создано сообщение об ошибке переполнения стека. Вся причина в том, что `fac` продолжает вызываться, даже когда аргумент `n` стал меньше 1. Каким-то образом нужно внести проверку, что `n` не меньше 1, и выйти, если это так. Нам повезло, потому что инструкция Julia `if` нам в этом поможет.

3.7 Инструкции `if`

Теперь перепишем рекурсивную функцию `fac`, используя инструкцию `if`. Следующий ниже исходный код является первой попыткой. Мы будем расширять его до тех пор, пока функция факториала не будет обрабатывать все пограничные случаи, такие как `fac(0)`.

Листинг 3.14 Почти рабочая функция факториала с использованием рекурсии

```
function fac(n)  
    if n <= 2  
        return n    ①  
    end  
    n*fac(n-1)    ②  
end
```

- ① Мы выходим из функции `fac`, используя инструкцию `return`, если аргумент `n` меньше или равен 2. Здесь `return` вызывается в явной форме, чтобы вернуть значение досрочно, не ожидая до тех пор, пока не будет достигнуто последнее выражение в функции.

- ② Снова вызвать `fac`, но с `n-1`, и умножить возвращенный результат на `n`. Напомню, что последнее выражение в функции Julia выполняет неявный возврат значения.

3.7.1 Инструкции `if-else`

Вместо того чтобы для досрочного выхода из функции использовать инструкцию `return`, можно выбирать между двумя разными подлежащими исполнению блоками кода путем добавления подчиненного компонента `else`, как показано в следующем ниже листинге.

Листинг 3.15 Инструкция `if-else`

```
function fac(n)
    if n <= 2
        n
    else
        n*fac(n-1)
    end
end
```

Если условие `n <= 2` не равно `true`, то будет вычислен исходный код между блоком `else-end`. Вся инструкция `if-else`, как и все другие инструкции в Julia, представляет выражение, результатом которого является значение. В результате вычисления инструкции получается значение вычисленного блока кода. Вы можете попробовать это сами в интерактивной среде REPL. Поэкспериментируйте с разными значениями `x` и посмотрите, как изменится значение `y`:

```
julia> x = 4
4

julia> y = if x > 3
        6
        else
        4
        end

6

julia> y
6
```

Однако функция `fac` еще не работает правильно:

```
julia> factorial(0)
1

julia> factorial(-1)
ERROR: DomainError with -1:
`n` must not be negative.

julia> fac(0)
0
```



```
julia> fac(-1)
-1
```

Вызов `fac(0)` возвращает 0, но должен возвращать 1. Кроме того, `fac(n)` с $n < 0$ даже нельзя допускать. Таким образом, необходимо по-разному обрабатывать ситуацию, когда $n == 0$, и ситуацию, когда $n < 0$.

3.7.2 Подчиненный компонент *elseif*

В этом случае на помощь приходит подчиненный компонент `elseif`. В любую инструкцию `if` можно добавлять несколько таких компонентов. Именно это и сделано в следующем ниже листинге, чтобы обрабатывать все уникальные случаи. Продолжите работу и проверьте в интерактивной среде REPL правильность поведения вызовов `fac(0)` и `fac(-1)` с учетом этого обновления.

Листинг 3.16 Инструкция `if-else`

```
function fac(n)
    if n > 2
        n*fac(n-1)
    elseif n > 0
        n
    elseif n == 0
        1
    else
        err = DomainError(n, "`n` must not be negative.")
        throw(err)
    end
end
```

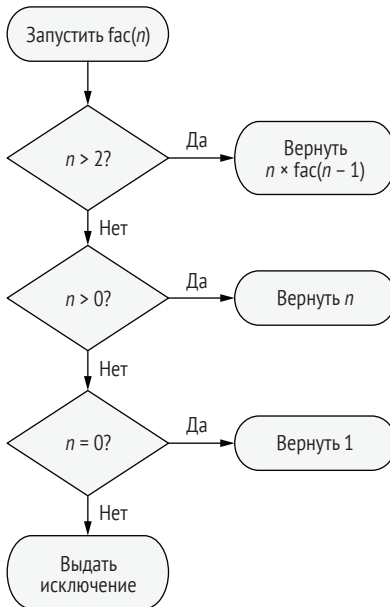
- ❶ Вернуть 1, если n равно нулю.
- ❷ Создать объект-исключение. Такие объекты используются для хранения информации о произошедшей ошибке.
- ❸ Сообщить об ошибке, указывающей, что n не может быть отрицательным.

Каждый подчиненный компонент `elseif` добавляет еще одну проверку условия. Сначала проверяется, что $n > 2$, затем проверяется, что $n > 0$. Каждая проверка `elseif` продолжает выполняться до тех пор, пока не наступит условие, результатом вычисления которого будет `true`. Если ни одно из условий не равно `true`, то вычисляется подчиненный компонент `else`, который сообщает об ошибке (рис. 3.7).

Прежде чем продолжить обсуждение темы обработки ошибок, давайте подытожим правило написания инструкции `if`:

- 1 Допускается ровно одно ключевое слово `if`, и оно должно быть в начале.
- 2 Подчиненный компонент `else` не является обязательным, но его можно использовать только один раз и только в самом конце.
- 3 Допустимо вставлять любое число подчиненных компонентов `elseif`, но они должны стоять после инструкции `if`.

Рис. 3.7 Инструкция if с elseif и else



3.8 Исключения для обработки ошибок

На программистском жаргоне функции возвращают значения, но выдают исключения. В Julia они используются для обработки ошибок программиста. Вы как программист не должны передавать в `fac` отрицательные числа. Однако ошибки случаются, и их нужно исправлять. Идея состоит в том, чтобы сообщать о проблеме как можно раньше – сразу после ее обнаружения. Благодаря этому упрощается диагностика проблем во время разработки и тестирования программного обеспечения.

Чем выдача исключения отличается от возврата значения? Поясню на примере (рис. 3.8).

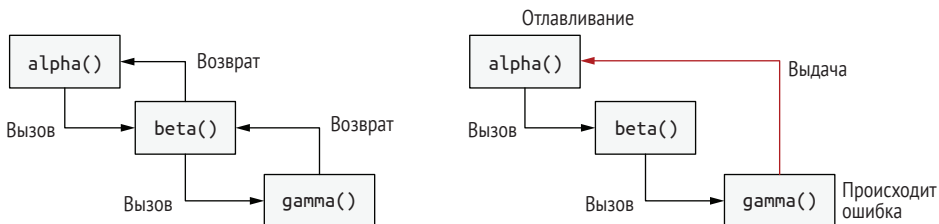


Рис. 3.8 Разница между обычными возвратами и выдачей исключений

Если функция `alpha` вызывает `beta`, которая вызывает `gamma`, то образуется так называемый *стек вызовов*. Стек вызовов – это место

в памяти, где хранятся вызовы функций. Он необходим вследствие того, что когда центральный процессор заканчивает обработку инструкций в `gamma`, ему нужно вернуться к месту в `beta`, где была вызвана `gamma`. Это место хранится в памяти. Оно называется обратным адресом. Точно так же нужно помнить, как вернуться к `alpha` из `beta`. Эти вложенные вызовы функций создают стек адресов возврата. Это и есть стек вызовов.

Как показано на рис. 3.8, инструкция `return` переносит нас назад тем же путем, которым мы пришли. Функция `throw` отличается; она позволяет пропускать шаги в стеке вызовов. Она пропускает каждую вызванную функцию до тех пор, пока не достигнет точки, в которой определена инструкция `catch`:

```
function alpha()
    try                                ❶
        beta()
    catch e
        # обработать исключение      ❷
    end
end
```

- ❶ Определяет блок кода, в котором где-то в стеке вызовов может возникнуть исключение.
- ❷ Если исключение все же возникнет, то оно будет отловлено, и этот блок кода предназначен для очистки или обработки исключения.

Информация об ошибке хранится в объекте-исключении, который был передан в функцию `throw`. Этот объект назначается переменной `e`; благодаря этому блок `catch` может получать доступ к информации о возникшей ошибке. На данный момент мы не сможем подробно обсудить тему исключений, так как для ее изложения требуется более глубокое понимание системы типов Julia. Однако вы можете поэкспериментировать с исключениями в интерактивной среде REPL, чтобы испытать результаты прерывания ими нормального потока управления:

```
julia> y = try
    fac(3)
catch e
    42
end
6

julia> y = try
    fac(-3)
catch e
    42
end
42

julia> fac(-3)
```

```
ERROR: DomainError with -3:
`n` must not be negative.
```

Напомню, почти все в Julia является выражением, включая блоки try-catch.

3.9 Поток управления в отличие от потока данных

Теперь, когда мы рассмотрели различные формы потока управления, мы обсудим смысл потока управления подробнее. Сравнение потока управления с потоком данных поможет лучше понять эту концепцию. Давайте рассмотрим простой фрагмент исходного кода:

```
alice = encrypt(bob)
```

На него можно посмотреть с двух разных точек зрения: хранящееся в bob сообщение *втекает* в функцию шифрования encrypt, а объект-шифротекст *вытекает* из функции (рис. 3.9).

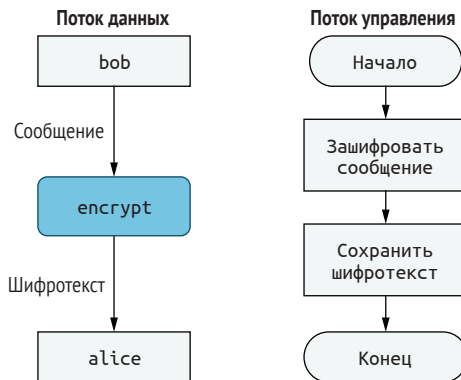


Рис. 3.9 Противопоставление потока данных и потока управления

В случае потока данных данные текут вдоль стрелки между прямоугольниками. На рис. 3.9 светлые прямоугольники – это истоки и стоки, а темный прямоугольник – фильтр. Он преобразовывает поступающие данные в другой тип данных.

На диаграмме потока управления (например, блок-схеме) стрелки представляют не перетекание данных, а передачу управления. Поток управления – это то, как управление переходит от одного блока к другому и как этот поток можно изменять и перенаправлять:

```
y = gamma(beta(alpha(x)))
```

В приведенном выше примере можно подумать о том, как управление передается от функции alpha к функции beta и, наконец,

к функции `gamma`: здесь мы имеем поток управления. С точки зрения потока данных мы думаем о данных, которые втекают в `alpha`, вытекают из нее и затем втекают в `beta`.

При анализе сложного исходного кода бывает полезно рисовать схему потока данных. Помечая стрелки типами данных, входящих и исходящих из функций (фильтров), можно получать более полное представление о потоках сложных данных в исходном коде.

3.10 Подсчет кроликов

Во многих книгах по программированию можно найти реализацию функции `fib`, которая расшифровывается как *Фибоначчи*. Почему эта функция так популярна в программировании? И зачем она нужна? Перечислим следующие ниже причины.

- 1 Это простой способ демонстрации переложения математической формулы в исходный код.
- 2 Ее реализация позволяет противопоставлять решение задач с помощью рекурсии и итераций (цикла).
- 3 Числа Фибоначчи всплывают в самых разных обстоятельствах реальной жизни: в числе лепестков цветка, спиралей у подсолнуха или раковине наутилуса и изломах, которые появляются в листорасположении (филлотаксисе).
- 4 Это простая демонстрация разработки модели явлений реального мира.

Вот так выглядит числовая последовательность, которая продолжается до бесконечности:

0, 1, 1, 2, 3, 5, 8, 13, 21, 34, 55, 89, 144, ...

Каждое число этой последовательности называется *числом Фибоначчи*. Математики любят обозначать каждое число буквой *F*. Первое число последовательности – F_0 , второе – F_1 и т. д. Другими словами, индексация начинается с 0. Математически числа Фибоначчи определяются следующим образом:

$$F_0 = 0, F_1 = 1,$$

$$F_n = F_{n-1} + F_{n-2}, n > 1.$$

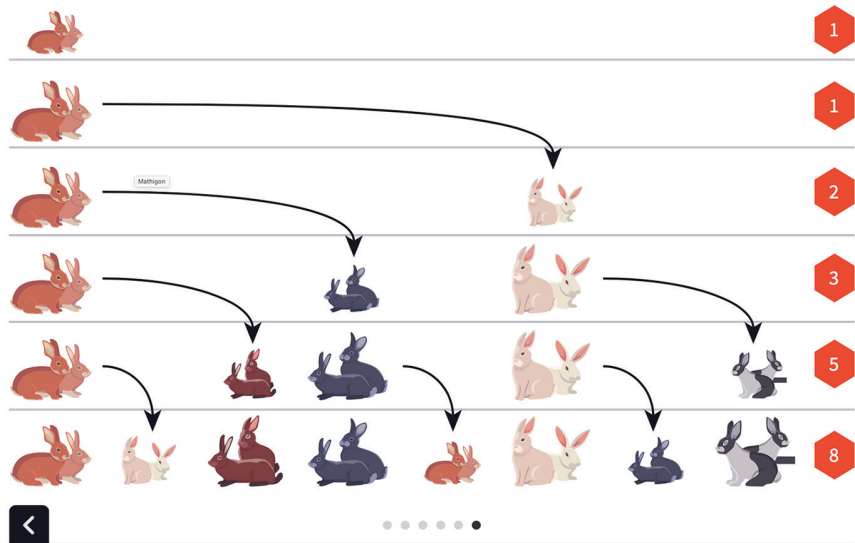
Возможно, это определение покажется столь же информативным, как и страница в Википедии (то есть не очень), поэтому, для того чтобы дать некоторое понимание этого математического определения на интуитивном уровне, я попытаюсь продемонстрировать наглядно на конкретном примере: росте популяции кроликов. Собственно говоря, так и были открыты числа Фибоначчи. Леонардо Пизанский,

он же Фибоначчи, около 800 лет назад задался вопросом, как популяция кроликов будет расти каждый месяц. Он задал следующий гипотетический вопрос:

Если в начале года имеется одна пара кроликов, то сколько их будет в конце года?

В целях ответа на этот вопрос нужно разработать модель реальности. При разработке моделей надо стараться выделять наиболее важные особенности конкретных признаков, которые будут моделироваться. Например, нас не интересует расселение кроликов, их внешний вид, их питание или добывание ими пищи. Модель касается только роста их популяции. Все модели строятся под определенную цель; если требуется проверить возможность класть новый мобильный телефон в карман, то модель не должна быть сложнее куска дерева. Здесь требуется симитировать лишь физические размеры телефона, а не цвет корпуса или четкость экрана.

Таким образом, модели всегда предусматривают значительные упрощения реальности. В модели роста числа кроликов на основе последовательности чисел Фибоначчи мы имеем дело с бессмертными кроликами. Они никогда не умирают. Они рождаются и через месяц после рождения начинают размножаться. Мы всегда моделируем их парами. Каждый месяц одна пара кроликов производит другую пару кроликов, после того как они достигают репродуктивного возраста (рис. 3.10).



На шестом месяце имеется три репродуктивные пары: изначальная, а также их две первые пары, или детеныши

Рис. 3.10 Ежемесячный рост популяции кроликов, как показано на Mathigon

Mathigon (<https://mathigon.org/course/sequences/fibonacci>) – отличный онлайн-источник, демонстрирующий рост популяции кроликов в интерактивном режиме в соответствии с последовательностью чисел Фибоначчи. Шестиугольники на снимке экрана показывают количество пар кроликов в данный месяц. В первый месяц имеется только 1 пара, а в шестой месяц их уже 8 пар. Реализовав функцию `fib` (листинг 3.17) вычисления чисел Фибоначчи, она будет работать следующим образом: `fib(1)` соответствует F_1 ; `fib(n)` соответствует F_n .

Листинг 3.17 Вычисление чисел Фибоначчи

```
function fib(n)
    if n == 0                ❶
        0
    elseif n == 1           ❶
        1
    else
        fib(n-1) + fib(n-2)  ❷
    end
end
```

- ❶ Математическое определение гласит, что $F_0 = 0$, $F_1 = 1$, что здесь как раз и выражено.
- ❷ Для всех других значений n они равны двум предыдущим числам Фибоначчи. Таким способом выражается рекурсия. Функция `fib` вызывает саму себя с другим аргументом.

Попробуем разобраться с работой этой функции на практике. Что происходит при попытке вычислить `fib(3)`? В этом случае задается $n = 3$. Всякий раз, когда $n > 1$, вычисляется следующая ниже строка:

```
fib(n-1) + fib(n-2)
```

Функция будет вычисляться снова и снова, но аргумент n каждый раз будет уменьшаться на 1 и 2, а это означает, что рано или поздно первые условия функции `fib` станут истинными. Затем результат всплывает вверх, выполняя более ранние запросы. Таким образом, в этом случае у нас получается своего рода двойная рекурсия. Следующие ниже примеры в интерактивной среде REPL дают представление о механизме работы функции Фибоначчи:

```
julia> fib(3) == fib(2) + fib(1)
true

julia> fib(4) == fib(3) + fib(2)
true

julia> n = 5
5
```

```
julia> fib(n) == fib(n-1) + fib(n-2)
true
```

3.10.1 Базовый случай

Во избежание выполнения рекурсии до полного израсходования всей стековой памяти необходимо определить *базовый случай*. Это инструкция `if`, которая позволяет в какой-то момент выходить из рекурсии.

Листинг 3.18 Вычисление чисел Фибоначчи

```
function fib(n)
    if 0 <= n <= 1      ❶
        n
    else
        fib(n-1) + fib(n-2)
    end
end
```

❶ Проверить, не находится ли `n` в диапазоне от 0 до 1, то есть равно или больше нуля, а также меньше или равно единице.

Условие `0 <= n <= 1` определяет базовое условие, или точку выхода. Что-то подобное требуется в каждой рекурсивной функции. Рекурсивная функция – это функция, которая вызывает сама себя.

3.10.2 Итерации по сравнению с рекурсией

Ранее было продемонстрировано, что рекурсия – это лишь один из многих способов решения задачи; она никогда не является обязательной. Рекурсию всегда можно заменить итерациями. Под итерациями я подразумеваю цикл (например, использование цикла `for` или цикла `while`). Например, с помощью следующего ниже исходного кода прокручивается диапазон от 0 до 4:

```
for i in 0:4
    println(i)
end
```

Если ту же задачу можно решить с помощью итераций, то зачем использовать рекурсию? Давайте посмотрим на итерационное решение, чтобы обсудить его плюсы и минусы.

Листинг 3.19 Вычисление числа Фибоначчи с использованием итераций

```
function fibi(n)
    if 0 <= n <= 1      ❶
        return n
    end
end
```



```

end
prev = 0
x = 1
for i in 2:n
    tmp = x
    x += prev
    prev = tmp
end
x
end

```

- ❶ Досрочный выход; во избежание глубокой вложенности используется ключевое слово `return`, чтобы немедленно выйти из функции со значением `n`.
- ❷ Переменная `prev` используется для представления `fib(n-2)`.
- ❸ Для хранения окончательного результата `fib(n)`.
- ❹ Это сокращенная форма для `x = x + prev`, что эквивалентно вычислению `fib(n-1) + fib(n-2)`.

Хотя этот исходный код, возможно, выглядит концептуально проще, хорошо видно, что итеративная версия все намного усложняет. Требуется гораздо больше служебной работы, а это приводит к большему числу переменных, которые нужно надлежаще поддерживать и обновлять. На создание этого примера исходного кода я потратил заметно больше времени, при этом допустив несколько ошибок в первоначальной версии. Рекурсивный вариант, напротив, был написан правильно с первой попытки.

Таким образом, хотя к рекурсии придется некоторое время привыкать, она зачастую значительно упрощает исходный код. Одним из недостатков рекурсии является нередко более медленное ее исполнение. Таким образом, как правило, сначала решение реализуется с использованием рекурсии, и если оно оказывается слишком медленным, то оно переписывается с использованием итераций.

3.10.3 Возвращать или не возвращать

В последнем примере для досрочного выхода из функции использовалась инструкция `return`. Это делать тоже не обязательно, но, как видите, написание исходного кода без `return` затрудняет его чтение. Это связано с тем, что в итоге получается глубокая вложенность инструкций потока управления, что наглядно показано в следующем ниже листинге.

Листинг 3.20 Вычисление чисел Фибоначчи без досрочного возврата

```

function fibi(n)
    if 0 <= n <= 1
        n
    else
        prev = 0
        x = 1
    end
end

```

```
    for i in 2:n
        tmp = x
        x += prev
        prev = tmp
    end
    x
end
end
```

В подобной ситуации нет жестких правил. Просто нужно руководствоваться здравым смыслом и полагаться на собственное чувство хорошего вкуса. Как правило, я стараюсь избегать вложений глубже трех уровней; вместе с тем следует избегать создания строгих правил. Одержимость высеченными в камне правилами всегда преследовала индустрию программного обеспечения. Лучше проявлять гибкость и полагаться на здравый смысл. Разработчики языка Julia изначально стремились к его прагматичности.

Резюме

- В инструкциях потока управления используются условия с применением булевых выражений. Циклы повторяются до тех пор, пока условие остается истинным.
- В Julia даже инструкции управления потоком являются выражениями, а значит, они возвращают значение. В Julia даже `nothing` не является значением.
- Способность компьютеров повторять аналогичные вычисления большое число раз делает их пригодными для вычислений, которые трудно выполнять вручную, например для вычисления тригонометрических функций.
- Рекурсивная функция – это функция, которая вызывает саму себя. Рекурсивные функции должны иметь *базовый случай*, иначе они никогда не прекратят свою работу.
- Рекурсия и итерации могут решать одни и те же задачи. Рекурсия нередко упрощает написание исходного кода, а итерации обычно обеспечивают более высокую производительность.

Julia в качестве электронной таблицы

Эта глава охватывает следующие ниже темы:

- работа с коллекциями чисел с использованием типов `Array` и `Tuple`;
- полезные типы для включения в коллекции, такие как числа, знаки и текстовые строки;
- выполнение статистических расчетов;
- преобразование списков чисел с помощью функции `map`;
- применение предикатов с помощью функции `filter`.

Во второй главе мы обсудили способы работы с Julia в качестве калькулятора. Однако сегодня люди, которым приходится работать с числами, обычно не пользуются настольными калькуляторами; они используют инструменты для работы с электронными таблицами, такие как Microsoft Excel или Apple Numbers (рис. 4.1).

В этих приложениях числа хранятся в таблицах. Вся серьезная научная работа предусматривает работу с большими таблицами данных, включая целые столбцы чисел. Ученые и аналитики данных получают данные статистических опросов или измерений, которые они хотят анализировать. Они не работают с графическим инструментом электронных таблиц в буквальном смысле, а манипулируют

данными в табличной форме, как в современном приложении для работы с электронными таблицами. И Julia отлично подходит для такой работы.

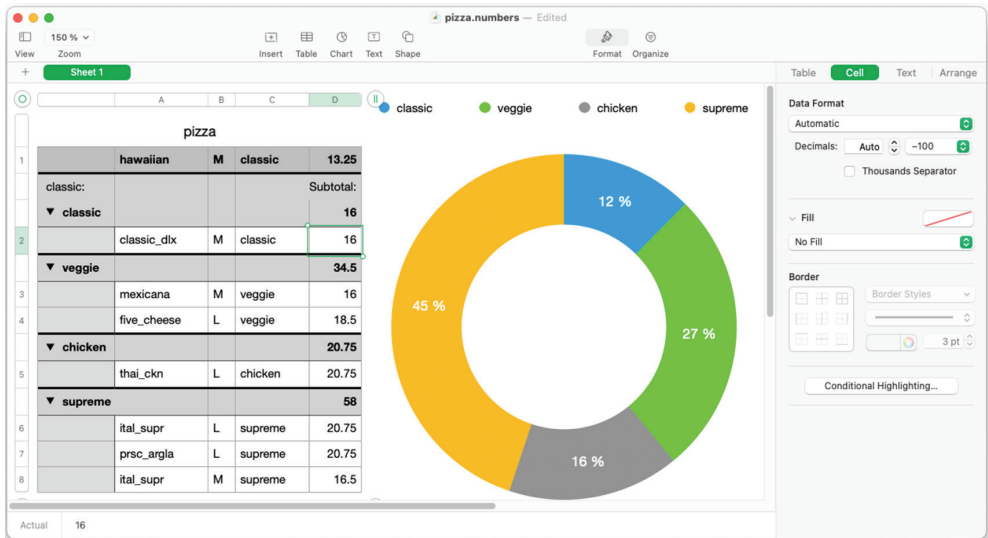


Рис. 4.1 Apple Numbers – приложение для работы с электронными таблицами, предназначенное для работы со строками и столбцами чисел

В этой главе вы коснетесь только поверхности того, что можно делать. Главной же целью станет введение типов данных `Array` и `Tuple`. Поскольку они являются контейнерами для других значений, вы также коснетесь типов `Char` (буквенный символ) и `String` (строковый литерал), чтобы иметь что-то интересное внутри массивов и кортежей. На самом деле в эти два типа коллекций вы будете помещать числа, буквенные символы и строковые литералы, а также булевы значения.

4.1 Анализ продажи пиццы

В целях более глубокого понимания предназначения различных операций на массивах я воспользуюсь примером продажи пиццы. В табл. 4.1 показаны разные виды пиццы, продаваемые в разном количестве и по разным ценам. Вы научитесь использовать исходный код Julia для ответа на следующие вопросы:

- сколько всего пицц было продано?
- какова общая выручка от продажи пицц?
- какова средняя цена проданных пицц?
- каково среднее число пицц, проданных в каждой категории?

Таблица 4.1. Данные о продаже пиццы; в каждой строке указано количество проданных пицц каждого типа

Пицца	Количество	Цена
Пеперони	4	15.0
Маргарита	1	11.5
Курица барбекю	5	13.0
Гавайская	3	12.75
Прощутто	2	14.25

4.2 Различные типы массивов

Массив в Julia может представлять строку таблицы, ее столбец или всю таблицу чисел. На самом деле массивы могут содержать элементы любого типа, а не только числа. Это могут быть массивы булевых значений, буквенных символов или строковых литералов. При этом можно запрашивать элементы, находящиеся в определенной позиции, например «получить третий элемент в массиве А».

Давайте создадим столбец чисел, содержащий количество проданных пицц. Обратите внимание, как числа расположены вертикально. На языке Julia так сообщается о том, что мы только что создали *вектор-столбец*. При создании вектора-столбца каждый элемент отделяется запятой:

```
julia> amounts = [4, 1, 5, 3, 2]
5-element Vector{Int64}:
 4
 1
 5
 3
 2
```

Отделяя каждый элемент пробелом, также можно создавать вектор-строку:

```
julia> row = [4 1 5 3 2]
1×5 Matrix{Int64}:
 4  1  5  3  2
```

Каждое значение в векторе имеет связанный с ним *индекс элемента*, как показано на рис. 4.2.

Индексы элементов				
1	2	3	4	5
4	1	5	3	2

row = [4 1 5 3 2]

Рис. 4.2 Иллюстрация организации элементов в вектор-строку

Слово *вектор* обычно используется для обозначения одномерных массивов, а двумерный массив называется *матрицей*. В приложении для работы с электронными таблицами матрицу можно рассматривать как таблицу. В Julia таблицы конструируются путем укладки векторов-строк друг над другом.

Обратите внимание, что каждая строка отделена точкой с запятой. Ниже показана таблица со столбцами количеств (amounts) и цен (prices):

```
julia> pizzas = [4 15.0;
                  1 11.5;
                  5 13.0;
                  3 12.75;
                  2 14.25]
5×2 Matrix{Float64}:
 4.0  15.0
 1.0  11.5
 5.0  13.0
 3.0  12.75
 2.0  14.25
```

Новые строки не требуются; тем самым облегчается чтение исходного кода. Точно такую же матрицу можно было бы получить, написав следующее:

```
pizzas = [4 15.0; 1 11.5; 5 13.0; 3 12.75; 2 14.25]
```

В целях концептуализации принципа организации столбцов и матриц ниже приведена наглядная иллюстрация (рис. 4.3). В случае одномерных векторов мы обычно говорим об индексах элементов, но в случае матрицы нумеруются как строки, так и столбцы.

		Индексы столбцов	
		1	2
Индексы строк	Количества	4	15.0
	Цены	1	11.5
		5	13.0
		3	12.75
		2	14.25
		Пиццы	

Рис. 4.3 Сравнение векторов-столбцов и матриц

Однако в этой главе мы сосредоточимся в первую очередь на векторах-столбцах. Они наиболее точно соответствуют тому, что в дру-

гих языках называется *массивами*. Многомерные массивы не являются центральной функциональностью других языков, как в Julia.

4.3 Выполнение операций на массивах

Списки чисел не очень интересны, если только они не позволяют делать что-то полезное. К счастью, многие функции оперируют на массивах. Например, функция `sum` используется для сложения всех элементов массива. Ниже подсчитывается общее количество проданных пицц:

```
julia> no_pizzas_sold = sum(amounts)
15
```

Если требуется узнать количество элементов в `amounts`, то для этого применяется функция `length`. Она также позволяет рассчитывать среднее количество проданных пицц каждого типа:

```
julia> length(amounts)
5
```

```
julia> avg = sum(amounts) / length(amounts)
3.0
```

Давайте поместим цены в переменную, чтобы было с чем экспериментировать:

```
julia> prices = [15.0, 11.5, 13.0, 12.75, 14.25]
5-element Vector{Float64}:
 15.0
 11.5
 13.0
 12.75
 14.25
```

В целях упрощения обзора имеющихся цен их можно отсортировать:

```
julia> sorted = sort(prices) ❶
5-element Vector{Float64}:
 11.5
 12.75
 13.0
 14.25
 15.0
```

```
julia> prices ❷
5-element Vector{Float64}:
 15.0
 11.5
```

```
13.0
12.75
14.25
```

- ❶ Отсортированные цены хранятся в `sorted`.
- ❷ Функция `sort` не видоизменила `prices`.

При вызове функции `sort` создается новый вектор. Вектор `prices` не видоизменяется. По принятому соглашению функции Julia никогда не видоизменяют свои входные аргументы. Однако иногда такая потребность возникает. Разработчики языка Julia установили условное обозначение в виде восклицательного знака (!), прикрепляемого к имени любой функции, изменяющей свои входные аргументы. Отсюда многие функции Julia, которые не видоизменяют свои входные аргументы, имеют родственные функции, которые это делают. Например, функция `sort!` будет сортировать свой входной вектор, а не возвращать новую отсортированную версию:

```
julia> sort!(prices)
5-element Vector{Float64}:
 11.5
 12.75
 13.0
 14.25
 15.0
```

```
julia> prices
5-element Vector{Float64}:
 11.5
 12.75
 13.0
 14.25
 15.0
```

❶

- ❶ Цены были видоизменены функцией `sort!`

Что делать, если вы живете в стране с налогом на добавленную стоимость? В целях вычисления стикерной цены пиццы необходимо добавлять налог. Если вы живете в Норвегии, то налог на добавленную стоимость составляет 25 %. Рассчитаем новые цены с налогами:

```
julia> prices = [15.0, 11.5, 13.0, 12.75, 14.25];

julia> prices_with_tax = prices * 1.25
5-element Vector{Float64}:
 18.75
 14.375
 16.25
 15.9375
 17.8125
```


Но что, если требуется узнать количество денег, заработанных на каждом виде пиццы? Можно попытаться умножить количество (amounts) на цены (prices):

```
julia> amounts * prices
ERROR: MethodError: no method matching *(::Vector{Int64},
::Vector{Float64})
```

Не беспокойтесь о сообщении об ошибке. В последующих главах я объясню концепции, которые нужно усвоить, чтобы его читать.

На данный момент нужно усвоить, что нет очевидного определения того, что именно должно генерировать умножение между двумя столбцами чисел. Его можно интерпретировать самыми разными способами. И следовательно, необходимо сообщить Julia в явной форме о том, что нужны поэлементные операции. Это достигается путем добавления точки к математическому оператору. Операторы +, -, * и / предназначены для выполнения арифметических действий на отдельных числах (скалярах). А для выполнения поэлементных операций на массивах чисел нужно использовать операторы .+, .-, .* и ./:

```
julia> amounts .* prices
5-element Vector{Float64}:
 60.0
 11.5
 65.0
 38.25
 28.5
```

Этот результат можно передать функции sum, чтобы вычислить общую прибыль от продажи пиццы:

```
julia> sum(amounts .* prices)
203.25
```

4.4 Работа с модулем Statistics

Профессионалы, занимающиеся статистикой и анализом данных, обычно работают с таблицами данных; в связи с этим Julia позволяет легко реализовывать свои собственные функции статистических вычислений на отдельных столбцах данных. Ниже приведен базовый пример функции среднего арифметического значения average.

Листинг 4.1 Вычисление среднего арифметического значения

```
average(A) = sum(A) / length(A)
```

Вместо того чтобы изобретать велосипед, можно воспользоваться готовыми статистическими функциями. Они идут в комплекте с Ju-

lia, но размещены в модуле *Statistics* (см. <https://docs.julialang.org/en/v1/stdlib/Statistics/>). Модули будут рассмотрены подробнее позже, но их можно рассматривать как наборы готовых функций, которые можно использовать в своих программах. Для того чтобы использовать модуль, надо написать

```
using Statistics
```

Это приведет к тому, что определенные в модуле функции, типы и константы будут загружены и станут доступны для работы. Кроме того, станет доступна документация модуля. Напомню, вход в режим документации Julia осуществляется путем ввода вопросительного знака (?) в начале строки:

```
julia> using Statistics

help?> Statistics
search: Statistics

Statistics

Standard library module for basic statistics functionality.

help?> middle(3, 4)
middle(x, y)

Compute the middle of two numbers x and y, which is equivalent in both value and type to computing their mean  $((x + y) / 2)$ .
```

Для того чтобы получить общее представление о существующих в модуле типах и функциях, надо написать имя модуля и точку, а затем дважды нажать на клавишу **Tab**. В результате будут показаны все возможные завершения (для ясности некоторые результаты были опущены):

```
julia> Statistics.
corn          mean          realXcY
corzm         mean!         sqrt!
cov           median        std
cov2cor!      median!       stdm
covm          middle        unscaled_covzm
covzm         quantile       var
eval          quantile!      varm
include       range_varm    varm!
```

Давайте обследуем несколько статистических функций в интерактивной среде REPL:

```
julia> mean(amounts) ❶
3.0

julia> mean(prices)
13.3
```

```
julia> median(amounts) ②
3.0

julia> median(prices)
13.0

julia> std(amounts) ③
1.5811388300841898

julia> std([3, 3, 3])
0.0
```

- ① Вычислить среднее арифметическое массива `amounts`.
- ② Срединное значение после сортировки значений.
- ③ Стандартное отклонение.

Функции `mean` и `median` используются для вычисления средних значений, но работают немного по-разному. В случае функции `mean` все значения складываются и сумма делится на количество значений. Если среди значений есть несколько экстремальных значений, то среднее сильно искажается. И следовательно, когда, например, рассчитывается средний доход семьи, то обычно используется медиана (функция `median`). Медианный доход рассчитывается путем сортировки всех доходов домохозяйств и последующего выбора дохода в середине отсортированного списка. Благодаря такому подходу несколько сверхбогатых семей не будут искажать результат.

С помощью функции `std` находят стандартное отклонение в наборе значений. Стандартное отклонение служит мерой отличия значений. Если все элементы одинаковы, то стандартное отклонение будет равно нулю. До сих пор мы рассматривали работу с массивами в целом, но для того чтобы иметь возможность создавать свою собственную функциональность обработки массивов, нужно уметь обращаться к отдельным элементам массива.

4.5 Доступ к элементам

Каждый элемент в массиве *Julia* нумеруется, начиная с 1. Такой подход называется индексацией с отсчетом от 1 и очень распространен в числовых и математически ориентированных языках. Однако в мейнстримных языках, таких как *Python*, *C*, *C++* и *Java*, используется индексация с отсчетом от 0.

Для определения литералов массива, а также для доступа к отдельным элементам по индексу используются квадратные скобки:

```
julia> amounts = [4, 1, 5, 3, 2]
5-element Vector{Int64}:
 4
 1
 5
```

Индексация с отсчетом от 1 по сравнению с индексацией с отсчетом от 0

Разработчики любят спорить о том, как лучше всего индексировать массивы. В математике принято нумеровать элементы, строки и столбцы с использованием индексации с отсчетом от 1. При обсуждении деталей аппаратного обеспечения, таких как адреса памяти компьютера, чаще используется индексация с отсчетом от 0. Таким образом, в языках с числовой ориентированностью, как правило, используется индексация с отсчетом от 1, тогда как в языках, более близких к аппаратным средствам, таких как C, используется индексация с отсчетом от 0.

```
3
2
julia> amounts[1] ❶
4
julia> amounts[2]
1
julia> amounts[5] ❷
2
```

- ❶ Доступ к первому элементу массива `amounts`.
- ❷ Получить пятый (последний) элемент в массиве `amounts`.

Квадратные скобки используются как для определения литерала массива, так и для доступа к отдельным элементам. Разумеется, вы также хотите иметь возможность изменять отдельные элементы. Это делается аналогичным образом:

```
julia> xs = [2, 3]
2-element Vector{Int64}:
 2
 3
julia> xs[1] = 42
42
julia> xs
2-element Vector{Int64}:
42
 3
julia> xs[2] = 12
12
julia> xs
2-element Vector{Int64}:
42
12
```

При каждом изменении элемента его можно распечатывать, чтобы показывать, как массив выглядит в данный момент. Все эти примеры чисты и аккуратны. Но что произойдет, если попытаться обратиться к элементу с недопустимым индексом?

```
julia> xs[3] = 5 ❶
ERROR: BoundsError: attempt to access 2-element Vector{Int64} at index [3]
```

```
julia> xs[0] ❷
ERROR: BoundsError: attempt to access 2-element Vector{Int64} at index [0]
```

- ❶ Массив `xs` имеет только два значения, поэтому установить третий элемент невозможно. Julia выполняет проверку на наличие допустимых индексов.
- ❷ Элементы начинаются с индекса 1. В индексе 0 значений нет.

Приведенное выше поведение характерно для большинства мейнстримных языков. Однако некоторые старые популярные языки позволяют устанавливать элементы с любым индексом, независимо от установленного заранее размера массива.

В этих примерах с доступом к элементам есть пара трудностей:

- 1 индекс последнего элемента не всегда известен, так как массивы могут иметь разный размер и могут увеличиваться;
- 2 хотя в Julia стандартом является индексация с отсчетом от 1, в нем можно создавать массивы с отсчетом от 0.

Для урегулирования случаев, когда не всегда известен адрес начала или конца массива, следует использовать ключевые слова `begin` и `end`, которые позволяют обращаться соответственно к первому и последнему элементам:

```
julia> amounts[1] ❶
4
```

```
julia> amounts[begin] ❶
4
```

```
julia> amounts[5] ❷
2
```

```
julia> amounts[end] ❷
2
```

```
julia> amounts[4] ❸
3
```

```
julia> amounts[end-1] ❸
3
```

- ❶ Доступ к первому элементу. `[1]` и `[begin]` идентичны.
- ❷ Доступ к последнему элементу. `[5]` и `[end]` совпадают.
- ❸ Вычитая, можно делать такие вещи, как обращаться к предпоследнему элементу.

4.6 Создание массивов

До сих пор массивы создавались с использованием литералов массивов. *Литерал массива* означает, что каждый элемент, из которого массив состоит, перечисляется буквально. Например, `[4, 8, 1]` и `[false, false, true]` – это примеры литералов массивов. Переменная `xs` может ссылаться на массив, но не является литералом массива. Однако литералы массивов не очень эффективны при создании больших массивов. Целый ряд функций, таких как `zeros`, `ones`, `fill` и `rand`, упрощают быстрое создание массивов, содержащих определенные значения.

Например, что, если нужен массив, содержащий 50 элементов со значением 0? Для этого используется функция `zeros`:

```
julia> xs = zeros(50)
50-element Vector{Float64}:
 0.0
 0.0
 0.0
 ⋮
 0.0
 0.0
 0.0
 0.0

julia> length(xs)
50
```

Элементы вектора настолько часто инициализируются значением 1, что существует функция `ones`, которая делает это в явной форме. Указанная функция создает массив указанной длины, каждому элементу которого дается значение 1:

```
julia> ones(5)
5-element Vector{Float64}:
 1.0
 1.0
 1.0
 1.0
 1.0
```

Но еще существует возможность заполнять большой массив любым значением, используя функцию `fill`. Здесь создается массив из шести элементов, каждый из которых имеет значение 42:

```
julia> fill(42, 6)
6-element Vector{Int64}:
 42
 42
 42
 42
 42
 42
```

42

42

Во многих ситуациях нужны массивы с большим количеством случайных значений. Вызов функции `rand(n)` создает вектор, содержащий `n` случайных чисел от 0 до 1:

```
julia> rand(3)
3-element Vector{Float64}:
 0.5862914538673218
 0.8917281248249265
 0.37928032685681234
```

При создании массивов описание массива в интерактивной среде REPL будет выглядеть примерно так:

```
5-element Vector{Float64}
```

Оно говорит о том, что созданный вектор содержит пять элементов, и каждый из этих элементов имеет тип `Float64`. Но что, если нужны элементы другого типа? Скажем, нужны 8-битовые целые числа со знаком. Как это сделать? Функции `ones`, `zeros` и `rand` позволяют указывать тип элемента. Вот несколько примеров:

```
julia> ones{Int8, 5} ❶
5-element Vector{Int8}:
 1
 1
 1
 1
 1
```

```
julia> zeros{UInt8, 4} ❷
4-element Vector{UInt8}:
 0x00
 0x00
 0x00
 0x00
```

```
julia> rand{Int8, 3} ❸
3-element Vector{Int8}:
 -50
 125
 58
```

- ❶ Создать массив, содержащий 8-битовые целые числа со знаком, со значением 1. Обратите внимание, что в описании вектора указано `Vector{Int8}`.
- ❷ Создать четыре нуля беззнакового 8-битового типа. Обратите внимание на то, как в шестнадцатеричной форме записывается ноль, это принятый в Julia по умолчанию способ форматирования целых чисел без знака.
- ❸ Создать три случайных 8-битовых целых числа со знаком. Значения будут выбраны в случайном порядке из всего диапазона: от -128 до 127.

Следует отметить, что даже литералы массива позволяют указывать тип элемента. Так, можно указать, что требуется вектор 8-битовых целых чисел со знаком:

```
julia> xs = Int8[72, 69, 76, 76, 79]
5-element Vector{Int8}:
 72
 69
 76
 76
 79
```

Литерал массива имеет префикс типа, который применяется для каждого элемента – в данном случае `Int8`. Если префикс типа не указан, то Julia вычислит тип элемента автоматически. Детали того, как это работает, станут очевидны при обсуждении типов в главе 7. Если требуется проверить тип, к которому относится каждый элемент в массиве, то для этого применяется функция `eltype` (сокращение от англ. *element type*, то есть тип элемента):

```
julia> eltype(xs)
Int8

julia> eltype([3, 4, 5])
Int64

julia> eltype([true, false])
Bool
```

4.7 Отображение значений в массиве

Помимо простого сложения и умножения значений, можно делать гораздо больше. Во всех языках программирования, поддерживающих программирование в функциональном стиле, есть три функции, именуемые `map`, `reduce` и `filter`. Сначала давайте вернемся к предыдущим вычислениям таблицы синусов и обследуем функцию `map`. Если помните, мы написали эту функцию в главе 3.

Листинг 4.2 Исходный код создания таблицы синусов, хранящийся в функции

```
function print_sin_table(increment, max_angle)
    angle = 0
    while angle <= max_angle
        rad = deg2rad(angle)
        x = sin(rad)
        println(x)
        angle = angle + increment
    end
end
```


Однако, вместо того чтобы распечатывать таблицу значений синуса, можно создать массив всех значений синуса. Для этого используется функция `map`, которая предназначена для преобразования (а точнее, отображения) коллекции значений в другой массив значений. Здесь функция `map` преобразовывает массив градусов в массив радиан. Она применяет функцию `deg2rad` к каждому элементу входного массива:

```
julia> degs = [0, 15, 30, 45, 60];

julia> rads = map(deg2rad, degs)
5-element Vector{Float64}:
 0.0
 0.2617993877991494
 0.5235987755982988
 0.7853981633974483
 1.0471975511965976
```

Функция `map` называется *функцией более высокого порядка*. Это такие функции, которые принимают другие функции на входе и/или возвращают функции на выходе. Они отличаются от встречавшихся до сих пор функций, которые на входе принимают исключительно числа. Базовая форма функции `map` принимает функцию `f` в качестве первого аргумента и применяет эту функцию к каждому элементу в коллекции `xs`, создавая на выходе новую коллекцию `ys`:

```
ys = map(f, xs)
```

Второй аргумент, представляющий коллекцию, не обязательно должен быть фактическим массивом. Он может быть чем угодно, что можно прокручивать в цикле и получать несколько элементов; так, например, можно использовать объект-диапазон. Ниже используется диапазон от 0 до 90 с шагом 15 градусов:

```
julia> map(deg2rad, 0:15:90)
7-element Vector{Float64}:
 0.0
 0.2617993877991494
 0.5235987755982988
 0.7853981633974483
 1.0471975511965976
 1.3089969389957472
 1.5707963267948966
```

Функции `map` можно скомбинировать и создать таблицу синусов:

```
julia> map(sin, map(deg2rad, 0:15:90))
7-element Vector{Float64}:
 0.0
 0.25881904510252074
 0.49999999999999994
 0.7071067811865475
```

```
0.8660254037844386
0.9659258262890683
1.0
```

Однако обычно это делается не так. Вместо этого все подлежащие выполнению преобразования собираются в одну функцию. Такой подход более эффективен по потреблению памяти, так как каждый вызов функции `map` создает новый массив. Еще одним решением является предварительное выделение массива того же размера, что и на выходе, и использование этого массива в повторяющихся преобразованиях (отображениях одного значения в другое). Мутирующая функция `map!` позволяет делать именно это.

Она записывает свои выходные данные прямо в массив, указанный в качестве второго аргумента. Третий аргумент – это входные данные, которые функция `map!` не будет видоизменять.

Однако если входной аргумент имеет тот же тип и длину, что и выходной, то входной аргумент можно реиспользовать в качестве выходного аргумента:

```
result = zeros(Float64, length(0:15:90))  ❶
map!(deg2rad, result, 0:15:90)           ❷
map!(sin, result, result)                 ❷
```

❶ Выделить массив для хранения результатов.

❷ Входной и целевой массивы должны иметь одинаковую длину.

Однако это не самый элегантный способ написания исходного кода, и лучше избегать мутирующих вызовов, так как они значительно усложняют анализ потока данных. И следовательно, все преобразования обычно собирают в одну функцию, тем самым уменьшая объемы выделяемой памяти, которые исходный код должен делать, и нередко упрощая его чтение:

```
deg2sin(deg) = sin(deg2rad(deg))
map(deg2sin, 0:15:90)
```

Первая строка – это однострочное определение функции. Вместо него можно было бы использовать многострочное определение, но это заняло бы больше места:

```
function deg2sin(deg)
    sin(deg2rad(deg))
end

map(deg2sin, 0:15:90)
```

Самостоятельная реализация встроенных функций обеспечивает отличную возможность разобраться в их работе. В целях более глубокого понимания функции `map` давайте создадим свою собственную функцию отображения под названием `transform`. Она содержит новые понятия, о которых мы поговорим подробнее.

Листинг 4.3 Поддельная версия встроенной функции `map`

```
function transform(fun, xs)
    ys = []           ❶
    for x in xs
        push!(ys, fun(x))  ❷
    end
    ys               ❸
end
```

- ❶ Пустой массив для хранения итогового результата.
- ❷ Добавить преобразованный элемент в массив `ys` результатов.
- ❸ Вернуть окончательный результат.

Функция `transform` принимает два аргумента, `fun` и `xs`, где первый – это функция, а второй – массив или другой итерируемый объект-коллекция. Функции можно хранить в переменных и использовать. Ниже приведена простая демонстрация:

```
julia> sin(1.0)
0.8414709848078965

julia> g = sin
sin (generic function with 13 methods)

julia> g(1.0)           ❶
0.8414709848078965

julia> add = +          ❷
+ (generic function with 190 methods)

julia> add(2, 3)
5
```

- ❶ Вызов функции `sin`.
- ❷ Напомню, что в *Julia* оператор `+` является функцией.

Вот почему `fun` можно использовать как функцию и ее вызывать, несмотря на то что она является аргументом функции `transform`. Следующая ниже часть нуждается в дополнительном пояснении:

```
push!(xs, x)
```

Эта функция добавляет элемент `x` в массив `xs`. Напомню, что восклицательный знак предупреждает о том, что функция `push!` потенциально видоизменяет свои входные аргументы. В этом случае видоизменяется аргумент `xs`.

Для того чтобы вызывать правильную функцию, необходимо добавлять восклицательный знак; восклицательный знак является частью имени функции, так что `push` и `push!` будут считаться двумя разными именами функций. В *Julia* нет функции `push`. Если бы она существовала, то, как можно себе представить, она возвращала бы новый массив с дополнительным элементом.

При этом надо отметить, что добавлять восклицательный знак к определяемым вами функциям вовсе не обязательно, но вы должны выработать привычку это делать, чтобы помогать другим разработчикам читать ваш исходный код. Благодаря этому будет видно, где переменная потенциально видоизменяется (подвергается мутации).

СОВЕТ Возможно, новичку мутирующие функции не покажутся чем-то важным. Однако при написании больших программ вы начнете замечать, что функции, которые мутируют (модифицируют) свои входные данные, часто затрудняют чтение и отслеживание программ. Восклицательный знак помогает снижать умственную нагрузку при чтении исходного кода. Без него каждый вызов функции потенциально может видоизменять свои входные данные, что значительно усложнило бы анализ исходного кода.

Ниже приведена простая демонстрация механизма работы функции `push!`. Создается пустой массив `ys`, и в него добавляются числа. Хорошо видно, как каждый раз массив увеличивается:

```
julia> ys = []           ❶
Any[]

julia> push!(ys, 3)      ❷
1-element Vector{Any}:
 3

julia> push!(ys, 8)
2-element Vector{Any}:
 3
 8

julia> push!(ys, 2)
3-element Vector{Any}:
 3
 8
 2
```

- ❶ Создать пустой массив.
- ❷ Добавить в массив число 3.

4.8 Знакомство с буквенными символами и строковыми литералами

До сих пор вы работали почти исключительно с числами, но ваша таблица с пиццей содержит не только числа. Таблица также содержит текст, например названия пицц. Как в Julia работают с текстом?

Начнем с самого базового строительного блока. Текст состоит из буквенных символов, и в Julia один буквенный символ представлен типом `Char`. В памяти компьютера все является числом, включая и символы. Вот небольшая задача: посмотрите на следующий ниже пример и постарайтесь его понять:

```
julia> x = Int8(65)           ❶
65

julia> ch = Char(65)         ❷
'A': ASCII/Unicode U+0041 (category Lu: Letter, uppercase)

julia> ch = Char(66)
'B': ASCII/Unicode U+0042 (category Lu: Letter, uppercase)

julia> 'A'                   ❸
'A': ASCII/Unicode U+0041 (category Lu: Letter, uppercase)

julia> Int8('A')            ❹
65
```

- ❶ Создание 8-битового целого числа со знаком из 64-битового числа со значением 65.
- ❷ Создание буквенного символа из числа 65.
- ❸ Буквенный символ.
- ❹ Создание 8-битового числа из символа.

Для создания буквенных символов отдельные буквы заключаются в одинарные кавычки¹. 'A' и 'Y' – это буквенные символы.

Приведенный выше пример исходного кода показывает, что в Julia буквенные символы – это просто числа другого типа. Напомню, что типы `UInt8` и `Int8` потребляют одинаковое число битов и могут хранить одни и те же данные, но интерпретироваться по-разному. То же самое относится и к символам. Хотя в памяти они выглядят одинаково, тип определяет, что конкретно с ними можно делать. Например, два символа нельзя складывать, но зато к символу можно прибавлять число:

```
julia> 'A' + 'B'
ERROR: MethodError: no method matching +(::Char, ::Char)

julia> 'A' + 3
'D': ASCII/Unicode U+0044 (category Lu: Letter, uppercase)
```

Массивы символов можно создавать точно так же, как и массивы чисел или булевых значений:

```
julia> chars = ['H', 'E', 'L', 'L', 'O']
5-element Vector{Char}:
'H': ASCII/Unicode U+0048
```

¹ Буква представляет собой в прямом смысле букву от A до Z, а не, например, переменную или константу, содержащую букву.

```
'E': ASCII/Unicode U+0045
'L': ASCII/Unicode U+004C
'L': ASCII/Unicode U+004C
'O': ASCII/Unicode U+004F
```

ПРИМЕЧАНИЕ В целях улучшения удобочитаемости и ясности я иногда редактирую результат интерактивной среды REPL. Например, я удаляю прилагаемое к символам описание (category Lu: Letter, uppercase), так как оно создает много визуального шума.

Строковый литерал, или текстовая строка, – это просто цепочка буквенных символов. Обратите внимание на результат: строковые литералы обозначаются двойными кавычками, тогда как отдельные символы обозначаются одинарными кавычками:

```
julia> join(chars)
"HELLO"
```

Функция `join` на входе может принимать любой итерируемый объект (объект, который можно использовать в цикле `for`). Следовательно, можно передавать и диапазон символов:

```
julia> join('A':'G')
"ABCDEFG"
```

Кроме того, используя шаг 2 в объекте-диапазоне, можно пропускать каждый второй символ:

```
julia> join('A':2:'G')
"ACEG"
```

Отдельные символы можно собирать в строковый литерал и возвращать массив букв:

```
julia> collect("HELLO")
5-element Vector{Char}:
'H': ASCII/Unicode U+0048
'E': ASCII/Unicode U+0045
'L': ASCII/Unicode U+004C
'L': ASCII/Unicode U+004C
'O': ASCII/Unicode U+004F
```

Функция `collect` характерна своей универсальностью; она может превращать в массив любой объект, который позволяет прокручивать множество его значений в цикле. Так, с помощью нее можно собирать диапазоны:

```
julia> collect(2:5)
4-element Vector{Int64}:
 2
 3
```

```

4
5
julia> collect('B':'D')
3-element Vector{Char}:
 'B': ASCII/Unicode U+0042
 'C': ASCII/Unicode U+0043
 'D': ASCII/Unicode U+0044

```

Строковые литералы и буквенные символы очень полезны для представления данных о пиццах. Давайте посмотрим на способы компоновки информации о каждой пицце.

4.9 Хранение данных о пиццах в кортежах

Для этого будут использоваться близкородственные массивы, именуемые *кортежами*. При написании кортежей квадратные скобки `[]` следует заменять на круглые скобки `()`. Ниже приведен пример кортежа, описывающего информацию о продаже гавайской пиццы. В нем говорится, что малая (S) гавайская пицца была продана за 10.50 доллара:

```
pizza_tuple = ("hawaiian", 'S', 10.5)
```

Поскольку в ряде популярных языков, таких как Python и JavaScript, для обозначения строковых литералов используются как одинарные, так и двойные кавычки, стоит напомнить, что в этом примере `'S'` обозначает буквенный символ, а не строковый литерал. В Julia строковые литералы нельзя писать с использованием одинарных кавычек:

```

julia> 'hawaiian'
ERROR: syntax: character literal contains multiple characters

```

Таким образом, между `'S'` и `"S"` пролегает важное различие. Последнее представляет строковый литерал, который необходимо трактовать как коллекцию буквенных символов. Это различие аналогично разнице между числом 42 и массивом `[42]`. Вместо кортежа с данными о пицце можно использовать массив:

```
pizza_array = ["hawaiian", 'S', 10.5]
```

Так в чем же разница? Массивы предназначены для однородных данных. Каждый элемент должен быть одного типа. Но совершенно ясно, что в данном случае они *не* однотипны. В результате тип элемента массива становится типом `Any`:

```

julia> pizza = ["hawaiian", 'S', 10.5]
3-element Vector{Any}:

```

```
"hawaiian"
'S': ASCII/Unicode U+0053
10.5
```

```
julia> eltype(pizza)
Any
```

Позднее мы рассмотрим систему типов Julia более подробно. А пока можете думать, что Any означает, что сойдет все, что угодно. В массив можно помещать любое значение. Это не сработало бы, если бы тип элемента был более конкретным, например Int64:

```
julia> xs = [4, 5, 3]
3-element Vector{Int64}:
 4
 5
 3
```

```
julia> xs[1] = "hi"
ERROR: MethodError: Cannot `convert` an object of type String to an
object of type Int64
```

- ❶ Julia делает вывод, что каждый элемент имеет тип Int64.
- ❷ Julia не знает, как превратить строковый литерал в число.

Массив pizza, напротив, совершенно неразборчив, то есть ему не важен тип объекта. Отдельным элементам можно присваивать что угодно, потому что типом элемента массива pizza является Any:

```
julia> pizza[3] = true
true

julia> pizza[1] = 42
42

julia> pizza
3-element Vector{Any}:
 42
 'S': ASCII/Unicode U+0053
 true
```

С другой стороны, кортежи намного строже. Кортеж отслеживает тип каждого элемента; это можно увидеть, если вызвать функцию typeof на кортеже:

```
julia> pza = ("hawaiian", 'S', 10.5)
("hawaiian", 'S', 10.5)

julia> typeof(pza)
Tuple{String, Char, Float64}
```

Второе важное замечание состоит в том, что кортежи *немутуируемы*, то есть их нельзя изменять. Значения можно только читать.


```
julia> pza[1]
"hawaiian"

julia> pza[1] = "pepperoni"
ERROR: MethodError: no method matching
  setindex!{::Tuple{String, Char, Float64}, ::String, ::Int64}
```

В других аспектах кортежи очень похожи на массивы. Кортеж можно прокручивать в цикле так же, как массив или диапазон:

```
julia> for item in pza
    println(item)
end
hawaiian
S
10.5
```

Если кортежи фактически содержат числа, то их можно передавать таким функциям, как `sum`, `median` и `mean`:

```
julia> nums = (3, 4, 1)
(3, 4, 1)

julia> sum(nums)
8

julia> median(nums)
3.0
```

Давайте посмотрим, как создается список данных о продаже пицц (табл. 4.2), комбинируя кортежи и массивы. Вот данные, которые требуется сохранить в Julia.

Таблица 4.2. Данные о продаже пицц, в которых каждая строка содержит информацию о проданной пицце

Пицца	Размер	Цена
Гавайская	S	10.5
Сицилийская	S	12.25
Гавайская	L	16.5
Курица барбекю	L	20.75
Курица барбекю	M	16.75

Мы хотим эти данные обрабатывать и получать информацию, например, о выручке в целом или числе проданных больших пицц. На языке Julia ее можно представить следующим образом:

```
julia> sales = [
    ("hawaiian", 'S', 10.5),
    ("sicilian", 'S', 12.25),
    ("hawaiian", 'L', 16.5),
```

```

        ("bbq chicken", 'L', 20.75),
        ("bbq chicken", 'M', 16.75)
    ]

```

Для представления валютных данных здесь используются числа с плавающей точкой, что не очень подходит для этих целей. Если пишется программное обеспечение, обрабатывающее валютные данные, то следует всегда использовать числа с фиксированной точкой¹, но в образовательных целях я здесь упрощаю.

В целях упрощения обработки данных о пиццах для разных свойств будут определены функции *доступа*². Функция доступа будет вызываться для размера порции (*portion*) пиццы, потому что в стандартной библиотеке Julia уже есть функция *size*:

```

name(pizza)    = pizza[1]
portion(pizza) = pizza[2]
price(pizza)   = pizza[3]

```

Это обычные функции. Здесь для определения функций я использую однострочный синтаксис Julia, но я мог бы использовать и многострочное определение, как показано ниже:

```

function price(pizza)
    pizza[3]
end

```

Напомню, что в функции Julia значение последнего выражения возвращается. Писать `return pizza[3]` не обязательно.

Какой тип у этих аргументов *pizza*? Кортежи или массивы? На самом деле это не имеет значения, потому что доступ по индексу работает одинаково. Приведенные выше функции доступа удобны в работе с функцией *map*, потому что они позволяют, например, получать названия всех пицц:

```

julia> map(name, sales)
5-element Vector{String}:
 "hawaiian"
 "sicilian"
 "hawaiian"
 "bbq chicken"
 "bbq chicken"

```

В приведенном выше фрагменте исходного кода функция *name* применена к каждому элементу массива *sales*, и все результирующие значения собраны в новый массив.

¹ Для работы с валютными данными обратитесь к файлу `FixedPointDecimals.jl` либо файлу `CurrenciesBase.jl`.

² Функции, используемые для установки и получения значений внутри более сложной структуры данных, называются *функциями доступа*, или *функциями-получателями*.

4.10 Фильтрация пицц на основе предикатов

Теперь, когда есть немного полезных для работы данных, можно познакомиться со следующей функцией более высокого порядка:

```
ys = filter(p, xs)
```

Функция `filter` принимает коллекцию значений `xs` и возвращает подмножество этих значений `ys`. Конкретные значения коллекции `xs`, которые включаются в результирующее подмножество `ys`, определяются предикатом `p`. Вы спросите, что такое предикат?

ОПРЕДЕЛЕНИЕ Предикат – это функция, которая принимает некоторое значение и всегда возвращает булево значение *истина* (`true`) либо *ложь* (`false`).

В стандартную библиотеку Julia входит ряд предикатных функций. Вот несколько примеров:

```
julia> iseven(3)    ❶  
false
```

```
julia> iseven(2)    ❶  
true
```

```
julia> isodd(3)     ❷  
true
```

```
julia> isodd(4)     ❷  
false
```

- ❶ Проверить четность чисел (делятся на два).
- ❷ Проверить нечетность чисел (не делятся на два).

Предикаты не ограничиваются числами. Существуют также предикаты для буквенных символов:

```
julia> isuppercase('A')  ❶  
true
```

```
julia> isuppercase('a')  ❶  
false
```

```
julia> isspace(' ')      ❷  
true
```

```
julia> isspace('X')      ❷  
false
```

- ❶ Предоставленный буквенный символ является заглавной буквой?
- ❷ Буквенный символ является пробелом? Например, X – это буква, а не пробел.

Предикаты очень удобно использовать с функцией `filter`. Ниже приведен пример получения четных чисел из диапазона:

```
julia> filter(iseven, 1:10)
5-element Vector{Int64}:
 2
 4
 6
 8
10
```

Но для работы с данными о пиццах нужно определить свои собственные предикаты, которые позволят получать данные о продаже пицц определенного размера или типа:

```
issmall(pizza) = portion(pizza) == 'S'
islarge(pizza)  = portion(pizza) == 'L'
isbbq(pizza)    = name(pizza) == "bbq chicken"
```

4.10.1 Комбинирование функций более высокого порядка

Следующий ниже подход можно использовать для комбинирования функции `map`, функций доступа, функции `filter` и предикатов с целью выяснения количества денег, заработанных, например, на продаже больших пицц или пицц с курицей барбекю. Сначала надо найти большие пиццы:

```
julia> filter(islarge, sales)
2-element Vector{Tuple{String, Char, Float64}}:
 ("hawaiian", 'L', 16.5)
 ("bbq chicken", 'L', 20.75)
```

Далее надо получить цену больших пицц:

```
julia> map(price, filter(islarge, sales))
2-element Vector{Float64}:
 16.5
 20.75
```

Используя функцию `sum`, можно вычислить количество денег, заработанных на продаже больших пицц:

```
julia> sum(map(price, filter(islarge, sales)))
37.25
```

В последнем примере определяется количество денег, заработанных на продаже пицц с курицей барбекю:

```
julia> bbq_sales = filter(isbbq, sales)
2-element Vector{Tuple{String, Char, Float64}}:
 ("bbq chicken", 'L', 20.75)
 ("bbq chicken", 'M', 16.75)
```

```
julia> sum(map(price, bbq_sales))
37.5
```

Оказывается, процедура отображения многочисленных значений в другой набор значений и последующее приведение всех этих значений к одному-единственному в программировании настолько распространена, что у нее есть собственное название: `mapreduce`. В последнем случае мы отображаем позиции продаж куриц барбекю в продажные цены, а затем их складываем. Такая процедура на языке функционального программирования называется *редукцией*.

4.11 Отображение и редукция массива

С помощью функции `mapreduce` последнюю часть можно было бы написать как один вызов функции:

```
julia> mapreduce(price, +, bbq_sales)
37.5
```

Функция `mapreduce` состоит из двух функций более высокого порядка: `map` и `reduce`. В целях демонстрации принципа ее работы давайте создадим свой собственный вариант `mapreduce` под названием `mapcompress`, чтобы избежать конфликта имен:

```
mapcompress(f, g, xs) = reduce(g, map(f, xs))
```

Поясню принцип работы функции `reduce`: в качестве первого аргумента она принимает некоторую двоичную функцию `g`, а затем использует эту функцию для комбинирования элементов коллекции `xs`, предоставленной в качестве второго аргумента.

```
g(x, y) = ...
y = reduce(g, xs)
```

В отличие от функции `map`, функция `reduce` требует, чтобы входная функция принимала два аргумента. Вот почему она называется двоичной функцией. Обычные математические операторы, такие как `+`, `-` и `*`, являются в Julia двоичными функциями. Следовательно, их можно использовать с функцией `reduce` для выполнения эквивалента функций `sum` и `factorial`:

```
julia> sum(2:4)
9

julia> reduce(+, 2:4)
9

julia> factorial(4)
24
```

```
julia> reduce(*, 1:4)
24
```

ПРИМЕЧАНИЕ Многие разработчики считают имя функции `reduce` не совсем понятным. Потенциально более подходящими именами были бы *accumulate* (накапливать), *aggregate* (агрегировать) или *compress* (сжимать). В некоторых языках она называется *inject* (внедрять, инъектировать).

4.11.1 Таблица синусов с использованием `map` и `reduce`

Сама синусная функция, по сути, является классическим случаем отображения и редукции. Для каждого аргумента при вызове `sin(x)` получается бесконечная последовательность чисел, которая путем сложения редуцируется в одно-единственное значение. Вот как была реализована собственная синусная функция под названием `sine`, без конфликта со встроенной функцией `sin`.

Листинг 4.4 Синусная функция с использованием ряда Тейлора

```
function sine(x)
    n = 5
    total = 0
    for i in 0:n
        total += (-1)^i * x^(2i+1) / factorial(2i + 1)
    end
    total
end
```

Это вычисление можно выразить элегантнее, используя функцию более высокого порядка `mapreduce`.

Листинг 4.5 Синусная функция посредством `mapreduce` на ряде Тейлора

```
function sinus(x)
    n = 5
    taylor(i) = (-1)^i * x^(2i+1) / factorial(2i + 1)
    mapreduce(taylor, +, 0:n)
end
```

Здесь делается нечто новое: внутри функции `sinus` определяется новая функция под названием `taylor`, которая принимает один аргумент `i`. Данная функция используется для вычисления члена в ряду Тейлора, то есть одно из чисел в ряду Тейлора, которые будут затем складываться. Обратите внимание на возможность определять функцию внутри других функций.

Но почему бы не определить эту функцию снаружи? Это связано с тем, что в ней используется переменная `x`, а значение переменной

х за пределами определения синусной функции неизвестно. Если от этого у вас кружится голова, то не волнуйтесь. Данные понятия будут пересматриваться в книге неоднократно и, вероятно, на каком-то этапе станут более понятными.

Вызов `mapreduce(taylor, +, 0:n)` сначала применит функцию `taylor` к каждому значению в диапазоне `0:n`. В результате этой операции будет сгенерирован массив значений, которые затем будут скомбинированы оператором `+`, являющимся вторым аргументом функции `mapreduce`.

4.12 Подсчет совпадений с использованием булевых массивов

В главе 3 я упоминал, что бывает полезно трактовать булевы значения как целые числа 0 или 1. Теперь, когда вы познакомились с массивами, мы рассмотрим конкретный пример.

Листинг 4.6 Подсчет пицц, совпадающих с предикатами

```
julia> matches = map(islarge, sales)
5-element Vector{Bool}:
 0
 0
 1
 1
 0

julia> sum(matches)
2
```

Поскольку мы имеем комбинацию функций более высокого порядка `map` и `sum`, эту комбинацию можно заменить одним вызовом `mapreduce`. Однако последовательность из операции сложения элементов после операции отображения настолько распространена, что в функцию Julia `sum` была встроена поддержка отображения и сложения. В результате получается элегантный способ подсчитывать число больших пицц, число пицц с курицей барбекю и т. д.:

```
julia> sum(islarge, sales)
2

julia> sum(isbbq, sales)
2
```

Функция `sum` преобразовывает все продажи пицц в булевы значения, применяя первый аргумент в качестве предиката, в результате чего генерируется массив нулей и единиц, которые складываются функцией `sum`.

Прочитав эту главу, вы узнали о самых фундаментальных концепциях программирования. Без потока управления и коллекций данных программирование не было бы столь полезным. Именно способность легко работать с многочисленными элементами данных делает компьютеры такими универсальными и мощными.

Резюме

- Язык Julia поддерживает много разных типов массивов. *Вектор* – это одномерный массив, и *матрица* – это двумерный массив.
- Как в математике, так и в Julia проводится различие между *векторами-столбцами* и *векторами-строками*. Векторы-столбцы больше всего напоминают то, что в других языках программирования называется массивами.
- Julia представляет векторы-строки как матрицы только с одной строкой, поэтому векторы-строки сильно отличаются от одномерных массивов в других языках.
- Массивы в Julia по умолчанию индексируются с отсчетом от 1. Это означает, что первый элемент начинается с индекса 1.
- Математические операции можно выполнять на целых массивах на каждом элементе. Для этого перед обычными математическими операторами надо ставить точку: `.*`, `./`, `.-` и `.-`.
- Операции на массивах описываются как отображение, фильтрация или редукция (например, функции `sum` и `mean` выполняют редукцию, поскольку несколько значений сводятся к одному-единственному).
- Функции `zeros`, `ones`, `fill` и `rand` упрощают создание массивов с большим числом элементов.
- Массив объектов `Char` отличается от объектов `String`. Для того чтобы формировать строковый литерал, буквенные символы должны быть конкатенированы. Однако строковые литералы ведут себя аналогично массивам.
- Поведение кортежей аналогично массивам, но они немутуируемы, то есть их нельзя изменять.

5

Работа с текстом

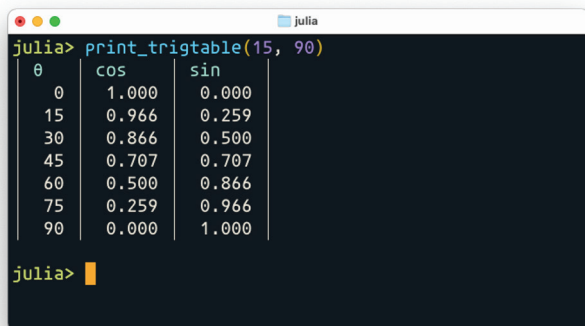
Эта глава охватывает следующие ниже темы:

- представление текста с помощью типа `String`;
- форматирование текста с помощью функций `lpad` и `rpad`;
- чтение текста с клавиатуры или из файла;
- вывод текста на экран или в файл;
- создание простой интерактивной программы.

В этой главе все внимание будет сконцентрировано на практических аспектах работы с текстом в Julia, например на том, как выводить текст на экран, а также читать его из файла или писать его в файл. Мы также рассмотрим простое интерактивное приложение, в котором пользователь пишет ответы на вопросы.

Однако сначала мы сосредоточимся на различных способах вывода текста на экран, вернувшись к примерам продажи пицц и таблицам синусов. Созданные ранее таблицы читались очень легко. А как насчет того, чтобы создать хорошо отформатированное отображение на экране, как на рис. 5.1?

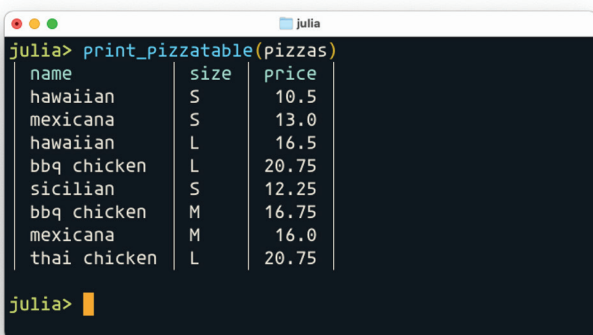
Здесь значения синуса и косинуса аккуратно расположены в отдельных столбцах. Было бы неплохо аккуратно отформатировать информацию о продаже пицц по аналогии с ними, расположив в четко разделенных столбцах, как показано на рис. 5.2.



```
julia> print_trigtable(15, 90)
  θ   | cos   | sin   |
  ---+---+---
  0    | 1.000 | 0.000 |
  15   | 0.966 | 0.259 |
  30   | 0.866 | 0.500 |
  45   | 0.707 | 0.707 |
  60   | 0.500 | 0.866 |
  75   | 0.259 | 0.966 |
  90   | 0.000 | 1.000 |

julia> 
```

Рис. 5.1 Аккуратно отформатированное отображение таблицы косинусов и синусов



```
julia> print_pizzatable(pizzas)
  name      | size | price |
  ---+---+---
  hawaiian  | S    | 10.5  |
  mexicana  | S    | 13.0  |
  hawaiian  | L    | 16.5  |
  bbq chicken | L    | 20.75 |
  sicilian  | S    | 12.25 |
  bbq chicken | M    | 16.75 |
  mexicana  | M    | 16.0  |
  thai chicken | L    | 20.75 |

julia> 
```

Рис. 5.2 Форматированное отображение продажи пицц с использованием выравнивания и цвета в окне терминала Unix

Мы будем использовать функцию `printstyled` для раскрашивания текста, функции `grad` и `lpad` для форматирования вывода, а также оператор `^` для повторения буквенных символов. Далее применим функцию `open`, которая позволяет использовать функции `print` и `println` для вывода текстового результата в файлы. Для чтения и обработки вводимых данных будут использоваться функции `getline`, `split` и `parse`.

5.1 Создание хорошо отформатированной таблицы продажи пицц

Мы начнем с просмотра окончательного результата, то есть исходного кода, который нужно написать, а затем двинемся в обратном направлении, получая объяснения по поводу того, как к этому

прийти. Приведенный ниже исходный код уже не должен выглядеть совершенно чужим, но есть несколько новых концепций, которые я объясню.

Листинг 5.1 Создание красивой таблицы продажи пицц

```
function print_pizzatable(pizzas)
    print("| ")
    printstyled(rpad("name", 12), color=:cyan)
    print(" | ")
    printstyled("size", color=:cyan)
    print(" | ")
    printstyled(rpad("price", 5), color=:cyan)
    println(" |")
    for pz in pizzas
        print("| ", rpad(name(pz), 12))
        print(" | ", rpad(portion(pz), 4), " | ")
        println(lpad(price(pz), 5), " |")
    end
end
```

Здесь мы имеем новые функции – `printstyled`, `rpad` и `lpad`, – работа которых требует дальнейшего обследования и объяснения. Для того чтобы функция `print_pizzatable` заработала, также потребуются функции доступа, которые были определены в главе 4. Все эти функции в качестве аргумента принимают кортеж `pizza`, например `("bbq chicken", 'L', 20.75)`, и возвращают один из элементов кортежа.

Листинг 5.2 Функции доступа к кортежу pizza

```
name(pizza)    = pizza[1]
portion(pizza) = pizza[2]
price(pizza)   = pizza[3]
```

Я расскажу о каждой функции, которая используется внутри функции `print_pizzatable`, подробнее и на нескольких простых примерах. В предыдущих главах функции `print` и `println` были затронуты лишь кратко, так что давайте рассмотрим их детали.

5.1.1 Функции `print`, `println` и `printstyled`

Это универсальные функции, которые можно использовать для вывода текста на экран или даже в файлы либо сетевое соединение (подробнее об этом см. в главе 18). Давайте рассмотрим несколько простых примеров, чтобы продемонстрировать механизм работы этих функций:

```
julia> println("hello world")
hello world
```

```
julia> print("hello world")
hello world
```

Хм? Разве они не делают то же самое? Нет, но на этом примере не-легко увидеть разницу. Вместо этого будем использовать точку с запятой (;), чтобы отделять инструкции на одной строке. Это поможет отчетливее увидеть разницу:

```
julia> println("hello"); println("world")
hello
world
```

```
julia> print("hello"); print("world")
helloworld
```

```
julia> print("hello\n"); print("world\n")
hello
world
```

Приведенный выше исходный код показывает, что инструкция `println` – это та же самая инструкция `print` с добавленным в конце символом новой строки `\n`. Julia, как и многие другие языки, позволяет отображать невидимые управляющие символы, такие как перевод строки, с помощью обратной косой черты и различных комбинаций букв. В табл. 5.1 показано несколько наиболее распространенных управляющих символов, которые можно использовать в Julia, чтобы влиять на внешний вид текста, выводимого в окно терминала Unix.

Таблица 5.1. Последовательности экранирования¹ для записи общих управляющих символов, используемых в терминалах Unix

Последовательность экранирования	Шестнадцатеричное значение	Эффект
<code>\n</code>	0x0A	Новая строка
<code>\t</code>	0x09	Горизонтальная табуляция
<code>\v</code>	0x0B	Вертикальная табуляция
<code>\r</code>	0x0D	Возврат каретки
<code>\b</code>	0x08	Возврат на одну позицию
<code>\\</code>	0x5C	Обратная косая черта
<code>\"</code>	0x22	Двойные кавычки

Двойная кавычка не является управляющим символом, но поскольку она используется для обозначения начала и конца строки, для ее представления необходимо использовать управляющую последовательность. Но какая польза от знания шестнадцатеричного значения? Его можно использовать непосредственно для создания буквенных символов. Ниже с помощью шестнадцатеричного значения 0x0a создается новая строка:

¹ Англ. escape sequence; син. escape-последовательность. – Прим. перев.

```
julia> newln = Char(0x0a)
'\n': ASCII/Unicode U+000A

julia> print("hi"); print(newln); print("world")
hi
world
```

Давайте посмотрим на другие примеры эффекта от использования приведенных выше разных последовательностей экранирования в сочетании с обычным текстом:

```
julia> println("hello \v world") ❶
hello
      world

julia> println("hello \n world")
hello
world

julia> println("hello \r world") ❷
world

julia> println("ABC\n\tABC\n\tABC")
ABC
      ABC
      ABC
```

- ❶ Здесь используется вертикальная табуляция `\v`, в отличие от более широко известной горизонтальной табуляции `\t`.
- ❷ Возврат каретки перемещает курсор в начало строки. Следовательно, слово `world` перезаписывает слово `hello`, написанное изначально.

Для понимания всего этого требуется определенный контекст. Почему буквенные символы в текстовых строках побуждают курсор перемещаться? Это связано с историей системы печати текста. Когда была разработана первая операционная система Unix, еще не было электронных дисплеев, подобных тем, которые используются сегодня. Вместо них пользователи компьютеров применяли электромеханические устройства, именуемые телетайпами (рис. 5.3). Телетайп очень похож на старомодную пишущую машинку.

Они также служили экраном. Когда компьютер передавал какую-то информацию, он посылал буквенные символы на пишущую машинку, что заставляло ту печатать отправленные символы на бумаге. Как следствие появилась потребность в управляющих символах, то есть таких, которые командовали телетайпом, приказывая ему создавать новую строку и перемещать каретку вниз или назад.

Используемые сегодня консольные приложения являются симуляторами тех старых телетайпов. Это было сделано для того, чтобы могли работать программы, написанные для работы с телетайпами. Команды Unix, такие как `ls`, `cp`, `cat` и `echo`, не подозревают, что работают на современных компьютерах с электронными дисплея-

ми. С их точки зрения они все еще взаимодействуют со старым добрым телетайпом.



Рис. 5.3 Телетайп Type 68D компании Siemens (Norsk Teknisk Museum)

Со временем эти бумажные терминалы были заменены электронными. На этом этапе управляющие символы были расширены новыми, чтобы управлять цветом. Например, когда электронный терминал будет получать последовательность экранирования `\u001b[33m`, он будет переключаться на написание желтых букв. Если бы он получил `\u001b[31m`, то он бы писал красными буквами.

Так, для того чтобы вывести `hello world` желтыми буквами, надо написать следующее:

```
julia> print("\u001b[33m hello world")
hello world
```

Однако запоминать эти последовательности экранирования для разных значений цвета очень обременительно. И поэтому Julia предоставляет функцию `printstyled`, которая позволяет указывать используемый цвет по имени. Цвет указывается с помощью именованного аргумента `color`:

```
julia> printstyled("hello world", color = :cyan)
hello world
```

Эта инструкция выводит `hello world` в сине-зеленом цвете. О функции `printstyled` и о возможных значениях цвета можно получить справку, переместив текстовый курсор в начало строки (**Ctrl+A**) и нажав `?`, чтобы перейти в режим справки:

```
help?> printstyled("hello world", color = :cyan)
printstyled([io], xs...; bold=false, color=:normal)
Print xs in a color specified as a symbol or integer,
optionally in bold.
color may take any of the values :normal, :default,
:bold, :black, :blink, :blue, :cyan, :green, :hidden,
:light_black, :light_blue, :light_cyan, :light_green,
:light_magenta, :light_red, :light_yellow, :magenta,
:nothing, :red, :reverse, :underline, :white, or :yellow
or an integer between 0 and 255 inclusive. Note that not
all terminals support 256 colors. If the keyword bold is
given as true, the result will be printed in bold
```

Все цвета заданы как *символические имена*¹. Символическое имя очень похоже на строковый литерал. Оно часто используется в работе со строковыми литералами, которые важны только для программистов, а не для пользователей программы. Объекты-символы можно создавать программно:

```
julia> sym = Symbol("hello")
:hello

julia> sym == :hello
true
```

5.1.2 Печать нескольких элементов

Все функции печати довольно универсальны в том, что они позволяют печатать и назначать количество элементов для вывода на печать:

```
julia> print("abc", 42, true, "xyz")
abc42truexyz
```

Числа и булевы значения при передаче в качестве аргументов различным функциям `print` конвертируются в строковые литералы:

¹ В языке Julia проводится различие между буквенным символом (`character`), строковым литералом (`string`) и символьным именем (`symbol`). Буквенный символ – это буквы, цифры, знаки препинания и другие литеры; он имеет тип `Char`. Строковый литерал – это цепочка буквенных символов; он имеет тип `String`. Символьное имя – это эффективный по производительности и потреблению памяти интернированный строковый литерал, в частности используемый в метапрограммировании и для быстрого доступа к полям данных; он имеет тип `Symbol`. – *Прим. перев.*

лы. Функция `string` работает точно так же, за исключением того, что вместо вывода на экран она возвращает значение типа `String`:

```
julia> string("abc", 42, true, "xyz")
"abc42truexyz"
```

Благодаря такому поведению одну функцию `println` можно использовать для вывода информации об одной пицце. Обратите внимание на использование определенных ранее функций доступа:

```
julia> pizza = ("hawaiian", 'S', 10.5)
("hawaiian", 'S', 10.5)

julia> println(name(pizza), " ", portion(pizza), " ", price(pizza))
hawaiian S 10.5
```

5.1.3 Печать нескольких пицц

Эту функцию можно использовать для написания простой таблицы продажи пицц.

Листинг 5.3 Примитивная распечатка таблицы пицц

```
pizzas = [
    ("hawaiian", 'S', 10.5),
    ("mexicana", 'S', 13.0),
    ("hawaiian", 'L', 16.5),
    ("bbq chicken", 'L', 20.75),
    ("sicilian", 'S', 12.25),
    ("bbq chicken", 'M', 16.75),
    ("mexicana", 'M', 16.0),
    ("thai chicken", 'L', 20.75),
]

for pz in pizzas
    println(name(pz), " ", portion(pz), " ", price(pz))
end
```

Однако с этим исходным кодом есть проблема – столбцы имени, размера и цены не выровнены, и в итоге мы получаем следующее:

```
hawaiian S 10.5
mexicana S 13.0
hawaiian L 16.5
bbq chicken L 20.75
sicilian S 12.25
bbq chicken M 16.75
mexicana M 16.0
thai chicken L 20.75
```

Для выравнивания данных нужно добавить отступы слева и справа, используя функции `lpad` и `rpad`.

5.1.4 Выравнивание с помощью функций `lpad` и `rpad`

С помощью функций дополнения можно указывать, что строковый литерал всегда должен иметь заданную длину. Если введенный текст меньше, то он будет дополнен выбранным знаком. Если знак не указан, то по умолчанию используется пробел.

```
julia> lpad("ABC", 6, '-') ❶  
"---ABC"
```

```
julia> rpad("ABC", 6, '-') ❷  
"ABC---"
```

- ❶ Дополнение слева.
- ❷ Дополнение справа.

Сначала делается дополнение знаком '-' с левой стороны до тех пор, пока вся строка не станет длиной 6 знаков. Второй пример идентичен, за исключением того, что знак '-' дополняется справа.

С помощью функций `lpad` и `rpad` можно задавать ширину каждого столбца таблицы и добавлять отступы, например пробелы, везде, где предоставленный строковый литерал короче. В приведенном ниже примере мы придерживаемся простоты, проверяя ширину самых широких строковых литералов в каждом столбце:

```
julia> length("thai chicken") ❶  
12
```

```
julia> length("size") ❷  
4
```

```
julia> max(length("16.75"), length("price")) ❸  
5
```

- ❶ Ширина столбца с названием пиццы.
- ❷ Ширина столбца размера.
- ❸ Ширина столбца цены.

Давайте видоизменим этот исходный код, чтобы задействовать отступы. Его можно вставить в терминал либо сохранить в файле и затем загрузить в терминал, например с помощью команды `include`. Обратите внимание, что в исходном коде знак дополнения не указан. Если его не указать, то по умолчанию будет применен пробел.

Листинг 5.4 Простая выровненная таблица пицц

```
function simple_pizatable(pizzas)  
    pname = rpad("name", 12)  
    psize = rpad("size", 4)  
    pprice = rpad("price", 5)
```

```

        printstyled(pname, " ",
                    psize, " ",
                    pprice,
                    color=:cyan)
    println()

    for pz in pizzas
        pname = rpad(name(pz), 12)
        psize = rpad(portion(pz), 4)
        pprice = lpad(price(pz), 5)
        println(pname, " ", psize, " ", pprice)
    end
end

```

- ❶ Для заголовка с описанием каждого столбца используется сине-зеленый цвет.
- ❷ Цифры дополняются слева, поэтому десятичные дроби выравниваются.

Работу приведенного выше исходного кода можно проверить в терминале:

```

julia> simple_pizzatable(pizzas)
name      size price
hawaiian  S    10.5
mexicana  S    13.0
hawaiian  L    16.5
bbq chicken L  20.75
sicilian  S    12.25
bbq chicken M  16.75
mexicana  M    16.0
thai chicken L  20.75

```

Обратите внимание на выравнивание точек: они выровнены не-точно. Эта проблема решается по-разному, но в следующем ниже примере с тригонометрическими таблицами мы сделаем так, чтобы каждое число имело одинаковое количество десятичных знаков после точки.

5.1.5 Добавление линий

Разделительные линии добавляются довольно просто. Для этого применяется символ Юникода в виде длинной вертикальной линии: '|':

```

julia> '|'
'|': Unicode U+2502 (category So: Symbol, other)

```

Это стилистика языка Julia – сообщать о том, что некий символ Юникода представлен шестнадцатеричным значением, в данном случае значением 0x2502. Следовательно, этот знак можно было бы получить двумя разными способами:

```
julia> Char(0x2502)
'|': Unicode U+2502

julia> '\U2502'
'|': Unicode U+2502
```

На самом деле существует очень много полезных символов Юникода, которые можно использовать для составления таблиц:

```
julia> collect("-|H")
3-element Vector{Char}:
'|': Unicode U+251C
'—': Unicode U+2500
'|': Unicode U+253C
'|': Unicode U+2524
```

Для того чтобы рисовать линии, полезно уметь повторять символы. В Julia оператор степени `^` используется для повторения символов:

```
julia> "A2"^3
"A2A2A2"

julia> "-"^4
"----"

julia> "-"^2
"__"
```

5.2 Распечатка тригонометрической таблицы

Все, что вы узнали, можно применить повторно, чтобы создать тригонометрическую таблицу. В приведенном ниже исходном коде используются те же функции, с которыми вы уже познакомились: `print`, `printstyled`, `lpad` и `grad`. Пока не пытайтесь выполнить эту функцию; я просто дам ее общий обзор.

Листинг 5.5 Создание тригонометрической таблицы

```
function print_trigtable(inc, maxangle)
    print(" | ")
    printstyled("θ ", color=:cyan)
    print(" | ")
    printstyled(rpad("cos", n),
                color=:cyan)
    print(" | ")
    printstyled(rpad("sin", n),
                color=:cyan)
    println(" | ")
```

```

angle = 0
while angle <= maxangle
    rad = deg2rad(angle)
    cosx = format(cos(rad))
    sinx = format(sin(rad))
    print(" | ")
    print(lpad(angle, 3), " | ",
          lpad(cosx, 6), " | ",
          lpad(sinx, 6))
    println(" | ")
    angle += inc
end
end

```

- ① Написать заголовок каждого столбца чисел.
- ② Написать строку тригонометрических значений каждого угла.

В приведенном выше исходном коде не так много новых идей; ключевая логика была рассмотрена в главе 3. Однако числа требуют специальной обработки на предмет выравнивания десятичной точки. После точки в каждом числе необходимо иметь одинаковое количество цифр, а слишком длинные числа не нужны. Если использовать результаты как есть, то получится слишком много цифр:

```

julia> rads = map(deg2rad, 0:15:90);

julia> map(sin, rads)
7-element Vector{Float64}:
 0.0
0.25881904510252074
0.49999999999999994
0.7071067811865475
0.8660254037844386
0.9659258262890683
1.0

```

Вместо этого требуется что-то вроде вот такого:

```

julia> print_trigtable(15, 180)
| θ | cos | sin |
| 0 | 1.000 | 0.000 |
| 15 | 0.966 | 0.259 |
| 30 | 0.866 | 0.500 |
| 45 | 0.707 | 0.707 |
| 60 | 0.500 | 0.866 |
| 75 | 0.259 | 0.966 |
| 90 | 0.000 | 1.000 |
| 105 | -0.259 | 0.966 |
| 120 | -0.500 | 0.866 |
| 135 | -0.707 | 0.707 |
| 150 | -0.866 | 0.500 |
| 165 | -0.966 | 0.259 |
| 180 | -1.000 | 0.000 |

```

Для получения подобного результата у нас есть вспомогательная функция, которую можно увидеть в главном листинге исходного кода: `format`.

Листинг 5.6 Вспомогательная функция форматирования чисел

```
n = length("-0.966")

function format(x)
    x = round(x, digits=3) ❶
    if x < 0
        rpad(x, n, '0')    ❷
    else
        rpad(x, n-1, '0')
    end
end
```

- ❶ Округлить вниз до трех знаков после точки.
- ❷ При наличии отрицательного числа необходимо разрешить дополнительный символ для знака -.

В переменной `n` хранится максимальная ширина числового литерала. Для того чтобы получить максимальное количество символов, необходимых для строкового литерала с числом, я, в сущности, использую сценарий развития по наихудшему варианту, в данном случае отрицательное число:

```
julia> format(3.1)
"3.100"
```

```
julia> format(-3.1)
"-3.100"
```

В приведенном выше примере вы видите, что при использовании отрицательных чисел необходимо допускать наличие большего количества символов. Позже вы будете печатать с помощью функции `lpad`, используя пробелы, то есть общая ширина изменяться не будет, независимо от наличия или отсутствия отрицательного числа:

```
julia> lpad(format(4.2), 6)
" 4.200"
```

```
julia> lpad(format(-4.2), 6)
"-4.200"
```

5.3 Чтение и запись продажи пицц в CSV-файлы

До сих пор вы работали с данными, представленными в табличном формате; именно для этого и было предназначено приложение по

обработке табличных данных. Очень распространенный файловый формат обмена данными между различными типами приложений по обработке табличных данных и научными приложениями называется CSV, который расшифровывается как *comma separated values*, то есть значения, отделенные друг от друга запятыми. Мы реализуем функцию `store_pizzatable`, чтобы записывать данные о пиццах в CSV-файл, и функцию `load_pizzatable`, чтобы читать из того же CSV-файла. Ниже приведен пример формата CSV-файла данных о пиццах, с которым будут работать обе функции:

```
name,size,price
hawaiian,S,10.5
mexicana,S,13.0
hawaiian,L,16.5
bbq chicken,L,20.75
sicilian,S,12.25
bbq chicken,M,16.75
mexicana,M,16.0
thai chicken,L,20.75
```

Первая строка называется *заголовком*. Он дает имя каждому столбцу в файле. Каждое значение в каждой строке отделяется запятой. На рис. 5.4 показан пример загрузки такого CSV-файла в Apple Numbers.

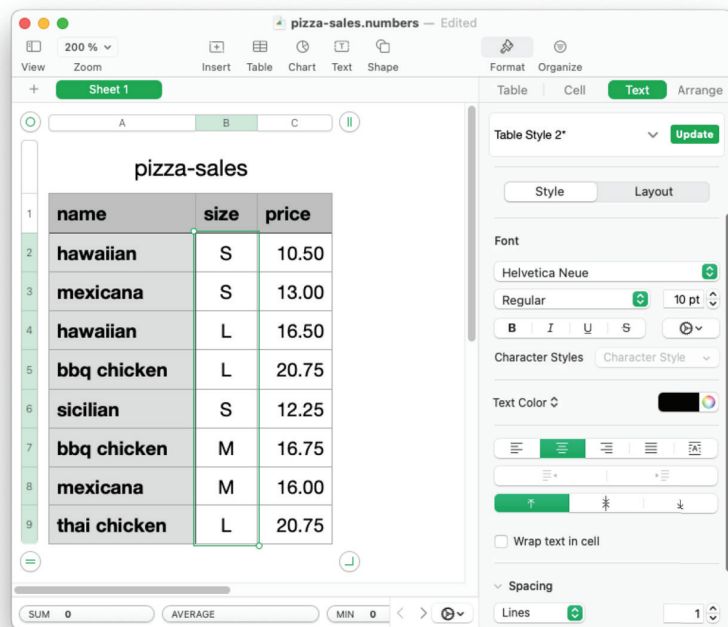


Рис. 5.4 Данные о продаже пицц, загруженные в табличное приложение Apple Numbers

Допустим, вы производите много полезных расчетов, которыми хотите делиться и которые хотите обследовать в таблице либо графически; в этом поможет экспорт расчетных данных в формат CSV. Для такой работы в Julia уже есть встроенные функции и очень хорошие внешние библиотеки, такие как CSV.jl, находящиеся по адресу csv.juliadata.org. Однако мы сосредоточимся на изучении основ чтения и записи файлов и поэтому не будем использовать ни внешние пакеты, ни функции.

5.3.1 Запись продаж пицц в файл

Давайте определим простую функцию `store_pizzatable`, которая выводит данные о продаже пицц в виде значений, отделенных запятыми.

Листинг 5.7 Функция экспорта данных о продаже пицц в виде значений, отделенных запятыми

```
function store_pizzatable(io, pizzas)
    println(io, "name,size,price") ❶
    for pz in pizzas                ❷
        println(io, name(pz), ",",
                portion(pz), ",",
                price(pz))
    end
end
```

- ❶ Записать заголовок CSV.
- ❷ Строка для каждой проданной пиццы.

Приведенная выше функция должна показаться уже знакомой. Новым является то, что функция `println` принимает новый первый аргумент с именем `io`. Он несет в себе неприятные неожиданности, поэтому давайте сначала применим эту функцию неправильно:

```
julia> store_pizzatable("-->", pizzas[1:3])
-->name,size,price
-->hawaiian,S,10.5
-->mexicana,S,13.0
-->hawaiian,L,16.5
```

Результат предсказуем. Она просто записывает объект `io` в виде обычного строкового литерала. Но если первый аргумент имеет особый тип – не строковый литерал, не число и не логическое значение, а объект `IO`, то мы меняем место, куда функции `print` и `println` записывают свой результат.

Вызов `println("hello")` на самом деле является сокращенным вызовом `println(stdout, "hello")`. Что такое `stdout`? Это сокращение от `standard out`, то есть *стандартный вывод*, обозначающий адрес, по

которому производится печать. `stdout` представляет используемое по умолчанию место для всего, что печатается. По умолчанию это окно терминала; однако адресом может быть файл или даже сетевое соединение. Вместо строкового литерала `"-->"` можно попробовать применить `stdout`, хотя результат будет довольно скучным:

```
julia> store_pizatable(stdout, pizzas[1:3])
name,size,price
hawaiian,S,10.5
mexicana,S,13.0
hawaiian,L,16.5
```

Интереснее становится, когда в качестве адреса указывается файл. Для этого нужно создать объект `IO`, который представляет файл.

```
julia> io = open("pizza-sales.csv", "w") ❶
IOStream(<file pizza-sales.csv>)
julia> store_pizatable(io, pizzas)        ❷
julia> close(io)                          ❸
```

- ❶ Открыть файл `pizza-sales.csv`. Вторым аргументом, `w`, сообщает Julia, что его нужно открыть для записи. Следовательно, файл будет создан, если он еще не существует.
- ❷ Использовать объект `io` для записи данных о продаже пицц в открытый файл.
- ❸ По окончании соединения с файлом должно быть закрыто. Чтение и запись в файл буферизуются. Следовательно, если не закрыть соединение, то не все данные обязательно будут записаны.

Написав точку с запятой в начале строки, можно перейти в режим оболочки. Всякий раз, когда вы хотите вернуться в режим Julia, нужно, находясь в начале строки, нажать клавишу ← (**Backspace**). Перейдите в режим оболочки и посмотрите на созданный файл с помощью команды Unix `cat`:

```
shell> cat pizza-sales.csv
name,size,price
hawaiian,S,10.5
mexicana,S,13.0
hawaiian,L,16.5
bbq chicken,L,20.75
sicilian,S,12.25
bbq chicken,M,16.75
mexicana,M,16.0
thai chicken,L,20.75
```

5.3.2 Чтение продаж пицц из файла

Обычно CSV-файл можно скачивать из интернета. В виде CSV-файла можно скачивать статистические данные о чем угодно, от школьных результатов до уровня безработицы и ВВП на душу населения.

Файл `pizza-sales.csv` можно открыть и попытаться его прочитать. Для этого существует ряд умных способов, которые будут рассмотрены подробно в главе 17. В данном же примере все просто: мы будем использовать функцию `getline`; она читает по одной строке за раз:

```
julia> io = open("pizza-sales.csv")
IOStream(<file pizza-sales.csv>)

julia> line = readline(io)
"name,size,price"

julia> line = readline(io)
"hawaiian,S,10.5"
```

Источник, из которого он читается, не обязательно должен быть файлом. Как мы обсуждали ранее, окно терминала трактуется как объект `IO`, именуемый стандартным выводом `stdout`. Существует соответствующий объект `IO`, представляющий клавиатуру, который называется `stdin`. Он дает возможность читать данные, вводимые с клавиатуры:

```
julia> s = readline(stdin)
hello
"hello"

julia> print(s)
hello
```

- ❶ Написанный текст был захвачен функцией `readline`.
- ❷ Значение, хранящееся в `s`. Обратите внимание на использование кавычек, указывающее на то, что это значение является строковым литералом.

В целях понимания механизма работы лучше всего попробовать другой пример. Чуть позже мы создадим простое приложение, в котором все это будет использовано.

Впрочем, давайте вернемся к пицце. Как превратить строковый литерал отделенных запятыми значений в кортеж или массив `pizza`? Для этого служит функция `split`. Она позволяет разбивать строковый литерал на несколько частей и собирать части в массив:

```
julia> pizza = split(line, ',')
3-element Vector{SubString{String}}:
"hawaiian"
"S"
"10.5"
```

Однако есть ряд проблем с трактовкой результата как пиццы. Допустим, мы хотим добавить налог на добавленную стоимость в размере 25 %. Приведенный ниже фрагмент не работает:

```
julia> p = price(pizza)
"10.5"
```

```
julia> p*1.25
ERROR: MethodError: no method matching *(::SubString{String},
::Float64)
```

Причина проблемы в том, что цена p – на самом деле не число, а строковый литерал (точнее, подстрока, но не будем вдаваться в подробности):

```
julia> typeof(p)
SubString{String}
```

Это относится ко всему, что читается из файла; Julia будет трактовать читаемое как текст. Julia не знает, какая часть файла должна быть представлена в виде чисел, булевых значений или чего-то еще. Однако в Julia есть функция синтаксического разбора `parse`, которая позволяет конвертировать строковый литерал во что-то еще. В следующем ниже фрагменте показана конвертация в число:

```
julia> parse{Int, "42"}
42

julia> parse{Float64, "42"}
42.0

julia> parse{Bool, "true"}
true

julia> parse{Bool, "1"}
true
```

Здесь синтаксический разбор числа 42 выполняется дважды. В первый раз оно преобразовывается в целое число, а во второй – в число с плавающей точкой. Таким образом, один и тот же текст можно интерпретировать по-разному. Интерпретация зависит от вашего выбора.

Существуют также совершенно разные строковые литералы, которые можно интерпретировать как один и тот же объект. Например, в результате синтаксического разбора `"true"` и `"1"` могут быть конвертированы в булево значение `"true"`. С помощью этих строительных блоков можно собрать функцию загрузки пицц.

Листинг 5.8 Загрузка данных о продаже пицц

```
function load_pizzatable(io)
    pizzas = []
    readline(io)
    while !eof(io)
        pz = split(readline(io), ',')
        pr = parse{Float64, price(pz)}
        sz = portion(pz)
        push!(pizzas, (name(pz), sz[1], pr))
    end
```

```
pizzas
end
```

- ❶ Место хранения списка пицц.
- ❷ Пропустить заголовок с названием, размером, ценой.
- ❸ eof – это сокращение от англ. end of file, то есть конец файла. Эта функция станет истинной при достижении конца файла, то есть когда больше нечего будет читать.
- ❹ Превратить строковый литерал цены в фактическое число с помощью функции синтаксического анализа parse.
- ❺ Добавить запись в список пицц. Обратите внимание на sz[1]; таким образом строковый литерал, например, "М" превращается в буквенный символ 'М'.

Если вы попробуете эту функцию в интерактивной среде REPL, то должны получить примерно вот такой результат:

```
julia> io = open("pizza-sales.csv");

julia> pizzas = load_pizzatable(io)
8-element Vector{Any}:
 ("hawaiian", 'S', 10.5)
 ("mexicana", 'S', 13.0)
 ("hawaiian", 'L', 16.5)
 ("bbq chicken", 'L', 20.75)
 ("sicilian", 'S', 12.25)
 ("bbq chicken", 'M', 16.75)
 ("mexicana", 'M', 16.0)
 ("thai chicken", 'L', 20.75)

julia> close(io)
```

5.4 Взаимодействие с пользователем

Давайте создадим интерактивное приложение, демонстрирующее возможность чтения данных, вводимых пользователем через стандартный ввод. На создание этого примера меня вдохновило простое приложение, которое я сделал, чтобы помочь своим детям учить таблицу умножения (рис. 5.5).



```
julia> practice(4)
5 * 7 = 42
Wrong, it is 35
9 * 4 = 36
2 * 9 = 18
7 * 5 = 40
Wrong, it is 35
Correct: 2 of 4

julia> 
```

Рис. 5.5 Работа приложения проверки знания таблицы умножения

Приложение снова и снова просит пользователя умножить два числа и проверяет ответ. В конце мы получим сводку о числе полученных правильных ответов. Давайте посмотрим на реализацию функции `practice`.

Листинг 5.9 Упражнение на умножение: предлагается написать `n` ответов

```
function practice(n)
    correct = 0
    for i in 1:n
        x = rand(2:9)
        y = rand(2:9)
        print(x, " * ", y, " = ")
        answer = readline(stdin)
        z = parse{Int, answer}
        if z == x*y
            correct += 1
        else
            printstyled("Wrong, it is ", x*y, color = :red)
            println()
        end
    end
    println("Correct: ", correct, " of ", n)
end
```

- ❶ Отслеживать число предоставленных пользователем ответов.
- ❷ Случайные числа в диапазоне от 2 до 9.
- ❸ Конвертировать написанное пользователем число в целое число.
- ❹ Проверить правильность ответа пользователя.

Эта программа запускается путем вызова функции `practice` из интерактивной среды REPL. Скажем, вы хотите попрактиковаться в восьми разных умножениях. Вы бы написали следующее:

```
julia> practice(8)
```

Всякий раз, когда вы сталкиваетесь с более сложными функциями, которые пытаетесь понять, принцип их работы можно обследовать, просто копируя и вставляя строки исходного кода, как показано ниже:

```
julia> x = rand(2:9)
3

julia> y = rand(2:9)
6

julia> print(x, " * ", y, " = ")
3 * 6 =

julia> answer = readline(stdin)
```

```
18
"18"
```

Преимущество такого подхода состоит в том, что он позволяет видеть значение каждого выражения, в результате чего вы можете наблюдать пошаговые преобразования значения. Например, ниже можно выяснить причину, по которой не работает прямое сравнение ответа:

```
julia> z = parse{Int, answer}
18

julia> answer == x*y
false

julia> z == x*y
true
```

С целью закрепления своего понимания можно поэкспериментировать с улучшением этой программы. Вот несколько идей с засеканием времени. Записывайте количество времени, которое тратится на ответы. Для этой цели можно использовать функцию `time()`. Записывайте время до того, как задавать вопросы, и после того, как будет дан ответ. Посмотрите на разницу. Возможно, вы захотите округлить до ближайшей секунды с помощью функции `round()`. Используйте справочную систему Julia, которая поможет применять эти функции наилучшим образом.

Кроме того, в качестве аргумента функции `practice` можно указать диапазон чисел, используемый вместе с функцией `rand`. Он может относиться к диапазону, на котором требуется попрактиковаться побольше. Также было бы интересно поэкспериментировать в создании красивых таблиц с использованием символов `–`, `†`, `‡` и `‡`.

Резюме

- Функции `print`, `println` и `printstyled` используются для отправки текста по адресу места назначения, такому как окно терминала или файл.
- В терминале можно использовать специальные последовательности управляющих символов для написания цветного текста. Функция `printstyled` упрощает эту задачу, вследствие чего требуется знать только названия разных значений цвета.
- Для того чтобы писать в файлы или читать из них, файлы необходимо открывать. По завершении работы их необходимо закрывать.
- При чтении файла достижение конца файла проверяется с помощью функции `eof`.

- Функции `lpad` и `grad` помогают выравнивать текст в столбцах. Это делается путем дополнения выбранным символом слева либо справа до тех пор, пока не будет достигнута желаемая ширина строки.
- Текст можно писать в объекты или читать из объектов ввода/вывода IO. Объекты IO – это местозаполнители для реальных физических объектов, таких как файлы на жестком диске, клавиатура, сетевое соединение или окно терминала – или даже телетайп.
- Строковые литералы можно конвертировать в различные объекты с помощью функции синтаксического разбора `parse`.

Хранение данных в словарях

Эта глава охватывает следующие ниже темы:

- хранение значений ключей в словарях;
- работа с парными объектами;
- использование кортежей для создания словарей;
- сравнение словарей и массивов;
- сравнение именованных кортежей и словарей.

В этой главе представлен новый тип данных, именуемый *словарем*. В некоторых других языках этот тип данных также называется *ассоциативным массивом*¹. В словарях значения ищутся по ключам, а не исключительно с использованием целочисленных индексов, как в массиве. Приведенный ниже пример исходного кода иллюстрирует разницу. Каждая строка выполняет следующие ниже операции:

- 1 получить 42-е значение *x* массива *xs*. Значения в массивах упорядочены. Однако *xs* мог быть и словарем, поскольку словарные ключи могут быть чем угодно, включая целые числа;
- 2 получить значение *y* словаря *ys* по ключу "foo";
- 3 использование буквенного символа 'D' вместо строкового литерала в качестве ключа в словаре *zs* для получения значения *z*.

¹ Англ. map; син. массив отображений ключ-значение. – Прим. перев.

```
x = xs[42]
y = ys["foo"]
z = zs['d']
```

Вы обнаружите полезность словарей, разобравшись в примере исходного кода, который конвертирует римские цифры в десятичные значения и обратно. Словарь будет использоваться для отслеживания соответствий римских букв, таких как I, V или X, значениям в десятичной системе.

6.1 Синтаксический разбор римских цифр

Хотя римские цифры сегодня не очень практичны в использовании, их полезно знать, чтобы понимать системы счисления. В частности, при программировании вы будете сталкиваться с различными системами счисления.

И римские цифры, и двоичная система – используемая в компьютерах система – возможно, покажутся очень громоздкими в использовании. Однако нередко это выглядит так, потому что числа не используются по назначению.

По сравнению с арабскими цифрами (которые общепринято использовать для записи чисел) производить вычисления с римскими цифрами, используя ручку и бумагу, довольно трудно. Однако римляне не вели расчеты на письме, а делали это с использованием римских счетов (рис. 6.1).

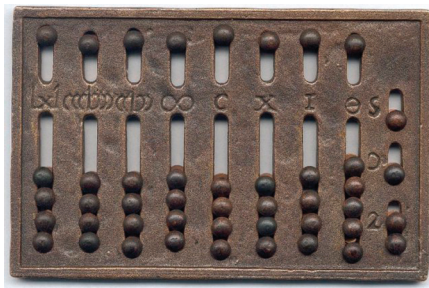


Рис. 6.1 Римские счета с камешками, представляющими разные значения. Столбец задает стоимость каждого камешка

Они поделены на несколько столбцов. Двигаясь справа налево, мы видим столбики камешков, отмеченные как I, X и C; в каждом из них по четыре камешка. Каждый из этих камешков представляет различное значение в зависимости от того, в каком столбце он находится:

- в столбце I каждый камешек соответствует 1;
- в столбце X каждый камешек соответствует 10;
- в столбце C каждый камешек соответствует 100.

Над ними находятся столбцы, которые содержат один камешек каждый. Они называются V, L и D и представляют значения 5, 50 и 500. (На римских счетах нет букв VLD.)

ПРИМЕЧАНИЕ Прелесть римской системы состоит в возможности быстро записывать именно то, о чем говорят камешки на счетах, равно как камешки на римских счетах, которые можно быстро располагать в соответствии с прочитанной римской цифрой. По этой причине римские цифры использовались в Европе вплоть до 1500-х годов, намного позже того, как были введены арабские цифры.

Давайте посмотрим, как эти знания можно было бы использовать для разбора римских цифр и превращения их в арабские. Мы поместим следующий ниже исходный код в текстовый файл и сохраним его. Не беспокойтесь о новом синтаксисе; мы его обязательно рассмотрим.

Листинг 6.1 Синтаксический разбор и конвертация римских чисел в десятичные

```
roman_numerals =
    Dict{'I' => 1, 'X' => 10, 'C' => 100,
        'V' => 5, 'L' => 50, 'D' => 500,
        'M' => 1000)

function parse_roman(s)
    s = reverse(uppercase(s))
    vals = [roman_numerals[ch] for ch in s]
    result = 0
    for (i, val) in enumerate(vals)
        if i > 1 && val < vals[i - 1]
            result -= val
        else
            result += val
        end
    end
    result
end
```

Теперь загрузим этот файл в интерактивную среду REPL, чтобы его протестировать. Ниже приводится пример применения функции `parse_roman` с разными римскими цифрами на входе:

```
julia> parse_roman("II")
2

julia> parse_roman("IV")
4

julia> parse_roman("VI")
6

julia> parse_roman("IX")
9
```

```
julia> parse_roman("XI")  
11
```

Давайте рассмотрим принцип работы исходного кода.

6.2 Использование *tuna Dict*

Латинские буквы I, V, X и т. д. отображаются, или транслируются, в числа, используя *словарь*. Словарь состоит из нескольких пар; пары создаются с помощью оператора стрелки `=>`. При этом нельзя использовать оператор равенства `=`, потому что он применяется для присваивания. Выражение `x = y` присваивает значение `y` переменной `x`, а `x => y` создает пару `x` и `y` (ключ-значение):

```
julia> 'X' => 10           ❶  
'X' => 10  
  
julia> pair = 'X' => 10   ❷  
'X' => 10  
  
julia> dump(pair)         ❸  
Pair{Char,Int64}  
  first: Char 'X'  
  second: Int64 10  
  
julia> pair.first         ❹  
'X': ASCII/Unicode U+0058 (category Lu: Letter, uppercase)  
  
julia> pair.second  
10
```

- ❶ Пара из буквы X и числа 10.
- ❷ Пары можно сохранить в переменной и посмотреть позже.
- ❸ Функция `dump` позволяет заглядывать в поля любого значения.
- ❹ Извлечение первого значения в паре.

Пары – это составные объекты с полями `first` и `second`. Указанные поля позволяют обращаться к обоим значениям, заданным при формировании пары. Тем не менее это нужно рассматривать как деталь реализации, и следует обращаться к полям пар посредством функций доступа `first` и `last`. Такое поведение делает пары очень похожими на объекты-диапазоны, описанные в главе 3, и кортежи, рассмотренные в главе 4:

```
julia> range = 2:4  
2:4  
  
julia> pair = 8=>9  
8 => 9  
  
julia> tuple = (3, 'B')
```

```
(3, 'B')

julia> first(range), first(pair), first(tuple)
(2, 8, 3)

julia> last(range), last(pair), last(tuple)
(4, 9, 'B')
```

В приведенном выше примере исходного кода мы обратились к нескольким значениям в одной строке, отделив их запятой, в результате чего был сгенерирован кортеж с тремя значениями.

Поначалу может показаться непонятным, почему доступ ко второму полю пары осуществляется посредством функции `last`. Причина в том, что последний элемент существует и для массивов и кортежей. Следовательно, функция `last` лучше обобщает несколько типов коллекций.

ПРИМЕЧАНИЕ Ради любопытства можете применить функцию `dump` на объекте-словаре. В нем есть такие поля, как `slots`, `idxfloor`, `maxprobe` и т. д., которые, скорее всего, не будут иметь большого смысла. Это связано с тем, что функция `dump` раскрывает детали реализации. Как пользователь типа данных, вам вовсе не нужно знать об имеющихся у словаря полях, а только то, какую функцию можно использовать для оперирования на нем.

Для создания словаря надо предоставить список пар. В следующем ниже исходном коде показана процедура создания словаря для соотнесения букв, используемых в римских цифрах, с соответствующим им десятичным значением.

```
julia> roman_numerals =
    Dict{'I' => 1, 'X' => 10, 'C' => 100,
        'V' => 5, 'L' => 50, 'D' => 500,
        'M' => 1000}
Dict{Char,Int64} with 7 entries:
  'M' => 1000
  'D' => 500
  'I' => 1
  'L' => 50
  'V' => 5
  'X' => 10
  'C' => 100
```

При использовании словаря первые значения в каждой паре называются *ключами* словаря. Вторые значения в каждой паре образуют значения словаря. Так, I, X и C – это ключи, а 1, 10 и 100 – значения.

Словарь можно опрашивать по ключу и получать соответствующее значение. Здесь он принимает латинскую букву и возвращает соответствующее значение:

```
julia> roman_numerals['C']  
100  
  
julia> roman_numerals['M']  
1000
```

6.3 Прокручивание буквенных символов в цикле

Этот словарь можно использовать для конвертации латинских букв в соответствующие значения. В строке 8 функции `parse_roman` эта конвертация выполняется с помощью *включения в массив*¹. Каждый буквенный символ `ch` в строковом литерале `s` прокручивается в цикле. На каждой итерации вычисляется значение `roman_numerals[ch]`, и все эти значения собираются в массив:

```
vals = [roman_numerals[ch] for ch in s]
```

Выражение включения похоже на цикл `for`, в котором на каждой итерации значение вычисляется и добавляется в коллекцию. Включение можно создавать для любой коллекции, в том числе для словарей:

```
julia> Dict{'A'+i=>i for i in 1:4}  
Dict{Char, Int64} with 4 entries:  
  'C' => 2  
  'D' => 3  
  'E' => 4  
  'B' => 1
```

Но в исходном коде с римскими цифрами цикл `for` используется для формирования массива. В целях более глубокого понимания принципа работы включения в массив давайте рассмотрим обычный цикл `for`, выполняющий то же самое. В этом примере работа начинается с римских цифр "XIV", которые требуется конвертировать:

```
julia> s = "XIV"  
"XIV"  
  
julia> vals = Int8[]  
Int8[]  
  
julia> for ch in s  
    push!(vals, roman_numerals[ch])  
end  
  
julia> vals
```

¹ Англ. array comprehension. – Прим. перев.

```
3-element Vector{Int8}:
 10
  1
  5
```

Строковый литерал "XIV" превратился в массив значений [10, 1, 5] под названием `vals`. Однако работа еще не совсем закончена. Позже нужно объединить эти значения в одно число.

Перед конвертированием входных строковых литералов исходный код переводит каждую букву в верхний регистр. Строковый литерал "xiv" будет обработан неправильно, потому что все ключи словаря находятся в верхнем регистре.

Я проведу вас по механике процесса и оставлю объяснение причины выполнения этих шагов напоследок. Давайте перевернем порядок букв, чтобы можно было удобно обрабатывать цифры в цикле справа налево:

```
julia> s = "xiv"
"xiv"

julia> s = reverse(uppercase(s))
"vix"
```

6.4 Перечисление значений и индексов

При обработке значения `val` в цикле важно иметь возможность сравнивать его с предыдущим значением. Этого можно было бы добиться с помощью переменной, скажем `prev`, хранящей значение из предыдущей итерации. Вместо этого мы будем использовать функцию `enumerate`, чтобы получать индекс `i` каждого обрабатываемого значения переменной `val`. Тогда предшествующее значение переменной `val` будет просто выражаться как `vals[i-1]`:

```
for (i, val) in enumerate(vals)
    if i > 1 && val < vals[i - 1]
        result -= val
    else
        result += val
    end
end
```

Для того чтобы получше разобраться в работе функции `enumerate`, давайте воспользуемся несколькими примерами, посвященными исключительно указанной функции перечисления элементов:

```
julia> enumerate([4, 6, 8])
enumerate([4, 6, 8])
```

Приведенный выше результат не очень-то полезен. Причина в том, что функция `enumerate` является *ленивой*. Мы не видим ника-

ких значений, потому что этому выражению на самом деле не требуется каких-либо вычисляемых значений. Но зато можно применить функцию `collect`, чтобы собрать все значения, которые произвела бы функция `enumerate`, в массив. Ниже приведен простой пример сбора значений из диапазона:

```
julia> collect(2:3:11)
4-element Vector{Int64}:
 2
 5
 8
11
```

Здесь интереснее то, как собираются значения из функции `enumerate`:

```
julia> collect(enumerate(2:3:11))
4-element Vector{Tuple{Int64, Int64}}:
(1, 2)
(2, 5)
(3, 8)
(4, 11)

julia> collect(enumerate([4, 6, 8]))
3-element Vector{Tuple{Int64, Int64}}:
(1, 4)
(2, 6)
(3, 8)
```

Функция `collect` симулирует прокручивание элементов в цикле, как в цикле `for`, за исключением того, что она *собирает* все встречающиеся значения в массив, который она возвращает. И поэтому вы видите, что с функцией `enumerate` на каждой итерации получается пара значений: целочисленный индекс и значение по этому индексу.

6.5 Объяснение процесса конвертации

Неправильно просто складывать отдельные латинские буквы, конвертированные в соответствующие им значения. Возьмем римское число XVI. Оно превращается в [10, 5, 1]. Можно было бы сложить элементы и получить правильный результат: 16. Однако предполагается, что XIV означает 14, потому что, когда меньшая римская цифра стоит перед большей, например в IV, меньшее значение вычитается из большего.

Нельзя просто просуммировать соответствующий массив [10, 1, 5]. Его надо перевернуть и двигаться по значениям в обратном направлении. В каждом индексе текущее значение сравнивается с предыдущим. Если оно меньше, то значение из результата вычитается; в противном случае оно к результату добавляется, как показано ниже:

```

if i > 1 && val < vals[i - 1]
    result -= val
else
    result += val
end

```

Это именно то, что делает выражение `val < vals[i - 1]`. Оно сравнивает текущее значение `val` с предыдущим значением `vals[i - 1]`. Результат используется для накопления значений всех отдельных латинских букв.

6.6 *Использование словарей*

Теперь, когда мы рассмотрели практический пример исходного кода с использованием типа-словаря `Dict`, давайте рассмотрим еще несколько способов взаимодействия со словарем.

6.6.1 *Создание словарей*

Словари можно создавать несколькими способами. В этом разделе будет рассмотрено несколько примеров, начиная с нескольких аргументов, где каждый аргумент – это парный объект:

```

julia> Dict{"two" => 2, "four" => 4}
Dict{String,Int64} with 2 entries:
  "two"  => 2
  "four" => 4

```

Передать массив пар в конструктор словаря (функцию с таким же именем, что и тип, экземпляры которого она создает):

```

julia> pairs = ["two" => 2, "four" => 4]
2-element Vector{Pair{String, Int64}}:
  "two" => 2
  "four" => 4

```

```

julia> Dict(pairs)
Dict{String,Int64} with 2 entries:
  "two"  => 2
  "four" => 4

```

Передать массив кортежей в конструктор словаря. В отличие от пар, кортежи могут содержать более двух значений. Однако в случае словарей они должны содержать только ключ и значение:

```

julia> tuples = [("two", 2), ("four", 4)]
2-element Vector{Tuple{String, Int64}}:
  ("two", 2)
  ("four", 4)

```

```
julia> Dict{tuples}
Dict{String,Int64} with 2 entries:
  "two" => 2
  "four" => 4
```

Как узнать, какой вариант использовать? Применяемый вариант зависит от решаемой задачи. Например, в главе 5, когда мы читали данные о пиццах, мы получили массив кортежей:

```
pizzas = [
  ("mexicana", 13.0),
  ("hawaiian", 16.5),
  ("bbq chicken", 20.75),
  ("sicilian", 12.25),
]
```

Возможно, эти данные понадобится поместить в словарь, чтобы быстро сверяться с ценой на конкретную пиццу:

```
julia> pizza_dict = Dict{pizzas}
Dict{String, Float64} with 4 entries:
  "sicilian" => 12.25
  "bbq chicken" => 20.75
  "mexicana" => 13.0
  "hawaiian" => 16.5

julia> pizza_dict["mexicana"]
13.0
```

И если хранение данных о пиццах в упорядоченном виде не имеет особой важности, то этот словарь можно определить напрямую:

```
Dict{
  "sicilian" => 12.25,
  "bbq chicken" => 20.75,
  "mexicana" => 13.0,
  "hawaiian" => 16.5)
```

Иногда требуется пустой словарь, который потом заполняется. Одним из примеров может быть загрузка из файла прямо в словарь. Вместо того чтобы добавлять значения в конец массива, их можно вставлять в словарь:

```
julia> d = Dict{Any, Any}()
Dict{Any, Any}()
```

Обратите внимание на выражение `{Any, Any}`. Оно описывает типы ключа и значения словаря, которые были вычислены автоматически. Однако при создании словаря для пицц вы, возможно, заметили, что ему задан тип `Dict{String, Float64}`. Тип `String` относится к типу ключей словаря, а `Float64` – к типу значений. Однако типы ключей и значений можно указывать и для пустого словаря:


```
julia> d = Dict{String, Float64}()
Dict{String,Int64} with 0 entries

julia> d["hawaiian"] = 16.5
16.5
```

Преимущество указания типа ключа и значения состоит в том, что становится легче выявлять неправильное использование словаря во время выполнения программы. Если попытаться использовать неправильные типы ключа и значения, то Julia выдаст исключение, указывающее на ошибку (в главе 6 различные типы рассматриваются подробнее). В данном случае делается попытка использовать в качестве ключа целое число 5, когда ожидается ключ в виде строкового литерала:

```
julia> d[5] = "five"
ERROR: MethodError: Cannot `convert` an object of type Int64
to an object of type String
```

Иногда удобно получать ключи и значения в отдельных массивах. Однако с помощью функции `zip` их можно объединять в пары, создавая словари.

```
julia> words = ["one", "two"]
2-element Vector{String}:
"one"
"two"

julia> nums = [1, 2]
2-element Vector{Int64}:
 1
 2

julia> collect(zip(words, nums))
2-element Vector{Tuple{String,Int64}}:
("one", 1)
("two", 2)

julia> Dict(zip(words, nums))
Dict{String,Int64} with 2 entries:
"two" => 2
"one" => 1
```

6.6.2 *Доступ к элементам*

Вы уже рассмотрели один из способов получения и установки значений элементов словаря. Но что произойдет, если попытаться получить значение несуществующего ключа, например "seven"?

```
julia> d["hawaiian"]
16.5
```

```
julia> d["seven"]  
ERROR: KeyError: key "seven" not found
```

Будет получена ошибка. Можно, конечно, просто добавить:

```
julia> d["seven"] = 7;  
  
julia> d["seven"]  
7.0
```

Но как избежать ошибки, если мы не уверены в существовании ключа? Одним из решений является функция `get()`. Если ключ не существует, то возвращается сигнальное значение. Сигнальным значением может быть что угодно.

ПРИМЕЧАНИЕ В компьютерном программировании сигнальное значение (также именуемое *флаговым*, *контрольным*, *спусковым*, *дозорным* значением, или *фиктивными данными*) является специальным значением в контексте алгоритма, который использует его присутствие в качестве условия завершения, обычно в цикле или рекурсивном алгоритме.

Эта стратегия применяется во многих языках программирования при работе со словарями. В следующем ниже примере в качестве сигнального значения используется `-1`:

```
julia> get(d, "eight", -1)  
-1
```

Как вариант можно просто опросить словарь на предмет наличия у него ключа:

```
julia> haskey(d, "eight")  
false  
  
julia> d["eight"] = 8  
8  
  
julia> haskey(d, "eight")  
true
```

6.7 Зачем использовать словарь?

Для конвертации римских цифр в десятичные числа, в принципе, можно использовать массив. Вот пример того, как это можно было бы сделать.

Листинг 6.2 Отыскание значения по ключу в массиве пар ключ-значение

```
function lookup(key, table)
    for (k, v) in table           ❶
        if key == k
            return v             ❷
        end
    end
    throw(KeyError(key))        ❸
end
```

- ❶ Брать ключ *k* и значение *v* каждой пары в массиве.
- ❷ Найден совпадающий ключ, поэтому вернуть соответствующее значение.
- ❸ Если в процессе прокручивания всех пар в цикле совпадающий ключ не найден, то возвращаемое значение отсутствует, и вместо него необходимо выдать исключение. В Julia в случаях отсутствия ключей принято использовать исключение `KeyError`.

Вместо словаря можно определить сверочную таблицу в виде массива пар:

```
numerals = ['I' => 1, 'X' => 10, 'C' => 100,
            'V' => 5, 'L' => 50, 'D' => 500,
            'M' => 1000]
```

При этом аналогично словарю можно просматривать значения, основываясь на ключах.

```
julia> lookup('X', roman_numerals)
10
```

```
julia> lookup('D', roman_numerals)
500
```

```
julia> lookup('S', roman_numerals) ❶
ERROR: KeyError: key 'S' not found
```

- ❶ Демонстрация отыскания несуществующего ключа с созданием исключения.

При выполнении поиска по ключу следует избегать массивов, потому что время поиска растет линейно вместе с размером массива. Поиск элемента среди 30 элементов занимает в среднем в три раза больше времени, чем поиск элемента среди 10 элементов. Нетрудно заметить, что такой подход плохо масштабируется с большими массивами. Поиск одного элемента среди 1 миллиона элементов займет в 1000 раз больше времени, чем его поиск среди 1000 элементов.

Словари, напротив, сделаны так, что время поиска не зависит от числа содержащихся в нем элементов. Поиск 1 элемента среди 100 аналогичен поиску среди 1 миллиона.

Почему поиск по словарю такой быстрый?

Вопрос о том, почему словари обеспечивают быстрое отыскание значения по ключу, выходит за рамки этой книги. Книги о структурах данных и алгоритмах обычно подробно освещают эту тему, а словарь больше относится к интерфейсу данных, чем к фактической структуре данных, используемой для быстрого поиска. В Julia для обеспечения быстрого поиска используются хеш-таблицы, но также можно реализовать словарь с использованием структуры данных под названием *двоичное дерево поиска*.

Вместе с тем не следует сбрасывать со счетов массивы. Поиск в коротких массивах выполняется очень быстро – быстрее, чем в словаре сравнимого размера. Таким образом, если число элементов меньше 100, то массивы по-прежнему являются жизнеспособным выбором. На самом деле в примере исходного кода с римскими цифрами словарь использовался по той причине, что со словарями удобно работать при поиске по ключу, и вам никогда не придется беспокоиться о снижении производительности из-за добавления слишком большого числа элементов.

Однако есть особые случаи, в которых использование массива будет работать очень хорошо (например, если массив никогда не видоизменяется). Если элементы никогда не добавляются и не удаляются, то массив можно отсортировать. В отсортированном массиве можно проводить очень быстрый поиск с помощью функции Julia `searchsortedfirst`. На самом деле пример исходного кода с конвертацией римских цифр хорошо вписывается в этот подход, поскольку соответствие между цифрами и десятичными значениями фиксировано. Это можно сделать, храня ключи и значения в отдельных массивах, отсортированных по значениям ключей.

Листинг 6.3 Массив отсортированных ключей с соответствующим массивом значений

```
keys = ['C', 'D', 'I', 'L', 'M', 'V', 'X']  
vals = [100, 500, 1, 50, 1000, 5, 10]
```

С помощью функции `searchsortedfirst` можно отыскать индекс определенного ключа.

```
julia> i = searchsortedfirst(keys, 'I')  
3
```

Можно проверить, что значение ключа I расположено по тому же индексу `i` в массиве `vals`:

```
julia> vals[i]  
1
```

Вот еще один пример:

```
julia> j = searchsortedfirst(keys, 'V')
6

julia> vals[j]
5
```

6.8 *Использование именованных кортежей в качестве словарей*

Прежде чем закончить эту главу, хотел бы показать еще один изящный прием, позволяющий писать более удобочитаемый исходный код с большей производительностью. Вы уже видели кортежи, в которых к элементам можно обращаться по индексу. Чего вы еще не видели, так это доступа к значениям кортежей по ключу, как в словаре.

Напомним, что мы создали кортеж для пиццы следующим образом: `("hawaiian", 'S', 10.5)`. Каждому значению можно дать имя; назначаемые имена будут иметь тип `Symbol` (символический), а не `String` (строковый). В главе 5 вы использовали такие символические имена, как `:cyan`, `:green` и `:red`, чтобы указывать цвет печатаемого текста. К отдельным значениям в кортеже `pizza` можно обращаться аналогичным образом, используя такие символические имена, как `:name` и `:price`:

```
julia> pizza = (name = "hawaiian", size = 'S', price = 10.5)
(name = "hawaiian", size = 'S', price = 10.5)

julia> pizza[:name]
"hawaiian"

julia> pizza[:price]
10.5

julia> pizza.name
"hawaiian"

julia> pizza.size
'S': ASCII/Unicode U+0053
```

Обратите внимание на сокращения в последних двух выражениях; `pizza[:price]` эквивалентно `pizza.price`. Этот способ работы с данными покажется знакомым разработчикам на JavaScript.

Имейте в виду, что функциональность символических имен гораздо более ограничена, чем строковых литералов. В большинстве случаев они рассматриваются как атомарные значения. Обращаться к отдельным буквенным символам в буквенном символе невозможно, равно как и их комбинировать и манипулировать ими как

строковыми литералами. К счастью, конвертация между ключами и строковыми литералами выполняется очень легко в обе стороны:

```
julia> s = "price"; t = :name;
```

```
julia> Symbol(s)      ❶  
:price
```

```
julia> string(t)      ❷  
"name"
```

- ❶ Создать символическое имя из строкового литерала.
- ❷ Создать строковый литерал из символического имени.

Зная это, можно переписать функцию `parse_roman` из листинга 6.1, чтобы вместо словаря использовать именованный кортеж. Обратите внимание, что поисковое выражение `roman_numerals[ch]` следует поменять на `roman_numerals[Symbol(ch)]`, потому что теперь словарь `roman_numerals` в качестве ключей имеет не буквы, а символические имена.

Листинг 6.4 Разбор римских цифр с использованием именованных кортежей

```
roman_numerals =                                     ❶  
    (I = 1, X = 10, C = 100,  
     V = 5, L = 50, D = 500,  
     M = 1000)  
  
function parse_roman(s)  
    s = reverse(uppercase(s))  
    vals = [roman_numerals[Symbol(ch)] for ch in s]  ❷  
    result = 0  
    for (i, val) in enumerate(vals)  
        if i > 1 && val < vals[i - 1]  
            result -= val  
        else  
            result += val  
        end  
    end  
    result  
end
```

- ❶ Словарь заменен на именованный кортеж.
- ❷ Сверка со словарем посредством `Symbol(ch)` вместо `ch`.

6.8.1 Когда применять именованный кортеж?

Именованные кортежи очень похожи на словари. Тогда в чем смысл их наличия? Все типы кортежей немутуируемы, то есть их невозможно изменить. После создания кортежа в него нельзя добавлять новые значения и нельзя изменять существующие значения. Напротив,

и массивы, и словари позволяют добавлять значения. Словари предоставляют более широкий выбор типов, которые можно использовать в качестве ключей. Именованный кортеж позволяет использовать в качестве ключей только символические имена.

Преимущество кортежа любого типа по сравнению с массивом или словарем заключается в том, что JIT-компилятор Julia будет точно знать, какие элементы будут в кортеже в любой момент времени, что позволяет проводить более агрессивную оптимизацию. Таким образом, можно предположить, что кортеж обычно дает равную или более высокую производительность по сравнению с массивом либо словарем.

Хотя применение в качестве ключей только символических имен налагает ограничения, оно также позволяет именованным кортежам предлагать более удобный синтаксис для доступа к значениям. Например, `pizza.name` легче писать и читать, чем `pizza[:name]`.

6.8.2 Связывание всего воедино

В этой главе были рассмотрены все типы ключей, которые должен знать любой программист. С числами, диапазонами, строковыми литералами, массивами, кортежами и словарями можно делать практически все. Однако я еще мало рассказал о том, *что такое* тип на самом деле и как создавать свои собственные конкретно-прикладные типы. Эта функциональность имеет решающее значение для облегчения создания более крупных и многофункциональных приложений. Данной теме будут посвящены следующие две главы.

Резюме

- Словари содержат пары ключ-значение, в которых ключ должен быть уникальным.
- Пары ключ-значение можно быстро отыскивать, добавлять или удалять из словаря. Это отличается от больших массивов, поиск в которых обычно потребует много времени.
- Массивы обеспечивают более высокую производительность, когда число элементов мало либо когда невозможно обращаться к элементам по индексу.
- В Julia ключи и значения типизированы. Следовательно, Julia может обнаруживать использование ключей неправильного типа, а также попытки вставки значений неправильного типа.
- Именованные кортежи работают как немутуируемая версия словарей. В них можно отыскивать значения, но невозможно их изменять или добавлять новые записи.

Часть II

Типы

В части I «Основы» мы коснулись типов лишь поверхностно. Во второй части система типов Julia излагается подробнее путем создания различных примеров, таких как пример ступенчатой ракеты, которые демонстрируют принцип работы системы типов и предлагаемые ею преимущества.

Эта часть знакомит с теми сторонами языка Julia, которые делают его особенным. Более глубокое освещение системы типов в этой части также позволяет рассказать о принципах работы функций и методов в Julia. В частности, это позволяет правильно объяснить множественную диспетчеризацию, которая является уникальной особенностью языка Julia. Множественная диспетчеризация лежит в основе того, что делает язык Julia таким выразительным и высокопроизводительным, несмотря на динамическую типизацию.

Понимание типов

Эта глава охватывает следующие ниже темы:

- понимание иерархии типов;
- различия между абстрактными и конкретными типами;
- комбинирование примитивных типов для создания составных типов;
- использование возможностей множественной диспетчеризации для элегантного решения сложных задач;
- чем множественная диспетчеризация отличается от одиночной диспетчеризации в объектно ориентированных языках¹.

Все объекты в Julia относятся к определенному типу. Напомню, что для определения типа любого объекта можно использовать функцию `typeof`:

```
julia> typeof(42)
Int64
```

```
julia> typeof('A')
```

¹ Большинство современных языков являются объектно ориентированными. Они предназначены для связывания поведения с типами и реиспользования функциональности посредством так называемого *наследования*.

Char

```
julia> typeof("hello")  
String
```

Тип определяет то, что можно делать с объектом. Например, словарь позволяет искать значение по ключу, а массив хранит элементы по порядку. Выражение, возвращающее значение типа `Bool`, такое как `true` или `false`, можно использовать в инструкциях `if` и циклах `while`, тогда как выражения, вычисляющие значение с плавающей точкой, использовать нельзя:

```
julia> if 2.5  
    print("this should not be possible")  
end  
ERROR: TypeError: non-boolean (Float64) used in boolean context
```

Таким образом, если требуется создавать объекты с другим поведением и признаками, то необходимо определять новые типы. В программировании мы часто пытаемся имитировать реальный мир:

- банковские приложения содержат типы, представляющие банковские счета, клиентов и транзакции;
- видеоигры содержат объекты, изображающие монстров, героев, оружие, космические корабли, ловушки и т. д.;
- графические пользовательские интерфейсы содержат объекты, представляющие кнопки, элементы меню, всплывающие меню и переключатели;
- приложения для рисования содержат объекты, представляющие различные формы, штрихи, цвета и инструменты рисования.

Итак, какой бы тип приложения вы ни хотели создать, при разработке приложения нужно уметь создавать соответствующие типы. В этой и следующей главах будут определены типы, относящиеся к моделированию поведения в видеоигре и ракетном симуляторе.

7.1 Создание составных типов из примитивных

Начнем с основ: целые числа, буквенные символы и числа с плавающей точкой – все это примеры примитивных типов. Их невозможно разбить дальше на более мелкие части. В некоторых языках, таких как LISP, они называются *атомами*. С помощью функции `isprimitivetype` можно проверять тип на его принадлежность к *примитивному* типу:

```
julia> isprimitivetype{Int8}  
true
```

```
julia> isprimitivetype(Char)
true

julia> isprimitivetype(String)
false
```

Примитивные типы можно комбинировать, чтобы создавать составные типы. Составные типы можно даже создавать из других составных типов. Например, строковый литерал представляет собой составной тип, состоящий из нескольких буквенных символов, которые являются примитивными типами. Продемонстрируем это на конкретном примере, определив составной тип, который может быть полезен в видеоигре, чтобы представлять лучника, стреляющего в противников стрелами.

Листинг 7.1 Определение составного типа Archer (Лучник)

```
struct Archer
    name::String ❶
    health::Int   ❷
    arrows::Int    ❸
end
```

- ❶ Имя лучника – допустим, Робин Гуд.
- ❷ Оставшиеся очки здоровья.
- ❸ Оставшиеся стрелы в колчане.

Типы следует трактовать как шаблоны или формочки для печенья, которые используются для штамповки многочисленных объектов. Создаваемые из типа объекты называются *экземплярами*.

ПРЕДУПРЕЖДЕНИЕ Составные типы Julia могут показаться очень похожими на классы в Java, C++ или Python, но это не одно и то же. Они не поддерживают наследование в рамках реализации и не содержат прикрепленных методов.

В следующем ниже фрагменте показано создание экземпляров типа `Archer`. При этом нередко можно услышать такие фразы, как «создать экземпляр объекта `Archer`» или «инстанцировать объект `Archer`».

```
julia> robin = Archer("Robin Hood", 30, 24)
Archer("Robin Hood", 30, 24)

julia> william = Archer("William Tell", 28, 1)
Archer("William Tell", 28, 1)

julia> robin.name ❶
"Robin Hood"
```

```
julia> robin.arrows ②
24
```

- ① Обратиться к полю `name` объекта `robin`.
- ② Обратиться к полю `arrows` объекта `robin`.

Определение составного типа имеет некоторое сходство с использованием словаря. Например, в нем определяются поля для хранения значений, доступ к которым можно получать через имя поля. Однако, в отличие от словаря, для каждого поля можно указывать другой тип, используя *аннотацию типа*.

ВАЖНО В Julia `::` используется для аннотирования переменных и выражений определенным типом. `x::T` означает, что переменная `x` должна иметь тип `T`. Это помогает Julia выяснять количество байтов, необходимых для хранения всех полей в структуре.

В целях прояснения этого момента давайте определим словарь для хранения информации о лучнике.

Листинг 7.2 Использование словаря для хранения информации о лучнике

```
julia> robin = Dict{String, Any}("name" => "Robin Hood",
                                "health" => 30,
                                "arrows" => 24)
Dict{String, Any} with 3 entries: ①
  "name" => "Robin Hood"
  "health" => 30
  "arrows" => 24

julia> robin["name"] ②
"Robin Hood"

julia> robin["arrows"]
24
```

- ① Словарь с ключом типа `String`, значение которого имеет тип `Any`.
- ② Доступ к значению, хранящемуся по ключу `name`.

Проблема с использованием словаря заключается в том, что он требует, чтобы все значения были одного типа. Но подождите, как это может быть? `name` и `arrows` имеют совершенно разные типы?

Короткий ответ таков: значения в словаре имеют тип `Any`. Это означает, что можно хранить значения любого типа. Ключи имеют более строгие ограничения, поскольку они определяются как `String`. Но для того чтобы по-настоящему понять механизм работы типов, нужно обследовать иерархии типов Julia.

7.2 Обследование иерархий типов

Если вы знакомы с объектно ориентированными языками, то должны быть знакомы с иерархиями наследования классов¹. В Julia тоже есть иерархии: это иерархии типов, но существенное отличие состоит в том, что указанные иерархии существуют и для примитивных типов. Например, в таком языке, как Java или C++, целое число или число с плавающей точкой – это всего лишь один конкретный тип. Однако в Julia даже числа, коллекции и строковые литералы являются частью более глубоких иерархий типов (рис. 7.1).

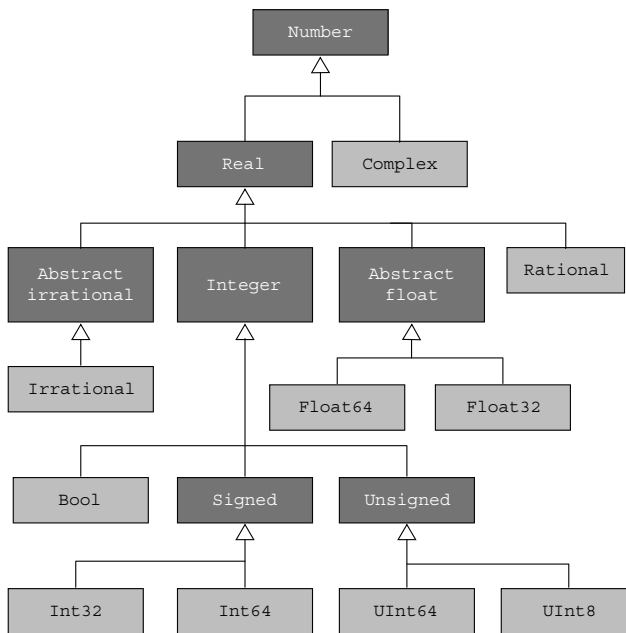


Рис. 7.1 Иерархия типов чисел, показывающая абстрактные и конкретные типы в темных и светлых прямоугольниках

Указанные иерархии можно обследовать с помощью функций `supertype` и `subtypes`. Для чисел на рис. 7.1 можно воссоздать иерархию типов, начав с вершины иерархии и продвигаясь вниз, используя функцию `subtypes` для отыскания подтипов, которые затем можно обследовать дальше:

```
julia> subtypes(Number)
2-element Vector{Any}:
 Complex
```

❶

¹ В объектно ориентированном программировании *класс* – это тип, который может быть частью иерархии типов и имеет ассоциированные с ним функции, именуемые *методами*.

Real

```
julia> subtypes(Real) ②
4-element Vector{Any}:
  AbstractFloat
  AbstractIrrational
  Integer
  Rational
```

```
julia> subtypes(Integer) ③
3-element Vector{Any}:
  Bool
  Signed
  Unsigned
```

- ① Найти непосредственные подтипы типа Number.
- ② Выяснить подтипы действительных чисел.
- ③ Целые числа могут быть со знаком либо без знака.

Но как узнать, что корнем иерархии чисел является тип Number? Можно двигаться вверх от типов чисел, которые уже знакомы:

```
julia> T = typeof(42) ①
Int64
```

```
julia> T = supertype(T) ②
Signed
```

```
julia> T = supertype(T)
Integer
```

```
julia> T = supertype(T)
Real
```

```
julia> T = supertype(T)
Number
```

- ① Сохранить тип числа 42 в переменной T.
- ② Заглянуть в надтип типа Int64 и сохранить его в T.

Можно даже продолжать передавать корень иерархии чисел вверх до тех пор, пока не будет достигнут корень всей иерархии типов Julia. По достижении типа Any станет ясно, что была достигнута вершина иерархии типов, потому что надтипом типа Any тоже является тип Any:

```
julia> T = supertype(T)
Any
```

```
julia> T = supertype(T)
Any
```

Важно понимать, что типы Julia – это первоклассные объекты, которые можно передавать в качестве аргументов или сохранять

в переменных. Например, здесь тип целого числа 42 сохраняется в переменной с именем `T`. Во многих языках `T` принято использовать в качестве имени произвольного типа. Давайте обследуем иерархию типов с помощью нескольких простых функций.

Листинг 7.3 Отыскание корня иерархии типов

```
function findroot(T)
    T2 = supertype(T)
    println(T)
    if T2 != T ❶
        findroot(T2)
    end
end
```

❶ Проверить на совпадение надтипа типа `T` с типом `T`.

Это рекурсивная функция¹, которую можно использовать для отыскания вершины иерархии типов:

```
julia> findroot(typeof(42))
Int64
Signed
Integer
Real
Number
Any
```

```
julia> supertype(Any)
Any
```

Вы видите, что иерархия типов останавливается на типе `Any`, поскольку надтипом типа `Any` является тип `Any`. Итак, каково значение этих типов иерархий? Как они помогают программисту? Следующий ниже пример в интерактивной среде REPL дает практическую подсказку:

```
julia> anything = Any[42, 8] ❶
2-element Vector{Any}:
 42
  8

julia> integers = Integer[42, 8] ❷
2-element Vector{Integer}:
 42
  8

julia> anything[2] = "hello" ❸
```

¹ Рекурсивная функция – это функция, которая вызывает сама себя, без использования цикла.

```
"hello"
```

```
julia> integers[2] = "hello" ④  
ERROR: MethodError: Cannot `convert` an object  
of type String to an object of type Integer
```

- ① Определить массив, который может содержать любое значение (то есть значение типа Any).
- ② Определить массив для хранения целочисленных значений.
- ③ Вставка строки в массив типа Any работает нормально.
- ④ Массив целых чисел не принимает текстовые строки.

Поскольку в Julia каждый объект подчиняется типу Any, в массив, каждый элемент которого, согласно вашему описанию, должен иметь тип Any, можно помещать любой объект. Однако не каждый объект Julia имеет тип Integer. Таким образом, помещение строкового литерала, к примеру такого, как "hello", в массив, в котором каждый элемент должен иметь тип Integer, работать не будет.

Как узнать, какие именно типы совместимы? Значение, которое вы пытаетесь сохранить, должно иметь тип, являющийся подтипом разрешенного типа элемента. Оказывается, это можно проверить программно с помощью оператора <:.

Листинг 7.4 Проверка на то, какие типы являются подтипами друг друга

```
julia> String <: Any  
true  
  
julia> String <: Integer ①  
false  
  
julia> Int8 <: Integer  
true  
  
julia> Float64 <: Integer ②  
false
```

- ① Текстовая строка не является целым числом.
- ② Float64 – это число, но не целое число.

Из этого примера видно, что типы могут и не быть чем-то связаны (например, числа с плавающей точкой и целые числа). Например, 4.5 – это число с плавающей точкой, а не целое число. Однако Int8(4) и Int32(5) – целые числа; они являются подтипами типа Integer.

Это должно навести на мысль о практических преимуществах определения составных типов для хранения родственно связанных данных по сравнению с использованием словаря. Каждое поле может иметь свой тип. За счет этого обеспечивается более точная проверка типов во время работы программы.

7.3 Создание симулятора сражений

В целях дальнейшего обследования этих концепций мы разработаем простой симулятор сражений между разными воинами, как это часто делается в настольных, карточных и видеоиграх.

Многие компьютерные игры работают по принципу всем известной детской игры «камень–ножницы–бумага». Здесь надо внести ясность: в нашей игре есть лучники, рыцари и копейщики, которых мы организуем так, чтобы

- лучники побеждали копейщиков,
- рыцари побеждали лучников, а
- копейщики побеждали рыцарей.

Исторически примерно так эти воины и вели себя в бою. Лучники стреляли стрелами в медлительных копейщиков и побеждали их до того, как те успевали подобраться достаточно близко, чтобы атаковать лучников. Эта стратегия потерпит неудачу с рыцарями, которые могли приближаться к лучникам до того, как тем удавалось выпустить много стрел и сразить их. Однако рыцари не могли использовать эту стратегию против копейщиков, так как стена копий не позволяла рыцарям атаковать копейщиков из опасения быть пронзенными.

Мы реализуем в исходном коде следующее:

- абстрактный тип `Warrior` для всех типов воинов;
- конкретные типы воинов `Archer` (Лучник), `Pikeman` (Копейщик) и `Knight` (Рыцарь);
- объяснение взаимосвязей между конкретными и абстрактными типами;
- поведение каждого типа воина путем определения таких функций, как `shoot!` (выпустить стрелу) и `mount!` (оседлать коня);
- функция `attack!` для симулирования нападения одного воина на другого;
- функция `battle!` для симулирования непрерывного сражения двух воинов между собой до тех пор, пока один из них не победит либо оба не погибнут.

7.3.1 Определение типов воинов

Давайте создадим файл с именем `warrrs.jl` для хранения исходного кода, который мы будем разрабатывать. Начнем с определения типов, которые будем использовать в своем симуляторе.

Листинг 7.5 Определение типов в симуляторе сражений

```
abstract type Warrior end
```

❶

```
mutable struct Archer <: Warrior
```

❷

```

        name::String
        health::Int
        arrows::Int
    end

    mutable struct Pikeman <: Warrior
        name::String
        health::Int
    end

    mutable struct Knight <: Warrior
        name::String
        health::Int
        mounted::Bool
    end

```

- ❶ Определяет абстрактный тип `Warrior`.
- ❷ Лучник определяется как мутируемый подтип типа `Warrior`.
- ❸ Рыцари могут скакать верхом на лошади или идти в пешем строю.

Исходный код в листинге 7.5 создает показанную ниже иерархию типов. В этой иерархии проводится различие между абстрактным и конкретными типами. `Archer`, `Pikeman` и `Knight` – это примеры конкретных типов, а `Warrior` – пример абстрактного типа. При этом объекты конкретного типа можно создавать, а объекты абстрактного типа – нельзя:

```

julia> robin = Archer("Robin Hood", 34, 24)
Archer("Robin Hood", 34, 24)

julia> Warrior()
ERROR: MethodError: no constructors have been defined for Warrior

```

Абстрактные типы предназначены для облегчения конструирования иерархии типов.

На рис. 7.2 в прямоугольник типа `Warrior` добавлены поля `name` и `health` (здоровье). Однако это сделано просто для того, чтобы уточнить, что все подтипы должны иметь эти поля. Julia не предлагает синтаксиса, который обеспечивал бы соблюдение такой организации. Это делается по соглашению.

Если тип Julia определяется как абстрактный, то он не может иметь никаких полей. Только конкретные типы могут иметь поля или значение. *Составной тип* – это конкретный тип с полями, а *примитивный тип* – это конкретный тип с одним значением.

Оператор подтипа `<:` используется не только для проверки того, что тип является подтипом другого, но и для определения типа как подтипа:

```

struct B <: A
    ...
end

```

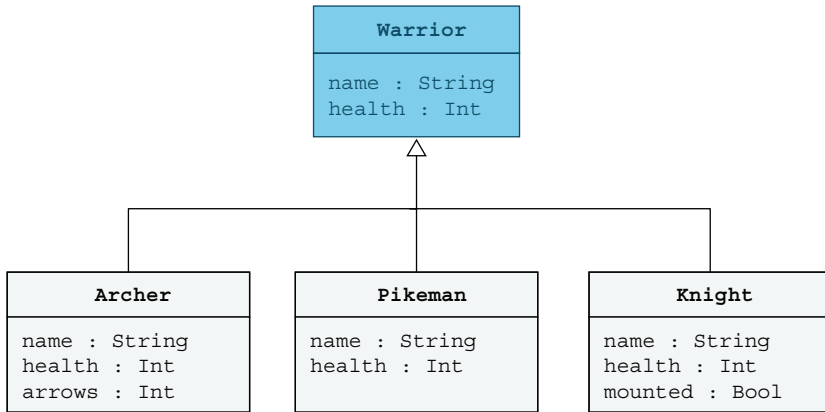


Рис. 7.2 Иерархия типов воинов. Темный прямоугольник является абстрактным типом, а светлые – конкретными типами

Приведенный выше фрагмент исходного кода определяет тип `B` как подтип типа `A`. В Julia нельзя подтипировать конкретные типы. Если вы использовали такие популярные объектно ориентированные языки, как Java, C++, C#, Python или Ruby, то этот подход, скорее всего, станет для вас неожиданностью. Если принять во внимание иерархию чисел, которую мы только что рассмотрели, то это имеет смысл. Например, сколько места требуется для типа `Int32` или `Float64`, известно заранее, но сколько байтов памяти потребуется для хранения целого или вещественного числа? Это невозможно знать заранее. Вот почему большинство числовых типов являются абстрактными.

7.3.2 Добавление воинам поведения

Воины, содержащие только данные, не интересны. И поэтому мы добавим к ним поведение, определив несколько функций с сопутствующими методами. Мы их добавим в файл исходного кода `warrior.jl` (листинг 7.6).

Все эти функции имеют в своем имени восклицательный знак, потому что они видоизменяют поле (напомню, что это всего лишь соглашение). Вот почему к определениям составных типов добавлено ключевое слово `mutable`. Если тип `struct` не определен как *мутабельный*, то он не будет поддерживать функции, которые видоизменяют поле. Без ключевого слова `mutable` составной тип по умолчанию будет *немутабельным*.

Листинг 7.6 Добавление поведения в типы воинов

```

function shoot!(archer::Archer)
    if archer.arrows > 0
        archer.arrows -= 1
    end
end
  
```

```
function resupply!(archer::Archer)
    archer.arrows = 24
end

function mount!(knight::Knight)
    knight.mounted = true
end

function dismount!(k::Knight)
    knight.mounted = false
end
```

Вот краткое описание того, что делает каждая функция:

- `shoot!` – лучник выпускает стрелу. Уменьшить число стрел на одну;
- `resupply!` – симулировать пополнение лучником запаса из 24 стрел;
- `mount!` – изменить состояние рыцаря, посадив его верхом на коня;
- `dismount!` – спешить рыцаря, чтобы приготовить его к пешему бою.

Мутируемые и немутуруемые типы

Следующее правило является важным открытием, сформулированным в сообществе функционального программирования: если объекты невозможно видоизменять, то вероятность наличия в программе ошибок будет меньше. Объекты, которые нельзя видоизменять, называются *немутуруемыми*. Если их можно видоизменять, то они называются *мутуруемыми*.

В старых языках объекты по умолчанию были мутуруемыми. Julia следует современному тренду: делать объекты *немутуруемыми*, если только они не помечены как *мутуруемые* в явной форме.

С помощью функции `shoot!` можно симулировать расходование лучником стрел в бою. Обычно у средневекового лучника в колчане было 24 стрелы. Когда они будут израсходованы, лучнику нужно будет пополнить запас стрел:

```
julia> robin = Archer("Robin Hood", 34, 24) ❶
Archer("Robin Hood", 34, 24)

julia> shoot!(robin)
23 ❷

julia> shoot!(robin)
22 ❸
```

- ❶ Лучник создан с 24 стрелами.
- ❷ Число оставшихся стрел.
- ❸ Осталось двадцать две стрелы.

Функцию `shoot!` можно улучшить приемом, который я часто использую при разработке программного обеспечения на Julia: я возвращаю объект, отображение которого на экране наиболее полезно при выполнении функции в интерактивной среде REPL (см. листинг 7.7). При вызове видоизменяющих объект функций имеет смысл показывать то, как этот объект выглядит после видоизменения. Так что неплохо взять за правило возвращать объект, видоизмененный в мутирующих функциях.

Листинг 7.7 Модификация мутирующих функций, чтобы сделать их дружелюбными для REPL

```
function shoot!(archer::Archer)
    if archer.arrows > 0
        archer.arrows -= 1
    end
    archer
end

function resupply!(archer::Archer)
    archer.arrows = 24
    archer
end
```

❶ Возвращает видоизмененный объект-лучник.

Это значительно упрощает тестирование разрабатываемых функций и проверку правильности выполнения ими операций:

```
julia> robin = Archer("Robin Hood", 34, 24)
Archer("Robin Hood", 34, 24)

julia> shoot!(robin)
Archer("Robin Hood", 34, 23) ❶

julia> shoot!(robin)
Archer("Robin Hood", 34, 22) ❶

julia> shoot!(robin)
Archer("Robin Hood", 34, 21) ❶

julia> resupply!(robin)
Archer("Robin Hood", 34, 24) ❷
```

❶ Показать уменьшающееся число стрелок.

❷ Число стрел увеличено до 24.

Эти функции можно использовать для конструирования новых функций, чтобы симулировать нападение воина на другого. Мы снова добавим приведенный ниже исходный код в файл `warrs.jl`. Похоже, функция `attack!` была определена дважды. Как это возможно?

Листинг 7.8 Два метода симулирования боя между лучниками и рыцарями

```

function attack!(a::Archer, b::Archer)           ❶
    if a.arrows > 0                               ❷
        shoot!(a)
        damage = 6 + rand(1:6)                   ❸
        b.health = max(b.health - damage, 0)      ❹
    end
    a.health, b.health
end

function attack!(a::Archer, b::Knight)           ❺
    if a.arrows > 0                               ❷
        shoot!(a)
        damage = rand(1:6)                       ❻
        if b.mounted
            damage += 3
        end
        b.health = max(b.health - damage, 0)      ❻
    end
    a.health, b.health
end
end

```

- ❶ Симулировать лучника, атакующего другого лучника.
- ❷ Не может атаковать, если стрелы закончились.
- ❸ Бросить шестигранный кубик (d6), чтобы вычислить урон от стрелы.
- ❹ Применить max во избежание отрицательного здоровья.
- ❺ Симулировать лучника, атакующего рыцаря.
- ❻ Бросить шестигранный кубик (d6), чтобы вычислить урон от стрелы.

Если бы это был обычный динамический язык, такой как JavaScript, Python, Ruby или Lua, то последнее определение функции `attack!` переписало бы первое. Если бы это был статически типизированный язык, такой как Java, C# или C++, то была бы создана *перезгрузка функции*¹. Но в Julia происходит совсем другое.

7.3.3 Использование множественной диспетчеризации для вызова методов

В Julia на самом деле были определены не две функции, а два *метода*, прикрепленных к функции `attack!`. Знаю, что это звучит запутанно, поэтому давайте остановимся на этой теме подробнее. В Julia функции фактически определяются, как показано в следующем ниже листинге.

¹ *Перезгрузка функций* является особенностью многих статически типизированных языков. Она позволяет определять одну и ту же функцию несколько раз с аргументами разных типов. При компиляции исходного кода компилятор выбирает нужную функцию.

Листинг 7.9 Определения функций в Julia без методов

```
function shoot! end
function resupply! end
function attack! end
```

Функции – это просто имена. Если к ним не прикрепить методы, то они ничего не смогут делать. Давайте запустим новую версию интерактивной среды REPL, вставим следующие ниже определения функций вместе с определениями типов `Warrior`, `Archer` и `Knight` (см. листинг 7.5) и создадим несколько объектов для работы:

```
julia> robin = Archer("Robin Hood", 34, 24)
Archer("Robin Hood", 34, 24)

julia> white = Knight("Lancelot", 34, true)
Knight("Lancelot", 34, true)
```

Теперь с этими объектами можно попробовать что-то сделать и посмотреть, что получится:

```
julia> attack!(robin, white) ❶
ERROR: MethodError: no method matching attack!(::Archer, ::Knight)

julia> shoot!(robin) ❶
ERROR: MethodError: no method matching shoot!(::Archer)

julia> mount!(white) ❷
ERROR: UndefVarError: mount! not defined
```

- ❶ Попытка вызвать функцию без определенных методов.
- ❷ Попытка вызвать неопределенную функцию.

Из этих ошибок видно, что Julia различает функции, которые вообще не были определены, такие как `mount!`, и функции, которые были определены, но не имеют методов, такие как `shoot!` и `attack!`. Но откуда известно, что у них нет никаких методов? В Julia есть функция под названием `methods`, которая позволяет инспектировать число прикрепленных к функции методов:

```
julia> methods(attack!) ❶
# 0 methods for generic function "attack!":

julia> methods(mount!) ❷
ERROR: UndefVarError: mount! not defined
```

- ❶ Показать, что функция `attack!` не имеет методов.
- ❷ Функция `mount!` не найдена.

Как можно видеть, Julia сообщает, что функция `attack!` не имеет никаких методов. Давайте сравним этот результат, загрузив файл `warrs.jl` в интерактивную среду REPL.

```
julia> include("warriors.jl") ❶

julia> methods(shoot!)
# 1 methods for generic function "shoot!":
[1] shoot!(archer::Archer)

julia> methods(attack!)
# 2 methods for generic function "attack!":
[1] attack!(a::Archer, b::Archer)
[2] attack!(a::Archer, b::Knight)

julia> methods(mount!)
# 1 methods for generic function "mount!":
[1] mount!(knight::Knight)
```

❶ Загружает исходный код в интерактивную среду REPL.

Рисунок 7.3 иллюстрирует информацию в интерактивной среде REPL. Внутри Julia есть список функций. Каждая функция входит в еще один список, содержащий методы соответствующей функции. Методы могут охватывать разные типы, поскольку они привязаны не к типам, а к функциям. Ничто не мешает добавить метод `shoot!`, который работает со словарем или массивом.

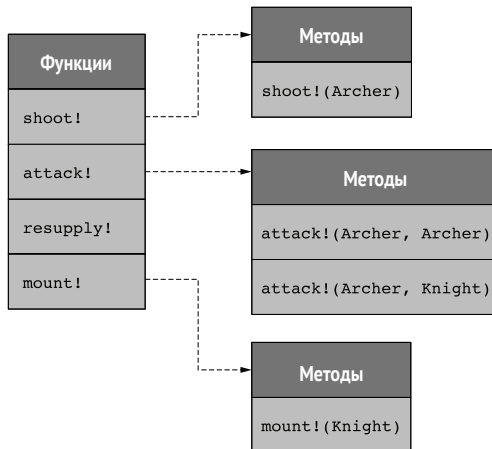


Рис. 7.3 Как Julia прикрепляет методы к функциям. Каждый метод оперирует уникальным набором аргументов

Давайте создадим несколько объектов, которые позволят немного поэкспериментировать:

```
julia> robin = Archer("Robin Hood", 34, 24)
Archer("Robin Hood", 34, 24)

julia> tell = Archer("William Tell", 30, 20)
Archer("William Tell", 30, 20)
```



```
julia> white = Knight("Lancelot", 34, true)
Knight("Lancelot", 34, true)
```

```
julia> black = Knight("Morien", 35, true)
Knight("Morien", 35, true)
```

С некоторыми объектами можно экспериментировать, используя при вызове функции `attack!` объекты разного типа:

```
julia> attack!(robin, white)      ❶
(34, 30)                          ❷

julia> attack!(robin, white)      ❶
(34, 26)                          ❷

julia> attack!(tell, robin)       ❸
(30, 22)                          ❷

julia> attack!(black, white)      ❹
ERROR: MethodError: no method matching attack!{::Knight, ::Knight}
Closest candidates are:
  attack!{::Archer, ::Knight}    ❺
```

- ❶ Атаковать рыцаря лучником.
- ❷ Оставшиеся очки здоровья атакующего и защищающегося.
- ❸ Дать лучнику возможность атаковать другого лучника.
- ❹ Дать рыцарю возможность атаковать рыцаря.
- ❺ Метод с ближайшим совпадением с предпринятым вызовом.

Советую немного поэкспериментировать самостоятельно. При этом можно понаблюдать за тем, как в разных атаках понижаются очки здоровья. В целях упрощения слежения за изменением очков здоровья каждый метод организован так, чтобы в конце боя возвращать кортеж с очками здоровья атакующего и защищающегося.

Интересным моментом здесь является последняя часть, когда делается попытка свести в бою двух рыцарей. Возможно, вы заметили, что мы еще не добавили метод сражения между двумя рыцарями. Мы добавляем его в следующем ниже листинге.

Листинг 7.10 Метод симулирования атаки рыцаря на другого рыцаря

```
function attack!(a::Knight, b::Knight)
    a.health = max(a.health - rand(1:6), 0)
    b.health = max(b.health - rand(1:6), 0)
    a.health, b.health
end
```

Приведенный выше метод можно добавить в файл `wagrs.jl` и перезагрузить файл. Вместо перезагрузки всего файла можно просто вставить определение в интерактивную среду REPL. После этого вы заметите, что черный рыцарь сэр Мориен атакует белого рыцаря сэра Ланселота:

```
julia> attack!(black, white)
(33, 22)
```

И вы заметите, что Julia сообщает, что функция `attack!` теперь имеет три метода:

```
julia> methods(attack!)
# 3 methods for generic function "attack!":
[1] attack!(a::Archer, b::Archer)
[2] attack!(a::Archer, b::Knight)
[3] attack!(a::Knight, b::Knight)
```

Давайте добавим еще один метод `attack!`, позволяющий лучникам атаковать копейщиков. И затем вы сами увидите, как изменилось число методов.

Листинг 7.11 Лучник атакует копейщика

```
function attack!(a::Archer, b::Pikeman)
    if a.arrows > 0
        shoot!(a)
        damage = 4 + rand(1:6)
        b.health = max(b.health - damage, 0)
    end
    a.health, b.health
end
```

❶ Разрешить атаку, только если у лучника есть стрелы (больше нуля).

7.4 Как Julia выбирает метод, который следует вызвать

При вызове функции `attack!(a, b)` Julia будет отыскивать тип каждого аргумента, чтобы найти кортеж из всех аргументных типов:

```
argtypes = (typeof(a), typeof(b))
```

Julia будет использовать этот набор аргументных типов, чтобы просматривать список всех методов и находить подходящий. Напомню, что в Julia функции не имеют исходного кода, его имеют методы. Если у функции нет методов, то эту функцию исполнить невозможно. Описанный процесс показан на рис. 7.4.

В этом примере мы исходим из того, что лучник атакует копейщика, поэтому `a` – это лучник, а `b` – копейщик. Давайте рассмотрим происходящее пошагово.

- 1 Julia пытается вычислить (исполнить) выражение `attack!(a, b)` в программе.
- 2 Julia берет имя функции `attack!` и просматривает таблицу всех функций до тех пор, пока не найдет запись для `attack!`.

- 3 Julia берет эквивалент (`typeof(a)`, `typeof(b)`), чтобы получить кортеж (`Archer`, `Pikeman`). Julia сканирует список хранящихся в функции `attack!` методов сверху вниз до тех пор, пока не найдет совпадение в 4-й записи.
- 4 Julia локализует метод. Метод кодируется как абстрактное синтаксическое дерево (AST)¹. Это общая структура данных² в динамических языках³, служащая для представления функций и методов во время работы программы.
- 5 JIT-компилятор Julia преобразовывает AST-дерево в машинный код⁴, который затем выполняется. Скомпилированный машинный код сохраняется в таблице методов, поэтому в следующий раз, когда просматривается `attack(Archer, Pikeman)`, будет исполняться кешированный машинный код.

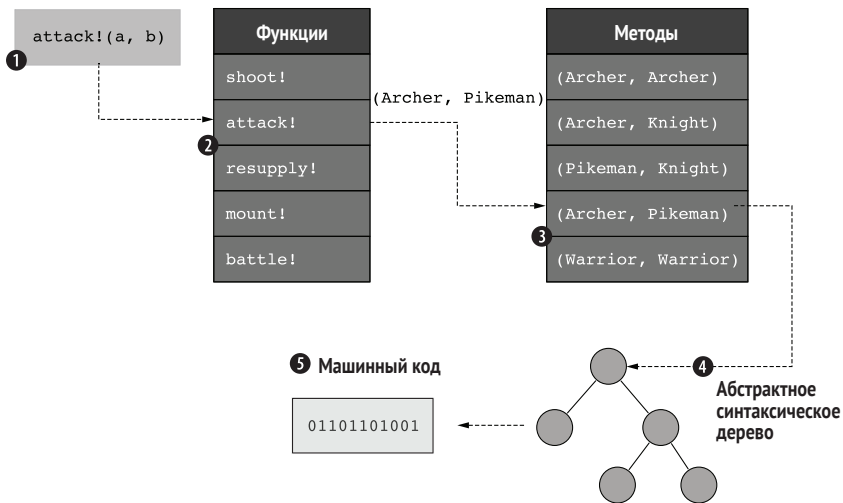


Рис. 7.4 Процесс вызова метода с использованием множественной диспетчеризации

¹ Англ. Abstract Syntax Tree. – Прим. перев.

² Структура данных – это особый способ организации данных в компьютерной программе. Массивы, текстовые строки, двоичные деревья, связанные списки и хеш-таблицы – все это примеры структур данных. Но почти любой составной тип можно трактовать как определение структуры данных.

³ В динамическом языке компилятор, который анализирует правильность типов, перед тем как программа будет разрешена к выполнению, отсутствует. В Julia компилятор есть, но он вызывается во время работы программы.

⁴ Микропроцессор не понимает языки программирования, к примеру такие, как Julia или Java. Он понимает только машинный код.

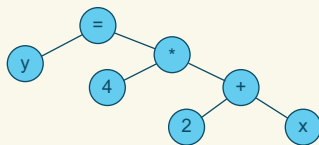
Полное понимание всего этого процесса потребует глубокого погружения в теорию компиляторов и интерпретаторов, что выходит за рамки данной книги. Так что самое лучшее в этой ситуации – думать о том, что все завершается шагом 4. Каким-то образом отыскивается представление метода, который можно исполнить. Последние шаги полезны в основном для тех, кто хочет понять причину, по которой Julia обладает такой высокой производительностью по сравнению с другими языками.

Абстрактные синтаксические деревья (AST) для любознательных

Эта книга посвящена не концепциям компиляторов, таким как AST-деревья. Но я предложу немного касающейся их информации, чтобы помочь понять язык Julia. Рассмотрим выражение, к примеру вот такое:

$$y = 4 * (2 + x)$$

Когда компилятор или интерпретатор читает такой исходный код, он обычно превращает его в древесную структуру, именуемую абстрактным синтаксическим деревом (AST), как показано на следующем ниже рисунке.



Абстрактное синтаксическое дерево (AST) выражения $y = 4 * (2 + x)$

В Julia каждый метод превращается в такую древесную структуру. Таблица методов каждой функции отслеживает каждую древесную структуру. Компилятор Julia использует эти структуры для создания фактического понятного компьютеру машинного кода.

7.4.1 Сравнение множественной диспетчеризации Julia с объектно ориентированными языками

Множественная диспетчеризация нередко дезориентирует разработчиков, имеющих опыт работы с объектно ориентированными языками программирования. Здесь я попытаюсь противопоставить подход Julia тому, как работают объектно ориентированные языки. В методе объектно ориентированного языка исполняемая реализация выбирается на основе типа одного аргумента. Вот почему мы называем этот подход *одиночной диспетчеризацией*. На объектно ориентированном языке вы бы написали не `attack!(archer, knight)`, а исходный код в следующем ниже листинге.

Листинг 7.12 Синтаксис, если бы язык Julia был объектно ориентированным

```
archer.attack!(knight)
archer.shoot!()
knight.mount!()
```

Хотя приведенный выше исходный код невозможно написать на Julia, такое поведение на языке Julia можно симитировать.

Листинг 7.13 Одиночная диспетчеризация на языке Julia

```
function attack(archer::Archer, opponent) ❶
    if typeof(opponent) == Archer
        ...
    elseif typeof(opponent) == Knight
        ...
    elseif typeof(opponent) == Pikeman
        end
    end
end

function attack(knight::Knight, opponent) ❷
    if typeof(opponent) == Archer
        ...
    elseif typeof(opponent) == Knight
        ...
    elseif typeof(opponent) == Pikeman
        end
    end
end
```

❶ Обработать все случаи, когда атакующим является лучник.

❷ Обработать все случаи, когда атакующим является рыцарь.

Приведенный выше исходный код иллюстрирует ограниченность одиночной диспетчеризации. Поскольку методы функции `attack!` могут быть выбраны только на основе типа первого аргумента, требуется длинный список инструкций `if-else`, чтобы учесть противников разных типов. Здесь следует внести ясность в принцип работы одиночной диспетчеризации, дав пошаговое объяснение (рис. 7.5).

- 1 При вычислении `a.attack!(b)` отыскивается объект, на который ссылается `a`.
- 2 В этом объекте-лучнике есть скрытое поле `isa`, которое указывает на тип объекта-лучника.
- 3 Тип `Archer` – это сам объект с различными полями. В нем есть поля для каждого метода: `shoot!`, `attack!` и т. д. Он похож на словарь, в котором имя функции `attack!` используется для отыскания правильного метода.
- 4 Метод представляет собой вычисляемое AST-дерево.

Таким образом, ключевое отличие Julia, которое следует высечь в граните, заключается в том, что в большинстве мейнстримных

языков методы хранятся в типе объекта, а в Julia методы хранятся в функциях.

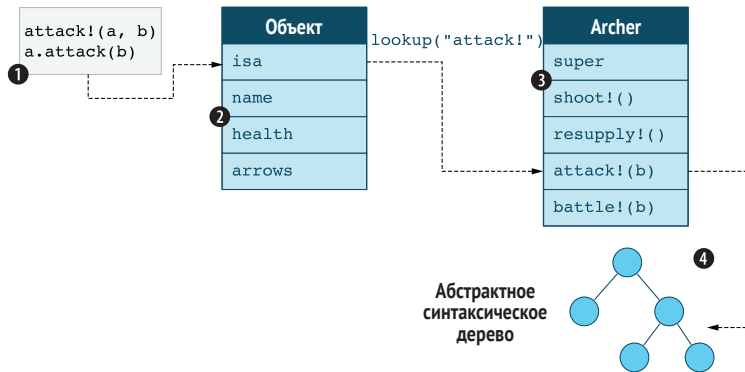


Рис. 7.5 Механизм вызова метода с использованием одиночной диспетчеризации

7.4.2 Чем множественная диспетчеризация отличается от перегрузки функций?

Статически типизированные языки, такие как Java, C# и C++, имеют так называемую перегрузку функций, которая внешне выглядит так же, как множественная диспетчеризация. Ключевое отличие состоит в том, что при перегрузке функций правильный подлежащий вызову метод определяется во время компиляции, а это означает, что метод, подобный показанному в следующем ниже листинге, в статически типизированном языке невозможен.

Листинг 7.14 Запись результатов битвы между двумя воинами

```
function battle!(a::Warrior, b::Warrior) ❶
    attack!(a, b)
    if a.health == 0 && b.health == 0
        println(a.name, " and ", b.name, " destroyed each other")
    elseif a.health == 0
        println(b.name, " defeated ", a.name)
    elseif b.health == 0
        println(a.name, " defeated ", b.name)
    else
        println(b.name, " survived attack from ", a.name)
    end
end
```

❶ а и b должны быть подтипами типа Warrior.

Давайте добавим этот метод в файл исходного кода `warriors.jl`, перезагрузим все и воссоздадим обычных подозреваемых, `white`

и `robin`, чтобы протестировать `battle!` в интерактивной среде REPL. Робин Гуд атакует сэра Ланселота несколько раз до тех пор, пока очки здоровья Ланселота не упадут настолько низко, что `battle!` напечатает, что он побежден:

```
julia> battle!(robin, white)
Lancelot survived attack from Robin Hood

julia> battle!(robin, white)
Lancelot survived attack from Robin Hood

julia> battle!(robin, white)
Robin Hood defeated Lancelot
```

При вызове `battle!(robin, white)` Julia будет искать метод с сигнатурой `battle!(a::Archer, b::Knight)`, но не сможет его найти. Однако `battle!(a::Warrior, b::Warrior)` является допустимым совпадением, потому что и `Knight`, и `Archer` являются подтипами типа `Warrior`.

Когда компилятор Julia компилирует метод `battle!`, он не знает, какой аргумент конкретного типа будет у `a` и `b`. Он только знает, что они имеют какой-то подтип типа `Warrior`. И следовательно, компилятор не может выбрать правильный метод `attack!`, чтобы его затем вызвать. Это решение может быть принято только во время работы программы. Именно это и отличает множественную диспетчеризацию от перегрузки функций. Встречающаяся в Java и C++ перегрузка функций зависит от способности компилятора подбирать правильный метод.

Резюме

- Числа в Julia являются частью сложной иерархии типов.
- В иерархии типов только листовые узлы могут быть конкретными типами. Все остальные типы являются абстрактными.
- Функции `typeof`, `supertype` и `subtypes` можно использовать для исследования иерархий типов.
- Функция – это просто имя. Без прикрепленных методов они ничего не могут делать. Исходный код всегда хранится внутри методов. Тип аргументов определяет метод, который будет исполняться во время работы программы.
- В объектно ориентированных языках используется одиночная диспетчеризация, то есть метод выбирается по типу первого аргумента функции. В Julia применяется множественная диспетчеризация, то есть на выбор метода влияют абсолютно все аргументы.
- Составной тип, в отличие от примитивного типа, состоит из нуля или более полей. Ключевое слово `struct` используется для определения составного типа.
- Добавляя в определение `struct` ключевое слово `mutable`, обеспечивается возможность видоизменять отдельные поля составного объекта во время работы программы.

Сборка ракеты

Эта глава охватывает следующие ниже темы:

- разработка сложных структур данных, состоящих из разных объектов разных типов;
- абстрагирование от различий между разными, но родственными типами.

В предыдущей главе мы создали несколько простых составных типов, чтобы представить разные типы воинов. Однако в более реалистичных приложениях приходится комбинировать много совершенно разных типов объектов в более сложные структуры данных.

В целях обследования данной темы мы соберем ракету в исходном коде. Почему ракета? Потому что ракеты состоят из многочисленных частей, благодаря чему появляется возможность создавать составные типы из других составных типов и показывать различные способы применения абстрактных типов в Julia, чтобы облегчать разработку сложных структур данных. Пример с ракетой будет использоваться далее в книге с целью обследования многих других тем, таких как механизм представления коллекций объектов в Julia.

Пример исходного кода начнется с определения простой ракеты типа `Rocket`, состоящей из объектов `Payload`, `Tank` и `Engine`. Позже мы видоизменим определение простого типа, создав более сложную многоступенчатую ракету, состоящую из нескольких объектов

StagedRocket. Далее исходный код будет видоизменен путем добавления типа Cluster, представляющего кластер ракетных двигателей, который можно прикреплять к любой ракетной ступени. В конце мы определим функцию launch!, чтобы симулировать запуск многоступенчатой ракеты.

8.1 Сборка простой ракеты в исходном коде

Начнем с моделирования простой космической ракеты в исходном коде. Это одноступенчатая ракета, состоящая из следующих ниже частей снизу вверх (см. рис. 8.1):

- *ракетный двигатель* – обеспечивает тяговое усилие;
- *топливный бак* – содержит выбрасываемое двигателем вещество;
- *полезная нагрузка* – например, капсула или спутник.



Рис. 8.1 Составляющие ракету детали

Полезная нагрузка – это полезные компоненты, которые требуется перемещать в пространстве. Это может быть модуль экипажа для астронавтов или зонд с приборами для обследования других планет.

Такая ракета может быть определена составным типом (листинг 8.1). Но пока рано набирать его на клавиатуре; это просто для того, чтобы вы подумали о типах, которые нужно определить. Необходимо будет реализовать разные типы для топливных баков и ракетных двигателей. Затем будут добавлены разные атрибуты и поведение, такие как заправка топливных баков и потребление ракетного топлива

Листинг 8.1 Первое определение простой ракеты

```
struct Rocket
  payload::Payload
  tank::Tank
  engine::Engine
end
```

Но пока мы сосредоточимся на топливном баке. Ракетное топливо – это вещество, которое ракетный двигатель выбрасывает, чтобы двигаться вперед. В простейшей форме это выпускаемый сжатый газ. Однако в реальных космических ракетах это комбинация топлива, к примеру такого как керосин или водород, и окислителя, такого как жидкий кислород. Однако эти подробности вовсе не требуется включать в модель. Вместо этого будет рассмотрено следующее:

- *сухая масса* – масса пустого топливного бака;
- *общая масса* – масса полного топливного бака;
- *масса ракетного топлива* – текущий остаток ракетного топлива в топливном баке;
- *масса* – сухая масса плюс оставшееся ракетное топливо.

Я покажу другой способ моделирования перечисленного выше на языке Julia и расскажу о плюсах и минусах этих подходов.

В целях упрощения организации исходного кода, который будет написан в этой главе, его можно распределить по нескольким файлам, а затем создать один файл (например, `Rockets.jl`), который включит их все в свой состав.

Благодаря этому в интерактивную среду REPL можно будет загружать только этот файл, чтобы тот импортировал весь исходный код. В листинге 8.2 подразумевается, что мы уже создали файлы `tank.jl`, `engine.jl` и `payloads.jl` и хотим загрузить их все за один раз.

Листинг 8.2 `Rockets.jl`

```
include("tanks.jl")
include("engines.jl")
include("payloads.jl")
```

Это всего лишь предложение. Если вы сочтете более практичным выгрузку всего исходного кода в один файл, то без проблем можете поступить по-своему.

ВАЖНО! При изменении определения типа в исходном коде потребуется полностью перезапускать интерактивную среду REPL и заново загружать исходный код. Другое дело – внесенные изменения в функции: для того чтобы они вступали в силу, требуется вставка в интерактивную среду REPL только нового исходного кода.

Чтобы предоставить ракете возможность содержать много разных типов топливных баков, надо сделать тип `Tank` абстрактным. Поскольку определенные в следующем ниже листинге средний и большой топливные баки являются подтипами типа `Tank`, их можно вставлять в любое поле, ожидающее объекта `Tank`.

Листинг 8.3 Определение разных топливных баков фиксированной емкости

```

abstract type Tank end ❶

mutable struct SmallTank <: Tank ❷
    propellant::Float64
end

mutable struct MediumTank <: Tank ❷
    propellant::Float64
end

mutable struct LargeTank <: Tank ❷
    propellant::Float64
end

# Функции доступа (геттеры)
drymass(::SmallTank) = 40.0 ❸
drymass(::MediumTank) = 250.0 ❸
drymass(::LargeTank) = 950.0 ❸

totalmass(::SmallTank) = 410.0 ❸
totalmass(::MediumTank) = 2300.0 ❸
totalmass(::LargeTank) = 10200.0 ❸

```

- ❶ Сделать тип Tank абстрактным.
- ❷ Мутируемый, чтобы дать массе ракетного топлива возможность изменяться.
- ❸ drymass (сухая масса) и totalmass (общая масса) не сохраняются.

Сухая масса и общая масса этих топливных баков привязаны к их типу. Однако также можно создать гибкий топливный бак, в котором можно задавать сухую массу и общую массу по своему усмотрению, как показано в следующем ниже листинге.

Листинг 8.4 Топливный бак с переменной вместимостью

```

mutable struct FlexiTank <: Tank
    drymass::Float64
    totalmass::Float64
    propellant::Float64
end

# Функции доступа (геттеры/получатели)
drymass(tank::FlexiTank) = tank.drymass
totalmass(tank::FlexiTank) = tank.totalmass

```

На данный момент топливные баки служат примитивными контейнерами информации. Они не делают ничего полезного, поэтому давайте добавим полезное поведение.

Листинг 8.5 Добавление способностей и поведения топливного бака

```

# Функции доступа (сеттеры и геттеры)
propellant(tank::Tank) = tank.propellant           ❶
function propellant!(tank::Tank, amount::Real)      ❶
    tank.propellant = amount
end

isempty(tank::Tank) = tank.propellant <= 0          ❷
mass(tank::Tank) = drymass(tank) + propellant(tank)  ❸

# Действия
function refill!(tank::Tank)                        ❹
    propellant!(tank, totalmass(tank) - drymass(tank))
    tank                                             ❺
end

function consume!(tank::Tank, amount::Real)         ❻
    remaining = max(propellant(tank) - amount, 0)
    propellant!(tank, remaining)
    remaining
end

```

- ❶ Функции доступа для поля `propellant`.
- ❷ Проверить, что топливный бак является пустым.
- ❸ Рассчитать текущую общую массу топливного бака.
- ❹ Наполнить топливный бак топливом.
- ❺ Сделать измененное состояние топливного бака доступным для REPL.
- ❻ Потребить топливо.

Давайте создадим несколько топливных баков, чтобы продемонстрировать поведение этих функций:

```

julia> small = SmallTank(50)           ❶
SmallTank{50.0}

julia> consume!(small, 10)              ❷
40.0

julia> consume!(small, 10)              ❷
30.0

julia> small                            ❸
SmallTank{30.0}

julia> refill!(small)
SmallTank{370.0}

julia> flexi = FlexiTank(5, 50, 0)      ❹
FlexiTank{5.0, 50.0, 0.0}

```

```
julia> refill!(flexi)
FlexiTank(5.0, 50.0, 45.0)
```

- ❶ Сделать малый топливный бак танк с 50 кг ракетного топлива.
- ❷ Потребить 10 кг ракетного топлива.
- ❸ Проверить объем оставшегося ракетного топлива.
- ❹ 5 кг сухой массы, 50 кг общей массы и 0 кг ракетного топлива.

В примере исходного кода используется два разных топливных бака: малый топливный бак и гибкий топливный бак. Хотя функции `consume!` и `refill!` были написаны только для работы с типом `Tank`, эти функции можно использовать как с типом `SmallTank`, так и с типом `FlexiTank`, потому что `drymass` и `totalmass` были реализованы для всех конкретных подтипов типа `Tank`.

Функции `propellant` и `propellant!` были реализованы, исходя из допущения, что все подтипы типа `Tank` имеют поле `propellant`. Так может быть не всегда. Однако это не проблема. Если определить функции доступа `propellant` для конкретного подтипа типа `Tank`, то они всегда будут иметь приоритет. Когда язык Julia просматривает список методов функции, он всегда ищет методы с наиболее конкретными типами аргументов.

ПРИМЕЧАНИЕ В объектно ориентированных языках функции доступа, также именуемые сеттерами и геттерами, имеют большую важность. Например, если в Java у типа есть поле `propellant`, то можно написать методы `GetPropellant` и `SetPropellant`. В Julia эквивалентом является функция `propellant` для получения значения и функция `propellant!` для установки значения. Однако Julia не является объектно ориентированным языком, поэтому злоупотреблять этим шаблоном не следует.

Методы функции `refill!` можно использовать для автоматической заправки топливных баков до максимума при их конструировании.

Листинг 8.6 Дополнительные конструкторы, облегчающие создание полных топливных баков

```
function SmallTank()
    refill!(SmallTank(0))
end

function MediumTank()
    refill!(MediumTank(0))
end

function LargeTank()
    refill!(LargeTank(0))
end
```

Примеры исходного кода показали, что существуют разные способы моделирования топливного бака. Как понять, какой подход использовать в своем собственном исходном коде? Первый подход, заключающийся в наличии конкретных типов для каждой емкости, таких как `SmallTank`, `MediumTank` и `LargeTank`, потенциально экономит память, в случае если создается много таких объектов. Число байтов, которое объект должен хранить в памяти, зависит от числа полей, которые он имеет, и размера каждого поля. Поле `Int8` занимает только один байт, а поле `Int64` занимает восемь байт. Каждый объект `FlexiTank` занимает в памяти в три раза больше места, чем объекты фиксированного топливного бака.

Однако в исходном коде, который мы пишем, это нисколько не волнует. У нас не так много топливных баков, а если бы у нас были тысячи топливных баков, то это не имело бы значения. `FlexiTank` был бы самым лучшим вариантом, поскольку он обеспечивает большую гибкость в использовании. Тогда почему здесь содержатся определения фиксированных топливных баков?

Сравнивая эти разные подходы к моделированию топливного бака, лучше понимаешь возможности системы типов Julia. Будут случаи, когда такой компромисс того стоит. Например, представьте себе симуляцию города с населением в миллионы человек. У каждого человека могут быть такие атрибуты, как местоположение, голод, усталость, деньги в кармане, одежда, обувь и т. д. Имея дело с таким количеством объектов, вы, возможно, захотите подумать об уменьшении размера объектов.

8.2 *Поддержание инвариантов в исходном коде*

При написании исходного кода важно учитывать понятие *инвариантности*. Инварианты – это то, что всегда должно оставаться истинным во время выполнения всей программы или какой-то ее части. Возможно, это понятие прозвучит очень абстрактно, поэтому давайте я обосную необходимость выражения инвариантов путем реализации функции `propellant!`, которая задает объем топлива в баке.

```
julia> tank = FlexiTank(5, 50, 10) ❶  
FlexiTank{5.0, 50.0, 10.0}
```

```
julia> propellant!(tank, 100)      ❷  
100
```

```
julia> totalmass(tank)  
50.0
```

```
julia> mass(tank)  
105.0
```

❶ Сухая масса 5 кг, общая масса 50 кг и 10 кг топлива.

❷ Установить массу топлива равной 100 кг.

Что здесь не так? Масса топлива задается больше, чем максимальная общая масса топливного бака. Этого нельзя делать. В любой момент должен быть верен следующий ниже инвариант топливного бака:

```
0 <= propellant(t) + drymass(t) <= totalmass(t)
```

Один из подходов к обеспечению истинности инварианта состоит в видоизменении метода-сеттера `propellant!` в части генерирования исключения, в случае если входные данные неверны.

Листинг 8.7 Метод задания значения ракетного топлива, поддерживающий инвариант топливного бака

```
function propellant!(tank::Tank, amount::Real)
    if 0 <= amount + drymass(tank) <= totalmass(tank) ❶
        tank.propellant = amount
    else
        msg = "Propellant mass plus dry mass must be less than total
mass"
        throw(DomainError(amount, msg)) ❷
    end
end
```

- ❶ Проверить, не нарушает ли новый объем ракетного топлива инвариант топливного бака.
- ❷ Сигнализировать об ошибке области значений путем выдачи исключения.

`DomainError` – это тип исключения, определенный в стандартной библиотеке языка Julia. Область значений, или домен, относится к набору допустимых входных значений аргумента функции. Таким образом, предоставление объема, превышающего общую массу, является ошибкой области значений.

Однако это не единственный способ нарушить инвариант топливного бака. Ниже топливный бак делается с 90 кг ракетного топлива, а общая масса может составлять только 50 кг:

```
julia> t = FlexiTank(5, 50, 90)
FlexiTank(5.0, 50.0, 90.0)

julia> mass(t), totalmass(t)
(95.0, 50.0)
```

Решение этих проблем зависит от умения создавать составные объекты.

8.3 Создание объектов с помощью функций-конструкторов

При определении составного типа Julia создает специальную функцию, именуемую конструктором, с тем же именем, что и тип. Кон-

структор отвечает за создание экземпляра типа (объекта), с которым он связан. Julia добавляет в функцию-конструктор два метода, которые принимают столько же аргументов, сколько имеется полей. В аргументах одного метода используются аннотации типов, как указано для каждого поля в `struct`. Другой метод принимает аргументы типа `Any`.

Однако в эту функцию-конструктор можно добавлять методы так же, как и в любую другую функцию. Если объем ракетного топлива не указан, то можно добавить методы, которые создают полный топливный бак.

Листинг 8.8 Создание полных топливных баков, когда объем ракетного топлива не указан

```
function FlexiTank(drymass::Number, totalmass::Number)
    FlexiTank(drymass, totalmass, totalmass - drymass)
end
```

```
MediumTank() = refill!(MediumTank(0))
LargeTank() = refill!(LargeTank(0))
```

Если применить функцию `methods`, то можно увидеть, что в функцию `FlexiTank` добавлен третий метод:

```
julia> methods(FlexiTank)
# 3 methods for type constructor:
[1] FlexiTank(drymass::Float64, totalmass::Float64,
propellant::Float64) ❶
[2] FlexiTank(drymass::Number, totalmass::Number) ❷
[3] FlexiTank(drymass, totalmass, propellant) ❶
```

- ❶ Существующие методы, определенные языком Julia.
- ❷ Новый метод, добавленный разработчиком.

Ниже приведен пример использования этих новых методов для создания полных топливных баков:

```
julia> FlexiTank(5, 50)
FlexiTank(5.0, 50.0, 45.0) ❶

julia> MediumTank()
MediumTank(2050.0) ❷

julia> LargeTank()
LargeTank(9250.0)
```

- ❶ Автоматически заправляется 45 кг ракетного топлива.
- ❷ Заправленный 2050 кг ракетного топлива.

Но что делать, если вы не хотите, чтобы пользователи типов самостоятельно устанавливали массу ракетного топлива? Возможно, вы хотите уменьшить вероятность нарушения важного инварианта

топливного бака, о котором говорилось ранее. Есть ли способ не давать Julia создавать собственные методы конструктора?

8.4 Различия между внешними и внутренними конструкторами

То, с чем вы только что познакомились, в терминологии Julia называется *внешними конструкторами*. Конструкторы определяются *за пределами* определения составного типа. Внешние конструкторы добавляют методы ко встроенным конструкторам. Если вместо этого вы хотите заменить методы конструктора своими собственными, то необходимо определить функции конструктора *внутри* определения `struct`, как показано ниже.

Листинг 8.9 Определение внутреннего конструктора для `FlexiTank`

```
mutable struct FlexiTank <: Tank
    drymass::Float64
    totalmass::Float64
    propellant::Float64

    function FlexiTank(drymass::Number, totalmass::Number)
        new(drymass, totalmass, totalmass - drymass)
    end
end
```

❶ Обратите внимание на новые замены `FlexiTank`.

Внутренние конструкторы знакомят со специальной функцией, именуемой `new`. Она доступна только внутри внутреннего конструктора и больше нигде. Ее необходимость обусловлена тем, что создание внутреннего конструктора удаляет все методы конструктора, созданные языком Julia. Другими словами, ни один из них больше вызвать невозможно.

Функция `new` очень похожа на предоставляемые языком Julia по умолчанию методы конструктора, но с несколькими важными отличиями: `new` можно указывать с нулем или более аргументов, но никогда не большим числом аргументов, чем число полей в составном типе. Что происходит с полями, для которых значение не указывается? Они получают случайное значение.

Откуда известно, что внутренний конструктор заменяет все предоставленные языком Julia методы конструктора? Можно перезагрузить интерактивную среду REPL и протестировать:

```
julia> t = FlexiTank(5, 50)
FlexiTank{Float64}(5.0, 50.0, 45.0)

julia> t = FlexiTank(5, 50, 150)
```

❶

```
ERROR: MethodError: no method matching FlexiTank{::Int64, ::Int64,
::Int64}
Closest candidates are:
  FlexiTank{::Number, ::Number}
```

```
julia> methods(FlexiTank)
# 1 method for type constructor:
[1] FlexiTank(drymass::Number, totalmass::Number)
```

2

- ① Julia больше не может найти метод, принимающий третий аргумент.
- ② Проверить число методов, прикрепленных к конструктору.

Теперь видно, что функция `methods` сообщает только об одном методе конструктора `FlexiTank`.

8.5 Моделирование ракетных двигателей и полезной нагрузки

Давайте переключимся с этой темы и поговорим о полезной нагрузке, которую требуется отправить в космос, и о ракетных двигателях, которые будут обеспечивать движение ракеты. Полезной нагрузкой может быть зонд, спутник, капсула экипажа или, полагая, четырехместный спортивный электромобиль Tesla Roadster, если вы – Илон Маск.

Листинг 8.10 Определение полезной нагрузки ракеты

```
struct Payload
    mass::Float64
end
```

Возможно, такое определение покажется упрощенным, но напомним, что мы создаем модели. Модели содержат только те свойства, которые необходимы для ответа на интересные вопросы. Например, первоначальная модель смартфона может быть просто деревянным бруском без кнопок, экрана и цветовой схемы. Почему? Потому что изначально вопросы, на которые требуется получить ответы, звучат так: «Удобно ли носить эту форму и размер в кармане? Сколько места будет для создания экрана и электроники внутри корпуса?»

То же самое относится и к конструированию и сборке ракеты. Изначально интересуют только бюджеты по массе. Важно знать такие вещи, как:

- 1 сколько топлива потребуется?
- 2 какую полезную нагрузку можно запустить на орбиту?
- 3 как далеко данная ракета сможет улететь?

В целях ответа на такие вопросы *не требуется* указывать ни инструменты, которые есть в космическом зонде, ни батареи или солнечные элементы, которые установлены в модели. Важными атрибу-

тами ракетного двигателя являются масса, тяга и удельный импульс (Isp)¹. Тяга может трактоваться как мощность двигателя, а удельный импульс – как его экономичность.

Листинг 8.11 Определение конкретно-прикладного ракетного двигателя

```
abstract type Engine end

struct CustomEngine <: Engine
    mass::Float64           ❶
    thrust::Float64         ❷
    Isp::Float64            ❸
end

mass(engine::CustomEngine) = engine.mass
thrust(engine::CustomEngine) = engine.thrust
Isp(engine::CustomEngine) = engine.Isp
```

- ❶ Масса ракетного двигателя в килограммах.
- ❷ Тяга ракетных двигателей в ньютонах.
- ❸ Удельный импульс (эффективность ракетного топлива).

Кроме того, можно определить типы конкретных двигателей с известными свойствами, такие как двигатель Резерфорда, используемый в ракете Electron, и двигатель Merlin, используемый в ракете Falcon 9.

Листинг 8.12 Определение ракетных двигателей Резерфорда и Мерлина

```
struct Rutherford <: Engine end ❶
struct Merlin <: Engine end      ❶

mass(::Rutherford) = 35.0         ❷
thrust(::Rutherford) = 25000.0    ❷
Isp(::Rutherford) = 311.0         ❷

mass(::Merlin) = 470.0
thrust(::Merlin) = 845e3
Isp(::Merlin) = 282.0
```

- ❶ Двигатели являются пустыми структурами struct.
- ❷ Использовать функции доступа для получения свойств двигателя.

Тяга (thrust) – это сила, создаваемая ракетным двигателем. Если известна общая масса ракеты, то можно рассчитать ускорение всей ракеты после запуска ракетных двигателей. Ускорение вычисляется из второго закона Ньютона, который гласит, что сила F пропорциональна массе m , умноженной на ускорение a :

¹ Англ. specific impulse. – Прим. перев.

$$F = ma \iff a = \frac{F}{m}.$$

Однако, для того чтобы знать объем массы, который выталкивается в любой момент времени, нужно знать объем ракетного топлива, который двигатель потребляет каждую секунду. Одна только тяга не может этого сказать; нужен удельный импульс (*Isp*). Двигатель с высоким удельным импульсом более эффективен по топливу, а это означает, что он будет потреблять меньше топлива при той же величине тяги.

ПРИМЕЧАНИЕ В физике импульс обычно обозначается буквой *I*. Таким образом, *Isp* поясняет, что имеется в виду удельный (или особый – special) импульс.

Удельный импульс аналогичен расходу бензина автомобилем. Однако, в отличие от автомобиля на дороге, ракета в открытом космосе продолжает движение даже без тяги, поэтому невозможно измерить эффективность потребления топлива (или эффективность ракетного топлива) дальностью доставки полезной нагрузки одним килограммом ракетного топлива. Вместо этого она измеряется с точки зрения числа секунд, которые единица ракетного топлива может выдерживать силу в 1 G (силу земного притяжения). Такой подход позволяет рассчитывать поток массы (потребление ракетного топлива в секунду):

```
g = 9.80665
function mass_flow(thrust::Number, Isp::Number)
    thrust / (Isp * g)
end
```

- ① м/с² ускорения земного притяжения (гравитации).
- ② Объем выходящей из двигателя массы измеряется в кг/с.

Например, это можно использовать для расчета ракетного топлива, потребляемого в секунду ракетой Falcon 9. Она имеет девять двигателей Merlin 1D, каждый с удельным импульсом 282 с и тягой 845 кН:

```
julia> engine_thrust = 845e3
845000.0

julia> isp = 282
282

julia> thrust = engine_thrust * 9
7.605e6

julia> flow = mass_flow(thrust, isp)
2749.979361594732
```

Таким образом, получается, что каждую секунду ракета Falcon 9 потребляет примерно 2.7 т ракетного топлива.

8.6 Сборка простой ракеты

Теперь у нас имеются все детали для сборки ракеты. Итак, начнем сборку ракеты!

Листинг 8.13 Ракета с полезной нагрузкой, топливным баком и двигателем

```
struct Rocket
    payload::Payload
    tank::Tank
    engine::Engine
end
```

Мы изготовим ракету, похожую на ракету Electron производства компании Rocket Lab¹. Указанная ракета может выводить на низкую околоземную орбиту полезный груз массой 300 кг. Мы сделаем одну ступень этой ракеты с малым топливным баком и двигателем Резерфорда. Двигатель Резерфорда, небольшой ракетный двигатель, используется в первой и второй ступенях ракеты Electron.

Для компактности, добавляя точку с запятой ;, я проинструктировал интерактивную среду REPL не печатать значение первых трех присваиваний. Вы можете убрать точку с запятой, чтобы увидеть разницу:

```
julia> payload = Payload(300);

julia> tank = SmallTank();

julia> engine = Rutherford();

julia> rocket = Rocket(payload, tank, engine)
Rocket(Payload(300.0), SmallTank(370.0), Rutherford())
```

Если вы немного разбираетесь в ракетной технике, то понимаете, что с приведенной выше ракетной конфигурацией было несколько проблем:

- 1 реальная ракета Electron имеет девять двигателей Резерфорда, а не один;
- 2 космические ракеты имеют несколько ступеней, которые отделяются по мере того, как ракета поднимается все выше; данная ракета имеет только одну ступень.

¹ Rocket Lab – новозеландская космическая компания, которая запускает на орбиту малые спутники массой в несколько сотен килограммов.

8.7 Создание ракеты с несколькими ступенями и двигателями

Давайте исправим эти проблемы. Важно понимать, что многоступенчатая ракета немного похожа на русскую матрешку. Полезной нагрузкой ракеты делается еще одна ракета. Полезной нагрузкой этой следующей ракеты будет другая ракета и т. д. На рис. 8.2 показана компоновка многоступенчатой ракеты в виде нескольких вложенных друг в друга ракет.

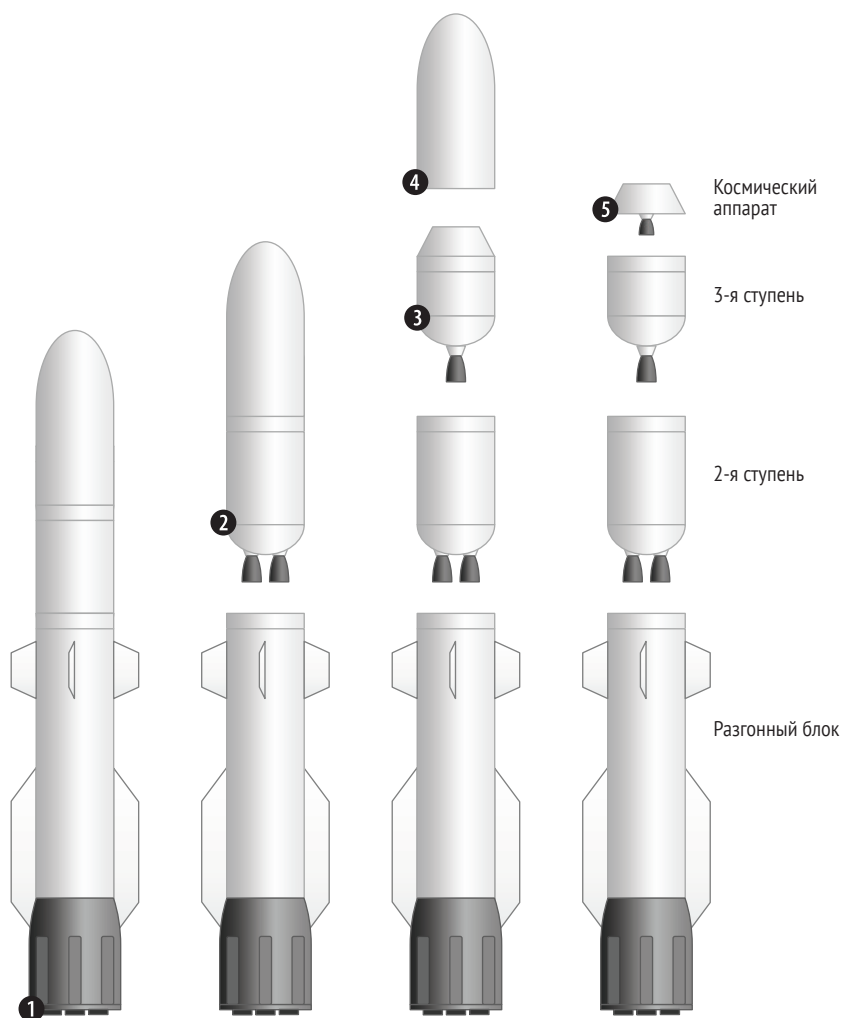


Рис. 8.2 Ступени ракеты

На нем показана многоступенчатая ракета, в которой постоянно выстреливается верхняя часть, обнажая полезную нагрузку ракеты.

Давайте остановимся на каждой пронумерованной ступени подробнее.

- 1 Вся многоступенчатая ракета со всеми частями называется *космическим кораблем*. Первая ступень называется *разгонным блоком* (или бустером).
- 2 Отстрелив разгонный блок, вы увидите вторую ступень ракеты.
- 3 Полезной нагрузкой второй ступени является третья ступень.
- 4 Третья ступень защищена *обтекателем*, то есть защитной оболочкой полезной нагрузки (в данной модели обтекатель будет проигнорирован).
- 5 По завершении запуска в космос доставляется последняя полезная нагрузка. Этой полезной нагрузкой будет *космический аппарат*, к примеру такой, как спутник, лунный посадочный модуль или капсула.

Для того чтобы можно было размещать ракеты внутри ракет, мы заменим тип `Rocket` на абстрактный и определим новые конкретные подтипы, как показано ниже.

Листинг 8.14 Наделение ракеты возможностью быть полезной нагрузкой другой ракеты

```
abstract type Rocket end ❶

struct Payload <: Rocket ❷
    mass::Float64
end

struct StagedRocket <: Rocket ❸
    nextstage::Rocket
    tank::Tank
    engine::Engine
end

function Rocket(payload::Rocket, tank::Tank, engine::Engine) ❹
    StagedRocket(payload, tank, engine)
end

thrust(r::Payload) = 0.0 # двигатели отсутствуют
thrust(r::StagedRocket) = thrust(r.engine)
```

- ❶ Сделать тип `Rocket` абстрактным.
- ❷ Превратить тип `Payload` в подтип типа `Rocket`.
- ❸ Для ясности переименовать полезную нагрузку в следующую ступень.
- ❹ Разрешить использование старого конструктора.

Но перед тем, как собрать ракету, нужно еще кое-что скорректировать. Реальная ракета *Electron* имеет девять двигателей Резерфорда на первой ступени, которую мы называем разгонным блоком. В на-

стоящее время у нас нет возможности добавлять более одного двигателя. В целях решения этой проблемы надо определить новый подтип двигателя под названием `Cluster`. Этот новый тип предназначен для моделирования кластера из нескольких одинаковых двигателей.

На рис. 8.3 не показаны все типы. Например, хватило места только для типов `MediumTank` и `FlexiTank` под абстрактным типом `Tank`.

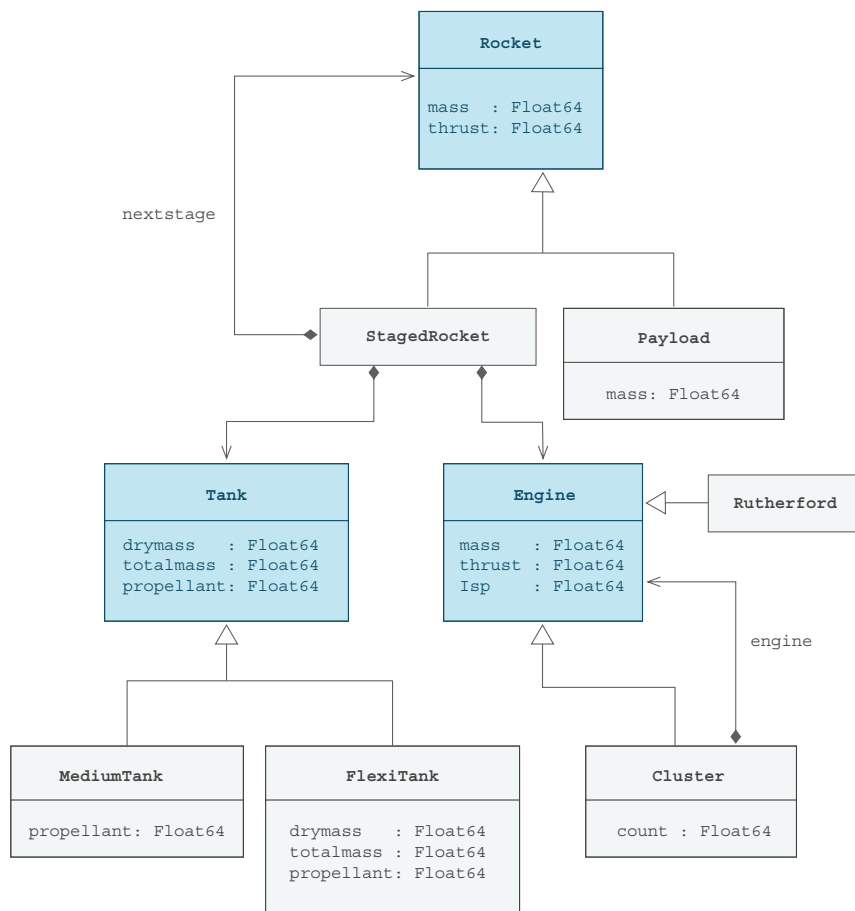


Рис. 8.3 UML-диаграмма взаимосвязей между различными частями ракеты

С помощью полых стрелок на диаграмме показано, как типы `StagedRocket` и `Payload` являются подтипами абстрактного типа `Rocket`. Закрашенная стрелка показывает, что в типе `StagedRocket` есть поле `nextstage`, указывающее на еще один объект типа `Rocket`.

Давайте посмотрим, как именно реализовать тип `Cluster` (листинг 8.15). Диаграмма UML говорит, что это одновременно подтип типа `Engine`, и указывает на еще один двигатель через поле `engine`.

Листинг 8.15 Определение кластера ракетных двигателей

```

struct Cluster <: Engine
    engine::Engine
    count::Int      ❶
end

Isp(cl::Cluster) = Isp(cl.engine)

mass(cl::Cluster) = mass(cl.engine) * cl.count
thrust(cl::Cluster) = thrust(cl.engine) * cl.count

```

❶ Число одинаковых двигателей в кластере.

Вы заметите, что удельный импульс не меняется. Топливная эффективность не меняется только потому, что добавляется больше двигателей. Однако добавление большего числа двигателей увеличивает массу кластера, а также общую тягу.

О КЛАСТЕРЕ РАЗНОРОДНЫХ ДВИГАТЕЛЕЙ Нельзя ли сделать кластер двигателей разных типов? Задача состоит в том, чтобы решить, как рассчитывать удельный импульс (Isp) кластера, в котором каждый двигатель имеет другой удельный импульс. Однако в главе 12 будет создан кластер разнородных двигателей.

Эти абстракции можно применить для определения функции `update!`, которая берет на себя истощение запасов ракетного топлива при симулировании полета ракеты. Симуляция осуществляется путем выполнения малых временных шагов Δt .

Эту стратегию принято использовать при написании симуляций реально-временных систем. При симулировании чего-то сложного, состоящего из многих частей, слишком усложняется выполнение аналитического решения путем решения одного математического уравнения. Видеоигры тоже делаются подобным образом. Каждый перемещающийся в игре объект будет получать функцию `update!`, аналогичную той, которая показана в следующем ниже листинге.

Листинг 8.16 Обновление массы ракетного топлива по истечении времени Δt

```

function update!(r::StagedRocket, t::Number, Δt::Number)
    mflow = mass_flow(thrust(r), Isp(r.engine))
    consume!(r.tank, mflow * Δt)
end

# Полезная нагрузка не имеет баков с ракетным топливом
update!(r::Payload, t::Number, Δt::Number) = nothing

```

Скажем, вы хотите сделать трехступенчатую ракету Electron. Третья ступень крошечная и, следовательно, нуждается только в очень малом двигателе. Для этой цели разрабатывающая ракету Electron

компания создает крошечный двигатель под названием Curie. Полные спецификации указанного двигателя еще неизвестны, поэтому этот двигатель будет определен на основе некоторых догадок.

Листинг 8.17 Определение крошечного двигателя для третьей ступени

```
struct Curie <: Engine end
```

```
mass(::Curie) = 8.0      ❶
thrust(::Curie) = 120.0  ❷
Isp(::Curie) = 317.0    ❶
```

- ❶ Масса и удельный импульс (Isp) должны быть угаданы.
- ❷ Единственная известная спецификация.

Теперь имеется достаточно функциональности, чтобы определить ракету Electron, состоящую из нескольких ступеней:

```
julia> payload = Payload(300)
Payload(300.0)

julia> thirdstage = Rocket(payload, SmallTank(), Curie())
StagedRocket(Payload(300.0), SmallTank(370.0), Curie())

julia> secondstage = Rocket(thirdstage, MediumTank(), Rutherford())
StagedRocket(StagedRocket(Payload(300.0),
                          SmallTank(370.0),
                          Curie()),
             MediumTank(2050.0),
             Rutherford())

julia> booster = Rocket(secondstage, LargeTank(),
                        Cluster(Rutherford(), 9))
StagedRocket(StagedRocket(StagedRocket(
    Payload(300.0),
    SmallTank(370.0),
    Curie()),
    MediumTank(2050.0),
    Rutherford()),
    LargeTank(9250.0),
    Cluster(Rutherford(), 9))
```

Позже, при выполнении физических расчетов, целесообразно абстрагироваться от того, как свойство, такое как масса, определяется для ракеты или части ракеты.

Листинг 8.18 Расчет общей массы многоступенчатой ракеты

```
mass(payload::Payload) = payload.mass

function mass(r::StagedRocket)
    mass(r.nextstage) + mass(r.tank) + mass(r.engine)
end
```

Выгоды от абстракции массы можно увидеть в том, как определяется `mass(r::StagedRocket)`. После реализации этой функции не нужно беспокоиться о деталях полезной нагрузки (`nextstage`). Это может быть полезная нагрузка или другая многоступенчатая ракета с 20 ступенями. Знать не обязательно; различия абстрагируются.

Точно так же не нужно беспокоиться о том, получаете ли вы массу одного двигателя или кластера двигателей. Представьте, что вы реализовали эту функцию до создания типа `Cluster`. Вам не нужно менять эту реализацию, потому что до тех пор, пока `Cluster` имеет тип `Engine` и реализует `mass`, все работает.

8.8 Запуск ракеты в космос

Функция `update!` позволяет отслеживать расход топлива. При вызове она дает оставшееся топливо. Когда эта величина достигает нуля, будет понятно, что ракета не может лететь намного выше, так как скорость будет неуклонно снижаться до тех пор, пока не станет отрицательной. Мы реализуем новую функцию, `launch!`, вычисляющую высоту, которую ракета достигнет до того, как у нее закончится топливо, а также истекшее время в момент, когда это произойдет.

Ракета выталкивается вверх с определенной силой, определяемой тягой двигателей T . Однако эта сила должна работать против сил земного притяжения – гравитации. Гравитация действует на ракету с силой, пропорциональной массе m ракеты и ускорению g земного притяжения:

$$F = T - mg.$$

Отсюда можно определить ускорение ракеты при ее пуске:

$$a = \frac{F}{m} \iff a = \frac{T - mg}{m}.$$

Во время обсуждения функции `update!` я отметил, что пуск симулируется как набор малых приращений времени Δt . Для одного малого приращения времени можно найти величину изменения скорости за это приращение (рис. 8.4).

Эти наблюдения являются основой для реализации функции `launch!`. Изменения скорости Δv суммируются по всем малым приращениям времени. При этом принято допущение, что по каждому приращению скорость остается постоянной, и это используется для расчета пройденного расстояния $\Delta h = v \times \Delta t$. Просуммировав все расстояния, пройденные за каждый малый шаг, получится общее пройденное расстояние, h .

Этот расчет можно визуализировать с помощью графика. Каждый столбик равен Δh , потому что каждый столбик имеет ширину

Δt и высоту v . Таким образом, сумма площадей всех столбиков равна пройденному расстоянию. Чем меньше делаются приращения Δt , тем точнее будет расчет:

$$\Delta v = a \Delta t.$$

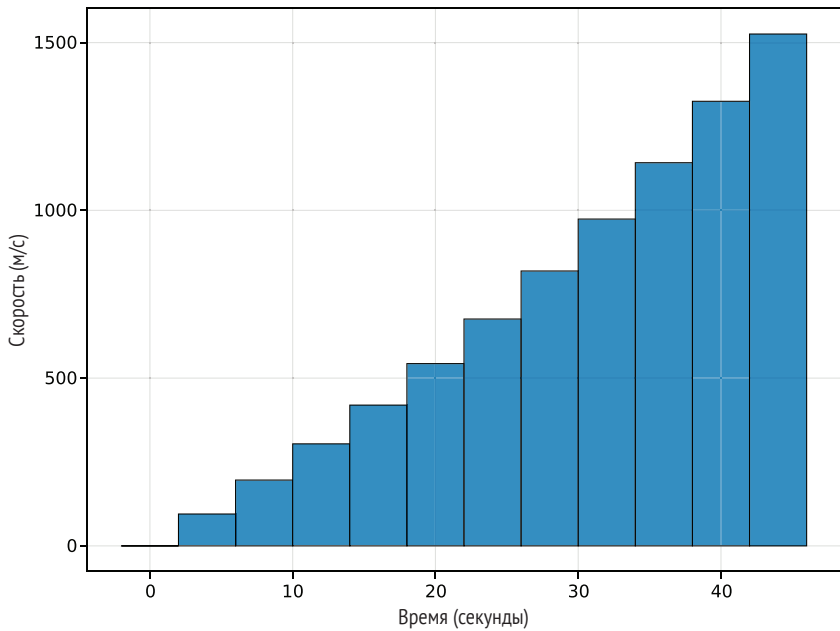


Рис. 8.4 Скорость ракеты после пуска. Ось x показывает число секунд после пуска

В математических терминах здесь выполняется аппроксимация интеграла скорости. Исходный код в листинге 8.19 является реализацией этих идей.

ПРИМЕЧАНИЕ В исходном коде используется точка отсечения в 1000 секунд. Если используются большие топливные баки, то ракета вполне может не израсходовать все топливо за указанное время. Вместо этого можно использовать цикл `while`, но тогда возникает риск получить бесконечный цикл.

Листинг 8.19 Симулирование пуска ракеты и определение максимальной высоты полета ракеты

```
function launch!(rocket::Rocket, Δt::Real)
    g = 9.80665 # вызываемое гравитацией ускорение
    v = 0.0     # скорость
    h = 0.0     # высота
    for t in 0:Δt:1000
```

```

m = mass(rocket)
F = thrust(rocket) - m*g
remaining = update!(rocket, t, Δt)
# Топливо и тяга остались?
if remaining == 0 || F <= 0
    return (t, h)
end
h += v*Δt
a = F/m
v += a*Δt
end
end

```

- ❶ Остановить симуляцию после 1000 секунд.
- ❷ Масса изменяется из-за потребления ракетного топлива.
- ❸ Время возврата и пройденное расстояние.
- ❹ Использовать скорость из предыдущего приращения времени.

Я вставил функцию `launch!` в файл с именем `simulation.jl`, но вы можете просто вставить ее в интерактивную среду REPL. Затем построить ракету и ее запустить:

```

julia> engine = Rutherford();
julia> tank = SmallTank();
julia> payload = Payload(300);
julia> rocket = Rocket(payload, tank, engine)

julia> launch!(rocket, 0.5)
(45.0, 31117.8036364191)

```

Из результата видно, что для того чтобы подняться на высоту около 31 километра, ракете потребовалось 45 секунд. Для настоящих ракет это будет иначе, потому что им приходится иметь дело с сопротивлением воздуха. По сути, мы запустили эту ракету на Земле без атмосферы. Ниже показано, что все топливо в баке израсходовано:

```

julia> tank
SmallTank(0.0)

```

В качестве упражнения можно попробовать запустить ракету с разной полезной нагрузкой и топливными баками. Заметили ли вы, что большие топливные баки не всегда уносят ракету дальше? Это связано с тем, что они увеличивают земное притяжение ракеты. Таким образом, сила притяжения может оказаться сильнее, чем сила тяги ракетного двигателя.

Помимо неучета сопротивления воздуха, этот симулированный ракетный пуск имеет ряд ограничений. Симуляция также не может справляться с многоступенчатыми ракетами.

Резюме

- В Julia можно определять свои собственные конкретно-прикладные конструкторы, чтобы обеспечивать инициализацию объектов допустимыми значениями.
- Внутренний конструктор заменяет конструктор, предоставляемый языком Julia по умолчанию. Внешние конструкторы – это просто удобные конструкторы, определяемые вне определения составного типа.
- Абстрактный тип определяется, например, с помощью конструкции `abstract type Payload end`. Абстрактные типы не имеют полей, и из них нельзя создавать объекты (нельзя создавать экземпляры абстрактного типа).
- Абстрактный и конкретный типы могут быть подтипами другого абстрактного типа. Однако ни один конкретный тип не может быть подтипом другого конкретного типа. Оператор `<:` является оператором подтипа.
- Комбинирование абстрактных типов с множественной диспетчеризацией позволяет абстрагироваться от различий между родственными типами, чтобы иметь возможность их использовать взаимозаменяемо.

Конвертация и приведение типов

Эта глава охватывает следующие ниже темы:

- конвертация одного родственного типа в другой;
- как приведение типов находит наименьший общий знаменатель среди родственных типов в выражении;
- обследование стандартной библиотеки Julia с помощью макрокоманды `@edit`.

Julia и другие мейнстримные языки программирования столь легко обрабатывают арифметические операции с использованием разных типов чисел, что большинство из нас, вероятно, не обращают особого внимания на этот факт:

```
julia> 3 + 4.2 ❶  
7.2
```

```
julia> UInt8(5) + Int128(3e6) ❷  
3000005
```

```
julia> 1//4 + 0.25  
0.5
```

❶ Числа в выражении конвертированы в числа с плавающей точкой.

❷ Все целые числа конвертированы в `Int128`.

На самом деле конвертация предусматривает много сложной работы. Под капотом в большинстве языков программирования определен набор правил приведения типов, которые устанавливают, что следует делать, если комбинируются числа разных типов. Правила приведения типов обеспечивают конвертацию всех чисел в разумный общий числовой тип, который можно использовать в окончательных расчетах. Конвертацию чисел не следует путать с синтаксическим разбором строковых литералов, служащим для извлечения чисел.

Возможно, вы задааетесь вопросом, почему важно разбираться в этих понятиях. Освоение системы приведения типов и конвертации в языке Julia открывает путь к более глубокому пониманию принципа работы чисел в Julia. Это наделит вас способностью выполнять широкий спектр задач, таких как следующие:

- определять конкретно-прикладные типы чисел;
- определять системы физических единиц и выполнять конвертации между разными единицами.

Главный пример программирования в этой главе будет предназначен именно для этого: для определения системы единиц измерения для разных типов углов, таких как градусы и радианы. Затем будет показано, как система приведения и конвертации типов языка Julia комбинирует разные типы углов в одном выражении.

Но зачем создавать числа с единицами измерения? Почему бы не дать числам просто быть числами? Потому что при разработке программного обеспечения из-за путаницы между футами, метрами и другими единицами измерения случается много ошибок. Благодаря числам с единицами измерения уменьшается число ошибок в программном обеспечении.

В Julia во многих случаях порядок следования аргументов не имеет значения. Например, если нужно проверить, не накладываются ли два геометрических объекта друг на друга, то вызов `overlap(circle, triangle)` должен дать тот же результат, что и вызов `overlap(triangle, circle)`. Подобную ситуацию можно представить в симуляции боя между разными типами воинов в видеоигре. Система приведения типов языка Julia предлагает элегантную технику обработки таких случаев без повторной реализации одних и тех же алгоритмов.

9.1 Обследование системы приведения чисел

Внутри микропроцессора математические операции всегда выполняются между одинаковыми типами чисел. У микропроцессора нет инструкции для прибавления целого числа к числу с плавающей точкой.

Таким образом, при работе с выражениями, состоящими из разных числовых типов, все языки программирования более высокого уровня должны конвертировать все аргументы выражения в один и тот же числовой тип. Но каким должен быть этот общий

числовой тип? Выяснение этого общего для всех типа и есть суть приведения типов¹.

Оно выражается как приведение аргументов математических операторов к общему типу. В большинстве мейнстримных языков механизмы и правила, управляющие приведением чисел, жестко встроены в язык и подробно описаны в спецификациях языка.

ПРИМЕЧАНИЕ В англоязычной документации других языков программирования используется так называемое *type coercion*. По форме данный термин тоже переводится как «приведение типов». Но по сути там приведение типов – это неявная конвертация, выполняемая компилятором. В отличие от них, компилятор Julia этого не делает.

В языке Julia принят совершенно иной подход: числа в нем являются первоклассными объектами. Они не являются специальными типами с уникальными запрограммированными правилами. Правила приведения типов в Julia определены в стандартной библиотеке, а не во внутренних компонентах JIT-компилятора Julia. За счет этого разработчик получает возможность расширять систему конвертации и приведения типов самостоятельно. Появляется возможность добавлять новые типы чисел, а также новые правила.

Но не увеличивает ли это риск того, что разработчики испортят систему типов чисел? Нет, потому что разработчик лишь *расширяет* существующую систему; он ее не *видоизменяет*.

Правила приведения обрабатываются традиционными функциями Julia. Подключение к существующей системе состоит в простом добавлении собственных методов в существующие функции. При этом с помощью макрокоманды `@edit` можно обследовать исходный код Julia на предмет приведения типов.

Настройка переменной среды Julia

Для работы макрокоманды `@edit` необходимо задать средовую переменную `JULIA_EDITOR`. Ее значение будет зависеть от операционной системы. Я, например, использую оболочку `fish`. Для этого требуется изменить конфигурацию запуска, `$HOME/.config/fish/config.fish`, добавив следующую ниже строку:

```
set -x JULIA_EDITOR mate
```

Если вы используете оболочку `bash`, то вместо этого необходимо видоизменить файл `$HOME/.profile`:

```
export JULIA_EDITOR=mate
```

¹ В языке Julia эта операция называется *type promotion* (повышение уровня типов). – Прим. перев.

Оба примера работают в macOS и Linux. Для видоизменения среды переменной JULIA_EDITOR пользователи Windows могут использовать диалоговое окно. Кроме того, пользователи Windows могут установить оболочку Unix.

В следующем ниже фрагменте исходного кода складываются целое число и число с плавающей точкой. Если присоединить макроккоманду @edit в качестве префикса, то Julia перейдет к определению вызываемой для обработки этого выражения функции, что позволит взглянуть на исходный код.

```
julia> @edit 2 + 3.5
```

Повсюду одни функции!

Следует всегда помнить, что в Julia почти все является вызовом функции. Арифметическое выражение $3 + 5$ является синтаксическим сахаром для вызова функции с именем `+`, например `+(3, 5)`. Каждая функция, в которой используются символы, такие как `+`, `-`, `*` и т. д., поддерживает ее использование в префиксной форме.

В следующем ниже исходном коде показано, как в Julia каждая арифметическая операция на некотором числе (типа `Number`) сперва вызывает функцию `promote` и только потом выполняет фактическую арифметическую операцию.

Листинг 9.1 Определение арифметических операций на числах в стандартной библиотеке Julia

```
+(x::Number, y::Number) = +(promote(x,y)...)
*(x::Number, y::Number) = *(promote(x,y)...)
-(x::Number, y::Number) = -(promote(x,y)...)
/(x::Number, y::Number) = /(promote(x,y)...)

```

Трехточечный оператор¹ ... используется для превращения (по сути, деструктуризации) массивов или кортежей в аргументы функции. Это означает, что `foo([4, 5, 8]...)` совпадает с `foo(4, 5, 8)`. Кроме того, его можно использовать для превращения возвращаемого функцией `promote` кортежа в аргументы различных арифметических функций, включая `+`, `-`, `*` и т. д. Для того чтобы лучше понять принцип работы функции `promote`, можно немного поэкспериментировать в интерактивной среде REPL:

```
julia> promote(2, 3.5)
(2.0, 3.5)
```

¹ Англ. *splat operator*. – Прим. перев.

```
julia> typeof(1//2), typeof(42.5), typeof(false), typeof(0x5)
(Rational{Int64}, Float64, Bool, UInt8)

julia> values = promote(1//2, 42.5, false, 0x5)
(0.5, 42.5, 0.0, 5.0)

julia> map(typeof, values)
(Float64, Float64, Float64, Float64)
```

Эти примеры показывают, что функция `promote` возвращает кортеж чисел, конвертированных в наиболее подходящий общий тип. Однако в повседневном программировании вместо этого можно использовать функцию `typeof`, чтобы выяснять, к какому типу будет приведено выражение.

9.2 Понимание конвертации чисел

Конвертация чисел означает конвертирование одного числового типа в другой. Ее не следует путать с синтаксическим разбором строкового литерала. Например, в результате синтаксического разбора из строкового литерала можно получить число, но тип `String` и тип чисел никак не связаны между собой; и поэтому его нельзя называть конвертацией.

Рекомендуемый и самый простой способ конвертации чисел в Julia состоит в использовании конструктора того типа, в который требуется конвертировать. Так, если дано значение `x` и его требуется конвертировать в некоторый тип `T`, то надо просто написать `T(x)`. Приведу несколько примеров:

```
julia> x = Int8(32)
32

julia> typeof(x)
Int8

julia> Int8(4.0)
4

julia> Float64(1//2)
0.5

julia> Float32(24)
24.0f0
```

Имейте в виду, что конвертация не всегда возможна:

```
julia> Int8(1000)
ERROR: InexactError: trunc{Int8, 1000}

julia> Int64(4.5)
ERROR: InexactError: Int64{4.5}
```

8-битовое число не может содержать значения больше $255 (2^8 - 1)$, а целые числа не могут представлять десятичные дроби.

Во многих случаях в Julia конвертация выполняется в неявной форме. Для этого используется функция `convert`, а не конструктор. Однако вызывать функцию `convert` в явной форме невозможно. Вот иллюстрация:

```
julia> convert{Int64}(5.0) ❶  
5
```

```
julia> convert{Float64}(5) ❷  
5.0
```

```
julia> convert{UInt8}(4.0)  
0x04
```

```
julia> 1//4 + 1//4  
1//2
```

```
julia> convert{Float32}(1//4 + 1//4)  
0.5f0
```

- ❶ Конвертировать число с плавающей точкой 5.0 в целое число 5.
- ❷ Конвертировать целое число 5 в число с плавающей точкой 5.0.

В показанных выше вызовах функций `Int64`, `Float64` и т. д. обратите внимание на первый аргумент. Это объекты-типы. В Julia типы – это первоклассные объекты, то есть с ними можно обращаться так же, как с любыми другими объектами. Их можно передавать, хранить и определять методы, которые оперируют на них. Объекты-типы даже имеют тип. Типом типа `Int64` является `Type{Int64}`, а для типа `Float64` это будет `Type{Float64}`:

```
julia> 3 isa Int64 ❶  
true
```

```
julia> Int64 isa Type{Int64} ❷  
true
```

```
julia> "hi" isa String  
true
```

```
julia> String isa Type{String}  
true
```

```
julia> Int64 isa Type{String} ❸  
false
```

- ❶ Число 3 имеет тип `Int64`.
- ❷ Тип `Int64` имеет тип `Type{Int64}`.
- ❸ Тип `Int64` не является типом `Type{String}`.

Типе можно трактовать как особый вид функции. Передача этой «функции» аргумента `T` возвращает аргументный тип `T`. Формаль-

но Type – это параметрический тип, который можно параметризовать с помощью аргумента-типа T, чтобы создать конкретный тип. Не волнуйтесь, если это не имеет смысла; данная тема достаточно сложна, и она будет объяснена подробнее в следующей главе.

Функция `convert` вызывается в неявной форме при выполнении различных видов присваиваний, включая следующие:

- присваивание значения элементу массива;
- установка значения поля составного типа;
- присваивание аннотаций типов локальным переменным;
- возврат из функции с аннотацией типа.

Давайте посмотрим на несколько примеров, демонстрирующих неявную конвертацию:

```
julia> values = Int8[3, 5]
2-element Vector{Int8}:
 3
 5
```

```
julia> typeof(values[2])
Int8
```

```
julia> x = 42
42
```

```
julia> typeof(x)
Int64
```

```
julia> values[2] = x ❶
42
```

- ❶ Присваивание значения `Int64` элементу массива, определенному с типом `Int8`, в результате чего вызывается `convert(Int8, x)`.

В следующем ниже примере создается составной тип `Point` с полями `x` и `y`. Затем создается экземпляр `p` объекта `Point`, и его полю `x` присваивается 8-битовое целое число. Поскольку поле имеет тип `Float64`, происходит неявная конвертация числа.

```
julia> mutable struct Point
    x::Float64
    y::Float64
end
```

```
julia> p = Point(3.5, 6.8)
Point{Float64, Float64}(3.5, 6.8)
```

```
julia> p.x = Int8(10) ❶
10
```

- ❶ Заставляет вызвать `convert(Float64, Int8(10))`.

Ниже к функции добавляется аннотация типа с целью обеспечения того, чтобы возвращаемое значение имело определенный тип.

Если это не получается обеспечить, то делается попытка конвертации с помощью функции `convert`:

```
julia> foo(x::Int64) :: UInt8 = 2x  
foo (generic function with 1 method)
```

```
julia> y = foo(42)  
0x54
```

```
julia> typeof(y)  
UInt8
```

Далее мы подробно рассмотрим механизм выполнения конвертации и приведения типов на примере более крупного фрагмента исходного кода.

9.3 Определение конкретно-прикладных единиц измерения углов

В общем и целом если перепутать единицы измерения, то расчеты в науке могут легко пойти не так. Например, в нефтяной промышленности легко спутать футы и метры, потому что координаты нефтяной скважины обычно задаются в метрах, а глубина скважины – в футах.

Широко известным примером является запущенный НАСА автоматический космический зонд под названием Mars Climate Orbiter (<http://mng.bz/m2l8>)¹, который был потерян из-за того, что НАСА и Локхид использовали разные единицы измерения. НАСА использовало метрические единицы, а Локхид использовала единицы измерения, общепринятые в США, такие как футы и фунты. Как видим, есть конкретные выгоды от разработки исходного кода, в котором единицы измерения не будут случайно перепутаны.

В примере этого раздела мы продемонстрируем работу с разными единицами измерения углов. В математике углы обычно задаются в радианах, тогда как ориентирующиеся по картам люди склонны использовать градусы. При использовании градусов круг делится на 360 градусов, вследствие чего один градус составляет $1/360$ длины окружности этого круга.

С радианами, напротив, мы имеем дело с количеством раз, которое радиус круга дублируется по окружности, чтобы получить этот угол (рис. 9.1), вследствие чего 1 радиан – это угол, который получится, если отметить расстояние по окружности, равное радиусу круга.

Градусы, по сравнению с радианами, более тесно связаны с навигацией, в частности с астронавигацией. Каждый день Земля переме-

¹ В переводе: орбитальный климатический спутник Марса. – Прим. перев.

щается примерно на 1 градус вокруг Солнца, так как год составляет 365 дней. Угол далее делится на 60 угловых минут, а угловая минута делится на 60 угловых секунд.



Рис. 9.1 Определение радиана

На самом деле можно работать как с метрическими градусами, так и с градусами, минутами, секундами (DMS)¹, но здесь, ради интереса, мы будем работать с DMS.



Рис. 9.2 Как подразделяются градусы, угловые минуты (аркминуты) и угловые секунды (арксекунды)

В этом примере исходного кода будет реализована следующая ниже функциональность:

- типы `Radian` и `DMS` для представления разных типов единиц измерения углов;
- конструкторы, облегчающие создание объектов-углов с заданными градусами, угловыми минутами и угловыми секундами;
- операции на типах углов, такие как сложение и вычитание;
- методы доступа для извлечения градусов, минут и секунд;
- расширение функции `show` для создания красивого отображения разных единиц измерения углов;
- расширение функции `convert` для поддержки конвертации из одной единицы измерения углов в другую;

¹ От англ. degrees, minutes, seconds. – Прим. перев.

- переопределение функций `sin` и `cos` для работы только с единицами измерения углов;
- сочетание нескольких приемов для создания красивых числовых литералов для единиц измерения углов;
- расширение функции `promotion_rule`, чтобы использовать разные единицы измерения углов в одном выражении.

Начнем с реализации типов углов `Radian` и `DMS`.

Листинг 9.2 Определение радиан и градусов как подтипов абстрактного типа `Angle`

```
abstract type Angle end

struct Radian <: Angle
    radians::Float64
end

# Градусы, минуты, секунды (DMS)
struct DMS <: Angle
    seconds::Int
end
```

Как и в примере с ракетой, мы определили абстрактный тип `Angle`, подтипами которого являются все конкретные единицы измерения углов. Польза от этого станет ясна позже.

9.3.1 Определение конструкторов углов

То, что DMS-значения хранятся в секундах, следует рассматривать как деталь реализации и не раскрывать пользователю. Следовательно, пользователи не должны использовать этот конструктор напрямую. Вместо этого мы определим более естественные конструкторы, как показано ниже.

Листинг 9.3 Конструкторы углов для градусов, минут и секунд

```
Degree(degrees::Integer) = Minute(degrees * 60)

Degree(deg::Integer, min::Integer) = Degree(deg) + Minute(min)

function Degree(deg::Integer, min::Integer, secs::Integer)
    Degree(deg, min) + Second(secs)
end

function Minute(minutes::Integer)
    DMS(minutes * 60)
end

function Second(seconds::Integer)
    DMS(seconds)
end
```


9.3.2 Определение арифметических операций на углах

Для того чтобы иметь возможность выполнять эти конструкции, нужно уметь складывать числа DMS. Фрагмент исходного кода `Degree(deg) + Minute(min)`, в сущности, делает `DMS(deg, 0, 0) + DMS(0, min, 0)`. Однако оператор `+` для типов DMS не был определен. Он также не был определен для радианов, поэтому в следующем ниже листинге давайте сделаем и то, и другое.

Листинг 9.4 Арифметика для углов в DMS и радианах

```
import Base: -, +

+(θ::DMS, α::DMS) = DMS(θ.seconds + α.seconds)
-(θ::DMS, α::DMS) = DMS(θ.seconds - α.seconds)

+(θ::Radian, α::Radian) = Radian(θ.radians + α.radians)
-(θ::Radian, α::Radian) = Radian(θ.radians - α.radians)
```

Поясню, как это работает. Как обсуждалось в разделе 7.3.3, определение метода в Julia создает функцию автоматически, если соответствующая функция еще не существует. Например, если функция `+` не импортирована, то при определении методов `+` Julia не будет знать, что она уже существует. Таким образом, Julia создаст совершенно новую функцию `+` и прикрепит к ней методы, специфичные для угла.

МОДУЛИ MAIN И BASE Все типы и функции Julia принадлежат модулю. Модуль можно рассматривать как пространство имен или библиотеку. Большая часть функциональности, которая поставляется в комплекте с Julia, находится в модуле под названием `Base`. Ранее вы использовали модуль `Statistics`. Функции и типы, которые не были сделаны частью именovanного модуля в явной форме, становятся частью модуля `Main`. Каждая функция и тип, которые создаются в интерактивной среде REPL, являются частью модуля `Main`.

Если затем попытаться вычислить `3 + 4`, то Julia попытается найти соответствующие методы для этой только что определенной функции `+`. Но у функции нет методов для работы с обычными числами, только для углов. Таким образом, если бы вы забыли написать `import Base: +`, то получили бы вот это сообщение об ошибке:

```
julia> 3 + 4
ERROR: MethodError: no method matching +(::Int64, ::Int64)
You may have intended to import Base.+
```

По сути, определенная в `Base` функция `+` и прикрепленные к ней методы в конечном итоге затеняются. Выполняя импорт, вы, по сути, сообщаете Julia, что хотите добавить методы к функции, определенной в существующем модуле, к примеру такому, как `Base`. Если это-

го не сделать, то только что определенная функция + станет частью модуля Main. Все, что определено в интерактивной среде REPL и не было импортировано откуда-то еще, становится частью модуля Main.

Рисунок 9.3 иллюстрирует проблему затенения. Список функций A – это то, что мы получаем, если не импортируем + и - из Base перед добавлением методов к этим функциям. В результате получаются две отдельные таблицы методов для каждого оператора: одна для Base, а другая для Main. В примере B + и - были импортированы, вследствие чего методы добавляются в таблицу методов, определенную модулем Base, а не в новую таблицу методов в создаваемом модуле Main.

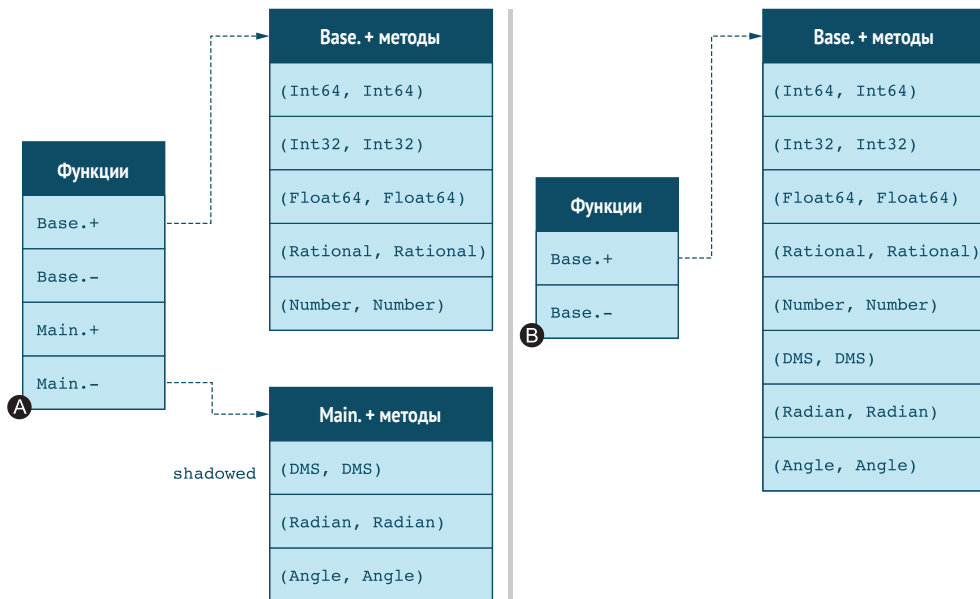


Рис. 9.3 Затенение функций, возникающее при импортировании функций перед их расширением

9.3.3 Определение функций доступа для извлечения градусов, минут и секунд

В следующем ниже листинге давайте найдем градусы, минуты и секунды при наличии угла в DMS.

Листинг 9.5 Функции доступа к градусам и минутам для объектов DMS

```
function degrees(dms::DMS)
    minutes = dms.seconds ÷ 60
    minutes ÷ 60
end
```

```
function minutes(dms::DMS)
    minutes = dms.seconds ÷ 60
    minutes % 60
end

seconds(dms::DMS) = dms.seconds % 60
```

Эти функции можно использовать для адаптированного отображения этих углов в интерактивной среде REPL. Для того чтобы написать, что угол составляет 90 градусов, 30 угловых минут и 45 угловых секунд, необходимо использовать обозначение $90^{\circ} 30' 45''$.

9.3.4 Отображение углов DMS на экране

Если применить конструкторы сейчас, то полученное стандартное отображение будет не очень хорошим. Оно показывает внутреннее представление градусов DMS в угловых секундах:

```
julia> α = Degree(90, 30, 45)
DMS(325845)

julia> degrees(α)
90

julia> minutes(α)
30

julia> seconds(α)
45

julia> β = Degree(90, 30) + Degree(90, 30)
DMS(651600)

julia> degrees(β)
181

julia> minutes(β)
0
```

Добавив метод в функцию Julia `show`, можно определить альтернативное представление. В интерактивной среде REPL вызов `show(io::IO, data)` используется для вывода пользователю данных определенного типа. Напомню, что в Julia можно определять методы для работы на обобщенных абстрактных типах; однако также можно добавлять методы, работающие с более конкретными типами, что и нужно было сделать в этой ситуации.

Листинг 9.6 Определение строкового представления объектов `Radian` и `DMS`

```
import Base: show

function show(io::IO, dms::DMS)
```

```

    print(io, degrees(dms), "° ", minutes(dms), "' ", seconds(dms), "'")
end

function show(io::IO, rad::Radian)
    print(io, rad.radians, "rad")
end

```

В главе 11 вы узнаете об объектах `show` и `IO` подробнее. А пока это даст вам хороший способ взглянуть на углы DMS:

```

julia> α = Degree(90, 30, 45)
90° 30' 45''

julia> β = Degree(90, 30) + Degree(90, 30)
181° 0' 0''

```

9.3.5 Определение конвертаций типов

Теперь, когда имеются все основы, с этими углами можно сделать что-то полезное. Их можно использовать с такими функциями, как `sin` и `cos`, но они принимают обычные числа в радианах. И поэтому необходимо определить конвертации, чтобы иметь возможность преобразовывать углы DMS в радианы.

Листинг 9.7 Определение методов прямой и косвенной конвертаций типов

```

import Base: convert

Radian(dms::DMS) = Radian(deg2rad(dms.seconds/3600))
Degree(rad::Radian) = DMS(floor{Int, rad2deg(rad.radians) * 3600})

convert{::Type{Radian}, dms::DMS} = Radian(dms)      ❶
convert{::Type{DMS}, rad::Radian} = DMS(rad)          ❷

```

- ❶ Конвертировать значение DMS в значение Radian.
- ❷ Конвертировать значение Radian в значение DMS.

Показанный выше листинг содержит ряд новых деталей, о которых стоит рассказать подробнее. Обратите внимание, что в определениях методов функции `convert` не указывается имя аргумента, а только его тип. Это похоже на пример с ракетой, в котором масса двигателей Резерфорда определялась как

```
mass(::Rutherford) = 35
```

Можно было бы написать `engine::Rutherford`, но какой в этом смысл? Составной тип `Rutherford` не имел полей, к которым можно было бы обращаться. Точно так же `Type{Radian}` и `Type{DMS}` не имеют полей, к которым требуется обращаться в определении `convert`. Имея эти конвертации, можно реализовать версии функций `sin`

и `cos`, которые принимают числа с единицами измерения в качестве аргументов.

Листинг 9.8 Переопределение стандартных функций `sin` и `cos` под использования типов `DMS` и `Radian`

```
sin(rad::Radian) = Base.sin(rad.radians)
cos(rad::Radian) = Base.cos(rad.radians)

sin(dms::DMS) = sin(Radian(dms))
cos(dms::DMS) = cos(Radian(dms))
```

В данном случае функции `sin` и `cos` перед созданием методов не импортируются. Это связано с потребностью затенить *настоящие* функции `sin` и `cos`, так как мы не хотим, чтобы люди вызывали эти функции, непреднамеренно используя обычные числа. Вы хотите, чтобы они использовали радианы или градусы в явной форме:

```
julia> sin(?/2)
ERROR: MethodError: no method matching sin(::Float64)
You may have intended to import Base.sin

julia> sin(90)
ERROR: MethodError: no method matching sin(::Int64)
You may have intended to import Base.sin

julia> sin(Degree(90))
1.0

julia> sin(Radian(?/2))
1.0
```

Теперь на входе в тригонометрическую функцию невозможно случайно применить угол, не указав единицу измерения углов в радианах либо градусах.

9.3.6 Создание красивых литералов углов

Все это хорошо, но выглядело бы намного лучше, если бы можно было писать `sin(90°)` вместо `sin(Degree(90))` и `sin(1.5rad)` вместо `sin(Radian(1.5))`.

На самом деле это делается легко. Обратите внимание, что Julia интерпретирует `1.5rad` как `1.5*rad`, вследствие чего, определив умножение обычных скаляров на градусы или радианы, задача будет волшебным образом решена.

Листинг 9.9 Операции и константы, позволяющие использовать красивые литералы углов

```
import Base: *, /

*(coeff::Number, dms::DMS) = DMS(coeff * dms.seconds)
```

```

*(dms::DMS, coeff::Number) = coeff * dms
/(dms::DMS, denom::Number) = DMS(dms.seconds/denom)

*(coeff::Number, rad::Radian) = Radian(coeff * rad.radians)
*(rad::Radian, coeff::Number) = coeff * rad
/(rad::Radian, denom::Number) = Radian(rad.radians/denom)

const ° = Degree(1)
const rad = Radian(1)

```

Изюминка заключена в последних двух строках. Они означают, что 90° читается в Julia как `90 * Degree(1)` и при вычислении будет получено `Degree(90)`:

```

julia> sin(90°)
1.0

julia> sin(1.5rad)
0.9974949866040544

julia> cos(30°)
0.8660254037844387

julia> cos(90°/3)
0.8660254037844387

julia> sin(3rad/2)
0.9974949866040544

```

9.3.7 Приведения типов

Простой, но трудоемкий способ добавления поддержки арифметических операций с разными единицами измерения углов означал бы определение большого числа функций со всеми возможными комбинациями. Представьте, что есть еще один тип угла: `MetricDegree`. Как показано в следующем ниже листинге, это быстро вызовет комбинаторный взрыв.

Листинг 9.10 Комбинаторный взрыв арифметических операций

```

+(α::DMS, β::Radian)           = Radian(α) + β
+(α::MetricDegree, β::DMS)      = α + MetricDegree(β)
+(α::Radian, β::MetricDegree)   = α + Radian(β)
+(α::Radian, β::DMS)            = α + Radian(β)

```

Я даже не стал показывать все комбинации. Главная мысль в том, что в результате получается много комбинаций, которыми трудно управлять. Вместо этого более оптимальным решением является определение обобщенных функций для разных единиц измерения, как показано в следующем ниже листинге.

Листинг 9.11 Упрощение арифметических операций за счет приведения типов

```
+(θ::Angle, α::Angle) = +(promote(θ, α)...)
-(θ::Angle, α::Angle) = -(promote(θ, α)...)

```

Осталась лишь одна проблема, которая заключается в том, что мы не сообщили функции `promote`, как делать приведение типов углов. Функция знает только о типах чисел. Первой догадкой о том, как добавить тип угла, было бы добавить еще один метод `promote`, но это так не работает. На самом деле `promote` выполняет свою работу, вызывая функцию `promotion_rule`. Нужно зарегистрировать свои типы, определив методы `promotion_rule` для *своих* типов.

Листинг 9.12 Определение приведения типов `Radian` и `DMS`

```
import Base: promote_rule

promote_rule(::Type{Radian}, ::Type{DMS}) = Radian

```

Эти методы необычны, так как все аргументы являются объектами-типами. Кроме того, ни одному аргументу не было дано имя, потому что объекты-типы не используются ни для чего, кроме выбора правильного метода функции `promotion_rule` множественной диспетчеризацией.

Функция `promotion_rule` на входе принимает два объекта-типа и на выходе возвращает еще один объект-тип:

```
julia> promote_rule(Int16, UInt8)
Int16

julia> promote_rule(Float64, UInt8)
Float64

julia> promote_rule(Radian, DMS)
Radian

```

Правило приведения можно сформулировать в виде следующего вопроса: при наличии двух разных типов к какому типу их все следует приводить? Теперь все части были поставлены на свое место. Мы подключились ко встроенному в Julia механизму конвертирования и приведения типов, реализовав методы для функций `convert` и `promotion_rule`:

```
julia> sin(90° + 3.14rad/2)
0.0007963267107331024

julia> cos(90° + 3.14rad/2)
-0.9999996829318346

julia> 45° + 45°
90° 0' 0''

```

```
julia> Radian(45° + 45°)
1.5707963267948966rad

julia> 45° + 3.14rad/4
1.5703981633974484rad
```

Этот пример наводит на мысль о преимуществах использования языка множественной диспетчеризации, такого как Julia. Реализовать такое поведение с помощью объектно ориентированного программирования сложнее и становится все труднее по мере того, как в смесь добавляется все больше типов. Если бы вы определили каждый угол как класс, то вам бы для каждой операции потребовалось несколько методов – по одному для каждого типа.

И с объектно ориентированным подходом еще есть более практические проблемы. Если вам когда-нибудь понадобится добавить еще одну единицу измерения углов, то для этого потребуется следующее:

- 1 добавить еще один класс с четырьмя методами для каждого оператора;
- 2 видоизменить каждый класс угла, включая базовый класс `Angle`, путем добавления версии каждого оператора, обрабатывающего новую единицу измерения углов;
- 3 добавить в каждый класс еще один конструктор, чтобы обрабатывать новую единицу измерения углов (в целях поддержки конвертации).

Очевидно, что это не масштабируется и нарушает принцип открытия-закрытия¹ в объектно ориентированном программировании. Принцип открытия-закрытия можно резюмировать в следующей фразе: открыто для расширения, закрыто для видоизменения.

Если бы единицы измерения углов предоставлялись в виде библиотеки, то вы не смогли бы расширять ее другими единицами измерения без изменения самой библиотеки. Это явно непрактично.

Julia элегантно решает эту проблему, не делая функции частью типов, вследствие чего в тип можно добавлять новые функции-конструкторы, не изменяя само определение типа. Добавление функций `convert` и `promotion_rule` не требует видоизменения библиотеки, предоставляющей типы, для которых вы пытаетесь определить правила приведения и конвертации.

Резюме

- Приведение типов обрабатывается в Julia путем определения правил приведения. Это делается посредством добавления методов в функцию `promotion_rule`.

¹ *Бертран Мейер*. Конструирование объектно ориентированного программного обеспечения (Object Oriented Software Construction, Bertrand Meyer, Prentice Hall, 1988, p. 23.).

- Конвертация значения x в тип T осуществляется двумя разными способами: $T(x)$ и `convert(T, x)`. Последний используется при работе с неявными конвертациями.
- Объект x может иметь тип T . `Type{T}` – это тип объекта-типа T . Эти знания помогут правильно добавлять методы в функцию `convert`.
- Определив свои собственные правила приведения и функции конвертации, можно добавлять новые типы чисел в числа Julia или добавлять такие единицы измерения, как градусы, метры, футы, градусы Цельсия либо Фаренгейта в числа Julia. Единицы измерения помогают делать числовой исходный код более надежным. Новые типы чисел помогают повышать точность вычислений, снижать потребность в памяти и повышать производительность.
- В целях добавления методов в функцию, определенную в другом модуле, эти функции необходимо импортировать в явной форме из модуля. Если этого не сделать, то в конечном итоге эти функции будут затенены, что обычно не дает желаемого результата.

10

Представление неизвестных значений

Эта глава охватывает следующие ниже темы:

- понимание правил использования неопределенных значений;
- представление отсутствия значения типом `Nothing`;
- работа со значениями, которые существуют, но неизвестны, с помощью типа `Missing`.

В любом языке программирования важно иметь возможность как-то регулировать отсутствие значения. Долгое время в большинстве мейнстримных языков программирования, таких как C/C++, Java, C#, Python и Ruby, имелось специальное значение под названием `null` либо `nil`, то есть то, что содержала бы переменная, если бы она не имела никакого значения. Точнее говоря, `null` либо `nil` указывает на то, что переменная не привязана к конкретному объекту.

В каких случаях это бывает полезно? В качестве примера давайте применим функцию Julia `findfirst`.

Она находит первое вхождение подстроки:

```
julia> findfirst("hello", "hello world") ①  
1:5
```

```
julia> findfirst("foo", "hello world") ②
```

① Найдена подстрока `hello`.

② Подстрока `foo` не найдена.

Но как указать, что подстроку найти невозможно? Для обозначения такой ситуации в языках Java, C#, Python и аналогичных им будут использоваться ключевые слова `null` либо `nil`. Однако недаром изобретатель этого значения, британский компьютерщик Тони Хоар (Tony Hoare), назвал указатель `null` своей ошибкой на миллиард долларов.

В силу этого затрудняется написание безопасного исходного кода, потому что в любой момент времени любая переменная может быть `null`. В программах, написанных с использованием языков с поддержкой `null`, требуется много стереотипного исходного кода, выполняющего проверки на `null`. Это связано с небезопасностью выполнения операций на объектах `null`.

По этой причине в современных языках стараются избегать объектов или указателей `null`. В языке Julia нет ни обобщенного объекта `null`, ни указателя `null`. Вместо них в нем есть различные типы, представляющие неизвестные либо отсутствующие значения. В этой главе вы поближе познакомитесь с этими типами, приемами их использования и ситуациями, когда их следует использовать.

10.1 Объект *nothing*

Самым близким к `null` значением, которое есть в Julia, является объект `nothing` типа `Nothing`. Это простой конкретный тип, определенный в Julia, как показано в следующем ниже листинге.

Листинг 10.1 Тип `Nothing` и постоянная `nothing`, как определено в Julia

```
struct Nothing
    # заметьте, полей нет
end

const nothing = Nothing()
```

Объект `nothing` является экземпляром типа `Nothing`. Однако каждый экземпляр типа `Nothing` является тем же самым объектом. В этом можно убедиться в интерактивной среде REPL:

```
julia> none = Nothing()

julia> none == nothing
true

julia> Nothing() == Nothing()
true
```

Однако *ничего* волшебного здесь не происходит. При вызове конструктора составного типа с нулевыми полями всегда будет возвращаться тот же самый объект. Сформулируем это в более формальном виде: для типа `T` без полей каждый экземпляр `t` типа `T` является

тем же самым объектом. Приведенный ниже пример должен помочь прояснить ситуацию:

```
julia> struct Empty end

julia> empty = Empty()
Empty()

julia> none = Empty()
Empty()

julia> empty == none
true

julia> Empty() == Empty()
true
```

Однако экземпляры разных пустых составных типов считаются разными. Следовательно, `Empty()` не возвращает тот же самый объект, что и `Nothing()`:

```
julia> Empty() == Nothing()
false

julia> empty = Empty()
Empty()

julia> empty == nothing
false
```

В языке Julia пустые составные типы упрощают создание объектов специального назначения, которым требуется придавать особый смысл. По общепринятому правилу в Julia объект `nothing` используется для того, чтобы указывать, что нечто невозможно найти либо не существует:

```
julia> findfirst("four", "one two three four")
15:18

julia> findfirst("four", "one two three")

julia> typeof(ans)
Nothing

julia> findfirst("four", "one two three") == nothing
true
```

10.2 Применение объекта *nothing* в структурах данных

Многоступенчатая ракета похожа на более обобщенную структуру данных, именуемую *связным списком*. Как и в примере с ракетой, не-

редко бывает удобно соединять несколько объектов в цепочку. Например, этот подход можно использовать для представления поезда, состоящего из нескольких грузовых вагонов. Следующее ниже определение не работает. Сможете определить причину?

Листинг 10.2 Определение бесконечного грузового поезда

```
struct Wagon
    cargo::Int           ❶
    next::Wagon          ❷
end

cargo(w::Wagon) = w.cargo + cargo(w.next)  ❸
```

- ❶ Тонны груза в товарном вагоне.
- ❷ Следующий вагон связан с этим.
- ❸ Подсчитать общий груз во всех вагонах.

С тем определением, которое мы дали, составить поезд из этих вагонов невозможно. Поясню на примере:

```
train = Wagon(3, Wagon(4, Wagon(1, Wagon(2, ...))))
```

Закончить эту цепочку вагонов просто невозможно. Каждому конструктору `Wagon` в качестве второго аргумента требуется объект-вагон. В целях иллюстрации бесконечной цепочки вагонов в пример исходного кода был вставлен оператор `...`. Следующим полем всегда должен быть вагон. Но что, если вместо этого сделать `Wagon` абстрактным типом? Это одно из возможных решений, которое уже использовалось в примере с многоступенчатой ракетой.

Напомню, что не у каждого подтипа типа `Rocket` было поле следующей ступени. Однако в этой главе я представлю более обобщенное решение этой задачи на основе параметрических типов. Здесь будут рассмотрены только основы, поскольку глава 18 всецело посвящена параметрическим типам.

ВАЖНО Параметрические типы, возможно, покажутся дополнительной функциональностью, представляющей интерес только для продвинутых программистов на языке Julia. Однако я намеренно свел использование параметрических типов в примерах исходного кода к минимуму. Параметрические типы находят широкое применение в реальном исходном коде Julia. Они имеют решающее значение для обеспечения правильности типов, производительности и уменьшения дублирования исходного кода.

10.2.1 Что такое параметрический тип?

Вы познакомились с параметрическими типами, когда я определял диапазоны и пары. Если `P{T}` – это параметрический тип `P`, то `T` – это

параметр-тип. Знаю, это звучит очень абстрактно, но на нескольких примерах все станет намного яснее:

```
julia> ['A', 'B', 'D']
3-element Vector{Char}: ❶
 'A'
 'B'
 'D'
```

```
julia> typeof(3:4)
UnitRange{Int64} ❷
```

```
julia> typeof(0x3//0x4)
Rational{UInt8} ❸
```

- ❶ Параметрический тип `Vector` с параметром-типом `Char`.
- ❷ Параметрический тип `UnitRange` с параметром-типом `Int64`.
- ❸ Параметрический тип `Rational` с параметром-типом `UInt8`.

Тип `Vector` можно трактовать как шаблон создания фактического типа. Для того чтобы создать реальный конкретный вектор, необходимо знать типы элементов в векторе. В первом примере параметром-типом является `Char`, поскольку каждый элемент является буквой. Для типа `UnitRange` параметр-тип представляет тип начала и конца диапазона. Для типа `Rational` параметр-тип указывает тип числителя и знаменателя дроби.

Параметры-типы относятся к параметрическим типам так же, как значения к функциям. На вход в функцию вводят значение и на выходе из нее получают значение:

$y = f(x)$

Мы вводим x в функцию f и получаем значение y . Ту же аналогию можно провести и для параметрических типов:

$S = P\{T\}$

Мы вводим тип T в P и получаем тип S . Это можно продемонстрировать на нескольких фактических типах Julia:

```
julia> IntRange = UnitRange{Int} ❶
UnitRange{Int64}
```

```
julia> FloatRange = UnitRange{Float64} ❷
UnitRange{Float64}
```

```
julia> IntRange(3, 5)
3:5
```

```
julia> FloatRange(3, 5) ❸
3.0:5.0
```

```
julia> NumPair = Pair{Int, Float32}
```

```
Pair{Int64, Float32}
```

```
julia> NumPair(3, 5)
3 => 5.0f0
```

```
julia> 3 => 5
3 => 5
```

- ❶ Создать тип-диапазон с именем `IntRange`.
- ❷ Создать тип-диапазон на основе чисел с плавающей точкой.
- ❸ Создать диапазон, используя конкретно-прикладной тип-диапазон.

Приведенный выше пример показывает, что с типами можно обращаться так же, как и с объектами. Мы создаем объекты нового типа и привязываем их к переменным типа `IntRange`, `FloatRange` и `NumPair`. Эти конкретно-прикладные типы затем используются для создания экземпляров разных типов объектов.

10.2.2 Использование типов-объединений для завершения товарного поезда

Тип `Union` является параметрическим типом. Для создания нового типа ему можно предоставить несколько параметров-типов. Типы `Union` отличаются тем, что они могут служить местозаполнителями для любого типа, перечисленного в качестве параметров-типов. Типы-объединения, как вариант, можно трактовать как способ объединения двух или более типов в один тип.

Допустим, даны типы с именами `T1`, `T2` и `T3`. В этом случае можно создать объединение этих типов, написав `Union{T1, T2, T3}`. В результате этого объединения будет создан новый тип, который может быть местозаполнителем для любого из этих типов. Это означает, что если будет написан метод с сигнатурой `f(x::Union{T1, T2, T3})`, то этот конкретный метод будет вызываться всякий раз, когда `x` имеет тип `T1`, `T2` либо `T3`. Давайте рассмотрим конкретный пример:

```
julia> f(x::Union{Int, String}) = x^3
f (generic function with 1 method)
```

```
julia> f(3)
27
```

```
julia> f(" hello ")
" hello  hello  hello "
```

```
julia> f(0.42)
ERROR: MethodError: no method matching f(::Float64)
Closest candidates are:
  f(!Matched::Union{Int64, String}) at none:1
```

Последний пример не работает, потому что `x` – это число с плавающей точкой, а мы определили метод для функции `f`, принимающей только объединение типов `Int` и `String`. Тип `Float64` не включен.

Каждый включенный в объединение типов тип будет считаться подтипом этого объединения. Оператор `<:` используется для задания подтипа либо для проверки, что некий тип является подтипом:

```
julia> String <: Union{Int64, String}
true

julia> Int64 <: Union{Int64, String}
true

julia> Float64 <: Union{Int64, String}
false

julia> Union{Int64, String} == Union{String, Int64} ❶
true
```

❶ Порядок следования параметров-типов не имеет значения.

Как показано в последнем вычисленном выражении, упорядоченность параметров-типов в определении типа `Union` не имеет значения. Вооружившись типами-объединениями, можно решить проблему с бесконечными поездами.

Листинг 10.3 Определение конечного поезда

```
struct Wagon
    cargo::Int
    next::Union{Wagon, Nothing} ❶
end

cargo(w::Wagon) = w.cargo + cargo(w.next)
cargo(w) = 0 ❷
```

- ❶ Следующий связанный вагон не может быть `nothing`.
- ❷ Значения, не являющиеся вагонами, не имеют груза.

Давайте перезагрузим интерактивную среду REPL и вставим новое определение типа. Этот исходный код позволит создать конечную цепочку вагонов. Обратите внимание, как определены два метода `cargo`. Здесь имеется два разных случая, поскольку следующим может быть либо объект `Wagon`, либо объект `nothing`:

```
julia> train = Wagon(3, Wagon(4, Wagon(1, nothing)))
Wagon(3, Wagon(4, Wagon(1, nothing)))

julia> cargo(train)
8

julia> train = Wagon(3, Wagon(4, Wagon(1, 42))) ❶
ERROR: MethodError: Cannot `convert` an object of type Int64
to an object of type Wagon
```

- ❶ Попытка использовать 42 в качестве следующего вагона.

Последний пример был включен, чтобы продемонстрировать, что благодаря определению типа `Union` атрибут `next` может быть только объектом типа `Wagon` либо типа `Nothing`. Поэтому установка атрибута `next` равным целому числу, например 42, недопустима. Это приводит к тому, что система типов Julia громко запротестует, выдав исключение.

10.3 Отсутствующие значения

В языке Julia отсутствующие значения представлены объектом `missing`, который имеет тип `Missing`. Он выглядит очень похожим на объект `nothing`, и тогда возникает вопрос: зачем он вообще нужен?

Это напрямую связано с тем, что разработчики языка Julia стремятся сделать его удобным для ученых, занимающихся научными вычислениями, статистикой, большими данными и т. д. В статистике отсутствующие данные являются важной концепцией. Это происходит постоянно, потому что почти в любой коллекции данных для статистического анализа будут отсутствовать данные. Например, может возникнуть ситуация, когда некоторые из участников статистического исследования заполняют не все поля формы.

Некоторые участники могут покинуть исследование до его завершения, оставив тех, кто проводит эксперимент, с неполными данными. Данные также могут отсутствовать из-за ошибок при их вводе. Таким образом, в отличие от концепции ничтожности (`nothing`), отсутствующие данные действительно существуют в реальном мире. Мы просто не знаем, какие они.

Специализированное программное обеспечение для статистиков, такое как R (см. <https://www.r-project.org>) и SAS (см. <https://www.sas.com>), давно установило правило, что отсутствующие данные должны распространяться, а не выдавать исключения. Это означает, что если в любой части более крупного вычисления значение будет отсутствовать, то результатом всего вычисления будет объект `missing`. Разработчики языка Julia тоже решили следовать этому правилу. Давайте посмотрим, что это означает на практике.

Листинг 10.4 Сравнение поведения объектов `missing` и `nothing` в математических выражениях

```
julia> missing < 10
missing

julia> nothing < 10
ERROR: MethodError: no method matching isless(::Nothing, ::Int64)

julia> 10 + missing
missing
```

```
julia> 10 + nothing  
ERROR: MethodError: no method matching +(::Int64, ::Nothing)
```

В листинге 10.4 видно, что в отличие от объекта `nothing` в результате вычисления каждой математической операции с отсутствующим значением получается объект `missing`, что приводит к выдаче исключений. Это объясняется тем, что в прошлом в статистической работе было допущено много серьезных ошибок, связанных с учетом отсутствующих значений. Поскольку объект `missing` в Julia распространяется как вирус, необработанное отсутствующее (`missing`) значение будет быстро обнаружено.

Отсутствие может обрабатываться в явной форме. Например, если требуется вычислить сумму или среднее значение массива, в котором могут быть пропущены значения, то во избежание попыток включить отсутствующие значения в результат можно применить функцию `skipmissing`:

```
julia> using Statistics  
  
julia> xs = [2, missing, 4, 8];  
  
julia> sum(xs) ❶  
missing  
  
julia> sum(skipmissing(xs)) ❷  
14  
  
julia> median(skipmissing(xs))  
4.0  
  
julia> mean(skipmissing(xs))  
4.666666666666667
```

- ❶ Отсутствующее значение загрязняет весь расчет, приводя к отсутствию результата.
- ❷ Отсутствующие значения пропускаются, чтобы сложить присутствующие значения.

10.4 Не число

Число с плавающей точкой `NaN`¹ имеет некоторое отношение к отсутствующим значениям. Число `NaN` получается, когда результат операции не определен. Обычно это становится проблемой при делении нуля на ноль:

```
julia> 0/0  
NaN
```

¹ Англ. not a number; перев. не число. – Прим. перев.

```
julia> 1/0  
Inf  
  
julia> -1/0  
-Inf
```

В этом случае `Inf` означает *бесконечность*¹, и это то, что получается при делении на ноль числа, отличного от нуля. В этом есть определенный смысл. При приближении делителя к нулю результат стремится к увеличению.

Довольно заманчиво трактовать `NaN` как отсутствующее (`missing`) значение и рассматривать возможность их взаимозаменяемости. Ведь `NaN` тоже распространяется по всем вычислениям.

Листинг 10.5 Распространение `NaN` в математических операциях

```
julia> NaN + 10  
NaN  
  
julia> NaN/4  
NaN  
  
julia> NaN < 10  
false  
  
julia> NaN > 10  
false
```

Однако проверки на `NaN` возвращают `false`. Следующая причина показывает, почему не следует использовать `NaN` для отсутствующих (`missing`) значений: если в алгоритме есть ошибка, которая приводит к $0/0$, то будет получаться `NaN`, что не будет отличимо от отсутствующих значений на входе.

Листинг 10.6 Невозможно отличить вычисление, приводящее к `NaN`, от входного значения, являющегося `NaN`

```
julia> calc(x) = 3x/x;  
  
julia> calc(0)  
NaN  
  
julia> calc(NaN)  
NaN
```

Как следствие можно ошибочно полагать, что алгоритм работает правильно, потому что он удаляет пропущенные значения в расчетах, и в результате будет маскироваться дефект. Например, в листинге 10.6 перед делением не делается проверка, что входное значение `x` равно нулю. Таким образом, при `x`, равном 0, результатом будет

¹ Англ. Infinity. – Прим. перев.

NaN. Если передавать NaN на вход функции `calc`, чтобы указывать отсутствующее значение, то не получится провести различие между ошибкой программиста и отсутствующим значением.

10.5 Неопределенные данные

С неопределенными данными в Julia сталкиваются редко, но о таких ситуациях стоит знать. Неопределенные данные возникают, когда значения переменной или поля структуры не заданы. Обычно Julia старается быть в этом умнее; если определена структура с числовыми полями, то Julia, в случае если ничего не предпринимается, автоматически инициализирует их нулем. Однако если структура `struct` определена, не сообщая Julia тип ее полей, то Julia не сможет угадать значения, которыми поля должны быть инициализированы.

Листинг 10.7 Определение составного типа, созданного с неопределенными значениями

```
julia> struct Person
           firstname
           lastname
           Person() = new()
       end

julia> friend = Person()
Person{#undef, #undef}

julia> friend.firstname
ERROR: UndefRefError: access to undefined reference
```

Julia позволяет создавать составные объекты с неинициализированными полями. Однако при попытке обратиться к неинициализированному полю будет вызываться исключение. От использования неинициализированных значений нет никаких выгод, но они помогают обнаруживать ошибки программиста.

10.6 Сведение всего воедино

Проводить различие между всеми этими понятиями *несуществования* бывает довольно сложно, поэтому давайте кратко подытожим различия: `nothing` – это тип значения `null`, служащий для программиста. Это то, чего хочет программист, когда чего-то не существует. `missing` – это тип значения `null`, служащий для статистиков. Это то, чего они хотят, когда в их входных данных отсутствует значение. NaN указывает на то, что где-то в исходном коде была недопустимая математическая операция. Другими словами, NaN связано с расчетами,

а не с коллекцией статистических данных. Неопределенность – это когда вы, программист, забыли сделать свою работу и инициализировать все используемые данные. Скорее всего, это указывает на дефект в программе.

В качестве последнего напоминания: в Julia *нет* `null` в обычном смысле, и поэтому возможность ничтожного (`nothing`) значения нужно допускать в явной форме с использованием объединений типов. Без этого аргумент функции не сможет передавать ничтожное значение.

Резюме

- Неизвестные значения в Julia представлены значением `nothing`, значением `missing`, значением `NaN` и неопределенным значением.
- Значение `nothing` имеет тип `Nothing` и указывает на то, что нечто не существует. Его следует использовать в качестве возвращаемого значения при сбое функции поиска или в структурах данных (например, для завершения связного списка).
- Значение `missing` – это то, что существует, но отсутствует, например, в опросе. Указанные значения имеют тип `Missing`. При реализации исходного кода, читающего статистические данные из файла, объект `missing` следует использовать в качестве местозаполнителя отсутствующих данных.
- Значение `NaN` – это результат недопустимых математических операций. Если функция возвращает `NaN`, то следует проверить, не была ли допущена ошибка программирования. Например, насколько вы уверены, что в вашем исходном коде никогда не встречается $0/0$?
- Неопределенность случается, когда переменная не была инициализирована известным значением.
- Ничтожное (`nothing`) и отсутствующее (`missing`) значения не встроены в язык, но они определяются как составные типы без каких-либо полей.
- Параметризованный тип `Union` удобно использовать с типами `Nothing` и `Missing`. Например, если поле может быть строковым литералом либо ничем, то тип следует определять как `Union{Nothing, String}`.

Часть III

Коллекции

Коллекции – это объекты, в которых хранятся и организовываются другие объекты. В этой части вы познакомитесь с уникальными функциональностями и возможностями коллекций, таких как массивы, множества и строковые литералы. Однако вы также поближе познакомитесь с общими для всех коллекций возможностями, такими как прокручивание всех элементов в цикле `for` и функции более высокого порядка, такие как `map`, `filter` и `reduce`. Коллекции были кратко рассмотрены в части I, но здесь каждая тема будет охвачена углубленно.

11

Работа со строковыми литералами

Эта глава охватывает следующие ниже темы:

- понимание взаимосвязей между Юникодом, кодовыми точками и кодировкой UTF-8;
- сравнение строковых литералов, конвертирование их в нижний регистр и выполнение других строковых операций;
- когда и как использовать неформатированные строковые литералы;
- изучение различных типов строковых литералов, включая регулярные выражения, MIME-типы и литералы BigInt.

В предыдущих главах вы уже имели небольшой практический опыт работы со строковыми литералами; однако в этой главе я изложу еще больше подробностей, которые помогут правильно их использовать. Здесь вы обследуете эти детали более пристально. Пока вы работаете с буквами от A до Z, все будет просто. Однако в мире существует многообразие языков, каждый со своим уникальным набором символов, и с ними необходимо как-то уметь обращаться.

Это означает, что минимальные знания, необходимые для эффективной работы со строковыми литералами на языке Julia, предусматривают определенное знакомство с *Юникодом*. Юникод – это международный стандарт соотнесения чисел (кодовых точек или позиций кода) с буквенными символами.

Julia также поддерживает специальные строковые литералы, помогающие выполнять различные задачи. Например, существуют специальные строковые литералы, именуемые *регулярными выражениями*, которые позволяют проверять на совпадение текстовой строки с определенным шаблоном, таким как адрес электронной почты, IP-адрес или почтовый индекс.

11.1 UTF-8 и Юникод

Строковые литералы в Julia – это Юникод, кодированный в формате UTF-8. Но что это значит и почему это должно волновать? Я покажу простой пример, иллюстрирующий важность более глубокого понимания Юникода.

Æser – это форма множественного числа словосочетания *norse gods* (скандинавские боги) на норвежском языке. Это четырехбуквенное слово, что подтверждается функцией `length`:

```
julia> length("Æser")
4
```

Но при попытке доступа к отдельным символам в слове вы заметите кое-что странное:

```
julia> "Æser"[1] ❶
'Æ': Unicode U+00C6 (category Lu: Letter, uppercase)

julia> "Æser"[2] ❷
ERROR: StringIndexError("Æser", 2)

julia> "Æser"[3] ❸
's': ASCII/Unicode U+0073 (category Ll: Letter, lowercase)

julia> "Æser"[4]
'e': ASCII/Unicode U+0065 (category Ll: Letter, lowercase)
```

- ❶ Работает как положено.
- ❷ Выдается исключение. Попытка обратиться к символу `s` по индексу 2, по-видимому, не работает.
- ❸ Вместо этого второй символ находится в индексе 3. Не кажется ли это странным?

Как насчет другого слова? *Þrúðvangr* – это название царства скандинавского бога Тора:

```
julia> length("Þrúðvangr")
9

julia> "Þrúðvangr"[9]
'a': ASCII/Unicode U+0061 (category Ll: Letter, lowercase)
```

Это девятибуквенное слово, но по индексу 9 находится шестая буква `'a'`. Что здесь вообще происходит? Для того чтобы это понять,

необходимо понимать Юникод и то, как строковые литералы Julia его поддерживают посредством кодировки UTF-8:

```
julia> sizeof("Æser")
5

julia> sizeof("Þrúðvangr")
12
```

В UTF-8 каждый символ кодируется в один до четырех байт. Обычные буквы, такие как А, В и С, занимают всего 1 байт, а для кодирования, к примеру, таких букв, как Æ, Þ и ð, которые не используются в английском языке, обычно требуется более 1 байта. Однако прежде чем углубиться в механизм работы с данной кодировкой на языке Julia, полезно понять несколько ключевых концепций Юникода, которые не являются специфическими для языка программирования Julia.

11.1.1 Понимание взаимосвязи между кодовыми точками и кодовыми единицами

Юникод и кодировки символов – сложные темы, с которыми многим разработчикам рано или поздно придется столкнуться. Представление истории существования Юникода и UTF-8 обеспечит важный контекст для понимания.

Оба стандарта произошли от более старого стандарта ASCII, в котором каждый символ кодировался в 8 битах. Числа от 65 до 90 кодировали буквы от А до Z, а числа от 97 до 122 – строчные буквы от а до z. Взаимосвязь между ASCII-кодами и соответствующими им символами можно обследовать в интерактивной среде REPL, используя конструктор для типа Char:

```
julia> Char(65)
'A': ASCII/Unicode U+0041

julia> Char(90)
'Z': ASCII/Unicode U+005A
```

Для того чтобы работать с разными языками, нужно было оперировать разными интерпретациями этих чисел от 1 до 255. Однако это быстро стало непрактичным. Например, текст, написанный с использованием разных алфавитов, нельзя было смешивать на одной странице. Решением стал Юникод, который был ориентирован на присваивание уникального числа каждому символу в мире – не только латинскому алфавиту, но и кириллице, китайскому, тайскому и всем японским наборам символов.

В терминологии Юникода присвоенное каждому символу число называется *кодовой точкой* (https://www.unicode.org/glossary/#code_point). Первоначально считалось, что для хранения каждой кодо-

вой точки Юникода 16 бит будет достаточно. 16 бит дают $2^{16} - 1 = 65\,535$ уникальных чисел. Таким образом, в одной из первых кодировок Юникода, UCS, для кодирования каждой кодовой точки Юникода использовалось 16 бит (2 байта) (<https://www.unicode.org/glossary/#UCS>).

Позже было установлено, что этого будет недостаточно, и для кодирования всех возможных символов Юникода потребуется 4 байта (32 бита). В этот момент подход UCS-кодировки начал выглядеть ошибочным по следующим причинам:

- 1 UCS-кодировка уже была несовместима с 8-битовым ASCII-кодом;
- 2 в сумме 4 байта для каждого буквенного символа занимали много места.

UTF-8-кодировка решила эти проблемы за счет использования переменного числа байтов на символ (<https://www.cl.cam.ac.uk/~mgk25/ucs/utf-8-history.txt>). Так, часто используемые символы можно было кодировать одним байтом, экономя место. 1-байтовые символы были намеренно сделаны обратно совместимыми с ASCII.

При кодировании буквенных символов переменной длины необходимо проводить различие между кодовой точкой символа и *кодowymi единицами*, необходимыми для кодирования символа. У каждого символа Юникода есть одно число, кодовая точка, которая его идентифицирует. Кодовые единицы используются для хранения этих кодовых точек в памяти. В целях достижения такой возможности кодировка UTF-8 требует переменного числа кодовых единиц (рис. 11.1).



Рис. 11.1 Кодовые точки Юникода, кодированные в кодовые единицы UTF-8 (показаны не все индексы)

В UCS-кодировке, напротив, кодовые единицы имеют фиксированный размер. Каждая кодовая единица UCS-кодировки имеет длину 16 бит. На рис. 11.1 показано соотношение между символами,

кодowymi точками и кодowymi единицами. Каждый серый блок для кодовых точек представляет 4 байта. Для каждого символа требуется переменное число кодовых единиц. Поэтому они были сгруппированы, чтобы показывать, какие именно байты относятся к одному и тому же символу. Черные кружки обозначают байтовый индекс нескольких составляющих символы кодовых единиц. В целях прояснения этих концепций давайте проведем несколько практических экспериментов с символами Юникода в интерактивной среде REPL:

```
julia> codepoint('A') ❶
0x00000041
```

```
julia> Int(codepoint('A')) ❷
65
```

```
julia> ncodeunits('A') ❸
1
```

```
julia> isascii('A') ❹
true
```

- ❶ Получить кодовую точку буквы A в шестнадцатеричной форме.
- ❷ Кодовая точка A как десятичное число.
- ❸ Количество кодовых единиц, необходимых для кодирования кодовой точки кода для буквы A.
- ❹ Является ли эта буква частью изначального стандарта ASCII?

Давайте обследуем символы, которые не являются частью изначального стандарта ASCII. Они должны иметь более одной кодовой единицы и не возвращать true при вызове функции `isascii()`:

```
julia> codepoint('Æ')
0x000000c6
```

```
julia> ncodeunits('Æ') ❶
2
```

```
julia> isascii('Æ') ❷
false
```

```
julia> codepoint(' ') ❸
0x0001f60f
```

```
julia> ncodeunits(' ') ❹
4
```

```
julia> isascii(' ')
false
```

- ❶ Для кодирования символа Æ в стандарте UTF-8, используемом в языке Julia, требуется 2 байта.
- ❷ Æ не является частью изначального стандарта ASCII.
- ❸ Кодовая точка для смайлика.
- ❹ Для кодирования эмодзи смайлика требуется 4 байта.

Функция `isUnicode` отсутствует, поскольку каждый символ Julia является символом Юникода. Функция `isascii` является способом проверки того, что данный символ Юникода также является частью изначального стандарта ASCII.

Просто набирая буквы в интерактивной среде REPL, можно также получать полезную информацию при вычислении буквенного символа:

```
julia> 'A'
'A': ASCII/Unicode U+0041 (category Lu: Letter, uppercase)

julia> 'Æ'
'Æ': Unicode U+00C6 (category Lu: Letter, uppercase)

julia> ' '
' ': Unicode U+1F60F (category So: Symbol, other)
```

Обратите внимание на сообщения о числе кодовой точки Юникода.

СОВЕТ Для того чтобы легко писать необычные символы, которых нет на клавиатуре, можно использовать обратную косую черту `\` и клавишу **Tab**. Например, для написания ☺ надо в интерактивной среде REPL набрать `\:smirk:` и нажать **Tab**, чтобы получить завершение.

Кроме того, можно нажать **Tab** после того, как будет написано `\:`, и получить полный список возможных смайликов. Норвежские буквы, к примеру такие как *ÆØÅ*, которые я иногда использую в своих примерах, можно легко написать в Mac, просто удерживая нажатой клавишу выбора и введя символ *O*, *A* или *'* (в английской раскладке клавиатуры). В других операционных системах надо переключиться на норвежскую раскладку клавиатуры либо скопировать буквы.

Кодовые точки Юникода можно писать в явной форме различными способами:

```
julia> '\U41'
'A': ASCII/Unicode U+0041 (category Lu: Letter, uppercase)

julia> Char(0x41)
'A': ASCII/Unicode U+0041 (category Lu: Letter, uppercase)

julia> '\U00c6'
'Æ': Unicode U+00C6 (category Lu: Letter, uppercase)

julia> Char(0xc6)
'Æ': Unicode U+00C6 (category Lu: Letter, uppercase)

julia> '\U01f60f'
' ': Unicode U+1F60F (category So: Symbol, other)

julia> Char(0x01f60f)
' ': Unicode U+1F60F (category So: Symbol, other)
```

Их можно комбинировать с функцией `map`, чтобы создавать различные диапазоны. Например, диапазон необязательно записывать просто в виде чисел. Например, диапазон `'A':'F'` является вполне допустимым:

```
julia> map(lowercase, 'A':'F')
6-element Vector{Char}:
 'a'
 'b'
 'c'
 'd'
 'e'
 'f'
```

```
julia> map(codepoint, 'A':'F')
6-element Vector{UInt32}:
 0x00000041
 0x00000042
 0x00000043
 0x00000044
 0x00000045
 0x00000046
```

И конечно же, можно пойти и от обратного:

```
julia> map(Char, 65:70)
6-element Vector{Char}:
 'A'
 'B'
 'C'
 'D'
 'E'
 'F'
```

Учитывая, что число символа и его индекс не совсем коррелируют, при работе со строковыми литералами и индексами необходимо соблюдать осторожность. Следует использовать функции `lastindex` и `nextind`, которые работают не только для строковых литералов, как показано в следующем ниже примере:

```
julia> xs = [4, 5, 3]

julia> i = firstindex(xs) ❶
1

julia> while i <= lastindex(xs) ❷
    println((i,xs[i]))
    i = nextind(xs, i) ❸
end

(1, 4)
(2, 5)
(3, 3)
```

- ❶ Получить индекс первого элемента. Таким образом, не нужно исходить из допущения, что первый элемент находится в индексе 1. Массивы Julia можно создавать с разными стартовыми индексами.
- ❷ Проверить, не был ли достигнут последний индекс.
- ❸ Находит индекс элемента, следующего за индексом *i* в массиве *xs*.

В следующем ниже листинге показано, что если делать это со строковым литералом, то логика будет полностью такой же.

Листинг 11.1 Итеративное прокручивание символов в строковом литерале с использованием цикла `while`

```
julia> s = "Þrúðvangar"

julia> i = firstindex(s)
1

julia> while i <= lastindex(s)
    println((i, s[i]))
    i = nextind(s, i)
end
(1, 'Þ')
(3, 'r')
(4, 'ú')
(6, 'ð')
(8, 'v')
(9, 'a')
(10, 'n')
(11, 'g')
(12, 'a')
(13, 'r')
```

- ❶ Обратите внимание, что, переходя от *Þ* к *r*, делается переход от индекса 1 к индексу 3.
- ❷ Для перехода от *ð* к *v* требуется пропуск индекса 7.

Используя эти функции, мы абстрагируемся от разницы между строковыми литералами и обычными массивами. Это видно по тому, как, например, функция `nextind` реализована в стандартной библиотеке. Для массивов это просто приращение:

```
nextind(::AbstractArray, i::Integer) = Int(i)+1
```

Для строковых литералов это гораздо более сложная операция. Я покажу только части реализации, отредактированные для ясности.

Листинг 11.2 Фрагмент базовой реализации функции `nextind` для строковых литералов

```
function nextind(s::String, i::Int)
    i == 0 && return 1
    n = ncodeunits(s)
```

```

between(i, 1, n) || throw(BoundsError(s, i))
l = codeunit(s, i)
(l < 0x80) | (0xf8 <= l) && return i+1
if l < 0xc0
    i' = thisind(s, i)
    return i' < i ? nextind(s, i') : i+1
end
end
...
end

```

В обычном исходном коде не нужно обязательно работать с функциями `nextind` и `lastindex`. Вместо них используются циклы `for`, так как на каждой итерации они будут автоматически извлекать весь символ:

```

julia> for ch in "brúðvangar"
    print(ch, " ")
end
brúðvangar

```

Если требуются индексы каждого символа, то следует использовать функцию `eachindex`:

```

julia> for i in eachindex(s)
    print(i, s[i], " ")
end
1b 3r 4ú 6ð 8v 9a 10n 11g 12a 13r

```

11.2 Операции на строковых литералах

Работа с текстом настолько распространена, что следует четко понимать существующие в языке возможности. Я не собираюсь показывать все существующие операции на строковых литералах, моя цель – дать представление о том, что вообще возможно.

Я часто использую Julia в качестве помощника при работе с другими языками программирования и для преобразования исходного кода разными способами. Я покажу все на примере. Во многих языках программирования нередко встречаются вот такие варианты форматирования идентификаторов:

- `FooBar` – регистр в стиле языка Pascal (верхний верблюжий регистр); используется для типов или классов; иногда применяется для констант;
- `foo_bar` – змеиный регистр; часто используется для имен переменных, методов и функций;
- `fooBar` – верблюжий регистр (нижний верблюжий регистр); часто используется для имен методов и переменных;
- `FOO_BAR` – верхний змеиный регистр; нередко используется для констант и значений перечисления;

- `foo-bar` – регистр Кебаб (шашлычный шампур); его можно найти в программах LISP и именах конфигурационных файлов.

Я продемонстрирую несколько способов преобразования между этими стилями и то, как их можно превращать в удобные функции-утилиты, облегчающие программирование.

Вот мой типичный процесс разработки простой функции, которая что-то делает: поскольку я не уверен в том, как незнакомая функция работает, я сначала ее испытываю. Затем постепенно комбинирую ее с дополнительными вызовами, чтобы получить то, что хочу. В конце я получаю достаточно материала, чтобы реализовать свою функцию.

Листинг 11.3 Итеративная разработка функции верблюжьего регистра

```
julia> s = "foo_bar"
"foo_bar"

julia> split(s, '_')
2-element Vector{SubString{String}}:
"foo"
"bar"

julia> uppercasefirst("foo")
"Foo"

julia> map(uppercasefirst, split(s, '_'))
2-element Vector{String}:
"Foo"
"Bar"

julia> join(["Foo", "Bar"])
"FooBar"

julia> join(map(uppercasefirst, split(s, '_')))
"FooBar"

julia> camel_case(s::AbstractString) = join(map(uppercasefirst,
                                                split(s, '_')))
camel_case (generic function with 1 method)
```

Теперь мы имеем функцию, которая будет выполнять преобразование, но зачастую мы хотим делать все это быстро: выбрать текст в редакторе исходного кода, который находится в змеином регистре и который мы хотим превратить в верблюжий, и вставить обратно.

Именно здесь пригодится функция Julia `clipboard()`. Она может как читать, так и писать в буфер обмена. Буфер обмена – это место, где хранится все, что вы копируете.

ВНИМАНИЕ В Linux функция буфера обмена не будет работать, если не установлены команды `xsel` или `xclip`. В Debian или Ubuntu они устанавливаются следующим образом:


```
$ sudo apt-get install xsel
```

В Red Hat или Fedora Linux они устанавливаются, как показано ниже:

```
$ sudo yum install xsel
```

В листинге 11.4 в функцию `camel_case` добавлен метод, который не принимает никаких строковых аргументов, а вместо этого читает данные из буфера обмена. Надо просто пометить текст и скопировать его перед выполнением функции `clipboard()`. Я пометил первую часть этого абзаца.

Листинг 11.4 Преобразование текста в буфере обмена в верблюжий регистр

```
julia> s = clipboard()
"HiHowAreYou"

julia> function camel_case()
    s = camel_case(clipboard())
    clipboard(s)
    s
end
```

Функция `clipboard()` получает содержимое буфера обмена, тогда как `clipboard(s)` сохраняет содержимое `s` в буфере обмена. Во время программирования всякий раз, когда требуется изменить текст со змеиным регистром на верблюжий, надо выполнить следующие ниже шаги:

- 1 скопировать текст;
- 2 переключиться на открытую интерактивную среду REPL;
- 3 начать набирать на клавиатуре `came...` и нажать клавишу со стрелкой вверх/вниз, в результате чего набранный текст должен дополниться результатом функции `camel_case()`, если вы вызывали ее раньше. Либо нажать **Tab**;
- 4 вернуться в редактор и вставить итоговый результат.

Более эффективное использование интерактивной среды REPL языка Julia

Для быстрой работы с Julia важно привыкнуть ко всем горячим клавишам. Клавиша со стрелкой вверх/вниз используется для быстрого поиска по вашей истории. Если начать писать несколько букв, эта история будет фильтроваться, чтобы показывать исторические совпадения, начинающиеся только с этих первых букв.

Клавиша **Tab** используется для завершения слова, совпадающего с функцией, о которой Julia знает. Она может быть встроенной либо определенной вами самими.

Ctrl+A и **Ctrl+E** используются для перехода к началу и концу строки в интерактивной среде REPL. Допустим, вы только что написали

```
map(uppercasefirst, split(s, '_'))
```

И хотите поменять это на

```
join(map(uppercasefirst, split(s, '_')))
```

Нажмите стрелку вверх, чтобы вернуться к строке, которую вы только что написали. Нажмите **Ctrl-A**, чтобы перейти к началу строки. Напишите `join`. Наконец, нажмите **Ctrl-E**, чтобы перейти в конец, и напишите `)`.

11.2.1 Конвертирование верблюжьего регистра в змеиный регистр

Давайте посмотрим на исходный код движения в другом направлении. В данном случае применение функции `split` не работает, но почему? Все дело в невозможности выполнить разбивку на конкретном символе; однако вместо символов функция `split` может принимать функции, которые могли бы принимать решение, на каком символе делать разбивку. Для разбивки на пробельных символах используется вызов `split(s, isspace)`, и поэтому можно попробовать применить функцию `isuppercase`. Она проверяет, не находится ли символ в верхнем регистре. Это бывает удобно тем, что разбивка делается в месте, где символы находятся в верхнем регистре:

```
julia> isuppercase('a')
false
```

```
julia> isuppercase('A')
true
```

```
julia> s = "oneTwoThreeFour"
"oneTwoThreeFour"
```

```
julia> split(s, isuppercase)
4-element Vector{SubString{String}}:
"one"
"wo"
"hree"
"our"
```

Как видите, этот подход не работает, потому что функция `split` удаляет символ, который используется для разбивки. Вместо этого мы будем использовать одну из многочисленных функций Julia `find`. Если в интерактивной среде REPL написать `find` и нажать **Tab**, то можно увидеть ряд возможных вариантов:

```
julia> find
findall findfirst findlast
```

```
findmax    findmax!    findmin
findmin!   findnext    findprev
```

Функция `findfirst` находит первый случай совпадения, а функция `findall` находит все такие случаи. Давайте посмотрим на пример, чтобы прояснить ситуацию:

```
julia> s = "The quick brown fox";

julia> findfirst(isspace, s)
4

julia> indices = findall(isspace, s)
3-element Vector{Int64}:
 4
10
16

julia> s[indices]
"  "
```

При этом все индексы заглавных букв можно прокручивать в цикле и извлекать (или захватывать) подстроки, используя диапазоны.

Листинг 11.5 Распечатка каждого слова, написанного с заглавной буквы

```
function snake_case(s::AbstractString)
    i = 1
    for j in findall(isuppercase, s)
        println(s[i:j-1])
        i = j
    end
    println(s[i:end])
end
```

Листинг 11.5 является демонстрацией процесса постепенной разработки функции. В данном случае функция `println` обеспечит правильный результат. Диапазоны `i:j-1` будут извлекать подстроку:

```
julia> snake_case("oneTwoThreeFour")
one
Two
Three
Four
```

В следующем ниже листинге показан полный пример. Функция `println` была удалена, и был добавлен массив строковых литералов, именуемый `words`, для хранения каждого отдельного слова с заглавной буквы.

Листинг 11.6 Преобразование строкового литерала в верблюжем регистре в строковый литерал в змеином регистре

```
function snake_case(s::AbstractString)
    words = String[]
    i = 1
    for j in findall(isuppercase, s)
        push!(words, lowercase(s[i:j-1]))
        i = j
    end
    push!(words, lowercase(s[i:end]))
    join(words, '_')
end
```

После того как слова были собраны в массив, надо соединить их в один строковый литерал, используя `join(words, '_')`. Второй аргумент, `'_'`, служит для присоединения каждого слова разделителем `_`.

11.2.2 Конвертирование между числами и строковыми литералами

Глава 5 посвящена чтению вводимых пользователем данных. Вне зависимости от того, поступают ли входные данные с клавиатуры либо из файла, они обычно имеют форму текстовых строк; однако на входе могут понадобиться и числа. В той главе мы рассмотрели функцию `parse`, которая служит именно для этого; давайте посмотрим на нее снова, но теперь более подробно. В качестве первого аргумента мы предоставим объект-тип, чтобы указать тип числа, синтаксический разбор которого нужно выполнить. Это может быть что угодно, от разных типов целых чисел до чисел с плавающей точкой:

```
julia> parse(UInt8, "42")
0x2a

julia> parse(Int16, "42")
42

julia> parse(Float64, "0.42")
0.42
```

❶ 42 в шестнадцатеричной форме (см. главу 2).

Можно даже указать основание. При синтаксическом разборе чисел Julia по умолчанию берет основание 10, что относится к цифрам от 0 до 9. Однако по желанию можно выполнять синтаксический разбор и извлечение чисел, как если бы они были двоичными, с основанием 2. Это подразумевает, что для формирования чисел имеются только цифры 0 и 1:

```
julia> parse{Int, "101"} ❶  
101  
  
julia> parse{Int, "101", base=2} ❷  
5  
  
julia> parse{Int, "101", base=16} ❸  
257
```

- ❶ Интерпретировать строковый литерал 101 как десятичное число.
- ❷ Как двоичное число (по основанию 2).
- ❸ Выполнить синтаксический разбор числа 101, как если бы оно представляло собой шестнадцатеричное число.

Эти конвертации можно выполнять и в обратном порядке. Можно взять число и решить, какое именно основание нужно использовать при конвертировании в строковый литерал:

```
julia> string(5, base=2) ❶  
"101"  
  
julia> string(17, base=16) ❷  
"11"  
  
julia> string(17, base=10) ❸  
"17"
```

- ❶ Создать строковый литерал с цифрами двоичного числа из десятичного числа 5.
- ❷ Превратить десятичное число 17 в строковый литерал в шестнадцатеричной форме.
- ❸ Конвертировать в строковый литерал с использованием десятичной системы числения.

Из упомянутой ранее главы о строковых литералах вы, возможно, помните именованный аргумент `color=:green`. Здесь имеется еще один именованный аргумент `base=2`. Это типичный случай использования именованного аргумента, поскольку указывается нечто, что нужно указывать лишь изредка. Касаясь ракетной темы, я расскажу о нескольких манипуляциях со строковыми литералами, связанных с ракетным двигателем РД-180 производства Энергомаш (рис. 11.2).

11.2.3 Интерполяция и конкатенация строковых литералов

В Julia строковые литералы можно комбинировать громадным количеством способов; ниже будет показано несколько способов в сравнении. Нередко возникают объекты, такие как числа, которые требуется преобразовывать в строковые литералы. В следующем ниже фрагменте исходного кода определено несколько переменных разных типов для использования в примерах со строковыми литералами:

```
julia> engine = "RD-180"  
"RD-180"
```

```
julia> company = "Energomash"  
"Energomash"  
  
julia> thrust = 3830  
3830  
  
julia> string("The ", engine,  
    " rocket engine, produced by ",  
    company,  
    " produces ", thrust,  
    " kN of thrust")  
  
"The RD-180 rocket engine, produced  
by Energomash produces 3830 kN of thrust"
```



Рис. 11.2 Ракетный двигатель Энергомаш РД-180

В приведенном выше исходном коде функция `string()` использовалась для конкатенации строковых литералов и конвертирования нестроковых литералов в строковые литералы. В качестве альтернативы можно использовать оператор конкатенации строковых лите-

ралов *. Если вы пришли из других языков, то, возможно, вам больше знаком оператор `+`, который используется для этой цели. Вместо него в Julia был сделан выбор в пользу оператора, широко используемого для обозначения конкатенации в математической нотации:

```
julia> "The " * engine *
      " rocket engine, produced by " *
      company *
      " produces " *
      string(thrust) *
      " kN of thrust"

"The RD-180 rocket engine, produced by Energomash
produces 3830 kN of thrust"
```

При работе с большим количеством переменных обычно лучше использовать строковую интерполяцию. Строковая интерполяция выполняется с помощью знака `$`:

```
julia> "The $engine rocket engine, produced by $company produces
      $thrust kN of thrust"

"The RD-180 rocket engine, produced by Energomash produces 3830 kN
of thrust"
```

Обратите внимание, что когда нет пробела, который мог бы четко отличать имя переменной от окружающего текста, нередко вместо `$variable` нужно использовать `$(variable)`. То же самое применимо при попытке интерполировать не переменную, а выражение. Например, возьмем случай, когда требуется написать 3830kN без пробела:

```
julia> "produces $thrustkN of thrust" ❶
ERROR: UndefVarError: thrustkN not defined

julia> "produces $(thrust)kN of thrust" ❷
"produces 3830kN of thrust"

julia> "two engines produces $(2 * thrust) kN of thrust" ❸
"two engines produces 7660 kN of thrust"
```

- ❶ Переменную `thrust` нельзя интерполировать таким образом.
- ❷ Правильный способ строковой интерполяции, когда нет окружающих пробелов.
- ❸ Строковая интерполяция выражения.

11.2.4 Форматирование с использованием функции `sprintf`

Если вы знакомы с программированием на C, то, возможно, вы знакомы с функциями `printf` и `sprintf`. В Julia есть имитации этих функций в виде макрокоманд `@printf` и `@sprintf`. В отличие от строковой интерполяции, эти макрокоманды позволяют подробнее задавать внешний вид выводимой на экран переменной.

ПРИМЕЧАНИЕ Макрокоманды отличаются от функций Julia префиксом `@`. Макрокоманда подобна генератору исходного кода; место вызова макрокоманды заменяется другим исходным кодом. Макрокоманды обеспечивают возможность продвинутого метапрограммирования, которое в данной книге не рассматривается.

Например, можно указывать количество цифр, которые должны использоваться при печати десятичного числа. Макрокоманда `@printf` выводит результат на консоль, но во всегда подгружаемом базовом модуле Julia макрокоманды `@sprintf` и `@printf` отсутствуют. Следовательно, для того чтобы использовать эти макрокоманды, необходимо включать модуль `Printf`; этим и объясняется первая строка исходного кода:

```
julia> using Printf

julia> @printf("π = %0.1f", pi)
π = 3.1

julia> @printf("π = %0.2f", pi)
π = 3.14

julia> @printf("π = %0.5f", pi)
π = 3.14159
```

Ниже приводится краткий обзор нескольких распространенных опций форматирования:

- `%d` – целые числа;
- `%f` – числа с плавающей точкой;
- `%x` – целые числа в шестнадцатеричном представлении;
- `%s` – показывает аргументы в виде строкового литерала.

С каждым из этих параметров форматирования можно указывать такие компоненты, как количество цифр, десятичных знаков или отступы. Сначала давайте рассмотрим несколько примеров базовых опций форматирования:

```
julia> @sprintf("|%d|", 29)
"|29|"

julia> @sprintf("|%f|", 29)
"|29.000000|"

julia> @sprintf("|%x|", 29)
"|1d|"
```

Перед числами и после них были добавлены полосы, чтобы следующие ниже примеры отступов легче читались:

```
julia> @sprintf("|%2d|", 42)
"|42|"
```



```
julia> @sprintf("%4d", 42)
"| 42|"

julia> @sprintf("%-2d", 42)
"|42|"

julia> @sprintf("%-4d", 42)
"|42  |"
```

Обратите внимание, что заполнение можно применять как к правой, так и к левой стороне. Правильное заполнение достигается путем добавления дефиса. Заполнение удобно использовать, если требуется отображать столбцы чисел, которые нужно выравнивать. Отступы можно добавлять в виде нулей, а не в виде пробелов, ставя перед числом отступов префикс 0:

```
julia> @sprintf("|%02d|", 42)
"|42|"

julia> @sprintf("|%04d|", 42)
"|0042|"
```

Заполнение говорит не о количестве добавляемых пробелов или нулей, а об общем количестве символов, выделяемых под заполнение числа. Если отступ равен двум, а число имеет две цифры, то ничего не произойдет. Однако если отступ равен четырем, то будут добавлены два пробела, в результате чего получится четыре символа.

11.3 Использование строковой интерполяции для генерации исходного кода

С тем, что вы только что узнали, можно создавать небольшие функции-утилиты. В приведенном ниже примере рассматривается генерирование исходного кода C++. Возможно, Julia не является вашим первичным рабочим языком; на работе вместо него вы, вероятно, пользуетесь более многословным языком, таким как C++ или Java, но Julia можно использовать в качестве комплементарного языка с целью упрощения работы. Далее мы рассмотрим пример того, как разработчик на C++ может упрощать свою работу, пользуясь преимуществами языка программирования Julia.

Инструментарий визуализации VTK (<https://vtk.org>) – это замечательная библиотека C++, предназначенная для визуализации научных данных. Она используется для создания визуализаций, подобных той, что показана на рис. 11.3.

К сожалению, писать исходный код C++ в VTK утомительно из-за большого объема типового стереотипного исходного кода, необхо-

димого в C++. Ниже приведен пример исходного кода C++, используемого в VTK для определения геометрической линии. Здесь не требуется понимание выполняемой исходным кодом работы; он был отредактирован, чтобы удалить ненужные детали.

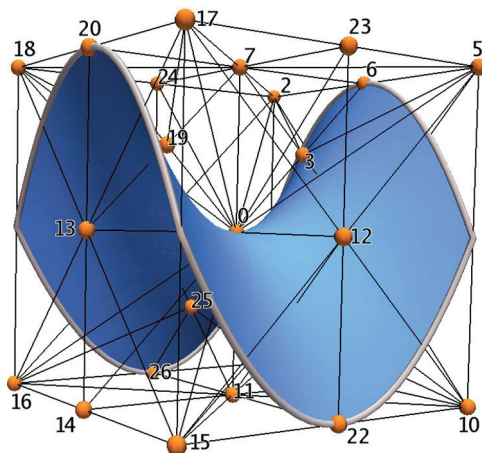


Рис. 11.3 Визуализация в VTK

Листинг 11.7 Класс Line в VTK

```
#ifndef vtkLine_h
#define vtkLine_h

class VTKCOMMONDATAMODEL_EXPORT vtkLine : public vtkCell
{
public:
    static vtkLine *New();
    vtkTypeMacro(vtkLine,vtkCell);
    void PrintSelf(ostream& os, vtkIndent indent) override;

    int GetCellType() override {return VTK_LINE;};

protected:
    vtkLine();
    ~vtkLine() override {}

private:
    vtkLine(const vtkLine&) = delete;
    void operator=(const vtkLine&) = delete;
};
```

Сравните этот исходный код со следующим ниже блоком кода в листинге 11.8, в котором задается многоугольник; вы заметите много повторов. Это касается всего исходного кода VTK, написанного для определения геометрических примитивов.

Листинг 11.8 Класс Polygon в VTK

```

#ifndef vtkPolygon_h
#define vtkPolygon_h

class VTKCOMMONDATAMODEL_EXPORT vtkPolygon : public vtkCell
{
public:
    static vtkPolygon *New();
    vtkTypeMacro(vtkPolygon,vtkCell);
    void PrintSelf(ostream& os, vtkIndent indent) override;

    int GetCellType() override {return VTK_POLYGON;};

protected:
    vtkPolygon();
    ~vtkPolygon() override;

private:
    vtkPolygon(const vtkPolygon&) = delete;
    void operator=(const vtkPolygon&) = delete;
};

#endif

```

Представьте, что вы часто пишете новые классы (типы) C++ для разных геометрических типов; было бы утомительно повторять весь этот шаблон. К счастью, на Julia можно создавать небольшие функции-утилиты, которые в этом помогают (листинг 11.9).

При создании текста, состоящего из нескольких строк, целесообразно использовать тройные кавычки: `"""`. Благодаря им можно писать строковые литералы, занимающие несколько строк.

Листинг 11.9 Генератор исходного кода VTK C++ в исходном коде Julia

```

function create_class(class::AbstractString)
    s = """
        #ifndef vtk$(class)_h
        #define vtk$(class)_h

        class VTKCOMMONDATAMODEL_EXPORT vtk$class : public vtkCell
        {
        public:
            static vtk$class *New();
            vtkTypeMacro(vtk$class,vtkCell);
            void PrintSelf(ostream& os, vtkIndent indent) override;

            int GetCellType() override {return VTK_$(uppercase(class));};

        protected:
            vtk$class();
            ~vtk$class() override;
    """
end

```

```

        private:
            vtk$class(const vtk$class&) = delete;
            void operator=(const vtk$class&) = delete;
        };

        #endif
        """
        clipboard(s) ❶
        println(s)
    end

```

- ❶ Упрощает вставку сгенерированного исходного кода класса в редактор исходного кода.

Ниже приведен пример использования этой функции-утилиты для создания класса Hexagon. Обратите внимание, что в двух предыдущих строках сгенерированный исходный код сохраняется в буфере обмена.

Листинг 11.10 Использование Julia для создания класса C++ Hexagon

```

julia> create_class("Hexagon")
#ifndef vtkHexagon_h
#define vtkHexagon_h

class VTKCOMMONDATAMODEL_EXPORT vtkHexagon : public vtkCell
{
public:
    static vtkHexagon *New();
    vtkTypeMacro(vtkHexagon,vtkCell);
    void PrintSelf(ostream& os, vtkIndent indent) override;

    int GetCellType() override {return VTK_HEXAGON;};

protected:
    vtkHexagon();
    ~vtkHexagon() override;

private:
    vtkHexagon(const vtkHexagon&) = delete;
    void operator=(const vtkHexagon&) = delete;
};

#endif

```

11.4 Работа с нестандартными строковыми литералами

Во многих языках программирования есть полезные объекты, такие как даты, регулярные выражения, типы MIME и числа, которые начи-

нают свою жизнь в качестве строковых литералов. Например, в Julia отсутствует возможность выражать очень большие числа в виде числовых литералов, поэтому их приходится выражать в виде строкового литерала, который затем подвергается синтаксическому разбору. Например, в Julia числовые литералы с плавающей точкой являются 64-битовыми, и этого недостаточно для хранения таких чисел, как 1.4e600. В Julia есть типы, к примеру такие, как `BigInt` и `BigFloat`, которые могут хранить сколь угодно большие числа. Но как создать такое число, если числовые литералы ограничены 64-битовыми значениями с плавающей точкой? Решение заключается в синтаксическом разборе строкового литерала, содержащего определение числа, и его извлечении:

```
julia> 1.4e600
ERROR: syntax: overflow in numeric constant "1.4e600"

julia> x = parse(BigFloat, "1.4e600")
1.3999...9994e+600

julia> typeof(x)
BigFloat
```

Еще один пример показателен при работе с датами. Возьмем сценарий, в котором из файла читается несколько дат; требуется выполнить их синтаксический разбор, чтобы извлечь даты. Для этого задается формат даты, к примеру такой, как `uuu-mm-dd`. Этот формат даты указывает, что первыми идут годы, а последними – дни и что каждый компонент отделяется дефисом: -. Ниже приведен пример конвертирования одного формата даты в другой (опции форматирования даты будут рассмотрены позже).

Листинг 11.11 Конвертирование из одного формата данных в другой

```
using Dates

dates = ["21/7", "8/12", "28/2"];

for s in dates
    date = Date(s, DateFormat("dd/mm"))
    dstr = Dates.format(date, DateFormat("E-u"))
    println(dstr)
end
```

Если выполнить этот исходный код, то будет получен следующий ниже результат:

```
Saturday-Jul
Saturday-Dec
Wednesday-Feb
```

Приведенный выше пример форматирования даты иллюстрирует проблему со всеми объектами, производными от строковых

литералов. Если исходный код пишется естественным образом, то в итоге придется снова и снова выполнять синтаксический разбор одного и того же строкового литерала. На каждой итерации цикла разбираются строковые литералы "dd/mm" и "E-u", но в этом нет необходимости. Эти строковые литералы одинаковы на каждой итерации; меняются только сами строковые литералы даты. Во избежание необходимости выполнять синтаксический разбор строковых литералов для создания таких объектов, как `BigFloat` и `DateFormat`, Julia предоставляет специальные строковые литералы, такие как `big"1.4e600"` и `dateformat"dd/mm"`.

Когда Julia выполняет синтаксический разбор исходного кода программы и встречает эти строковые литералы, он создает не объекты `String`, а объекты `BigInt`, `BigFloat` или `DateFormat`. Преимущество такого подхода заключается в том, что объекты создаются при синтаксическом разборе исходного кода, а не при его выполнении.

Возможно, это прозвучит как не имеющая значения странноватая деталь; однако указанный подход имеет существенное значение. Julia будет выполнять синтаксический разбор исходного кода цикла `for` программы всего раз. Но исходный код внутри цикла будет потенциально исполнять много раз. Таким образом, благодаря созданию объектов, таких как `DateFormat`, во время синтаксического разбора, а не во время выполнения, повышается производительность.

Ниже будет рассмотрен каждый нестандартный строковый литерал по очереди и в отдельных разделах. Прочитав эти разделы, станет яснее последствия данной стратегии.

11.4.1 Строковые литералы `DateFormat`

Вернемся к примеру `DateFormat`. Если нужно избежать снижения производительности при многократном синтаксическом разборе строковых литералов `DateFormat`, то в других языках программирования исходный код реструктурируется, как показано в следующем ниже листинге.

Листинг 11.12 Оптимизированный, но менее удобочитаемый исходный код форматирования даты

```
using Dates

informat = DateFormat("dd/mm")
outformat = DateFormat("E-u")
dates = ["21/7", "8/12", "28/2"]

for s in dates
    date = Date(s, informat)
    dstr = Dates.format(date, outformat)
    println(dstr)
end
```

С точки зрения производительности это работает. Проблема в том, что исходный код становится менее понятным. Взглянув на строки синтаксического разбора и форматирования дат, сразу не увидишь, какой именно формат используется. Для удобочитаемости гораздо лучше иметь возможность помещать определение формата даты именно там, где оно используется. Именно это и можно сделать со строковым литералом формата даты, как показано в следующем ниже листинге.

Листинг 11.13 Оптимизированный и удобочитаемый исходный код форматирования даты

```
using Dates

dates = ["21/7", "8/12", "28/2"]

for s in dates
    date = Date(s, dateformat"dd/mm")
    dstr = Dates.format(date, dateformat"E-u")
    println(dstr)
end
```

Я еще не рассказывал, как надо указывать строковый литерал формата даты. К счастью, интерактивная справочная система Julia дает отличный обзор; надо просто войти в режим справки (?) и написать `DateFormat`, в результате чего будет предоставлен список всех применяемых букв и их смысл. В принципе, для обозначения года, месяца и дня используются буквы `y`, `m` и `d`. Если нужно писать годы четырьмя цифрами, то надо указывать их как `yyyy`. Несколько примеров исходного кода в интерактивной среде REPL должны дать представление о том, как это работает:

```
julia> d = Date(2022, 8, 28)
2022-08-28

julia> Dates.format(d, dateformat"dd-mm-yyyy")
"28-08-2022"

julia> Dates.format(d, dateformat"mm-yy")
"08-22"

julia> Dates.format(d, dateformat"yy/m")
"22/8"
```

Не все форматы работают с числами. Форматы `u` и `U` дают название месяца, например `January` и `February`. Форматы `e` и `E` дают название дня, например `Monday` и `Tuesday`:

```
julia> Dates.format(d, dateformat"e u yyyy")
"Sun Aug 2022"

julia> Dates.format(d, dateformat"E U yyyy")
"Sunday August 2022"
```

11.4.2 Неформатированные строковые литералы

Одна проблема с обычными строковыми литералами Julia заключается в том, что такие символы, как `$` и `\n`, имеют особый смысл. В определенных типах текста это бывает неудобно. Данная проблема решается за счет использования символа экранирования `\\`; так, `$` будет записано как `\\$`, а `\n` как `\\n`. Однако если вы не хотите этого делать и вам не нужна строковая интерполяция, то можно использовать неформатированные (сырые) строковые литералы:

```
julia> thrust = 3830
3830
```

```
julia> raw"produces $thrust kN of thrust" # Не работает
"produces \\$thrust kN of thrust"
```

В данном случае нестандартный строковый литерал не создает новый тип объекта. Вместо этого при конструировании строкового объекта содержимое строкового литерала интерпретируется по-другому.

11.4.3 Использование регулярных выражений для сопоставления текста

Регулярные выражения – это своего рода миниатюрный язык, который используется для отыскания совпадений в тексте. Регулярные выражения широко используются в инструментах обработки текста Unix и во многих редакторах исходного кода. Например, регулярные выражения используются для поиска того или иного строкового литерала в исходном коде.

В приведенном ниже примере переменная `s` содержит исходный код Julia, и мы решили изменить имя типа `Rocket` на `SpaceCraft` (Космический корабль). Для отыскания местоположения заменяемого текста применяется функция `replace`:

```
julia> s = """
    struct RocketEngine
        thrust::Float64
        Isp::Float64
    end

    mutable struct Rocket
        tank::Tank
        engine::RocketEngine
    end
    """; ❶

julia> result = replace(s, "Rocket"=>"SpaceCraft"); ❷

julia> println(result)
```



```
struct SpaceCraftEngine
    thrust::Float64
    Isp::Float64
end

mutable struct SpaceCraft
    tank::Tank
    engine::SpaceCraftEngine
end
```

- ❶ Текст исходного кода, в котором, по нашему мнению, необходимо заменить подстроку.
- ❷ Заменить вхождение подстроки `Rocket` на `SpaceCraft`.

Как вы помните из главы 6, оператор `=>` используется для создания пары; он применялся для создания и хранения в словаре пар ключ-значение. В данном случае пара представляет собой искомый текст и подстановочный текст. Так, `"Rocket"=>"SpaceCraft"` означает найти `"Rocket"` и заменить его на `"SpaceCraft"`.

Однако, как видно из примера, такой подход дает не совсем то поведение, которое ожидается, поскольку также заменяется и `"RocketEngine"`, становясь `"SpaceCraftEngine"`. Но в данном случае требуется изменить только тип `Rocket`. Регулярные выражения помогают уточнять искомую подстроку.

В регулярных выражениях знак `«.»` означает любой символ; выражение `[A-D]` означает любой символ от `A` до `D`; а выражение `[^A-D]` означает любой символ, не входящий в диапазон от `A` до `D`. Таким образом, `"Rocket[^A-Za-z]"` будет означать отыскание слова *Rocket*, где первый совпавший символ *не* является буквой, как показано ниже.

Листинг 11.14 Замена текста с помощью регулярных выражений

```
julia> result = replace(s, r"Rocket[^A-Za-z]"=>"SpaceCraft");

julia> println(result)

struct RocketEngine
    thrust::Float64
    Isp::Float64
end

mutable struct SpaceCraft
    tank::Tank
    engine::RocketEngine
end
```

В приведенном выше примере искомая подстрока превращается в регулярное выражение путем добавления к ней префикса `r`. Это означает, что она не будет строковым объектом, что можно продемонстрировать в интерактивной среде REPL:

```
julia> rx = r"Rocket[^\A-Za-z]"
r"Rocket[^\A-Za-z]"

julia> typeof(rx)
Regex
```

Указанный объект регулярного выражения создается во время синтаксического разбора, а не во время выполнения программы. Таким образом, как и в случае с `DateFormat`, во время выполнения избегается повторный синтаксический разбор одного и того же регулярного выражения.

Синтаксису регулярных выражений посвящено много замечательной документации, поэтому ниже будет приведен только краткий обзор самых базовых символов, используемых в регулярных выражениях. Ниже приведен список так называемых *классов буквенных символов*:

Символ	Смысл	Пример
<code>\d</code>	Сопоставляет с любой цифрой	"387543"
<code>\w</code>	Сопоставляет с любым буквенно-цифровым словом, содержащим знак подчеркивания	"foo_bar_42"
<code>\s</code>	Сопоставляет с любыми пробельными символами, знаками табуляции, перевода строки, пробела	" "
<code>.</code>	Сопоставляет с любым символом	"aA ;%4t"

Кроме того, на процедуру сопоставления классов буквенных символов влияют специальные символы; они называются *квантификаторами* и показывают допустимое число повторов классов буквенных символов:

Символ	Смысл
<code>*</code>	Повторить символ 0 или более раз
<code>+</code>	Повторить один или несколько раз
<code>?</code>	Повторить ноль или один раз

Более сложное взаимодействие с системой регулярных выражений Julia предусматривает работу с объектами `RegexMatch`. В приведенном ниже примере извлекается число, состоящее из нескольких цифр, `\d+`, и слово, состоящее из нескольких букв, `\w+`. Это делается с помощью функции `match`, которая вернет объект `RegexMatch`, содержащий все совпадения:

```
julia> rx = r"\w+ (\d+) (\w+) \d+" ❶

julia> m = match(rx, "foo 123 bar 42") ❷
RegexMatch{"foo 123 bar 42", 1="123", 2="bar"}

julia> m[1] ❸
"123"
```

```
julia> m[2]
"bar"
```

3

- ❶ Задать регулярное выражение.
- ❷ Сопоставить регулярное выражение со строковым литералом.
- ❸ Обратиться к первому и второму совпадениям.

Обратите внимание, что некоторые части регулярного выражения содержат круглые скобки; они извлекают эту часть строкового литерала. Мы настроили объект регулярного выражения `rx` для извлечения числа и слова. К результатам извлечения можно обращаться по целочисленным индексам, например `m[1]` и `m[2]`.

В случае более сложных регулярных выражений бывает трудно отслеживать позицию каждого извлечения. К счастью, регулярные выражения позволяют давать результатам извлечения имена. Допустим, требуется извлечь часы и минуты из строкового литерала `11:30`. Для этого можно применить регулярное выражение `rx("(\d+):(\d+)")`, но вместо него мы будем именовать каждое совпадение с помощью `?<s>`, где `s` – это имя результата извлечения:

```
julia> rx = r"(<hour>\d+):(<minute>\d+)"
r"(<hour>\d+):(<minute>\d+)"
```

```
julia> m = match(rx, "11:30 in the morning")
RegexMatch("11:30", hour="11", minute="30")
```

```
julia> m["minute"]
"30"
```

```
julia> m["hour"]
"11"
```

Объекты `RegexMatch` во многом похожи на коллекции `Julia`, поэтому объект `RegexMatch` можно прокручивать в цикле `for`. При именовании результатов извлечения объект `RegexMatch` работает со многими функциями, которые применимы к словарям:

```
julia> keys(m)
2-element Vector{String}:
 "hour"
 "minute"
```

```
julia> haskey(m, "hour")
true
```

```
julia> haskey(m, "foo")
false
```

Хотя регулярные выражения чрезвычайно мощны и универсальны, с ними можно легко переусердствовать. Роб Пайк (Rob Pike),

один из создателей Go, Plan 9, UTF-8 и многих других популярных технологий системного программирования, неоднократно предостерегал от чрезмерного использования регулярных выражений. По мере возникновения новых потребностей становится все сложнее и труднее вносить в них видоизменения.

Лично я ими пользуюсь редко. В Julia можно многого достичь с базовыми строковыми и буквенными функциями, такими как `split`, `endwith`, `startswith`, `isdigit`, `isletter` и `isuppercase`.

11.4.4 Создание больших целых чисел с помощью *BigInt*

Как показано в следующих ниже примерах, литеральный синтаксис существует для большинства типов чисел:

```
julia> typeof(42)
Int64

julia> typeof(0x42)
UInt8

julia> typeof(0x42000000)
UInt32

julia> typeof(0.42)
Float64

julia> typeof(0.42f0)
Float32

julia> typeof(3//4)
Rational{Int64}
```

В случаях, когда он не существует, можно выполнить конвертацию, например `Int8(42)`, при которой берется 64-битовое целое число со знаком и превращается в 8-битовое целое число со знаком. При записи целых чисел произвольной глубины точности (с любым количеством цифр) это также делается с использованием формулировки `BigInt(42)`; однако до некоторой степени такой подход может приводить к неэффективности. Везде, где она встречается, целое число должно конвертироваться в большое целое число. Вместо этого при написании `big"42"` большое целое число создается во время синтаксического разбора программы, а не при каждом ее выполнении.

Это поведение не встроено в сам язык. Определять числовой литерал может любой разработчик. Ниже приведен пример добавления поддержки написания `int8"42"` для создания 42 во время синтаксического разбора в виде 8-битового целого числа со знаком. Этот пример также служит в качестве демонстрации того, что, в отличие от функций, макрокоманды вызываются лишь один раз.

Листинг 11.15 Определение строковых литералов для 8-битового целого числа со знаком

```
macro int8_str(s) ❶
    println("hello") ❷
    parse{Int8, s} ❸
end
```

- ❶ Для строкового литерала с префиксом `foo`, например `foo"42"`, написать `foo_str`.
- ❷ Выписывая сообщение при каждом вызове макрокоманды, можно увидеть частотность ее вызовов.
- ❸ Синтаксический разбор строкового литерала числа и возврат 8-битового числа.

Теперь это определение можно попробовать в цикле. Если макрокоманды работают как функции, то мы будем получать вызов при каждом добавлении к переменной `total` в цикле:

```
julia> total = 0
0

julia> for _ in 1:4
    total += int8"10"
end
hello

julia> total
40
```

Однако сообщение `hello` будет показано только один раз, а не четыре раза. И на этом тема макрокоманд будет закрыта, так как она слишком обширна, чтобы ее освещать в книге для начинающих. И тем не менее о существующих в Julia мощных функциональных возможностях полезно знать, даже если они вряд ли понадобятся в первых программах на Julia.

11.4.5 MIME-типы

Различные операционные системы имеют разные системы отслеживания типа своих файлов. Например, в Windows для обозначения типа файла широко используется трехбуквенное расширение имени файла. Изначальная macOS сохраняла тип файла в специальных атрибутах.

Однако для отправки файлов разных типов между компьютерами в интернете необходим общий стандарт идентификации типов файлов; в этом и состоит суть MIME-типов. Обычно они описываются как тип и подтип, отделенные косой чертой. HTML-страницы обозначаются как `text/html`, а JPEG-изображения обозначаются как `image/jpeg`. Тип PNG-файла будет записан как `image/png` и т. д. Объект MIME-типа создается в Julia следующим образом:

```
julia> MIME("text/html")
MIME type text/html

julia> typeof(ans)
MIME{Symbol{"text/html"}}
```

Таким образом, объект MIME-типа `MIME("foo/bar")` будет иметь тип `MIME{Symbol{"foo/bar"}}`. Это будет выглядеть несколько загадочно, пока я не расскажу о параметрических типах в главе 18. Название `MIME{Symbol{"foo/bar"}}` выглядит длинным и довольно обременительным для письма, поэтому Julia предлагает сокращение `MIME"foo/bar"`.

Их легко перепутать. `MIME("foo/bar")` и `MIME"foo/bar"` не одно и то же. Первый случай – это объект, а второй – тип объекта. Ниже приведен простой пример их применения для создания методов, дающих разные результаты для разных MIME-типов.

```
say_hello(::MIME"text/plain") = "hello world"
say_hello(::MIME"text/html") = "<h1>hello world</h1>"
```

Удобство здесь в том, что такой подход позволяет определять функции, которые могут предоставлять разные форматированные текстовые результаты для разных контекстов:

```
julia> say_hello(MIME("text/plain"))
"hello world"

julia> say_hello(MIME("text/html"))
"<h1>hello world</h1>"
```

Исходный код Julia, исполняемый в графической среде, к примеру такой, как блокноты Jupyter (<https://jupyter.org>), будет передавать MIME-тип HTML, поэтому графики и таблицы могут отрисовываться как HTML.

Резюме

- Строковые литералы Julia кодируются в UTF-8, то есть каждая кодовая точка кодируется как переменное число кодовых единиц.
- Функция `parse` используется для конвертирования строковых литералов в другие типы, например числа.
- Функция `string` используется для конвертирования чисел в строковые литералы.
- Строковые литералы Julia можно комбинировать с другими типами объектов, используя либо строковую интерполяцию с символом `$`, либо функцию `string` с переменным числом аргументов.

- Строковые литералы можно конкатенировать с помощью оператора умножения `*`.
- Форматированный вывод в `stdout` достигается с помощью макрокоманды `@printf`. С целью получения возвращаемого строкового значения вместо нее следует использовать макрокоманду `@sprintf`. Обе находятся в модуле `Printf`.
- В Julia строковые литералы можно расширять, но есть несколько встроенных: неформатированные строки, большие целые числа и регулярные выражения.

12

Понимание коллекций Julia

Эта глава охватывает следующие ниже темы:

- понимание классификации коллекций в зависимости от поддерживаемого ими типа операций;
- превращение многоступенчатых ракет в итерируемую коллекцию;
- использование общих операций, поддерживаемых различными типами коллекций.

Мы уже рассмотрели такие коллекции, как массивы и словари, но есть целый ряд других типов коллекций, включая множества, связанные списки, кучи, стеки и двоичные деревья. В этой главе я расскажу об общих между разными типами коллекций свойствах. В каждой коллекции организовано хранится несколько элементов, и каждый тип коллекции предлагает уникальные способы доступа к этим элементам. Например, работая со словарем, можно обращаться к элементам по ключу, а для массива потребуется индекс.

Однако у коллекций также есть стержневая функциональность, которую должны поддерживать все коллекции без исключения, например *итерируемость*. Если что-то является итерируемым, то к отдельным элементам можно обращаться в цикле `for` или использовать функцию более высокого порядка, такую как `map` или `filter`.

Что именно делает нечто коллекцией? Каковы различия и сходства между разными типами коллекций? А как создавать свою собственную? Мы рассмотрим эти вопросы на примере многоступенчатой ракеты из главы 8. Поскольку ракета состоит из многочисленных и разных частей, ее можно превратить в нечто, что Julia будет распознавать как коллекцию.

В этой главе будет добавлен исходный код в абстрактный тип `Tank`, чтобы показать, как определяются интерфейсы. Мы изменим тип `Cluster` двигателей, чтобы поддерживать прокручивание двигателей в цикле. В последнем примере мы видоизменим тип `StagedRocket` с целью поддержания итеративной обработки многоступенчатой ракеты.

12.1 Определение интерфейсов

Так что же такое интерфейс? Интерфейсы удобно понимать в противопоставлении с реализациями. При взаимодействии с компьютером используются мышь и клавиатура; это интерфейс к компьютеру – вовсе не требуется знать внутреннее устройство конкретного компьютера (рис. 12.1). Одну и ту же мышь и клавиатуру можно использовать на разных компьютерах, собранных совершенно по-разному. Независимо от объема памяти или микропроцессора компьютера, с ним можно взаимодействовать, кликая по одним и тем же значкам и перемещаясь по одним и тем же окнам. Другими словами, между многочисленными моделями компьютеров существует общий интерфейс, который изолирует от конкретной аппаратной реализации каждого компьютера.

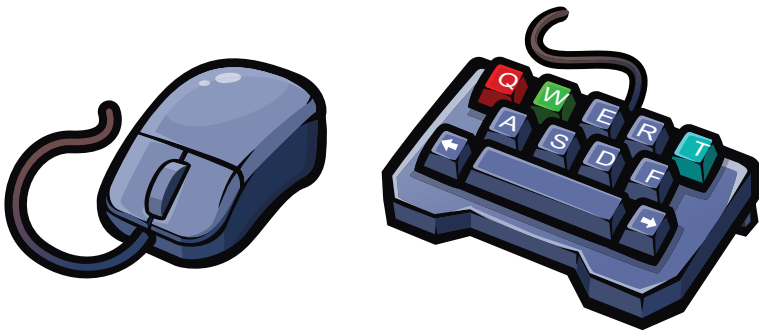


Рис. 12.1 Компьютеру не нужно знать механизмы работы устройств ввода

Выделение компонентов с четко определенными интерфейсами позволяет создавать большие сложные структуры. Если в каждой части системы используется четко определенный интерфейс, то части системы не должны обязательно знать подробности реализации.

Давайте свяжем это с программированием на языке Julia. Как показано на рис. 12.2, массивы и диапазоны являются подтипами типа `AbstractArray`.

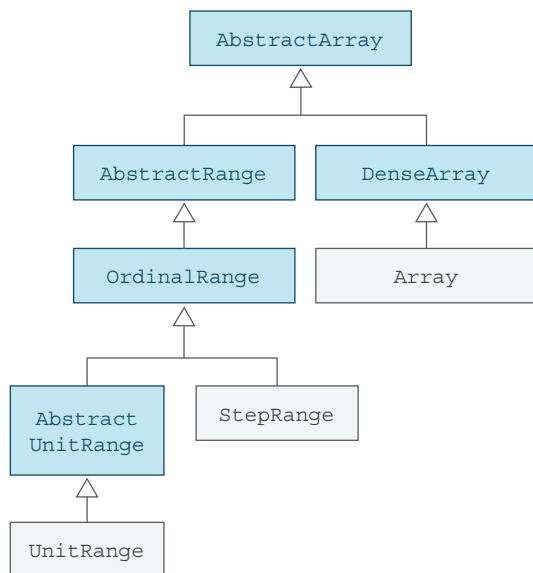


Рис. 12.2. Иерархия типов для массивов и диапазонов

Так, если определить функцию, оперирующую на `AbstractArray`, то не нужно учитывать разницу между массивом и диапазоном. Можно создать функцию `addup` (см. листинг 12.1), которая будет работать независимо от того, что именно, массив либо диапазон, было передано в качестве аргумента.

Листинг 12.1 Функция сложения элементов в абстрактном массиве

```
function addup(xs::AbstractArray)
    total = 0
    for x in xs
        total += x
    end
    total
end
```

- ① Переменная для хранения результата сложения.
- ② Вернуть результат сложения.

Давайте вызовем эту функцию с другими аргументами. Обратите внимание, что по действию она очень похожа на функцию `sum`, за исключением того, что она не позволяет складывать кортеж значений. Почему?

```
julia> addup(3:5) ❶
12

julia> addup([3, 4, 5]) ❷
12

julia> addup((3, 4, 5)) ❸
ERROR: MethodError: no method matching addup(::Tuple{Int64, Int64, Int64})

julia> sum((3, 4, 5))
12
```

- ❶ Сложение диапазона значений.
- ❷ Сложение массива значений.
- ❸ Попытка сложить кортеж значений.

Тип `Tuple` в иерархии типов `AbstractArray` отсутствует, поэтому значения типа `Tuple` не являются допустимыми аргументами сложения. Еще один пример общего интерфейса можно увидеть в доступе к элементам по индексу как для диапазонов, так и для массивов. Следует помнить, что первый элемент массивов Julia имеет индекс 1:

```
julia> r = 3:5 ❶
3:5

julia> r[2] ❷
4

julia> sum(r) ❸
12

julia> a = [3, 4, 5] ❶
3-element Vector{Int64}:
 3
 4
 5

julia> a[2] ❷
4

julia> sum(a) ❸
12
```

- ❶ Определить похожий диапазон и массив.
- ❷ Доступ ко второму элементу выглядит так же.
- ❸ Суммирование диапазона и массива работает одинаково.

Диапазон не имеет элементов; элементы существуют в неявной форме. Однако от этой разницы можно абстрагироваться, предоставив диапазону и массиву похожий интерфейс. За счет этого появляется возможность определять такие функции, как `sum`, для работы с обоими типами, не создавая два разных метода.

В объектно ориентированных языках, таких как Java, C++ или C#, интерфейсы для `AbstractArray` определяются в явной форме. Определение типа в этих языках включает список методов, которые должны быть реализованы в подтипах. В противном случае возникает ошибка компилятора.

Однако в Julia интерфейсы определяются *неформально*. Следовательно, компилятор не сообщает, что интерфейс был реализован неправильно.

12.2 Пример интерфейса топливного бака

В целях разъяснения правил определения интерфейсов и работы с ними на языке Julia мы рассмотрим пример топливного бака из главы 8 (см. листинг 12.2). Допустим, мы предоставляем интерфейс топливного бака, который может использоваться другими разработчиками, и хотим дать им возможность создавать свои собственные подтипы топливных баков для использования при сборке ракет.

Листинг 12.2 Определение абстрактного топливного бака

```
abstract type Tank end

propellant(tank::Tank) = tank.propellant

function refill!(tank::Tank) ❶
    propellant!(tank, totalmass(tank) - drymass(tank))
    tank
end
```

❶ Заправка топливного бака до максимума.

Теперь представьте, что разработчик пытается создать конкретный подтип типа `Tank` для использования в симуляции ракеты. Разработчик пишет следующий ниже исходный код.

Листинг 12.3 Определение подтипа топливного бака

```
mutable struct FlexiTank <: Tank
    drymass::Float64
    totalmass::Float64
    propellant::Float64
end
```

Он хочет опробовать свой новый подтип топливного бака и пишет в интерактивной среде REPL следующий ниже исходный код:

```
julia> tank = FlexiTank(10, 100, 0)
FlexiTank(10.0, 100.0, 0.0)
```

```
julia> refill!(tank)
ERROR: UndefVarError: totalmass not defined ❶
```

❶ Julia понятия не имеет, что такое `totalmass`.

Это сообщение об ошибке мешает разработчику, который пытается реализовать интерфейс `Tank`, понять, что он должен делать. Для решения данной проблемы в Julia принято определять функции в интерфейсе и их документировать.

Глядя на исходный код листинга 12.4, возникают следующие вопросы: Почему здесь основное внимание уделяется документированию исходного кода? Где определяется интерфейс? И каков синтаксис определения интерфейса? Ответ в том, что его нет. Вот почему я говорю, что интерфейсы в Julia определяются *неформально*. Следовательно, документация является *ключевой* частью определения интерфейса на языке Julia.

Вспомните из главы 7, что в Julia методы прикрепляются к функциям, а не к типам. Связывать функции с каким-либо конкретным типом просто невозможно. Единственная причина, по которой общая масса (`totalmass`), сухая масса (`drymass`) и ракетное топливо (`propellant`) относятся к интерфейсу `Tank`, состоит в том, что мы сказали так в документации. Все это понарошку.

Листинг 12.4 Определение хорошо задокументированного интерфейса топливных баков

```
"Хранит ракетное топливо"
abstract type Tank end

"""
    totalmass(t::Tank) -> Float64
Масса полностью заправленного топливного бака `t`.
"""
function totalmass end

"""
    drymass(t::Tank) -> Float64
Масса пустого топливного бака `t`.
"""
function drymass end

"""
    propellant(t::Tank) -> Float64
Получить оставшееся в баке топливо.
Ракетное топливо состоит из собственно топлива и окислителя
"""
propellant(tank::Tank) = tank.propellant

"""
    refill!(tank::Tank) -> Tank
Заполнить топливный бак до максимума. Возвращает полный бак
```

```

"""
function refill!(tank::Tank)
    propellant!(tank, totalmass(tank) - drymass(tank))
    tank
end

```

Система документации Julia работает путем предварения определения функции или типа обычной текстовой строкой. Внутри этой текстовой строки автор документирует свою функцию или печатает, используя синтаксис упрощенной разметки¹. По правилам указанного синтаксиса строки, которые нужно оформить как исходный код, форматируются отступами. Для выделения отдельных слов как исходного кода используются обратные кавычки `

СОВЕТ Иногда возникает потребность в написании определений функций прямо в интерактивной среде REPL. Однако при нажатии **Enter** после окончания документационной строки она вычисляется до того, как будет написано определение функции. Как это исправить? Если нажать клавишу **Alt** или **Option** при нажатой клавише **Enter**, то Julia позволит продолжить писать исходный код.

Для добавления документации в свои функции можно использовать текстовые строки в двойных или тройных кавычках (" или """). Следует учитывать, что это отличается от добавления комментария с помощью символа решетки #. В справочной системе Julia комментарии не сохраняются.

Тройные кавычки и двойные кавычки работают немного по-разному. Например, если нужно использовать двойные кавычки внутри текста, заключенного в двойные кавычки, то кавычки нужно *экранировать*, используя обратную косую черту. В случае тройных кавычек это делать не обязательно:

```

julia> print("file \"foo.txt\" not found")
file "foo.txt" not found

julia> print("""file "foo.txt" not found""")
file "foo.txt" not found

```

Документация не обязательно должна соответствовать синтаксису Julia. Например, в документации можно использовать стрелку, чтобы сообщать читателям возвращаемый функцией тип объекта:

```
"drymass(t::Tank) -> Float64"
```

Давайте поместим это новое определение типа Tank вместе с типом FlexiTank в файл и перезагрузим с ним интерактивную среду

¹ Язык упрощенной разметки (markdown) служит для создания форматированного текста с помощью текстового редактора.

REPL. Это можно организовать почти в любом виде. Я использую файл с именем `tank-interface.jl`, который выглядит следующим образом (для компактности строки документации были удалены):

```
abstract type Tank end

function totalmass end

function drymass end

propellant(tank::Tank) = tank.propellant

function refill!(tank::Tank)
    propellant!(tank, totalmass(tank) - drymass(tank))
    tank
end

mutable struct FlexiTank <: Tank
    drymass::Float64
    totalmass::Float64
    propellant::Float64
end
```

Давайте посмотрим на сообщения об ошибках, которые появятся на этот раз при попытке перезаправить гибкий топливный бак:

```
julia> t = FlexiTank(10, 100, 0)
FlexiTank{10.0, 100.0, 0.0}

julia> refill!(t)
ERROR: MethodError: no method matching totalmass(::FlexiTank)
```

На этот раз будет получено более приемлемое сообщение об ошибке. Julia сообщает о том, что `totalmass` действительно является функцией, но в ней отсутствует метод для типа `FlexiTank`. Проверив существующие методы, мы приходим к выводу, что необходим метод, работающий с типом `FlexiTank`:

```
julia> methods(totalmass)
# 0 methods for generic function "totalmass":
```

Как обсуждалось в главе 2, для входа в справочную систему Julia следует нажать клавишу `?`:

```
help?> totalmass
search: totalmass

totalmass(t::Tank) -> Float64

Mass of propellant tank t when it is full.
```

Обычно разработчик предоставляет руководство по своей библиотеке с объяснениями о том, как ее использовать. В этом руководстве объясняются существующие интерфейсы и способы их реализации.

В статически типизированном языке, таком как Java, компилятор и изолированная интегрированная среда разработки (IDE)¹ будут информировать разработчиков о методах и их аргументах, необходимых для реализации. Поскольку Julia – динамически типизированный язык, у нас нет такой роскоши. Мы должны документировать свои функции надлежащим образом, чтобы другие разработчики знали, какие аргументы ожидаются и что функции должны возвращать. Как показано в следующем ниже примере, до окончания написания вызова функции можно нажать клавишу **Tab**, чтобы получить список методов и их аргументов, соответствующих тому, что было написано:

```
refill!(tank::Tank)
julia> refill!(
```

❶

❶ Нажать клавишу Tab, после чего появятся доступные методы.

Однако эта стратегия бесполезна для функций `totalmass` и `drymass`, так как к ним не прикреплено никаких методов. Вот почему важно документировать аргументы, требующиеся для этих функций.

12.3 Интерфейсы по соглашению

Как показано в предыдущем примере, в Julia не все интерфейсы присоединены к определенному абстрактному типу. Например, есть *итерационный* интерфейс. Если реализовать этот интерфейс, то свою коллекцию можно будет прокручивать в цикле `for`. Он также позволит использовать ее с такими функциями, как `map`, `reduce` и `filter`, которые работают с итерируемыми коллекциями.

Итерационный интерфейс не представлен каким-либо конкретным абстрактным типом, который необходимо реализовывать. Он описан неформально. Как минимум, от вас как от разработчика ожидается, что функция `iterate` для вашего собственного типа-коллекции будет расширена следующими ниже методами:

Требуемые методы	Предназначение
<code>iterate(iter)</code>	Первый элемент и начальное состояние
<code>iterate(iter, state)</code>	Текущий элемент и следующее состояние

Несколько таких методов подробно описаны в официальной документации Julia. Наиболее полезными из них являются следующие:

Опциональные методы	Предназначение
<code>IteratorSize{IterType}</code>	Указывает на наличие у коллекции известной длины
<code>eltype{IterType}</code>	Тип каждого элемента
<code>length(iter)</code>	Число элементов в коллекции

¹ От англ. Integrated Development Environment. Примерами IDE-сред являются Visual Studio и IntelliJ IDEA.

Я приведу два разных связанных с ракетами примера, в которых мы реализуем некоторые из этих методов. В первом примере мы будем перебирать ракетные двигатели кластера в цикле. Во втором будем прокручивать ступени многоступенчатой ракеты.

12.4 Реализация итеративной обработки кластера двигателей

В главе 8 мы определили кластер ракетных двигателей, как показано ниже.

Листинг 12.5 Старое определение кластера ракетных двигателей

```
struct Cluster <: Engine
    engine::Engine
    count::Int
end
```

При таком определении все двигатели кластера должны быть одного типа. Но что, если требуется сочетание разных типов двигателей? Некоторые ракеты на самом деле имеют смешанные двигатели, и моделировать такие ракеты с имеющимся определением типа Cluster просто невозможно. В целях решения этой проблемы мы превратим тип Cluster в абстрактный. Этот абстрактный тип будет иметь два конкретных подтипа:

- UniformCluster, представляющий идентичные двигатели;
- MixedCluster, представляющий сочетание разных двигателей.

Но зачем вводить второй уровень абстракции? Почему типы UniformCluster и MixedCluster не могут быть прямыми подтипами типа Engine? По мере разработки исходного кода преимущества этого уровня абстракции станут очевидными. Давайте откроем исходный код типа Cluster из главы 8 и видоизменим его, как показано ниже.

Листинг 12.6 Переработанная иерархия типов кластеров

```
abstract type Cluster <: Engine end

struct UniformCluster <: Cluster
    engine::Engine
    count::Int
end

struct MixedCluster <: Cluster
    engines::Vector{Engine}
end

function Cluster(engine::Engine, count::Integer)
```

```

        UniformCluster(engine, count)
    end

    function Cluster(engine::Engine, engines::Engine...)
        sametype(e) = (typeof(engine) == typeof(e)) ❷

        if all(sametype, engines) ❸
            UniformCluster(engine, length(engines) + 1) ❹
        else
            MixedCluster([engine, engines...]) ❺
        end
    end
end

```

- ❶ Вектор элементов, которые являются подтипами типа `Engine`.
- ❷ Определить функцию, проверяющую на совпадение двигателя `e` с первым двигателем.
- ❸ Проверить, что все двигатели имеют одинаковый тип.
- ❹ Вернуть `UniformCluster`, если все двигатели имеют одинаковый тип.
- ❺ Вернуть `MixedCluster`, если двигатели имеют разные типы.

Мы добавляем методы в `Cluster`, которые смотрят на передаваемые на вход типы двигателей, чтобы определять необходимость создания однородного либо смешанного кластера. Здесь мы используем пару новых уловок.

Функция `sametype` определена внутри конструктора типа `Cluster`. Это означает, что она имеет доступ к аргументу `engine` без необходимости передавать его в качестве аргумента. Такой подход удобен тем, что функция более высокого порядка `all` ожидает функцию, которая принимает один аргумент и возвращает `true` либо `false`. Ниже приведено несколько примеров, чтобы дать общее представление.

Листинг 12.7 Демонстрация применения функции `all`

```

julia> iseven(3)
false

julia> iseven(4)
true

julia> all(iseven, [4, 8, 10]) ❶
true

julia> all(iseven, [3, 8, 10]) ❷
false

```

- ❶ В данном случае все числа четные.
- ❷ Число 3 нечетное.

Скрыв тип, используемый для представления кластера, можно создать иллюзию существования только одного типа `Cluster`. То, что внутри используется два разных типа, становится деталью реализа-

ции. Давайте продемонстрируем принцип работы такого подхода в интерактивной среде REPL:

```
julia> Cluster(Rutherford(), Rutherford())
UniformCluster(Rutherford(), 2) ❶

julia> Cluster(Rutherford(), Merlin())
MixedCluster(Engine[Rutherford(), Merlin()]) ❷
```

- ❶ Поскольку все аргументы имеют одинаковый тип, мы получаем `UniformCluster`.
- ❷ Смешанный кластер нужен, чтобы вместить двигатель Мерлина и Резерфорда.

При наличии этих изменений необходимо переопределить функции `Isp`, `mass` и `thrust`. Напомню, что в главе 8 они были определены следующим образом:

```
Isp(cl::Cluster) = Isp(cl.engine)
mass(cl::Cluster) = mass(cl.engine) * cl.count
thrust(cl::Cluster) = thrust(cl.engine) * cl.count
```

Теперь для этих типов кластеров надо реализовать *итерируемый* интерфейс, допускающий написание только одной реализации функций `mass` и `thrust`, которая будет работать для обоих типов кластеров.

12.4.1 Наделение кластеров итерируемостью

Можно попробовать выполнить итеративную обработку кластера в том виде, в каком он определен в настоящее время, но это не работает:

```
julia> cl = Cluster(Rutherford(), 3)
UniformCluster(Rutherford(), 3)

julia> for engine in cl
           println(mass(engine))
       end
ERROR: MethodError: no method matching iterate(UniformCluster)
```

ИТ-компилятор Julia будет конвертировать этот цикл `for` в цикл `while` более низкого уровня, который выглядит как исходный код в следующем ниже листинге.

Листинг 12.8 Реализация цикла `for` в Julia

```
cluster = Cluster(Rutherford(), 3)
next = iterate(cluster) ❶
while next != nothing ❷
    (engine, i) = next ❸
    println(mass(engine))
    next = iterate(cluster, i) ❹
end
```

- ❶ Инициализировать цикл итеративной обработки.
- ❷ Проверить достижение конца цикла.
- ❸ Извлечь значения из кортежа `next`.
- ❹ Продвинуться к следующему элементу коллекции.

Так что цикл `for` не работает, потому что еще не реализованы необходимые методы `iterate`. В следующем ниже листинге показано добавление этих методов, чтобы обеспечить циклическое прокручивание двигателей, составляющих смешанный кластер.

Листинг 12.9 Реализация интерфейса итераций для типа `MixedCluster`

```
import Base: iterate                                ❶

function iterate(cluster::MixedCluster)            ❷
    cluster.engines[1], 2
end

function iterate(cluster::MixedCluster, i::Integer) ❸
    if i > length(cluster.engines)
        nothing                                     ❹
    else
        cluster.engines[i], i+1                     ❺
    end
end
end
```

- ❶ Добавить методы в функцию `iterate`, определенную в модуле `Base`.
- ❷ Используется для запуска цикла итеративной обработки.
- ❸ Продвинуться к следующему элементу коллекции.
- ❹ Вернуть `nothing`, чтобы указать, что был достигнут конец.
- ❺ Текущий элемент и индекс следующего элемента.

Важно импортировать функцию `iterate` из модуля `Base`, так как цикл `for` создан для использования `iterate` из `Base`, а не одноименной функции итерации, определенной в другом модуле. При запуске итеративной обработки нужно вернуть самый первый элемент и индекс следующего элемента. Таким образом, при запуске цикла необходимо вернуть индекс второго элемента. Именно по этой причине мы возвращаем `cluster.engines[1], 2`. Для понимания принципа работы функцию `iterate` можно вызвать вручную:

```
cluster = Cluster(Rutherford(), Merlin());

julia> next = iterate(cluster) ❶
(Rutherford(), 2)

julia> (engine, i) = next      ❷
(Rutherford(), 2)

julia> next = iterate(cluster, i) ❸
(Merlin(), 3)
```

```
julia> (engine, i) = next           ❷
(Merlin(), 3)

julia> next = iterate(cluster, i) ❸
                                   ❹
```

- ❶ Получить первоначальное состояние.
- ❷ Извлечь из кортежа `next` двигатель и следующий индекс.
- ❸ Взять следующий двигатель.
- ❹ Достигнут конец, поэтому `next` равен `nothing`.

Теперь реализация функции `iterate` для `UniformCluster` в следующем ниже листинге должна иметь больше смысла.

Листинг 12.10 Реализация функции `iterate` для `UniformCluster`

```
import Base: iterate

function iterate(cluster::UniformCluster)
    cluster.engine, 2
end

function iterate(cluster::UniformCluster, i::Integer)
    if i > cluster.count
        nothing
    else
        cluster.engine, i+1
    end
end
```

Как видите, данная реализация стала проще, потому что всегда возвращается тот же самый двигатель. Индекс `i` используется только для отслеживания достижения конца цикла. Поскольку оба типа `Cluster` теперь поддерживают итеративную обработку, функции `mass` и `thrust` можно реализовать в терминах цикла, как показано в следующем ниже листинге.

Листинг 12.11 Определение функций `mass` и `thrust` для кластеров

```
mass(cluster::Cluster) = sum(mass, cluster)
thrust(cluster::Cluster) = sum(thrust, cluster)
```

Как это работает? Функция `sum` прокручивает коллекцию, переданную в качестве второго аргумента. При этом `sum` будет применять функцию, указанную в качестве первого аргумента, к каждому элементу, получаемому на каждой итерации цикла. Вызов `sum(thrust, cluster)` эквивалентен написанию `sum(map(thrust, cluster))`. Ни один из вызовов не будет работать до тех пор, пока не будет реализована функция `length` для кластеров (листинг 12.12); в противном случае Julia не сможет определять длину результирующего вектора перед началом итеративной обработки.

Листинг 12.12 Предоставление языку Julia способа выяснять количество двигателей в кластере

```
import Base: length

length(cluster::UniformCluster) = cluster.count
length(cluster::MixedCluster) = length(cluster.engines)
```

❶ Расширить функцию `length` для поддержания типов кластеров.

Напомню, что методы `sum` могут принимать как один, так и два аргумента. Для `Isp` значения не суммируются; там придется находить среднее значение, как показано ниже.

Листинг 12.13 Расчет удельного импульса кластера двигателей

```
Isp(cl::Cluster) = sum(Isp, cl)/length(cl)
```

Приведенный выше исходный код обеспечивает поддержку функции `length` со стороны коллекции, что разумно делать для большинства коллекций. И это совершенно естественно, так как разработчики хотят иметь функциональность проверки количества содержащихся в коллекции элементов.

При наличии таких изменений должна стать очевиднее причина, по которой тип `Cluster` был сделан абстрактным. Благодаря такому подходу появляется возможность использовать реализацию функций `mass`, `Isp` и `thrust` совместно в нескольких типах кластеров. Использование абстрактных типов – это неплохой способ реиспользования исходного кода.

Далее мы обследуем итеративную обработку ступеней ракеты. Она будет немного отличаться, так как невозможно обращаться к ступеням ракеты по индексу.

12.5 Реализация итеративной обработки ступеней ракеты

В следующем ниже листинге показано определение ступени ракеты, использованное в главе 8.

Листинг 12.14 Определение ступени ракеты

```
struct StagedRocket <: Rocket
    nextstage::Rocket
    tank::Tank
    engine::Engine
end
```

Обратите внимание, что здесь нет вектора, из которого можно извлекать отдельные ступени. И поэтому элемент кортежа, который будет возвращаться из функции `iterate`, не будет целочисленным индексом.

Листинг 12.15 Инициализировать цикл итеративной обработки многоступенчатой ракеты

```
import Base: iterate

iterate(r::StagedRocket) = (r, r.nextstage)
iterate(r::Rocket) = nothing
```

Исходный код листинга 12.15 обрабатывает два разных случая:

- многоступенчатая ракета с полезной нагрузкой;
- все другие ракеты, которые не имеют ступеней и, следовательно, не имеют элемента `next`.

Это означает, что методы `iterate` для каждого возможного подтипа типа `Rocket` добавлять не требуется. Вместо этого типы `Rocket` по умолчанию делаются без поддержки итеративной обработки. Кроме того, необходимо поддерживать продвижение по коллекции ступеней, что и будут делать методы `iterate` в следующем ниже листинге.

Листинг 12.16 Продвижение к следующей ступени ракеты

```
function iterate(first::StagedRocket, current::StagedRocket)
    current, current.nextstage ❶
end

function iterate(first::StagedRocket, current::Rocket) ❷
    nothing
end
```

- ❶ Вызывается, когда следующая ступень тоже является многоступенчатой ракетой.
- ❷ Применяемая по умолчанию обработка итеративного перехода к следующей ступени.

Мы определили эти новые методы таким образом, что по умолчанию итеративная обработка прекращается. Это делается путем задания типа аргумента `current` как `Rocket`. И тогда, когда `current` имеет тип `StagedRocket`, делается исключение. В этом случае мы знаем, что есть поле `nextstage`, к которому можно обратиться, чтобы продвигаться к следующему элементу коллекции.

Таким образом, хотя первый пример с типом `Cluster` создает впечатление, что `current` является целочисленным индексом, это не совсем так. Второй аргумент функции `iterate` не обязательно должен быть целым числом. Он может быть любым значением, которое поможет запоминать текущую позицию в прокручиваемой коллекции. Итеративную обработку можно протестировать с помощью неболь-

шой программы, поместив исходный код следующего ниже листинга в интерактивную среду REPL или файл, который загружается в интерактивную среду REPL.

Листинг 12.17 Прокручивание ступеней ракеты в цикле

```
payload = Payload(300)

thirdstage = Rocket(payload, SmallTank(), Curie())
secondstage = Rocket(thirdstage, MediumTank(), Rutherford())
booster = Rocket(secondstage, LargeTank(), Cluster(Rutherford(), 9))

for rocket in booster
    println("Mass of rocket: ", mass(rocket))
    println("Thrust of rocket: ", thrust(rocket))
    println()
end
```

Сначала мы проверяем, чтобы связанный с ракетой исходный код был загружен в интерактивную среду REPL. При выполнении этой программы должен быть получен следующий ниже результат:

```
Mass of rocket: 13568.0
Thrust of rocket: 225000

Mass of rocket: 3053.0
Thrust of rocket: 25000

Mass of rocket: 718.0
Thrust of rocket: 120
```

Он показывает, что итеративная обработка внутри цикла `for` работает, как и ожидалось. Однако ее нельзя использовать с такими функциями, как `sum`, `map` и `collect`. В следующем сеансе REPL показана безуспешная попытка применения функций `map` и `collect` на разгонном блоке.

```
julia> map(mass, booster)
ERROR: MethodError: no method matching length(::StagedRocket)

julia> collect(booster)
ERROR: MethodError: no method matching length(::StagedRocket)
```

Следующим шагом будет приведение этой функциональности в рабочее состояние.

12.5.1 Добавление поддержки функций `map` и `collect`

Функции `map` и `collect` не работают, потому что не хватает реализации метода `length` в типе `Rocket`. Для того чтобы выработать понимание этой задачи, я сначала покажу следующее ниже простое решение.

Листинг 12.18 Расчет количества ступеней многоступенчатой ракеты

```
import Base: length

length(::Rocket) = 0
length(r::StagedRocket) = 1 + length(r.nextstage)
```

Хотя оно работает, у него плохие характеристики производительности. Время, необходимое для расчета длины многоступенчатой ракеты, пропорционально ее длине. Такие алгоритмы называются *линейными*, или $O(n)$ в нотации «О» большое.

Нотация «О» большое

В информатике часто упоминаются требования к памяти и вычислительной мощности структур данных и алгоритмов в так называемой нотации «О» большое. Если время, необходимое для отыскания алгоритмом элемента, является линейным (например, зависит от числа элементов в коллекции), то мы записываем это как $O(n)$. При этом n относится к n элементам во всей коллекции. Таким образом, если n удваивается, то для завершения обработки алгоритму $O(n)$ потребуется вдвое больше времени. Алгоритм, в котором для просмотра каждого элемента используется цикл `for`, называется алгоритмом $O(n)$, а алгоритм с постоянным временем записывается как $O(1)$.

Julia предоставляет способ сообщать ее итерационной машине, что не существует эффективного метода определения длины коллекции, прокручиваемой в цикле. Разработчик делает это сам путем реализации метода `IteratorSize`, как показано ниже.

Листинг 12.19 Добавление черты `SizeUnknown` в подтипы типа `Rocket`

```
import Base: iterate, IteratorSize

IteratorSize{::Type{<:Rocket}} = Base.SizeUnknown()
```

На первый взгляд указанную концепцию не так просто усвоить. Это так называемый шаблон Холи по приданию типам черт¹. Шаблоны в программировании относятся к конкретным способам решения задач, которые реиспользуются в различных контекстах. В Julia

¹ Шаблон Холи по приданию типам черт (Holy traits pattern), также трюк Тима Холи, назван в честь Тима Холи, который давно участвует в развитии языка и экосистемы Julia. В двух словах, черта соответствует поведению объекта. Например, птицы и бабочки могут летать, поэтому у них обеих есть черта `CanFly`. Черты используются в качестве формального договора о том, как использовать тип данных. Например, если объект имеет черту `CanFly`, то можно быть уверенными в том, что он будет летать. — *Прим. перев.*

шаблон Холи по наделению чертами применяется для добавления черт в типы. *Черта* похожа на способность или характеристику (рис. 12.3). Например, у лучника может быть черта `CanShoot`; рыцарь может иметь черту `CanRide`; а конный лучник, такой как конный воин-кочевник, может одновременно иметь черты `CanShoot` и `CanRide`.

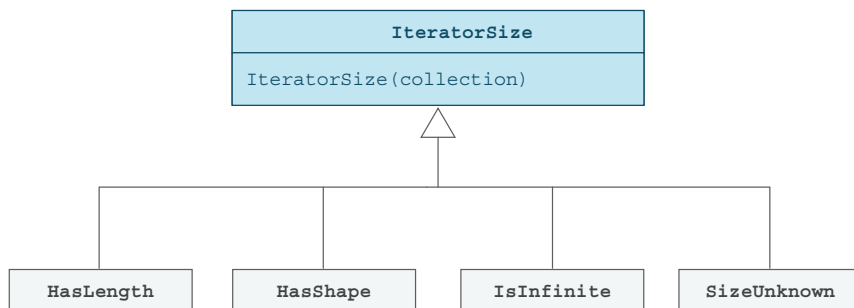


Рис. 12.3 Черта `IteratorSize` и ее подтипы

Итерируемые коллекции Julia могут иметь разные черты, одной из которых является `IteratorSize`. Как именно она работает? При вызове `collect(rocket)` она преобразовывается в следующий ниже исходный код.

Листинг 12.20 Реализация функции `collect`

```
_collect(rocket, IteratorEltype(rocket), IteratorSize(rocket)).
```

К функции `_collect` прикреплено несколько разных методов работы с коллекциями с разными чертами. Здесь видно, что поведение `collect` зависит от двух разных черт: `IteratorEltype` и `IteratorSize`. Черты своих собственных коллекций не требуется всегда регистрировать, потому что стандартного поведения, как правило, будет достаточно. В следующем ниже листинге показано определение стандартного поведения `IteratorSize`.

Листинг 12.21 Применяемое по умолчанию поведение `IteratorSize` для коллекций Julia

```
IteratorSize(x) = IteratorSize(typeof(x))
IteratorSize{::Type} = HasLength()
```

Когда черта `IteratorSize` определена как `HasLength`, Julia будет вызывать `length`, чтобы определять размер результирующего массива, сгенерированного в результате вызова `collect`. При определении указанной черты как `SizeUnknown` вместо этого на выходе будет использоваться пустой массив, который будет увеличиваться по мере необходимости.

В Julia черты определяются как абстрактные типы. Принимаемые

чертой значения определяются конкретным подтипом. `HasLength` и `SizeUnknown` являются подтипами типа `IteratorSize`. Возможно, вы узнали похожий шаблон из главы 9: обе функции `convert` и `promotion_rule` на входе принимают типы; однако в данном случае добавляется небольшая изюминка, состоящая из описания типа аргумента как `Type{<:Rocket}`.

Оператор подтипа `<:` указывает на то, что *все* подтипы типа `Rocket` имеют подтип `SizeUnknown` черты `IteratorSize`. Знаю, что это уже чересчур, но, к счастью, вы всегда сможете свериться с этим описанием, когда это будет необходимо. Знать о чертах гораздо важнее, чем помнить о том, как именно они работают. Определив черту `IteratorSize` для `Rocket`, появляется возможность использовать `map`, `sum` и другие функции более высокого порядка, оперирующие на коллекциях:

```
julia> booster = Rocket(secondstage,
                        LargeTank(),
                        Cluster(Rutherford(), 9)); ❶

julia> map(mass, booster) ❷
3-element Vector{Float64}:
 13568.0
  3053.0
   718.0

julia> sum(thrust, booster) ❸
250120
```

- ❶ Не забыть определить второе и третье состояния, если это еще не было сделано.
- ❷ Получить массу каждой ступени.
- ❸ Сложить тягу всех двигателей многоступенчатой ракеты.

Итеративная обработка – это самая базовая операция, которая должна поддерживаться коллекцией. Вы увидели ее реализацию в двух разных типах коллекций. В сущности, ракетные кластеры ведут себя как массивы; однако многоступенчатая ракета больше похожа на структуру данных, именуемую *связным списком*. В следующем разделе вы сравните связанные списки и массивы, чтобы лучше понять компромиссы, предлагаемые разными типами коллекций.

12.6 Сравнение связанных списков и массивов

Связывание ступеней друг с другом посредством `nextstage` аналогично тому, как работает *связный список*: они часто противопоставляются массивам. С массивом можно быстро определять длину или выбирать произвольный элемент по индексу. Здесь мы добавим поддержку индексного просмотра в свои подтипы типа `Cluster`, а позже сравним его со связным списком ступеней ракеты.

Напомним, что циклы `for` фактически превращаются в циклы `while`. На самом деле аналогичное происходит и при доступе и замене элементов в массиве. Допустим, мы создали кластер, в котором обращаемся к элементам.

Листинг 12.22 Доступ по индексу

```
cluster = Cluster(Rutherford(), Merlin())
engine = cluster[1]
cluster[2] = Rutherford()
```

- ❶ Прочитать первый элемент.
- ❷ Изменить второй элемент.

Компилятор Julia будет транслировать исходный код листинга 12.22 в несколько этапов. Один из этих этапов, именуемый *понижением*¹, превращает этот исходный код в следующий ниже.

Листинг 12.23 Доступ по индексу под капотом

```
cluster = Cluster(Rutherford(), Merlin())
engine = getindex(cluster, 1)
setindex!(cluster, Rutherford(), 2)
```

Таким образом, для того чтобы подтипы типа `Cluster` поддерживали доступ к элементам по индексу, нужно добавить методы в функции `getindex` и `setindex!`, находящиеся в модуле `Base`. Вот функции, используемые для реализации доступа к элементам посредством квадратных скобок: `[]`.

Листинг 12.24 Добавление доступа по индексу в ракетные кластеры

```
import Base: getindex, setindex!

function getindex(cluster::MixedCluster, i::Integer)
    cluster.engines[i]
end

function setindex!(cl::MixedCluster, engn::Engine, i::Integer)
    cl.engines[i] = engn
end

function getindex(cluster::UniformCluster, _)
    cluster.engine
end
```

- ❶ Получить *i*-й элемент кластера.
- ❷ Установить для *i*-го элемента кластера значение `engn`.
- ❸ Индекс для `UniformCluster` здесь не интересует.

¹ Англ. lowering. – Прим. перев.

Между `MixedCluster` и `UniformCluster` легко увидеть некоторые различия, и есть возможность легко поддерживать получение элементов из `UniformCluster`, потому что все элементы одинаковы. Но при этом нет возможности устанавливать значение элемента, потому что он больше не будет однородным. И значит, мы не добавили метод в функцию `setindex!` для работы с `UniformCluster`. Хотя для связного списка вполне можно определить индексный доступ, он не будет очень эффективным. Это показано в следующем ниже листинге.

Листинг 12.25 Доступ к ступени ракеты по индексу

```
import Base: getindex

function getindex(r::StagedRocket, i::Integer)
    for _ in 1:i-1
        r = r.nextstage
    end
    r
end
```

❶ `_` используется, когда имя переменной не интересует.

Такой поиск является операцией $O(n)$ (линейной). Чем больше ступеней, тем больше итераций придется делать в цикле `for`. Поиск в массиве, напротив, является операцией $O(1)$. Другими словами, это *постоянная операция*. Время поиска элемента в массиве из трех элементов точно такое же, как и в массиве с миллионами элементов.

С другой стороны, можно добавить поддержку других типов операций, которые работают быстрее. Давайте рассмотрим несколько поддерживаемых в Julia наиболее распространенных операций добавления элементов в коллекции и удаления элементов из коллекций.

12.6.1 Добавление и удаление элементов

Массивы позволяют добавлять элементы в начало и конец массива, а также удалять элементы с обоих концов. Обратите внимание на то, как функции `push!` и `pushfirst!` позволяют добавлять более одного элемента за один вызов функции.

```
julia> xs = Int[7]
1-element Vector{Int64}:
 7

julia> push!(xs, 9, 11)
3-element Vector{Int64}:
 7
 9
11
```

```
julia> pushfirst!(xs, 3, 5) ③
5-element Vector{Int64}:
 3
 5
 7
 9
11
 3

julia> pop!(xs) ④
11

julia> popfirst!(xs)
3
```

- ① Создать массив xs с одним элементом 7.
- ② Добавить 9 и 11 в конец массива.
- ③ Добавить 3 и 5 в начало массива.
- ④ Удалить последний элемент.
- ⑤ Удалить первый элемент.

Рисунок 12.4 поможет прояснить принцип работы этих операций.

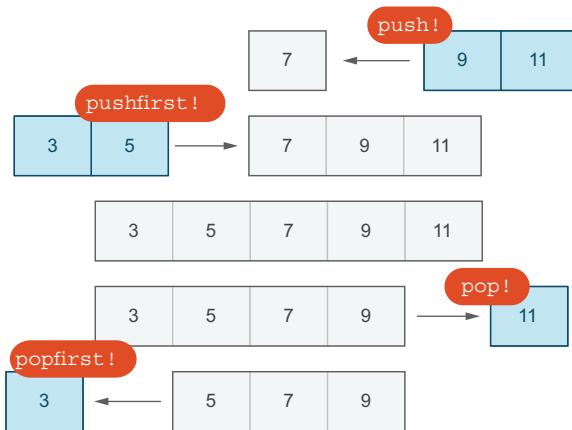


Рис. 12.4 Добавление элементов в массив и удаление из него

Для связанных списков эффективным будет удаление и добавление элементов, расположенных в начале списка (рис. 12.5). Таким образом, можно поддерживать такие операции, как `pushfirst!` и `popfirst!`. Однако, для того чтобы сделать реализацию этих функций удобнее, потребуется внести несколько корректив и выполнить подготовительную работу.

При работе со связными списками полезно иметь что-то очевидное для завершения цепочки объектов. Нередко это будет объект `nothing`, но в случае с многоступенчатой ракетой естественным подходом будет пустая полезная нагрузка (`emptypayload`).

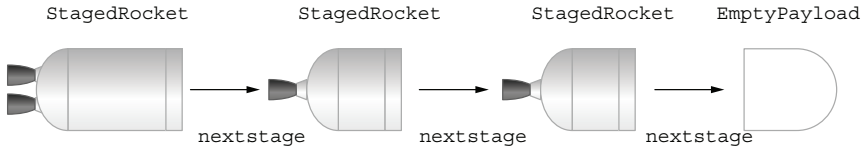


Рис. 12.5 Завершенный связный список ступеней ракеты

Листинг 12.26 Объект пустой полезной нагрузки для завершения списка ступеней ракеты

```

struct EmptyPayload <: Rocket end           ❶
const emptypayload = EmptyPayload()        ❷

mass(::EmptyPayload) = 0.0                 ❸
thrust(r::EmptyPayload) = 0                ❸
update!(r::EmptyPayload, t::Number, ?t::Number) = nothing ❸

function Rocket(tank::Tank, engine::Engine)
    StagedRocket(emptypayload, tank, engine)
end

```

- ❶ Полей нет.
- ❷ Каждый экземпляр имеет те же объекты, что и с `nothing`.
- ❸ Реализовать интерфейс ракеты.

Представление пустой полезной нагрузки дает ряд преимуществ, например наличие разумного стандартного конструктора одноступенчатой ракеты, как показано в листинге 12.26. Однако пример не совсем закончен. Нужен новый тип, и нужно видоизменить существующий. Тип `StagedRocket` изначально сделан немутуируемым, что будет мешать, например, работать функции `popfirst!`, так как нужно видоизменять поле `nextstage`. Поля немутуируемых объектов видоизменять нельзя.

ВАЖНО При каждом изменении определения типа, например делая структуру `struct` мутуируемой, необходимо перезапускать интерактивную среду REPL. Это связано с тем, что типы Julia фиксированы; как и во многих других динамических языках, их нельзя видоизменять во время выполнения программы.

Далее я представлю тип `SpaceVehicle` (листинг 12.27). На рис. 12.6 показана его концептуальная связь с другими частями ракеты.

Тип `SpaceVehicle` – это единое целое со всеми ступенями ракеты. Данная абстракция полезна для обертывания ступеней, чтобы иметь возможность отслеживать начало первой ступени ракеты. Это удобно при реализации функций `pushfirst!` и `popfirst!`, потому что позволяет добавлять и удалять ступени относительно чего-то еще.

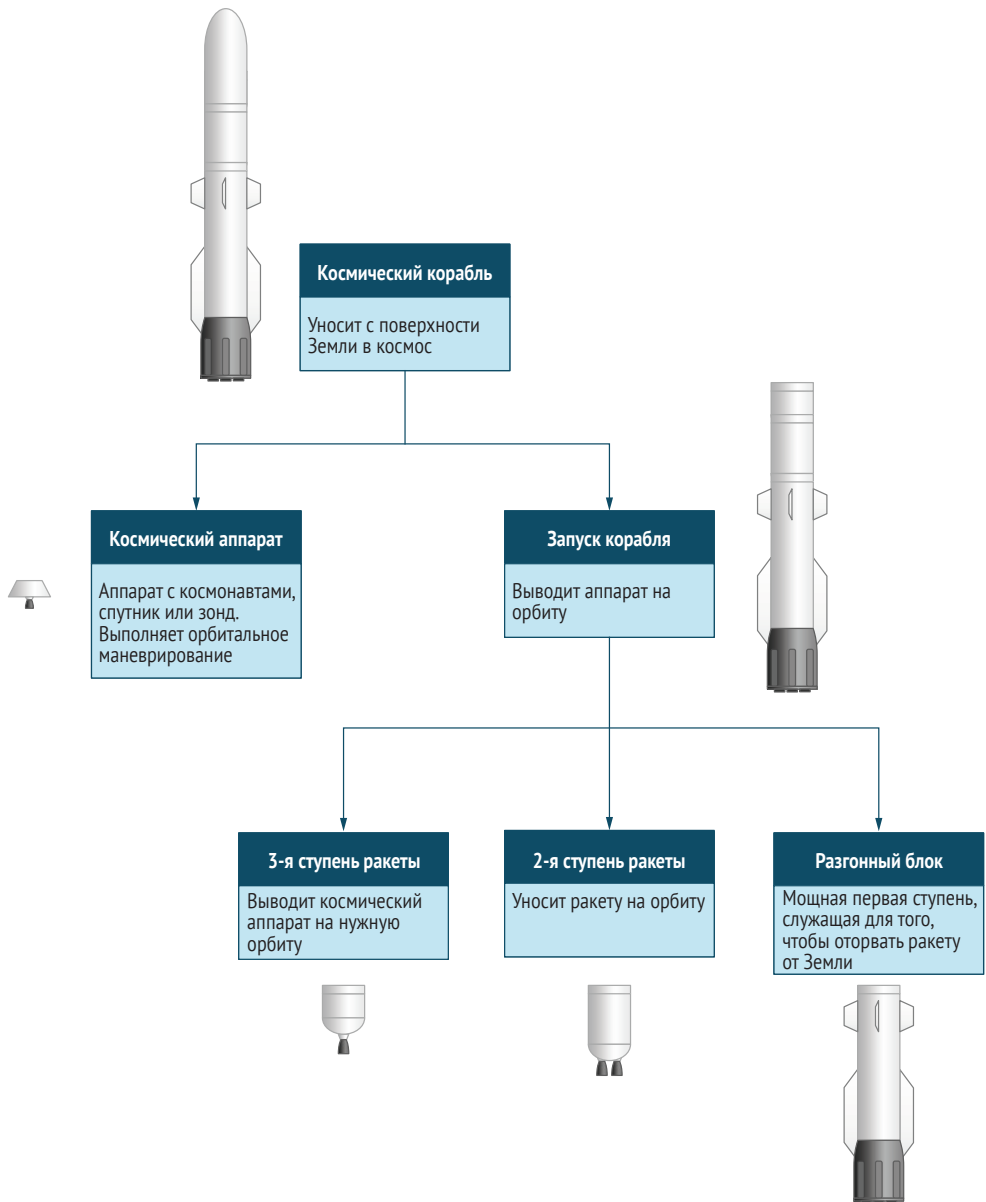


Рис. 12.6 Общий обзор ракетной терминологии

Листинг 12.27 Новое и видоизмененное определения типов

```

mutable struct StagedRocket <: Rocket
    nextstage::Rocket
    tank::Tank
    engine::Engine
end

```

1
2


```
mutable struct SpaceVehicle           ③
    activestage::Rocket               ④
end
```

- ① Изменить StagedRocket на мутируемый.
- ② Нужно, чтобы следующая ступень nextstage была изменяемой.
- ③ Обертка вокруг ступеней ракеты.
- ④ Ступень с работающими в данный момент двигателями.

При наличии указанных выше определений типов появляется основа для реализации своих собственных методов функций `popfirst!` и `pushfirst!` (листинг 12.28). Поскольку они являются стандартными для коллекций Julia функциями, они импортируются из `Base`, чтобы расширить их методами, работающими с конкретно-прикладной коллекцией: `SpaceVehicle`.

Листинг 12.28 Удаление ступеней снизу

```
import Base: popfirst!, pushfirst!

tail(r::StagedRocket) = r.nextstage
tail(r::Rocket) = nothing

function popfirst!(ship::SpaceVehicle)
    r = tail(ship.activestage)
    if r == nothing                               ①
        throw(throw(ArgumentError
            ➔("no rocket stages left")))          ②
    else
        discarded = ship.activestage              ③
        discarded.nextstage = emptypayload        ④
        ship.activestage = r                      ⑤
    end
    discarded                                    ⑥
end
```

- ① Проверить, не осталось ли еще ступеней.
- ② Не разрешается выталкивать ступени из пустого космического корабля.
- ③ Отбросить нижнюю ступень.
- ④ Отцепить отброшенную ступень.
- ⑤ Следующая ступень становится активной ступенью.
- ⑥ Показать в интерактивной среде REPL отброшенную ступень.

Функция `tail` требует пояснения. В нее добавляется два метода: один для обработки типа `StagedRocket` и другой для подтипов типа `Rocket`. Это простой способ проверки на наличие у космического корабля оставшихся ступеней. Поскольку поле `activestage` типа `SpaceVehicle` имеет тип `Rocket`, нельзя гарантировать, что существует значение поля `nextstage`. Почему тогда не присвоить ему тип `StagedRocket`? Потому что мы хотим отсоединять ступени до тех пор, пока не останется только полезная нагрузка, представляющая спутник или капсулу экипажа.

Листинг 12.29 Добавление ступени снизу

```
function pushfirst!(ship::SpaceVehicle, r::StagedRocket)
    r.nextstage = ship.activestage ❶
    ship.activestage = r             ❷
    ship
end
```

- ❶ Текущая ступень становится следующей ступенью.
- ❷ Новая ступень становится текущей ступенью.

Функция `pushfirst!` помещает новую ступень `r` перед существующей активной ступенью (листинг 12.29). Старая активная ступень меняет свою роль, становясь следующей ступенью новой активной ступени. После внесения всех этих правок и видоизменений можно запустить новую интерактивную среду REPL, чтобы понять принцип работы этих новых функций. Для большей наглядности я приведу пример ракеты со ступенями `a`, `b` и `c`. Каждая ступень имеет более крупные топливные баки и более крупный двигатель, чем предыдущая:

```
julia> a = Rocket(SmallTank(), Curie());
julia> b = Rocket(MediumTank(), Rutherford());
julia> c = Rocket(LargeTank(), Merlin());
```

Пример начинается с создания космического корабля с малой 40-килограммовой полезной нагрузкой:

```
julia> ship = SpaceVehicle(Payload(40));
```

Далее для добавления к этому космическому кораблю ступеней используется функция `pushfirst!`:

```
julia> pushfirst!(ship, a)
SpaceVehicle(StagedRocket(Payload(40.0), SmallTank(370.0), Curie()))

julia> pushfirst!(ship, b)
SpaceVehicle(StagedRocket(
    StagedRocket(
        Payload(40.0),
        SmallTank(370.0),
        Curie()),
    MediumTank(2050.0),
    Rutherford()))

julia> pushfirst!(ship, c)
SpaceVehicle(StagedRocket(
    StagedRocket(
        StagedRocket(
            Payload(40.0),
            SmallTank(370.0),
            Curie()),
        MediumTank(2050.0),
        Rutherford()),
    LargeTank(),
    Merlin()))
```

```

        MediumTank(2050.0),
        Rutherford()),
    LargeTank(9250.0),
    Merlin()))

```

Я отредактировал результат интерактивной среды REPL, добавив пробелы и отступы, чтобы четче показать создаваемую структуру при добавлении дополнительных ступеней. Вы видите, что вследствие своей максимальной вложенности самая малая ступень находится на самом глубоком уровне отступа. Внизу находится разгонная ступень с большим двигателем и топливным баком.

С помощью функции `popfirst!` можно отбросить все эти ступени снова:

```

julia> popfirst!(ship)
StagedRocket(EmptyPayload(), LargeTank(9250.0), Merlin())

julia> popfirst!(ship)
StagedRocket(EmptyPayload(), MediumTank(2050.0), Rutherford())

julia> popfirst!(ship)
StagedRocket(EmptyPayload(), SmallTank(370.0), Curie())

julia> ship
SpaceVehicle(Payload(40.0))

```

При каждом выталкивании возвращается ступень, которая была отброшена. Обратите внимание, как отделяется первой самая большая ступень с двигателем `Merlin`. Следующая ступень включает в себя топливный бак среднего размера. Верхняя ступень с малым топливным баком идет последней. Наконец, космический корабль остается только с начальной 40-килограммовой полезной нагрузкой.

12.7 Полезность конкретно-прикладных типов

В реальном исходном коде связный список используется редко, поскольку массивы более универсальны и в большинстве случаев имеют более высокую производительность. Тем не менее связные списки полезны для понимания, поскольку применяемые здесь принципы применимы к более сложным структурам данных. Древовидные структуры и графы также связывают элементы вместе.

Хотя вы вряд ли будете тратить много времени на написание совершенно обобщенных структур данных, таких как массивы, связные списки и словари, вы найдете для себя полезным превращать свои собственные конкретно-прикладные структуры данных (ступени ракеты) в коллекции. После того как в структуре данных будет реализован интерфейс хорошо зарекомендовавшей себя категории типов, такой как коллекция, вы получаете серию применимых к ним

функций. Например, сделав многоступенчатую ракету итерируемой, неожиданно для себя вы задаром получаете возможность вместе с ней использовать функции `map`, `reduce`, `filter`, `sum`, `collect` и другие.

Резюме

- Циклы и индексный доступ с квадратными скобками преобразовываются в вызовы функций `Julia iterate`, `getindex` и `setindex!`.
- Коллекции должны как минимум поддерживать итерации. Это делается путем реализации двух методов `iterate` для типа-коллекции: один – для запуска итерации, а другой – для получения следующего элемента.
- Типы `Julia` можно наделять различными возможностями с помощью шаблона Холи по наделению так называемыми чертами, то есть особым поведением. Коллекции `Julia` можно конфигурировать различными чертами, такими как `IteratorSize` и `IteratorEltypes`.
- Коллекции, у которых вычисление количества элементов происходит медленно, следует конфигурировать чертами `IteratorSize` на `SizeUnknown`.
- Массивы обеспечивают быстрый доступ к элементам по любому индексу, но не позволяют быстро вставлять элементы, кроме как в конец массива.
- Связные списки характеризуются медленным доступом к элементам по индексу, зато быстрой вставкой и удалением элементов в начале списка.
- Реализация хорошо зарекомендовавших себя интерфейсов `Julia` будет делать конкретно-прикладные типы данных более универсальными. Например, если превратить свою собственную структуру данных в коллекцию `Julia`, то появляется возможность использовать многие встроенные в `Julia` функции, связанные с коллекцией.

13

Работа с множествами

Эта глава охватывает следующие ниже темы:

- сопоставление различий между множествами и массивами;
- разные способы создания множеств;
- использование операций объединения и пересечения для отыскания элементов в разных типах программного обеспечения;
- понимание компромиссов между использованием множеств и поисковых операций.

Для того чтобы определить множество и показать, для каких операций его можно использовать, не нужно много времени. Что действительно требует времени, так это развитие интуитивного понимания характера задач, которые можно решать с помощью множеств.

Многие задачи, связанные с организацией и размещением данных, можно прекрасно решать, используя множества и операции на множествах, но это не всегда очевидно. В данной главе я расскажу о том, что такое множества, и о том, что с ними можно делать, а затем познакомлю с разными реалистичными примерами, демонстрирующими возможности хранения данных в множествах.

13.1 Какие задачи можно решать с помощью множеств?

Многие программы требуют организации больших объемов данных, в том числе следующих:

- фотоальбомы;
- почтовые клиенты;
- инструменты отслеживания программных дефектов;
- интернет-магазины;
- проекты по разработке программного обеспечения;
- специальное программное обеспечение, такое как ПО по геологическому моделированию.

Долгое время самым популярным способом организации данных были древовидные структуры. При отыскании элемента углубляются в подкатегории до тех пор, пока он не будет найден. Проблема данного подхода в том, что многие элементы потенциально могут существовать в нескольких подкатегориях, а не только в одной.

Интернет-магазин, такой как McMaster-Carr (рис. 13.1), в котором продается огромное количество механических компонентов, является отличным примером этой проблемы.

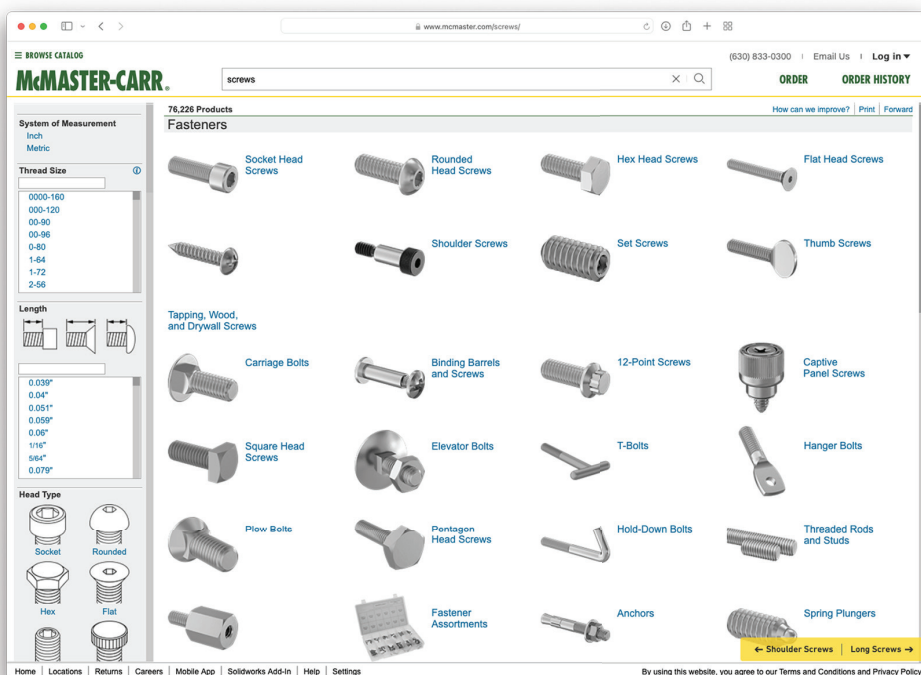


Рис. 13.1 Интернет-магазин McMaster-Carr, показывающий, что винты можно классифицировать по-разному

В боковой части экрана показаны различные категории винтов:

- *Система измерения* – указаны ли размеры винтов в имперских либо метрических единицах?
- *Размер резьбы* – каков фактический размер резьбы и резьбы на дюйм?
- *Материал* – изготовлен ли винт из стали, пластика или дерева?
- *Длина* – какова длина от головки винта до конца резьбы?
- *Тип головки* – плоский, закругленный или шестигранный?

Категорий гораздо больше, чем показано выше. Однако главное тут в том, что все это невозможно превратить в древовидную иерархию. Например, закругленная головка может иметься как у пластикового, так и у стального винтов.

Еще один случай – фотоальбомы. Как их организовывать? Можно организовать фотографии каждого члена семьи, чтобы у жены и у каждого ребенка был свой альбом. Либо, возможно, больше смысла будет иметь организация альбомов на основе событий, таких как посещение Барселоны или Гавайев. Или же, может быть, кто-то предпочтет организацию на основе деятельности, например создав альбомы для определенных типов достопримечательностей, таких как технологические музеи или зоопарки. Как видите, довольно трудно организовать фотографии в некую систему, поэтому давайте посмотрим, как множества и операции на множествах помогают в выполнении этой работы.

13.2 Что такое множество?

Множество – это тип-коллекция, подобный массивам или словарям. Ниже приведены примеры создания множества:

```
julia> fruits = Set(["apple", "banana", "pear", "orange"])
Set{String} with 4 elements:
```

```
"pear"
"orange"
"banana"
"apple"
```

```
julia> fruits = Set{Symbol}([:apple, :banana, :pear, :orange]) ❶
Set{Symbol} with 4 elements:
```

```
:pear
:apple
:banana
:orange
```

```
julia> odds = Set{Int64}([1, 3, 5, 7, 9])
Set{Int64} with 5 elements:
```

```
5
7
9
```

```

3
1

julia> nodups = Set([3, 3, 3, 1, 2])
Set{Int64} with 3 elements:
 2
 3
 1
    
```

- ❶ Специальные символы вместо строковых литералов.
- ❷ Попытка добавить дубликаты.

В первом случае создается множество фруктов, в котором каждый фрукт представлен строковым литералом. Второй случай аналогичен, но вместо строковых литералов для представления фруктов используются символические имена. Этот пример полезен тем, что показывает, как в Julia символические имена нередко используются для представления ключей.

Множества подобны ключам словаря; ни один элемент не встречается дважды. Обратите внимание, что в последнем примере `nodups` была сделана попытка добавить 3 несколько раз. Подобно словарю, элементы не существуют в каком-либо определенном порядке. При прокручивании множества в цикле элементы появляются в каком-то порядке. Однако у нас не будет никакого контроля над этим порядком. Если добавить или удалить элементы, то порядок может измениться.

Такое поведение отличается, скажем, от массива, в котором у нас есть полный контроль над тем, как добавление и удаление элементов влияет на порядок следования элементов массива. Если добавить элемент в массив с помощью функции `push!`, то каждый элемент остается в той же позиции, что и раньше. К каждому элементу можно обращаться по тому же индексу, что и раньше.

13.2.1 Сравнение свойств множеств и массивов

Сравнение свойств множества со свойствами массива (табл. 13.1) поможет лучше понять отличительные особенности множества.

Таблица 13.1. Различия и сходства между множествами и массивами

Свойство	Множества	Массивы
Дубликаты разрешены	Нет	Да
Элементы упорядочиваются	Нет	Да
Произвольный доступ	Нет	Да
Быстрая проверка на принадлежность	Да	Нет
Итерируемость	Да	Да

Ниже приведены два желательных свойства, предлагаемых множествами:

- 1 множества гарантированно не содержат повторяющихся элементов;
- 2 позволяют очень быстро проверять наличие конкретного объекта в множестве.

Массивы, напротив, не позволяют быстро проверять наличие в них конкретного объекта, потому что для определения наличия/отсутствия в массиве элемента необходимо просматривать каждый элемент массива. Поэтому поиск определенного элемента в массиве из двух миллионов элементов в среднем будет занимать в два раза больше времени, чем его поиск в массиве из одного миллиона элементов.

Это называется *линейной* взаимосвязью. С другой стороны, количество операций, необходимых для отыскания элемента в множестве, не увеличивается вместе с размером множества. Линейная взаимосвязь отсутствует.

Множества реализуются по-разному; так, в некоторых вариантах для отыскания элемента в множестве из n элементов в среднем требуется $\log(n)$ проверок. Для того чтобы помочь лучше разобраться в преимуществах использования множеств, давайте сравним множества с массивами на примере разных типов операций, под которые множества оптимизированы.

ПОИСК В ОТСОРТИРОВАННОМ МАССИВЕ

В отсортированном массиве можно выполнять двоичный поиск. Возьмем следующий ниже отсортированный массив чисел, чтобы понять, как работает двоичный поиск:

`A = [2, 7, 9, 10, 11, 12, 15, 18, 19]`

Приведенный выше отсортированный массив имеет 9 чисел. Допустим, мы ищем число 18. Число находится где-то в диапазоне 1:9 (индексном диапазоне). Обычно для отыскания числа требуется 8 сравнений, но при двоичном поиске мы начинаем с середины `A[5] == 11` и проверяем, не является ли `18 > 11` или `18 < 11`.

Поскольку массив отсортирован, можно сделать вывод, что 18 находится где-то в верхней половине массива, или, точнее, в диапазоне индексов 6:9. Указанный процесс поиска повторяется путем проверки середины этого диапазона. Поскольку в указанном диапазоне середины нет, то индекс округляется до `A[7] == 15`. Мы обнаружим, что 18 выше этого значения. Следовательно, ответ можно найти за 3 сравнения, а не за 8. Для этого типа поиска в Julia есть несколько функций:

```
julia> searchsorted(A, 18)
8:8
```

```
julia> searchsortedfirst(A, 18)
8
```

```
julia> searchsortedlast(A, 18)
8
```

Недостатком использования отсортированных массивов является то, что программист должен постоянно следить за тем, чтобы массив был отсортирован. В силу этого вставки становятся медленными, так как всякий раз массив нужно пересортировывать. Преимущество множеств состоит в том, что они позволяют не только выполнять быструю проверку на принадлежность (например, определять вхождение элемента в множество), но и быстро вставлять и удалять их.

ПРОВЕРКИ НА ПРИНАДЛЕЖНОСТЬ ОБЪЕКТА МНОЖЕСТВУ

Массив *A* можно превратить в множество *S*. Оба поддерживают проверку на принадлежность с помощью функции `in` либо ее греческого эквивалента `∈`. Кроме того, используя оператор `⊆` либо функцию `issubset`, можно выполнять проверку на принадлежность нескольких элементов:

```
julia> S = Set(A);

julia> 18 in S
true

julia> 18 ∈ A
true

julia> [11, 15] ⊆ A
true

julia> issubset([11, 15], S)
true
```

По поведению массивы похожи, но на множестве эти операции будут выполняться быстрее. Исключение составляют малые коллекции. При малом количестве элементов ни одна коллекция не работает так же быстро, как массив. Но при превышении 30–40 элементов множество начнет превосходить несортированный массив в проверках на принадлежность.

С другой стороны, множества рекомендуется использовать для малых коллекций, если важно поддерживать уникальный набор элементов, и порядок не важен. Это помогает *передавать* читателю вашего исходного кода *информацию* о том, как указанный исходный код должен работать. Использование более сложных типов коллекций, таких как `Dictionary` или `Set`, действительно начинает окупаться при наличии большого количества элементов.

МНОЖЕСТВА НЕ ДОПУСКАЮТ ДУБЛИРОВАНИЯ

Что происходит при попытке создать множество с дубликатами? Следующий ниже пример помогает ответить на этот вопрос:

```
julia> apples = ["apples", "apples", "apples"]
3-element Vector{String}:
"apples"
"apples"
"apples"

julia> appleset = Set(apples)
Set(["apples"])

julia> length(appleset)
1

julia> numbers = [1, 1, 1, 2, 2, 2, 3, 3];

julia> length(numbers)
8

julia> S = Set(numbers)
Set{2, 3, 1}

julia> length(S)
3
```

Дубликаты разрешены в массивах, но не в множествах.

Произвольный доступ и упорядоченность элементов

Я создам множество и массив с одними и теми же элементами, чтобы продемонстрировать всю разницу между случайным доступом и упорядоченностью:

```
A = [3, 5, 7]
S = Set(A)
```

Если применить функции `collect` либо `foreach`, то они будут прокручивать коллекции в цикле. Как видите, порядок другой; он не гарантируется и может меняться в зависимости от версий Julia:

```
julia> collect(S)
3-element Vector{Int64}:
 7
 3
 5

julia> collect(A)
3-element Vector{Int64}:
 3
 5
 7

julia> foreach(print, S)
735

julia> foreach(print, A)
357
```

Я имею возможность использовать скобки для доступа к элементам массива по индексу:

```
julia> A[2]
2
```

Но это невозможно сделать с множеством:

```
julia> S[2]
ERROR: MethodError: no method matching getindex(::Set{Int64}, ::Int64)
```

В случае массива функция `push!` добавляет каждый элемент в предсказуемое место:

```
julia> push!(A, 9)
4-element Vector{Int64}:
 3
 5
 7
 9
```

Однако в случае множеств элемент может оказаться где угодно:

```
julia> push!(S, 9)
Set{Int64}([7, 9, 3, 5])
```

В случае массива функция `pop!` будет удалять последний добавленный элемент:

```
julia> pop!(A)
9
```

Однако в случае `Set` этой операции лучше избегать, так как невозможно контролировать, какой элемент удаляется на самом деле:

```
julia> pop!(S)
7
```

В этом случае языка `Julia`, возможно, было бы уместнее выдавать исключение, чем позволять пользователю выполнять `pop!`.

13.3 Как использовать операции на множествах

Операции на множествах используются для комбинирования множеств, чтобы создавать новые множества. Однако операции на множествах на самом деле не ограничиваются множествами. Операции на множествах также можно выполнять между массивами. Разница в том, что множества предназначены для поддержания таких операций, а массивы – нет. Массивы эффективно выполняют операции на множествах только в случае малых коллекций элементов.

Операции на множествах позволяют отвечать на такие вопросы, как *предоставить фотографии Боба, когда он посещал Испанию и Грецию*. Если в вашем фотоприложении Bob представляет все снимки вашего дяди Боба, Spain представляет множество всех снимков из Испании, а Greece представляет множество всех снимков из Греции, то на такой вопрос можно ответить одним из двух следующих ниже эквивалентных способов:

```
S = Bob ∩ (Spain ∪ Greece)
```

```
S = intersect(Bob, union(Spain, Greece))
```

Приведенный выше исходный код демонстрирует применение операций объединения и пересечения. Их также можно записать с помощью символов $\cup$ и $\cap$. Лучше всего визуализировать поведение различных операций на множествах с помощью диаграмм Венна¹ (рис. 13.2).

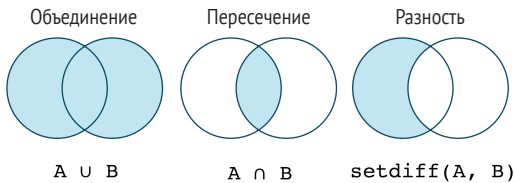


Рис. 13.2 Диаграммы Венна, помогающие объяснять операции на множествах

Две окружности в каждом примере представляют множества A и B. Это накладывающиеся друг на друга множества, то есть некоторые элементы в A также существуют в B.

Расцветченная область показывает элементы, которые включены в множество, полученное в результате операции на множестве. Например, при объединении множеств в результат включаются все элементы из A и B. С другой стороны, результатом пересечения множеств являются только общие для A и B элементы. В указанных операциях можно распознать аналогию с операторами И (&&) и ИЛИ (| |), используемыми в булевой логике. При *объединении* элементы должны находиться в множестве A *или* множестве B. При *пересечении* они должны находиться в множестве A *и* множестве B.

В операции *разности множеств* важен порядок. `setdiff(A, B)` возвращает элементы в A, оставшиеся после удаления элементов в A, которые также существуют в B. Давайте рассмотрим практический пример их применения. Представим, что у нас есть несколько множеств подписей к фотографиям:

```
bob = Set(["Bob in Spain", "Bob in Greece", "Joe and Bob in Florida"])
joe = Set(["Joe in Texas", "Joe and Eve in Scotland", "Joe and Bob in
```

¹ Диаграммы Венна обычно используются для иллюстрации логических взаимосвязей между двумя или более множествами элементов.

```
Florida"]])
eve = Set(["Eve in Wales", "Joe and Eve in Scotland", "Eve in Spain"])
```

То есть имеется три человека – Боб, Джо и Ева, – которые были в разных отпусках за границей, где они делали фотоснимки. В данном примере указанные фотоснимки представлены в виде текста подписи к ним.

В этом сценарии мы хотим применить операции на множествах, чтобы найти фотоснимки, содержащие более одного из этих людей. `bob` – это множество всех фотоснимков, на которых был Боб, `joe` – это множество всех фотоснимков, на которых был Джо, и `eve` – это множество всех фотоснимков, на которых была Ева. Приведенный ниже исходный код отыскивает фотоснимки, на которых Боб и Джо были в отпуске вместе:

```
julia> bob ∩ joe
Set{String} with 1 element:
"Joe and Bob in Florida"
```

Может оказаться так, что Ева расстанется с Джо, поэтому фотографии с Джо не будут нужны. Тогда Ева может применить `setdiff`, чтобы исключить изображения Джо:

```
julia> setdiff(eve, joe)
Set{String} with 2 elements:
"Eve in Wales"
"Eve in Spain"
```

Возможно, Джо захочет найти все отпуска, которые он провел вместе с кем-то еще:

```
julia> (bob ∪ eve) ∩ joe
Set{String} with 2 elements:
"Joe and Eve in Scotland"
"Joe and Bob in Florida"
```

Множества, конечно же, могут содержать объекты любого типа. Давайте сделаем несколько менее захватывающих операций на множествах с числами. А представляет множество в основном четных чисел, а В содержит в основном нечетные числа:

```
A = Set{1, 2, 4, 6}
B = Set{1, 3, 5, 6}
```

Пересечение множеств можно получить двумя разными способами:

```
julia> A ∩ B
Set{Int64} with 2 elements:
 6
 1

julia> intersect(A, B)
```

```
Set{Int64} with 2 elements:
```

```
6  
1
```

Можно также получить объединение множеств:

```
julia> A ∪ B
```

```
Set{Int64} with 6 elements:
```

```
5  
4  
6  
2  
3  
1
```

```
julia> union(A, B)
```

```
Set{Int64} with 6 elements:
```

```
5  
4  
6  
2  
3  
1
```

И наконец, можно получить разность множеств A и B:

```
julia> setdiff(A, B)
```

```
Set{Int64} with 2 elements:
```

```
4  
2
```

```
julia> setdiff(B, A)
```

```
Set{Int64} with 2 elements:
```

```
5  
3
```

Как видите, в разности множеств порядок имеет значение.

13.4 Как использовать множества в исходном коде

Базовые операции на множествах освоить несложно. Больше времени потребуется, чтобы понимать ситуации, когда использовать множества в исходном коде.

Меня часто удивляло, как множества могут давать мощные и элегантные решения сложных задач. Очень легко забыть о наличии множеств в своем инструментарии.

В следующих далее разделах мы рассмотрим задачи, которые можно решать с помощью множеств. Я также буду сравнивать использование множеств с другими решениями.

Сначала я покажу, как создавать множества для разных категорий товаров с помощью конструктора `Set`. После этого продемонстрирую процесс отыскания винтов, соответствующих разным критериям, с помощью операций `intersect` и `setdiff`. Альтернативное решение будет основано на определении составного типа `Screw` с другими свойствами.

Я покажу, как использовать функцию `filter` для отыскания объектов `Screw`, соответствующих заданным критериям.

13.5 Поиск товаров с помощью операций на множествах

При работе с товарами, скажем, в интернет-магазине обычно используется база данных SQL¹. Этот процесс концептуально похож на использование операций на множествах, поэтому я рассмотрю пример покупки винтов в интернет-магазине скобяных изделий.

Винт может иметь разные типы головок:

```
head_type = [rounded, flat, headless, tslot]
```

Вам может понадобиться плоский винт, если вы хотите, чтобы винт был заподлицо с поверхностью, или безголовый для таких вещей, как установочные винты для осевых колец. Винт может иметь тип углубления, который указывает на то, какой наконечник нужен на отвертке, чтобы закручивать винт²:

```
drive_style = [hex, phillips, slotted, torx]
```

Материал должен быть очевиден:

```
material = [aluminium, brass, steel, plastic, wood]
```

Это список категорий. Каждый элемент в списке на самом деле является множеством, и множество содержит номер, однозначно идентифицирующий этот винт. Из практических соображений будет продемонстрировано создание трехзначных номеров изделий. В следующем ниже примере с целью быстрого создания большого количества номеров изделий используются диапазоны:

```
rounded = Set(100:4:130)
flat     = Set(101:4:130)
```

¹ Язык структурированных запросов (Structured Query Language) – это специализированный язык, предназначенный для формулирования запросов к базе данных. Запрос – это поисковое предписание к базе данных, соответствующее одному или нескольким критериям.

² Шестигранное, крестообразное (плюсовое), щелевое (минусовое), звездочное углубления в головке винта. – *Прим. перев.*


```

headless = Set{102:4:130}
tslot     = Set{103:4:130}

hex       = Set{100:108}
phillips  = Set{109:115}
slotted   = Set{116:121}
torx      = Set{122:129}

aluminium = Set{100:3:120}
brass     = Set{101:3:120}
steel     = Set{102:3:120}
plastic   = Set{121:2:130}
wood      = Set{122:2:130}

```

Если внимательно присмотреться к числам, то можно увидеть, что они накладываются друг на друга. Например, некоторые номера изделий из алюминия совпадают с номерами изделий для шестигранных винтов.

Определив эти множества, можно начать задавать различные полезные вопросы, например такие: какие винты в каталоге продукции имеют закругленную головку, сделаны из дерева и могут закручиваться звездообразной отверткой? Для ответа на этот вопрос требуется простая операция на множестве:

```

julia> intersect(rounded, torx, wood)
Set{Int64} with 2 elements:
 124
 128

```

Или как насчет того, чтобы получить все стальные винты, которые можно закручивать с помощью крестообразной отвертки?

```

julia> intersect(phillips, steel)
Set{Int64} with 2 elements:
 114
 111

```

Или, может быть, вы просто хотите узнать о наличии непластиковых винтов с Т-образным пазом:

```

julia> setdiff(tslot, plastic)
Set{Int64} with 5 elements:
 107
 103
 115
 111
 119

```

Это один из способов применения множеств, но то же самое можно сделать с совершенно другими вариантами конструктивного исполнения, вообще не используя множества. Вместо них можно опре-

делить винт как более насыщенный тип данных со свойствами для каждого атрибута:

```
struct Screw
    prodnum::Int
    headtype::HeadType
    drivestyle::DriveStyle
    material::Material
end
```

Вместо того чтобы обращаться с винтами просто как с числами, существует тип данных со свойствами, с помощью которых можно пытаться находить совпадения с несколькими заданными критериями поиска. Как видим, различные свойства представлены конкретно-прикладными типами `HeadType`, `DriveStyle` и `Material`. В другом примере они могут быть представлены строковыми литералами либо символическими именами, но вместо этого они созданы как типы, чтобы улавливать случаи, когда любому из атрибутов назначается недопустимая категория.

13.5.1 Определение и использование перечислений

Для представления различных категорий используются перечисления, сокращенно `enum`. Перечисления существуют в разных языках. В Julia они немного своеобразны, потому что определяются с помощью макрокоманд. Теперь мы перезапустим Julia, так как эти переменные уже определены; иначе будут жалобы на уже определенные переменные:

```
@enum HeadType rounded flat headless tslot
@enum DriveStyle hex phillips slotted torx
@enum Material aluminum brass steel plastic wood
```

Подарком является префикс `@`. Углубления `hex`, `slotted` и `torx` можно трактовать как экземпляры типа `DriveStyle`. Фактически для их создания можно использовать конструктор `DriveStyle`:

```
julia> DriveStyle(2)
slotted::DriveStyle = 2

julia> DriveStyle(3)
torx::DriveStyle = 3

julia> DriveStyle(4)
ERROR: ArgumentError: invalid value for Enum DriveStyle: 4
```

Однако в последнем примере можно заметить добавленную типобезопасность. Для `DriveStyle` невозможно создавать другие значения, кроме тех, которые были указаны при определении перечисления.

13.5.2 Создание тестовых данных для выполнения запросов

В целях демонстрации применения этого типа для отыскания винтов с разными свойствами необходимо создать тестовые данные:

```
function make_screw(prodnum)
    headtype = rand(instances(HeadType))
    drivestyle = rand(instances(DriveStyle))
    material = rand(instances(Material))

    Screw(prodnum, headtype, drivestyle, material)
end

screws = map(make_screw, 100:150)
```

Приведенный выше исходный код создает массив винтов с номерами в диапазоне от 100 до 150, и значения каждого свойства выбираются в случайном порядке. Функция `instances` возвращает массив всех возможных значений перечисления:

```
julia> instances(DriveStyle)
(hex, phillips, slotted, torx)

julia> instances(Material)
(aluminium, brass, steel, plastic, wood)
```

13.5.3 Поиск винтов

В первом примере показаны винты, соответствующие желаемым критериям, с использованием операций на множестве. Теперь мы будем находить нужные винты, перебирая все винты в массиве и проверяя совпадение каждого винта всем требуемым критериям. В примере показано, как это сделать, указывая функции-предикаты. Функция-предикат на входе принимает винт и на выходе возвращает значение `true` либо `false`, в зависимости от соблюдения проверяемых функцией-предикатом критериев. Функция `isroundwood` будет проверять наличие у данного винта закругленной головки из дерева:

```
function isroundwood(screw)
    screw.headtype == rounded &&
    screw.material == wood
end
```

Затем этот предикат (функцию, возвращающую булево значение) можно использовать для фильтрации винтов:

```
julia> roundedwood = filter(isroundwood, screws)
3-element Vector{Screw}:
Screw(100, rounded, torx, wood)
Screw(113, rounded, slotted, wood)
Screw(129, rounded, torx, wood)
```

Как узнать, какие непластиковые винты с Т-образным пазом предлагаются в магазине?

```
julia> function isnonplastic(screw)
        screw.headtype == tslot &&
        screw.material != plastic
    end

julia> nonplastic = filter(isnonplastic, screws)
15-element Vector{Screw}:
Screw(105, tslot, hex, wood)
Screw(106, tslot, hex, wood)
Screw(107, tslot, hex, brass)
Screw(108, tslot, phillips, steel)
Screw(117, tslot, phillips, wood)
Screw(118, tslot, hex, wood)
Screw(125, tslot, phillips, wood)
Screw(128, tslot, phillips, wood)
Screw(130, tslot, phillips, wood)
Screw(131, tslot, torx, brass)
Screw(133, tslot, hex, wood)
Screw(134, tslot, slotted, wood)
Screw(138, tslot, hex, steel)
Screw(141, tslot, phillips, steel)
Screw(146, tslot, torx, brass)
```

13.5.4 Размещение объектов-винтов в множествах

Не всегда бывает легко определить самое лучшее решение для каждого случая, поэтому важно знать разные подходы. Иногда имеет смысл решения комбинировать. Кроме того, винты можно поместить в множества.

К ним можно применить функцию `filter`, чтобы сгенерировать множества, которые потом можно будет реиспользовать:

```
julia> issteel(screw) = screw.material == steel;
julia> steel_screws = Set(filter(issteel, screws));

julia> ishex(screw) = screw.drivestyle == hex
julia> hex_screws = Set(filter(ishex, screws))
```

Затем эти множества можно применять в операциях на множествах:

```
julia> steel_screws ∩ hex_screws
Set{Screw[
    Screw(126, headless, hex, steel),
    Screw(115, headless, hex, steel),
    Screw(121, flat, hex, steel),
    Screw(107, headless, hex, steel),
    Screw(108, flat, hex, steel)
]}
```

Однако это решение можно усовершенствовать и далее.

13.5.5 Поиск винтов с помощью словарей

Часто покупатели знают номер нужного им винта и хотят найти винт, используя именно этот номер, а не сложные критерии поиска. Этот вариант использования можно реализовать, храня винты в словаре, где ключом является номер изделия:

```
julia> screwdict = Dict{screw.prodnun => screw for screw in screws}
Dict{Int64,Screw} with 51 entries:
 148 => Screw(148, rounded, hex, brass)
 124 => Screw(124, rounded, hex, aluminium)
 134 => Screw(134, tslot, slotted, wood)
 136 => Screw(136, rounded, torx, aluminium)
 131 => Screw(131, tslot, torx, brass)
 144 => Screw(144, rounded, slotted, steel)
 142 => Screw(142, flat, slotted, steel)
 150 => Screw(150, rounded, hex, steel)
 ...

julia> screwdict[137]
Screw(137, headless, phillips, aluminium)

julia> screwdict[115]
Screw(115, flat, phillips, aluminium)
```

Приведенное выше изменение исходного кода позволяет вернуться к изначальному решению, в котором номера изделий использовались в множествах. Давайте сначала создадим несколько новых множеств на основе номеров изделий:

```
prodnums = keys(screwdict)

function isbrass(prodnun)
    screw = screwdict[prodnun]
    screw.material == brass
end
brass_screws = Set(filter(isbrass, prodnums))

function istorx(prodnun)
    screw = screwdict[prodnun]
    screw.drivestyle == torx
end
torx_screws = Set(filter(istorx, prodnums))
```

Теперь мы вернулись к элегантности использования операций на множествах, чтобы выбирать нужные изделия, основываясь на ключах изделий в множествах:

```
julia> brass_screws ∩ torx_screws
Set{[100, 122, 144]}
```

```
julia> [screwdict[pn] for pn in brass_screws ∩ torx_screws]
3-element Vector{Screw}:
Screw(100, rounded, torx, brass)
Screw(122, tslot, torx, brass)
Screw(144, flat, torx, brass)
```

13.6 Поиск в инструменте отслеживания дефектов с помощью множеств

При разработке крупных программных продуктов, в особенности в коллективе разработчиков, в компаниях обычно используется какой-то инструмент отслеживания программных дефектов. Обычно это веб-приложения, которые позволяют тестировщикам отправлять описания дефектов. Затем менеджеры или продуктовые специалисты просматривают эти дефекты и назначают приоритеты и серьезность, перед тем как дефекты будут окончательно переданы разработчикам программного обеспечения.

Некоторые общие атрибуты, регистрируемые по каждому дефекту, таковы:

- *проект* – частью какого программного проекта он является?
- *приоритет* – насколько важно исправить этот дефект?
- *серьезность* – это незначительная неприятность или системный сбой в критически важных функциях?
- *компонент* – находится ли он в пользовательском интерфейсе, в клиентской части, серверной части и т. д.?
- *уполномоченный* – кому в настоящее время поручено урегулировать этот дефект?

Как и в случае с изделиями, дефекты обычно однозначно идентифицируются по номеру. Таким образом, можно использовать подход, очень похожий на описанный ранее: дефекты можно использовать в словарях, в которых ключами являются номера дефектов.

Я продемонстрирую определение множеств, состоящих из разных номеров дефектов. Ниже приведено несколько вопросов, на которые, как вы понимаете, можно получить ответы с помощью множеств:

Какие самые важные дефекты назначены Бобу в проекте лунного посадочного модуля?

```
bob ∩ critical ∩ lunar_lander
```

Бывает нецелесообразно иметь такие имена для каждого множества, и множества должны быть организованы в соответствии с полями в инструменте отслеживания дефектов. Ниже показано использование словарей для группировки связанных множеств:

```
assignees["Bob"] ∩ severity[:critical] ∩ projects["Lunar Lander"]
```

При выполнении операции на нескольких объектах бывает практичнее не использовать символы операторов. Приведенное ниже выражение эквивалентно:

```
intersect(assignees["Bob"], severity[:critical], projects["Lunar  
Lander"])
```

Менеджер может задать следующий вопрос:

Какие первоочередные дефекты обрабатываются Бобом и Евой?

```
assignees["Bob"] ∪ assignees["Bob"] ∩ priorities[:top]
```

Можно было бы рассмотреть еще больше примеров, но, надеюсь, продемонстрированные примеры дадут хорошее представление о том, как использовать множества для упрощения задач в своих собственных приложениях.

13.7 Реляционные базы данных и множества

Если вы работали с SQL и реляционными базами данных раньше, то многое из того, что вы увидели в этой главе, возможно, покажется вам знакомым. В языке запросов к базе данных SQL можно выполнять много операций, аналогичных операциям на множествах. В мире баз данных так называемое *внутреннее соединение* эквивалентно *пересечению множеств*.

Реляционные базы данных построены на основе раздела математики, именуемого *реляционной алгеброй*, который охватывает моделирование данных и запросы к ним. В этой главе мы провели разведывательный анализ теории множеств, которая является более базовой. В реляционных базах данных можно создавать таблицы данных с несколькими столбцами, которые связаны с другими таблицами. Структура данных Julia, наиболее похожая на таблицы базы данных, называется кадром данных (DataFrame)¹ и существует в пакете DataFrames². Подробное описание пакета DataFrames см. в книге Богумила Каминьского «Julia для анализа данных»³.

¹ Кадр данных имеет несколько именованных столбцов. Каждый столбец может содержать разные типы данных.

² Дополнительная информация о пакете DataFrames находится по адресу <https://dataframes.juliadata.org/>.

³ Bogumil Kaminski. Julia for Data Analysis. Manning, 2022. – Прим. перев.

Резюме

- Множества помогают упорядочивать такие данные, как фотоальбомы, дефекты в инструменте отслеживания программных дефектов или товарные позиции, продаваемые в интернет-магазине.
- Множества не имеют дубликатов и позволяют очень быстро делать проверку на принадлежность, в отличие от массивов элементов.
- В отличие от массивов, элементы в множествах не имеют четко определенного порядка. Элементы множества невозможно вставлять в определенные позиции.
- Множество создается путем предоставления массива элементов, например `Set([4, 8, 10])`.
- Множества комбинируются с помощью таких операций на множествах, как `union`, `intersect` и `setdiff`.
- Проверка на принадлежность элемента `x` множеству `S` осуществляется с помощью функции `in`. Ее можно записать как `in(x, S)` или `x in S`.
- Тип-перечисление создается с помощью макрокоманды `@enum`. Выражение `@enum Fruit apple banana` создает перечисляемый тип `Fruit` с допустимыми значениями `apple` и `banana`.
- С помощью функции `filter` на массиве можно выполнять операции, аналогичные операциям на множествах. Однако в случае больших наборов данных производительность не будет одинаково хорошей.
- Теория множеств и реляционная алгебра (используемая в реляционных базах данных) позволяют выполнять аналогичные операции. Однако множества имеют дело со значениями, а реляционные базы данных – с таблицами и отношениями между ними.

14

Работа с векторами и матрицами

Эта глава охватывает следующие ниже темы:

- работа с числами в матрицах и выполнение вычислений;
- вертикальная и горизонтальная нарезки массивов;
- конкатенация массивов по разным измерениям с целью формирования более крупных массивов.

В главе 4 вы обследовали базовые операции, такие как `push!`, на одномерных массивах, именуемых векторами. В этой главе вы больше сосредоточитесь на работе с многомерными массивами, такими как матрицы.

Для чего можно использовать матрицу и вектор? Их можно комбинировать, чтобы решать огромное число задач. Например, векторы нашли широкое применение в геометрической интерпретации; в этом случае они представляют точки в пространстве. Матрицы используются для перемещения и поворота этих точек.

Матрицы применяются даже для решения математических уравнений, и они очень популярны в машинном обучении. Матрицы применяются для представления изображения. Каждый элемент матрицы может представлять цвет одного пиксела. Каждая из этих тем заслуживает отдельной главы или книги, поэтому в данной главе будут охвачены только основы работы с векторами и матрицами.

14.1 Векторы и матрицы в математике

Матрица или вектор – это не просто примитивный контейнер чисел. Например, в математике множества, кортежи и векторы могут выглядеть как список чисел и, следовательно, казаться похожими. Но то, что с ними можно делать, разительно отличается.

Наука о векторах и матрицах является частью области математики, именуемой *линейной алгеброй*. В линейной алгебре отдельные значения, такие как 1, 4 и 8, называются *скалярами*. В то время как несколько значений в строке или столбце являются *векторами*, таблицы – это *матрицы*, и если данные расположены в виде трехмерного массива, то они называются *кубом*. Векторы можно дополнительно подразделить на векторы-столбцы и векторы-строки (рис. 14.1).

Вектор-строка

5	7	8	9
---	---	---	---

5
7
8
9

Матрица

11	12	13	14
21	22	23	24
31	32	33	34

Вектор-столбец

Рис. 14.1 Массивы разных размеров

14.2 Конструирование матрицы из строк и столбцов

Матрицу можно сконструировать либо путем указания нескольких строк, уложенных друг над другом, либо путем выстраивания столбцов один за другим. При конструировании матрицы из векторов-строк каждая строка отделяется точкой с запятой `;`. Обратите внимание, что отдельные элементы не отделяются запятой. Если вы об этом забыли, то повторно обратитесь к изложению векторов-строк и векторов-столбцов в главе 4:

```
julia> table = [2  6 12;
                 3  4 12;
                 6  2 12;
                 12 1 12]
```

```
4×3 Matrix{Int64}:
```

```
 2  6 12
 3  4 12
 6  2 12
12  1 12
```

Для того чтобы создать матрицу из нескольких столбцов, каждый столбец можно определить отдельно, а затем скомбинировать их в матрицу:

```
julia> x1 = [2, 3, 6, 12]
julia> x2 = [6, 4, 2, 1]
julia> x3 = [12, 12, 12, 12]
```

```
julia> table = [x1 x2 x3]
```

```
4×3 Matrix{Int64}:
```

```
 2  6 12
 3  4 12
 6  2 12
12  1 12
```

Это идентично написанию векторов-столбцов внутрискрипчно, как показано ниже:

```
table = [[2, 3, 6, 12] [6, 4, 2, 1] [12, 12, 12, 12]]
```

Обратите внимание на то, как с помощью строки `4×3 Matrix{Int64}` Julia сообщает сводную информацию о полученном в результате массиве. Указанное сообщение говорит о том, что был создан массив из 4 строк и 3 столбцов, в котором каждый элемент имеет тип `Int64`.

Эти свойства можно запрашивать у произвольного массива: функция `eltype` предоставляет тип каждого элемента в массиве, `ndims` предоставляет число измерений, а `size` предоставляет число компонентов (элементов) в каждом измерении. Измерения¹ обычно принято трактовать как длину, высоту и глубину, но в данном случае я обычно буду говорить о строках и столбцах:

```
julia> eltype(table) ❶
Int64
```

```
julia> size(table) ❷
(4, 3)
```

```
julia> ndims(table) ❸
2
```

❶ Тип каждого элемента в массиве.

❷ Число строк и столбцов.

❸ Число измерений.

¹ Англ. dimension; син. мерность как в слове «двумерный». – Прим. перев.

Рисунок 14.2 помогает прояснить смысл этих свойств, показывая векторы и матрицы разных форм¹. Они имеют разное число строк и столбцов, а также разную ориентацию и размеры.

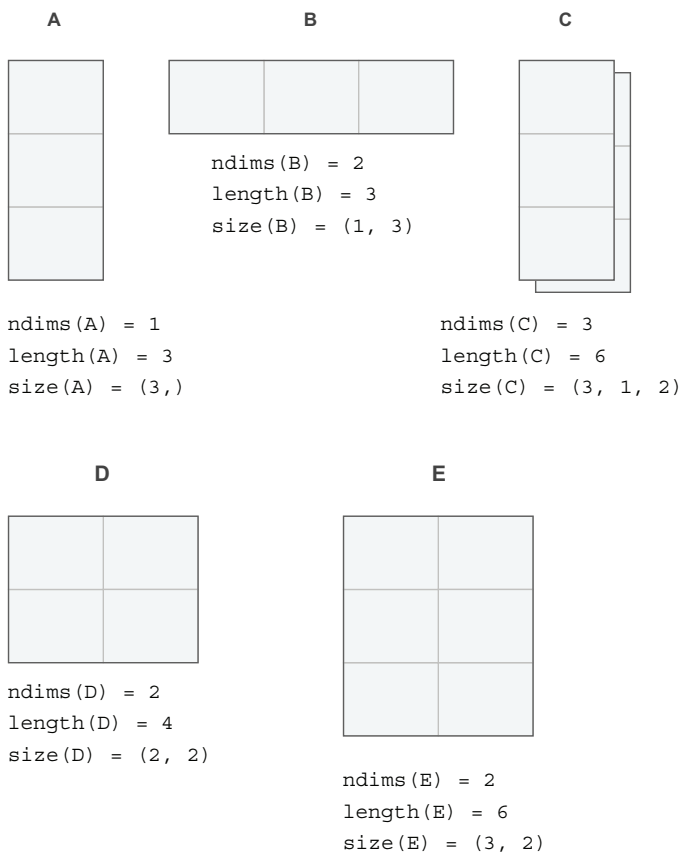


Рис. 14.2 Свойства массивов разной формы

14.3 Размер, длина и норма массива

Если вы пришли из других языков программирования, то можете легко спутать эти понятия массива:

- `size` – размеры массива;
- `length` – общее число элементов в массиве;
- `norm` – модуль² вектора.

Здесь была создана таблица с 4 строками и 3 столбцами, всего 12 элементов:

¹ Англ. *shape*; син. очертание. – Прим. перев.

² Англ. *magnitude*; син. величина, магнитуда. – Прим. перев.

```
julia> length(table)
12
```

Функцию `norm` понять сложнее. В целях ее объяснения я буду использовать небольшой вектор с элементами 3 и 4:

```
julia> using LinearAlgebra

julia> norm([3, 4])
5.0
```

Взглянув на прямоугольный треугольник, можно получить наглядное представление о том, что делает функция `norm`. Элементы вектора можно представить как стороны a и b треугольника (рис. 14.3). Функция `norm` дает длину наибольшей стороны, *гипотенузы*.

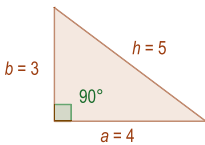


Рис. 14.3 Прямоугольный треугольник со сторонами длины a , b и h

Теорема Пифагора раскрывает взаимосвязь между всеми сторонами прямоугольного треугольника. Норму можно трактовать как применение теоремы Пифагора для определения длины *гипотенузы*:

$$5^2 = 3^2 + 4^2.$$

14.4 Вертикальная и горизонтальная нарезки массива

В Julia имеется отличная поддержка выборки срезов из массивов разных размеров. Эта гибкость обусловлена тем, что при использовании квадратных скобок `[]` для доступа к элементам или присваивания им значений вызываются функции `setindex!` и `getindex`. Давайте обследуем механизм работы нарезки (табл. 14.1).

Таблица 14.1. Связь между доступом к элементу и вызовами функций Julia

Синтаксический сахар	Транслируется в	Описание
<code>xs[i]</code>	<code>getindex(xs, i)</code>	Получить элемент с индексом i
<code>xs[i,j]</code>	<code>getindex(xs, i, j)</code>	Получить элемент в строке i и столбце j
<code>xs[i] = 42</code>	<code>setindex!(xs, 42, i)</code>	Задать значение элемента в индексе i
<code>xs[i,j] = 42</code>	<code>setindex!(xs, 42, i, j)</code>	Задать значение элемента в строке i и столбце j

Я просто начну с рассмотрения доступа к отдельным элементам одномерного массива. На рис. 14.1 показано, как выбирать один или

несколько элементов в одномерных массивах. Хотя на рисунке показаны выборки изнутри векторов-строк, те же принципы применяются к векторам-столбцам.

Для обозначения первого или последнего элемента вектора в строке или столбце можно использовать `begin` и `end` внутри квадратных скобок. В Julia первый элемент массива по умолчанию имеет индекс 1. Однако в Julia можно создавать массивы с любым начальным индексом, что делает ключевое слово `begin` очень полезным (рис. 14.4).

Обратите внимание, что к одним и тем же элементам можно обращаться разными способами. Например, если дан массив `A`, то `A[3]` и `A[begin+2]` будут представлять один и тот же элемент.

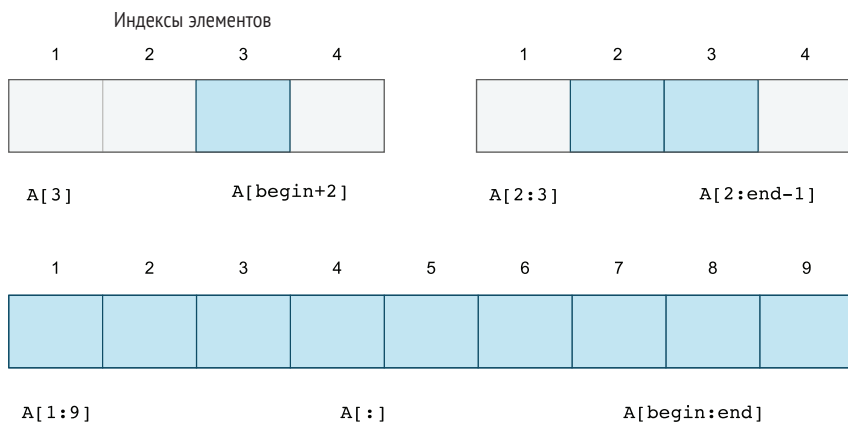


Рис. 14.4 Нарезка одномерного массива `A` разными способами

Для массива из четырех элементов, как и в первых двух примерах, `A[4]` и `A[end]` относятся к одному и тому же элементу. Точно так же `A[3]` и `A[end-1]` выбирают один и тот же элемент массива. С этими концепциями можно поэкспериментировать в интерактивной среде REPL:

```
julia> A = collect('A':'F')
6-element Vector{Char}:
 'A'
 'B'
 'C'
 'D'
 'E'
 'F'

julia> A[begin+1]
'B': ASCII/Unicode U+0042

julia> A[end-1]
'E': ASCII/Unicode U+0045
```

```
julia> A[2:5]
4-element Vector{Char}:
 'B'
 'C'
 'D'
 'E'
```

```
julia> A[begin+1:end-1]
4-element Vector{Char}:
 'B'
 'C'
 'D'
 'E'
```

Если не нужен конкретный индекс, а просто нужны все элементы, то можно написать `A[:]`. Чем это отличается от простого написания `A`? Все операции на срезах возвращают копии данных. Приведенный ниже пример поможет прояснить ситуацию:

```
julia> A = [8, 1, 2, 7];
```

```
julia> B = A[:];
```

```
julia> B[2] = 42
42
```

```
julia> B
4-element Vector{Int64}:
 8
42
 2
 7
```

```
julia> A
4-element Vector{Int64}:
 8
 1
 2
 7
```

Вы заметили, как изменился второй элемент вектора `B`, но не второй элемент вектора `A`? Если бы вместо `B = A[:]` было написано `B = A`, то второй элемент в векторе `A` тоже был бы изменен, поскольку `A` и `B` ссылались бы на один и тот же объект-массив.

Но что, если требуется выбрать срез массива, не создавая копию? Частое копирование тысяч элементов в тесном внутреннем цикле, в особенности при работе с очень большими объемами данных, может снижать производительность. В таких случаях в подразделе массива можно создать так называемое *представление*¹. Срезы – это не

¹ Англ. *view*; син. подвектор (подмассив, подматрица), проекция. – Прим. перев.

копии элементов массива, а сами эти элементы. Срез можно превратить в представление с помощью макрокоманды `@view`:

```
julia> B = @view A[3:end]
2-element view(::Vector{Int64}, 3:4) with eltype Int64:
 2
 7

julia> B[2] = 1331
1331

julia> A
4-element Vector{Int64}:
 8
 1
 2
1331
```

Последний результат показывает, что изменение второго элемента вектора `B` привело к изменению четвертого элемента вектора `A`.

Многие из этих примеров должны быть понятны, поскольку в нескольких предыдущих главах мы уже поработали с одномерными массивами. Интереснее становится, когда имеешь дело со срезами многомерных массивов, таких как матрицы.

Давайте создадим 2D-матрицу `A`, на которой мы проведем ряд экспериментов, используя функцию Julia `reshape`. Указанная функция на входе принимает `AbstractArray`. Следующий ниже пример исходного кода следует пояснить: в качестве входных данных используется диапазон `1:12`. Все диапазоны являются подтипами типа `AbstractArray`, поэтому Julia видит диапазон как одномерный массив с 12 элементами. Функция `reshape` реорганизовывает эти 12 элементов в матрицу с 3 строками и 4 столбцами, именуемую матрицей `3×4`.

```
julia> A = reshape(1:12, 3, 4)
3×4 reshape(::UnitRange{Int64}, 3, 4) with eltype Int64:
 1  4  7 10
 2  5  8 11
 3  6  9 12
```

Я продемонстрирую нарезку матрицы разными способами, но сначала дам несколько советов о том, как думать о нарезке, чтобы можно было понять смысл демонстраций.

ВАЖНО Форма матрицы определяется числом строк и столбцов. Следовательно, функция изменения числа строк и столбцов в Julia называется `reshape`. Следует учитывать, что функция `reshape` не может изменять длину матрицы (`length`). Массив `A` из шести элементов можно реформировать в матрицу `3×2` либо `2×3`, но реформировать его в матрицу `3×3` невозможно, так как он содержит девять элементов (см. рис. 14.5).

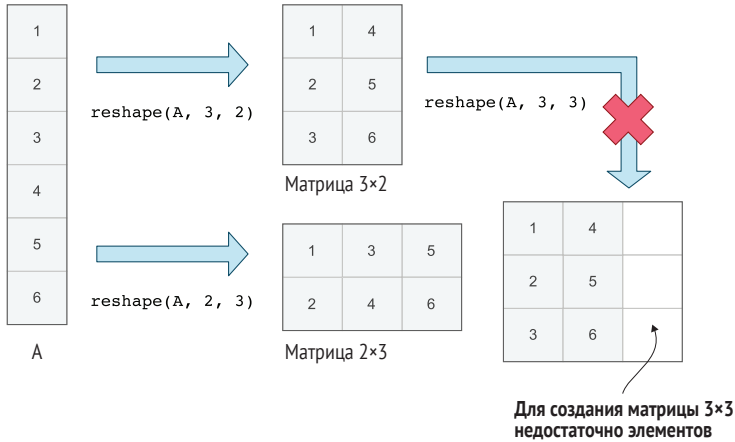


Рис. 14.5 Массив можно реформировать в матрицы с тем же числом элементов. Крестик означает, что реформировать массив из шести элементов в массив с девятью элементами невозможно

Мне нравится думать о нарезке массива с точки зрения операции пересечения множеств $\cap$. Так, $A[2, 3]$ можно прочесть следующим образом: *получить пересечение всех элементов строки 2 и столбца 3*.

На рис. 14.6 представлена визуализация этой идеи. Затененные квадраты представляют выбранные нами строки и столбцы, а квадраты более темного оттенка представляют пересечение между выбранными строками и столбцами.

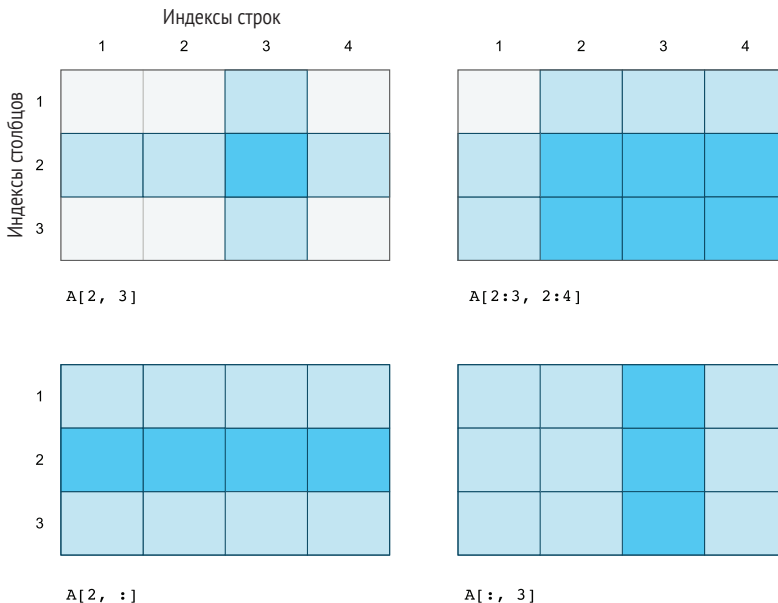


Рис. 14.6 Нарезка двумерных массивов

Эта концептуализация облегчает понимание выборки $A[2:3, 2:4]$. Ее можно прочесть следующим образом: *получить пересечение всех элементов в строках со 2 по 3 и в столбцах со 2 по 4*.

Следуя этой логике, становится очевидным, как выделять всю строку или столбец в матрице. С их выделением можно поэкспериментировать в интерактивной среде REPL:

```
julia> A[1, 2]
4

julia> A[3, 4]
12

julia> A[:, 4]
3-element Vector{Int64}:
 10
 11
 12

julia> A[2, :]
4-element Vector{Int64}:
 2
 5
 8
11
```

Также стоит отметить, что даже многомерные массивы можно рассматривать как одномерные:

```
julia> A[1]
1

julia> A[4]
4
```

14.5 Комбинирование матриц и векторов

Данные не всегда приходят в том очертании и форме, которые нужны для выполнения матричных операций. У нас может быть n векторов, но на самом деле требуется матрица с n столбцами.

К счастью, в Julia есть ряд функций, служащих для конкатенации матриц. В первом примере показан способ конкатенации двух векторов-строк по горизонтали с помощью функции `hcat` либо по вертикали с помощью функции `vcat` (рис. 14.7).

Функция `cat` позволяет указывать измерение, по которому следует выполнять конкатенацию. Это удобно при работе с массивами более высокой размерности. Аналогичные операции можно выполнять с векторами-столбцами (рис. 14.8).

1	3	5
---	---	---

A = [1 3 5]

7	9	11
---	---	----

B = [7 9 11]

1	3	5	7	9	11
---	---	---	---	---	----

hcat(A, B)
cat(A, B, dims = 2)

1	3	5
7	9	11

vcat(A, B)
cat(A, B, dims = 1)

Рис. 14.7 Горизонтальная и вертикальная конкатенации векторов-строк

1
3
5

A = [1, 3, 5]

7
9
11

B = [7, 9, 11]

1
3
5
7
9
11

hcat(A, B)
cat(A, B, dims = 2)

1	7
3	9
5	11

vcat(A, B)
cat(A, B, dims = 1)

Рис. 14.8 Горизонтальная и вертикальная конкатенации векторов-столбцов

Те же принципы применимы к комбинированию матриц; можно конкатенировать по любому измерению. Горизонтальная и вертикальная конкатенации имеют свои собственные функции, `hcat` и `vcat`, так как они применяются очень часто (рис. 14.9).

1	3	5
2	4	6

A = [1 3 5; 7 9 11]

7	9	11
8	10	12

B = [7 9 11; 8 10 12]

1	3	5	7	9	11
2	4	6	8	10	12

hcat(A, B)

cat(A, B, dims = 2)

1	3	5
2	4	6
7	9	11
8	10	12

vcat(A, B)

cat(A, B, dims = 1)

Рис. 14.9 Горизонтальная и вертикальная конкатенации матриц

Эти функции конкатенации могут принимать любое число аргументов, не ограничиваясь только двумя:

```
julia> x = [1, 2, 3]
3-element Vector{Int64}:
 1
 2
 3
```

```
julia> y = [8, 6, 4]
3-element Vector{Int64}:
 8
 6
 4
```

```
julia> hcat(x, y, x, y)
3×4 Matrix{Int64}:
 1  8  1  8
 2  6  2  6
 3  4  3  4
```

```
julia> hcat(x, 2y, 2x, 3y)
3×4 Matrix{Int64}:
 1 16  2 24
 2 12  4 18
 3  8  6 12
```

14.6 Создание матриц

При работе с матрицами часто требуются специальные виды матриц. Создание матриц только с нулями или единицами настолько распространено, что для этого существуют специальные функции:

```
julia> zeros{Int8, 2, 3}
2×3 Matrix{Int8}:
 0  0  0
 0  0  0

julia> ones{Float64, 2, 3}
2×3 Matrix{Float64}:
 1.0  1.0  1.0
 1.0  1.0  1.0
```

Обратите внимание, как в качестве первого аргумента опционально указывается требующийся тип каждого элемента. Если тип не указывается, то по умолчанию будет применен тип `Float64`.

На практике также нередко требуется создавать целый массив случайных чисел. Например, в глубоком обучении широко используются большие матрицы со случайными значениями. Случайные значения часто используются для создания тестовых данных:

```
julia> rand{UInt8, 2, 2}
2×2 Matrix{UInt8}:
 0x8e  0x61
 0xcf  0x0d
```

Иногда требуется заполнить всю матрицу просто определенным значением:

```
julia> fill{Int64, 3, 3}
3×3 Matrix{Int64}:
 12  12  12
 12  12  12
 12  12  12
```

Векторы и матрицы являются обширными темами, и если бы у нас было больше места, то можно было бы рассмотреть геометрическую интерпретацию векторов и матриц. Каковы возможные варианты их использования? Ориентацию ракеты можно было бы представить с помощью матрицы, а ее положения – в виде вектора. Матрицы можно было бы использовать для поворотов или перемещения ракеты в системе координат. Отличным ресурсом углубленного изложения работы с матрицами является книга «Julia для анализа данных» (Manning, 2022) Богумила Каминского.

Резюме

- Массивы можно использовать для определения векторов-столбцов, векторов-строк и матриц.
- Матрицы – это двумерные массивы, которые можно конструировать разными способами. Чаще всего их определяют как набор строк, но матрицы можно также определять как набор столбцов.
- Массивы обладают такими свойствами, как `ndims`, `size` и `length`. Они описывают число измерений, число элементов по каждому измерению и общее число элементов в массиве.
- Массивы можно нарезать, указывая диапазоны, что также обобщается и на матрицы. Диапазоны можно указывать для строк и столбцов, чтобы вырезать подматрицу.
- Срезы – это копии данных. Если вы не хотите, чтобы срезы были копиями, а напрямую ссылались на данные в изначальном массиве, то с помощью макрокоманды `@view` можно создавать представление (подраздел) массива.
- С помощью функций `hcat` и `vcat` матрицы и векторы можно комбинировать по горизонтали и вертикали. Для массивов более высоких размерностей можно использовать функцию `cat` и в качестве аргумента указывать измерение, по которому следует выполнять конкатенацию.
- Матрицы можно быстро создавать с помощью таких функций, как `rand`, `fill`, `ones` и `zeros`.

Часть IV

Технология разработки программного обеспечения

Технология разработки программного обеспечения¹ – это принципы организации и структурирования больших программ, служащие для упрощения их поддержания, модификации и усовершенствования. В главе 15 рассказывается о функциональном программировании и о том, как оно помогает разрабатывать более удобное в сопровождении программное обеспечение и поощряет новые взгляды на то, как думать об исходном коде.

Глава 16 посвящена физической организации программного обеспечения в виде модулей, каталогов и файлов, связывая ее с управлением зависимостями. Большое программное обеспечение чаще всего состоит из многочисленных пакетов, созданных разными коллективами разработчиков. Эта часть посвящена разработке устойчивой системы, служащей для поддержания версий программных пакетов, зависящих друг от друга.

¹ Син. программная инженерия. – Прим. перев.

15 Функциональное программирование на языке Julia

Эта глава охватывает следующие ниже темы:

- причина важности понимания функционального программирования на языке Julia;
- различия между функциональным и объектно ориентированным подходами к конструированию программ;
- практическое использование функций более высокого порядка;
- придание исходному коду большей удобочитаемости с помощью выстраивания функций в цепочку;
- разработка службы хранения паролей.

Julia – это многопарадигмальный язык программирования, но в Julia функциональный стиль программирования получил гораздо большее распространение, чем в других мейнстримных языках, с которыми вы, возможно, знакомы, таких как Python, Ruby, Java или C++. Следовательно, вполне естественно иметь хорошее понимание принципов функционального программирования, без которого невозможно стать хорошим разработчиком на языке Julia.

Функциональное программирование не всегда является самым лучшим подходом к решению каждой задачи. В этой главе вы на-

учитесь создавать службу хранения паролей как в объектно ориентированном, так и в функциональном стилях, что даст вам возможность обследовать плюсы и минусы двух разных стилей (парадигм) программирования. Перед созданием большого примера исходного кода мы рассмотрим ключевые строительные блоки функционального программирования, такие как функции более высокого порядка, замыкания, выстраивание функций в цепочки и композиция.

15.1 Чем функциональное программирование отличается от объектно ориентированного?

Давайте взглянем на сущность функционального программирования в более широкой перспективе и на причину, по которой оно описывается в книге по программированию на языке Julia. При обсуждении функционального программирования первая проблема заключается в том, что единого четкого его определения не существует. В этой главе я буду использовать определение, которое считаю прагматичным. На рис. 15.1 показано то, как функциональное программирование сочетается с другими парадигмами программирования.

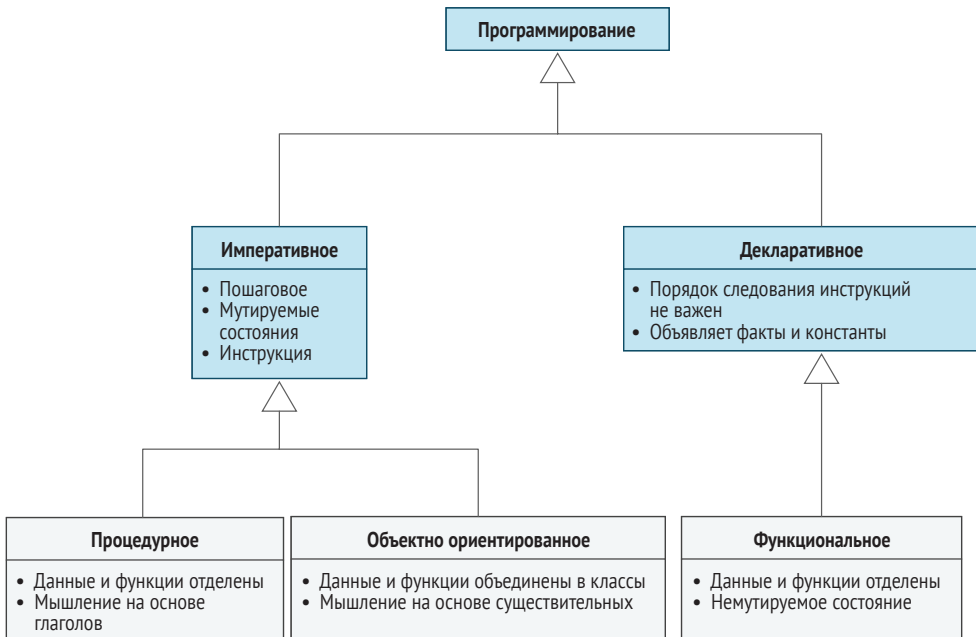


Рис. 15.1 Схема разных парадигм программирования и их взаимосвязей

Самое главное, что следует вынести из рис. 15.1, так это то, что *процедурное* программирование и *функциональное* программирование представляют собой совершенно *разные* стили. Написание исходного кода с набором функций вместо использования объектно ориентированного подхода не делает исходный код автоматически функциональным. Эта техника долгое время использовалась в таких языках, как C, Fortran и Pascal, в подходе, именуемом процедурным программированием.

В отличие от него, функциональное программирование обычно предусматривает целый ряд других практических приемов и подходов:

- трактовка функций как *первоклассных объектов*, то есть функциями можно манипулировать – передавать их в другие функции и хранить в переменных – как если бы они были обычными данными;
- поддержка *функций более высокого порядка*, то есть функций, принимающих на входе другие функции;
- использование функций `map`, `filter` и `reduce` вместо циклов `for` при прокручивании коллекций в цикле, чтобы выполнять на них различные операции;
- предпочтение *замыканий* или *лямбда-функций* вместо объектов и их методов, служащих для управления состоянием.

Функциональное программирование предлагает целый ряд способов комбинирования функций разными способами и модуляризации исходного кода на функциональном уровне.

15.2 Как и зачем учиться мыслить функционально

В функциональном программировании общепринято избегать видоизменения (мутации) входных данных, за счет чего становится легче рассуждать о потоке данных в исходном коде. Функции принимают входные данные и преобразовывают их в выходные данные. Благодаря этому программы можно трактовать как сложные конвейеры, по которым текут данные.

Следующий ниже листинг иллюстрирует эту концепцию с помощью функции `camel_case` из главы 11, которая представляет собой вложение нескольких вызовов функций. Каждый вызов производит данные, поступающие на вход следующей функции.

Листинг 15.1 Преобразование змеиногo регистра в верблюжий

```
"Превращает hello_to_you в HelloToYou"
function camel_case(input::AbstractString)
    join(map(uppercasefirst, split(input, '_')))
end
```

Для визуализации перетекания данных между функциями можно использовать диаграмму потока данных. На диаграмме потока данных круглые прямоугольники представляют преобразования данных, а стрелки отмечены типом протекающих по ним данных. Например, строковые и символьные данные втекают в функцию `split`, а затем из `split` вытекает массив строковых литералов и направляется в `map`.

Императивные стили программирования, такие как объектно ориентированное программирование, зачастую очень затрудняют выполнение такого анализа потока данных, поскольку функции, или методы, изменяют свои входные данные (подвергают их мутации). Вместо того чтобы думать о преобразовании потоков данных, в этих стилях принято концептуализировать процесс в виде объектов, отправляющих друг другу сообщения о видоизменении их состояний.

15.3 Избегание глубокой вложенности вызовов с помощью выстраивания функций в цепочки

Прежде чем разрабатывать более крупные примеры исходного кода, демонстрирующие плюсы и минусы функционального программирования, я хотел бы, чтобы вы лучше разобрались с несколькими фундаментальными строительными блоками, которые есть в вашем распоряжении. Функция `camel_case` из предыдущего раздела была реализована в довольно функциональном стиле, но ее не очень легко читать, поскольку она глубоко вложена. В ней нет ничего похожего на аккуратный конвейер, изображенный на рис. 15.2.

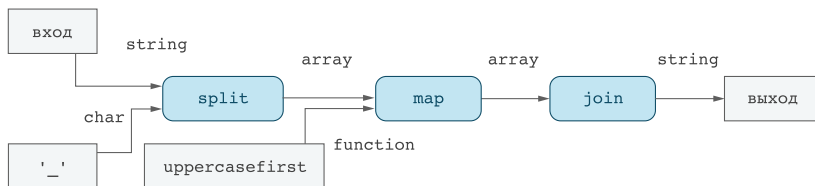


Рис. 15.2 Поток данных для функции `camel_case`

Однако, используя конвейерный оператор Julia `|>`, можно построить нечто подобное этому рисунку. Он позволяет передавать данные на выходе из одной функции на вход в другую функцию. Ниже приведен полный пример, который будет детально разобран позже.

Листинг 15.2 Верблюжий регистр посредством выстраивания функций в цепочку

```

splitter(dlm) = str -> split(str, dlm)
mapper(fn) = xs -> map(fn, xs)

function camel_case(input::AbstractString)
    input |> splitter('_') |> mapper(uppercasefirst) |> join
end

```

С работой функций `splitter` и `mapper` можно поэкспериментировать в интерактивной среде REPL:

```

julia> f = splitter('_')
#13 (generic function with 1 method)
julia> words = f("hello_how_are_you")
4-element Vector{SubString{String}}:
"hello"
"how"
"are"
"you"

julia> g = mapper(uppercasefirst)
#15 (generic function with 1 method)
julia> g(words)
4-element Vector{String}:
"Hello"
"How"
"Are"
"You"

```

Для понимания листинга 15.2 сперва требуется разобраться в операторах `->` и `|>`. Оператор `->` используется для определения так называемых *анонимных функций* Julia.

15.3.1 Понимание анонимных функций и замыканий

Анонимная функция – это функция без имени. Однострочные анонимные функции создаются с помощью оператора `->`. Без анонимных функций пришлось бы написать `splitter` и `mapper`, как показано в следующем ниже листинге.

Листинг 15.3 Функции `splitter` и `mapper` без анонимных функций

```

"Создать функцию, которая делает разбивку на буквенном символе `dlm`"
function splitter(dlm)
    function f(str)
        split(str, dlm)
    end
    return f
end

```

"Создать функцию, применяющую функцию `fn` ко всем ее входным данным"

```
function mapper(fn)
    function g(xs)
        map(fn, xs)
    end
    return g ②
end
```

- ① Возвращаемое значение не нужно; оно добавлено, просто чтобы подчеркнуть, что возвращается `f`.
- ② Это возвращаемое значение тоже не нужно, но последнее выражение все равно возвращается.

Приведенный выше пример показывает, что имена возвращаемых функций не важны. Пользователям функций `splitter` и `mapper` не важно, что эти функции внутри получили имена `f` и `g`. Таким образом, анонимные функции используются всякий раз, когда имя не важно. Давайте возьмем один из примеров исходного кода главы 4, посвященный градусам и синусу, и сделаем его более аккуратным, как показано в следующем ниже листинге.

Листинг 15.4 Упрощение исходного кода с помощью анонимных функций

```
deg2sin(deg) = sin(deg2rad(deg)) ①
map(deg2sin, 0:15:90) ①

map(deg->sin(deg2rad(deg)), 0:15:90) ②

map(0:15:90) do deg ③
    rads = deg2rad(deg) ③
    sin(rads) ③
end ③
```

- ① Вариант именованной функции.
- ② Однострочный вариант.
- ③ Многострочный вариант.

Если уместить анонимную функцию в одну строку невозможно, то оператор `->` нецелесообразен. В этих случаях можно использовать форму `do-end`, показанную в конце листинга 15.4.

Функции `f` и `g`, возвращаемые функциями `splitter` и `mapper`, называются замыканиями. *Замыкание* – это функция, которая захватила какое-то внешнее состояние, не переданное в качестве аргумента. Функция `f` принимает на входе в качестве аргумента только строковый литерал `str`. Разделитель `dlm`, используемый для разбивки строкового литерала `str`, был захвачен из окружающей функцию области видимости. В этом случае определение функции `splitter` задает область `f` внутри.

Функция `g` принимает только обрабатываемую коллекцию `xs`. Функция `fn` применяется к каждому элементу коллекции `xs`, за-

хваченной из объемлющей функцию области, определенной функцией `mapreg`.

Замыкания не нуждаются в именах. Функции `splitter` и `mapreg` в листинге 15.2 возвращают анонимные функции. Эти анонимные функции тоже являются замыканиями, поскольку они захватывают переменные из своей области видимости. Собственно говоря, даже существует довольно распространенное заблуждение о том, что замыкание – это просто причудливый термин для обозначения *анонимной функции*. Такое заблуждение не особо удивляет, если учитывать, что анонимные функции довольно часто используются для определения замыканий.

15.3.2 Использование конвейерного оператора `|>`

Конвейерный синтаксис Julia используется для выстраивания в цепочку функций, принимающих на входе одиночные аргументы. За счет этого появляется возможность переписывать вызовы наподобие `f(g(x))` как `x |> g |> f`.

Этот факт помогает объяснять причину, по которой пришлось создать функции `splitter` и `mapreg`. Обычные функции `split` и `map` требуют нескольких входных аргументов, и поэтому их нельзя использовать с конвейерным оператором `|>`. А вот функции (замыкания), возвращаемые функциями `splitter` и `mapreg`, использовать в конвейере можно:

```
julia> f = splitter('_');

julia> g = mapreg(uppercasefirst);

julia> "hi_there_world" |> f
3-element Vector{SubString{String}}:
"hi"
"there"
"world"

julia> "hi_there_world" |> f |> g
3-element Vector{String}:
"Hi"
"There"
"World"

julia> "hi_there_world" |> f |> g |> join
"HiThereWorld"
```

Следующая важная функциональная концепция, *частичное применение*, получается, если задать следующие два вопроса: зачем нужно использовать имена `splitter` (разделитель) и `mapreg` (преобразователь) и разве нельзя их просто назвать `split` и `map`?

15.3.3 Удобное генерирование новых функций с помощью частичного применения

В вычислительной науке *частичное применение функции* относится к процессу фиксации нескольких аргументов функции, генерируя еще одну функцию, принимающую меньшее число аргументов. Это определение частичного применения, возможно, кому-то покажется сложным, но на практическом примере станет видно, что оно намного проще, чем кажется.

Импортировав функции `split` и `map`, к ним можно добавить новые методы, принимающие только одиночные аргументы, и, таким образом, как показано в следующем листинге, определить более элегантную версию функции `camel_case`.

Листинг 15.5 Верблюжий регистр с частичным применением

```
import Base: split, map ❶

split(dlm::Char) = s -> split(s, dlm) ❷
map(fn::Function) = xs -> map(fn, xs) ❸

function camel_case(s)
    s |> split('_') |> map(uppercasefirst) |> join
end
```

- ❶ Импортировать функции `split` и `map`, для того чтобы их расширить.
- ❷ Фиксация аргумента `dlm` функции `split`.
- ❸ Фиксация функции отображения функции `fn`.

По сути, пользователю функции `split` предоставляется возможность фиксировать аргумент `dlm`. Новая функция `split` возвращается как вызов с фиксированным значением `dlm`. Тот же принцип применим и к функции `map`. Этот процесс фиксации конкретных аргументов функции и есть частичное применение функции.

Поскольку такая возможность нашла столь широкое применение на практике, многие функции стандартной библиотеки Julia были расширены за счет методов, которые принимают подмножество всех необходимых аргументов и возвращают функцию, принимающую остальные аргументы. Поясню на нескольких функциях модуля `Base` (встроенного модуля Julia, который всегда подгружается):

```
julia> images = ["bear.jpg", "truck.png", "car.jpg", "duck.png"];

julia> findfirst(img->img == "truck.png", images)
2

julia> findfirst(=="truck.png"), images)
2

julia> filter(img->endswith(img, ".png"), images)
```

```
2-element Vector{String}:  
"truck.png"  
"duck.png"  
  
julia> filter(endswith(".png"), images)  
2-element Vector{String}:  
"truck.png"  
"duck.png"
```

Можно убедиться, что при получении функцией `endswith` только одного аргумента она вызывает метод, возвращающий функцию:

```
julia> ispng = endswith(".png")  
  
julia> ispng isa Function  
true  
  
julia> ispng("truck.png")  
true  
  
julia> ispng("car.jpg")  
false
```

Умелое смешивание и сочетание функций, примеры которых были приведены выше, составляет значительную часть того, чем занимаются функциональные программисты.

15.4 Реализация шифра Цезаря и подстановочного шифра

Как и было обещано, далее будет продемонстрирован процесс разработки службы шифрования паролей как в объектно ориентированном, так и в функциональном стиле. Но перед этим следует объяснить способы применения шифров для шифрования и дешифрования паролей.

Шифр – это алгоритм, который на входе принимает так называемое *сообщение* и с помощью секретного ключа шифрует сообщение, чтобы получить так называемый *шифротекст*. Способ, когда один и тот же ключ можно использовать для шифрования и дешифрования, называется *симметричным* шифрованием (рис. 15.3).

Далее будет продемонстрирована реализация двух разных шифров: шифра Цезаря и подстановочного шифра (рис. 15.4). Создав два шифра, будет показано, как конфигурировать службу хранения паролей под использование разных шифров. С целью обеспечения взаимозаменяемости шифров необходимо сконструировать абстракции. Создание абстракций позволит сравнить применение принципов объектно ориентированного программирования и принципов функционального программирования.

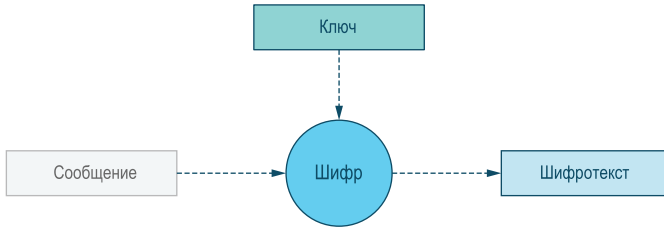


Рис. 15.3 Симметричное шифрование секретным ключом

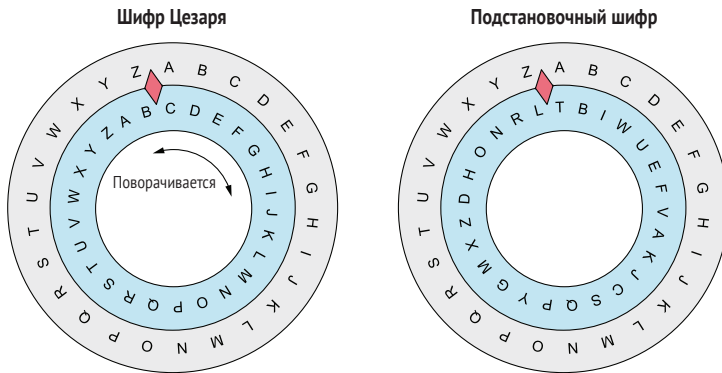


Рис. 15.4 Внутренний диск шифра Цезаря может вращаться. Внутренний диск подстановочного шифра фиксирован, но алфавитный порядок будет случайным

Каждый алгоритм шифрования основан на поиске буквы входного сообщения на внешнем диске. Далее берется соответствующая буква на внутреннем диске, чтобы перенести в шифротекст.

Как видно из шифра Цезаря, буква А будет переведена в букву С, буква В – в букву D и т. д. Как внутренний, так и внешний диск шифра Цезаря расположены в алфавитном порядке. *Шифроключ* – это количество позиций, на которое нужно повернуть внутренний диск (например, сдвинуть против часовой стрелки на две буквы).

Когда римские полководцы отправляли друг другу секретные сообщения, каждый полководец должен был знать этот секретный ключ, чтобы знать, насколько внутренний диск был повернут. Если бы враг узнал этот секрет, то он мог бы расшифровать любое перехваченное сообщение.

Подстановочный шифр характеризуется большей сложностью, поскольку секретным ключом является сам внутренний диск. Диск не нужно поворачивать – его нужно заменять полностью. Для того чтобы две стороны могли отправлять секретные сообщения друг другу, им необходимо было устанавливать один и тот же внутренний диск. Подстановочный шифр характеризуется как отображение между двумя алфавитами. Буквы на внешнем диске образуют один алфавит, который сопоставляется с буквами алфавита на внутреннем диске; он называется подстановочным алфавитом.

Я начну с демонстрации простой реализации обоих шифров, а затем покажу негибкую службу хранения паролей, жестко привязанную к одному шифру. Следующий шаг предусматривает демонстрацию видоизменения шифров и службы хранения паролей, которое позволит менять шифр, используемый для шифрования и дешифрования паролей. В первом подходе будет показано, как эта цель достигается посредством объектно ориентированного стиля программирования, а затем будет продемонстрирован функциональный подход.

15.4.1 Реализация шифра Цезаря

В листинге 15.6 демонстрируется реализация шифра Цезаря. Число повернутых букв передается в аргументе сдвига `shift`, а `ch` – это шифруемая либо дешифруемая буква. В функциях `caesar_encrypt` и `caesar_decrypt` нужно выполнить проверку на наличие входной буквы `ch` в алфавите. Благодаря этому обеспечивается исключение специальных и пробельных символов из процесса шифрования и дешифрования.

Листинг 15.6 Шифрование и дешифрование шифром Цезаря

```
# подсчитать число букв в английском алфавите
const n = length('A':'Z')

function caesar_encrypt(ch::Char, shift::Integer)
    if ch in 'A':'Z'
        'A' + mod((ch - 'A') + shift, n)
    else
        ch
    end
end

function caesar_decrypt(ch::Char, shift::Integer)
    if ch in 'A':'Z'
        'A' + mod((ch - 'A') - shift, n)
    else
        ch
    end
end
```

- ❶ Игнорировать символы не из алфавита.
- ❷ Применить функцию `mod`, чтобы циклически возвращаться в начало, находясь в конце алфавита.
- ❸ Символ в алфавите отсутствует, поэтому возвращаемое значение остается без изменения.

По рис. 15.4 видно, что мы получаем эффект циклического возврата в начало. Буквы в конце алфавита, к примеру такие, как Y и Z, отображаются в буквы A и B в начале алфавита. Вот почему нельзя добавлять сдвиг к каждой букве, `ch + shift`. Функция `mod` (оператор

деления по модулю) побуждает числа работать подобно тому, что мы видим на 12-часовых аналоговых часах:

```
julia> mod(1, 12)
1

julia> mod(9, 12)
9

julia> mod(13, 12) ❶
1

julia> mod(21, 12) ❶
9

julia> mod(17, 12) ❶
5
```

❶ Входное значение больше 12, поэтому оно отсчитывается с начала.

В этом примере в функцию `mod` вводится `ch - 'A'`, чтобы преобразовать буквы в числа в диапазоне от 0 до 25. Благодаря этому упрощается вычисление значения возврата в начало. Затем нужно превратить числа от 0 до 25 в букву. К счастью, как показано в следующем ниже сеансе REPL, математические операции Julia между числами и буквами настроены так, чтобы делать это предсказуемым образом:

```
julia> 'A' + 1
'B': ASCII/Unicode U+0042

julia> 'A' + 4
'E': ASCII/Unicode U+0045

julia> ch = 'Z'; n = 26; shift = 2 ❶
2

julia> 'A' + mod((ch - 'A') + shift, n) ❷
'B': ASCII/Unicode U+0042
```

❶ Инструкции можно отделять точкой с запятой.

❷ Z циклически возвращается в начало и становится B.

Возможно, вы заметили, что здесь использовано то же значение сдвига, что и на рис. 15.4.

Теперь вы знаете, как зашифруется одна буква, но как применить эти знания для шифрования и дешифрования целых сообщений? Для шифрования сообщения можно применить функцию `map`, а затем попытаться расшифровать его позже, чтобы убедиться, что будет возвращено то же, что было введено:

```
julia> message = "THE QUICK BROWN FOX JUMPS OVER THE LAZY DOG";

julia> shift = 2;
```

```
julia> cipher_text = map(ch -> caesar_encrypt(ch, shift), message)
"VJG SWKEM DTQYP HQZ LWORU QXGT VJG NCBA FQI"
```

```
julia> map(cipher_text) do ch
    caesar_decrypt(ch, shift)
end
"THE QUICK BROWN FOX JUMPS OVER THE LAZY DOG"
```

Обратите внимание, что для вызова функции `decrypt` была использована форма `do-end`, чтобы сравнить удобочитаемость исходного кода. Иногда мне кажется, что форма `do-end` читается легче, даже если речь идет только об одной строке исходного кода.

15.4.2 Реализация подстановочных шифров

Для создания подстановочного шифра нужно создать отображение между двумя алфавитами. Для этого нужно использовать находящуюся в модуле `Random` функцию `shuffle`.

Обратите внимание на использование диапазона `'A':'Z'` в следующем ниже исходном коде; он позволяет быстро создать строковый литерал, содержащий все буквы алфавита. Примененная к этому диапазону функция `collect` дала бы массив букв, но в данном случае требуется строковый литерал, поэтому вместо нее мы используем функцию `join`.

Функция `shuffle` будет переставлять элементы массива в случайном порядке. Напомню, что диапазон является подтипом типа `AbstractArray`, поэтому диапазон можно перемешивать, как если бы он был обычным массивом:

```
julia> using Random

julia> join('A':'Z')
"ABCDEFGHIJKLMNOPQRSTUVWXYZ"

julia> shuffle([1, 2, 3])
3-element Vector{Int64}:
 2
 3
 1

julia> shuffle(1:5)
5-element Vector{Int64}:
 1
 2
 5
 4
 3

julia> join(shuffle('A':'Z'))
"PKTAVEQDXGWJMBZOFSLICRUNYH"
```

Эти функции предоставляют инструменты, необходимые для создания отображения между двумя алфавитами на основе словаря. При создании словаря обычно нужны пары ключ-значение, но в следующем ниже исходном коде ключи и значения создаются как отдельные массивы. Тогда как из всего этого сделать словарь?

```
julia> alphabet = join('A':'F') ❶
"ABCDEF"
```

```
julia> substitute = join(shuffle('A':'F'))
"ACFBDE"
```

❶ Использовать диапазон A–F, чтобы сделать примеры короче.

Функция `zip` решает эту задачу. Указанная функция может брать два массива элементов и превращать их в итерируемый объект, который при их передаче в функцию `collect` превращается в массив пар:

```
julia> collect(zip(alphabet, substitute))
6-element Vector{Tuple{Char, Char}}:
 ('A', 'A')
 ('B', 'C')
 ('C', 'F')
 ('D', 'B')
 ('E', 'D')
 ('F', 'E')
```

Возвращенный функцией `zip` итерируемый объект передается конструктору `Dict`. Поскольку этот итерируемый объект на каждой итерации возвращает пары значений, возвращенный из `zip` объект будет использован для создания словаря:

```
julia> mapping = Dict(zip(alphabet, substitute))
Dict{Char, Char} with 6 entries:
 'C' => 'F'
 'D' => 'B'
 'A' => 'A'
 'E' => 'D'
 'F' => 'E'
 'B' => 'C'
```

Эти данные будут переданы на вход в функции шифрования подстановочным шифром.

Листинг 15.7 Шифрование подстановочным шифром на основе поиска по словарю

```
function substitution_encrypt(ch::Char, mapping::Dict{Char, Char})
    get(mapping, ch, ch)
end
```

У этого подхода есть одна вопиющая проблема: для дешифрования требуется обратный поиск. Рассматривать mapping как эквивалент шифроключа – плохое решение. В симметричном шифровании лучше всего использовать одинаковый ключ как для шифрования, так и для дешифрования. Поэтому вместо поиска по словарю мы будем выполнять поиск в массиве пар, как показано в следующем ниже листинге.

Листинг 15.8 Шифрование подстановочным шифром на основе поиска в массиве

```
using Random

alphabet = join('A':'Z')
substitute = join(shuffle('A':'Z'))
mapping = collect(zip(alphabet, substitute))

function substitution_encrypt(ch::Char, mapping::Vector)
    i = findfirst(row->first(row) == ch, mapping) ❶
    if isnothing(i)                                ❷
        ch
    else
        mapping[i][2]                             ❸
    end
end

function substitution_decrypt(ch::Char, mapping::Vector)
    i = findfirst(row->last(row) == ch, mapping)
    if isnothing(i)
        ch
    else
        mapping[i][1]
    end
end
```

- ❶ Найти индекс, в котором первая буква равна ch.
- ❷ Если буква в отображении отсутствует, то ничего не возвращается.
- ❸ Возвращает вторую букву в строке i отображения.

Это программное решение похоже на решение с шифром Цезаря в том смысле, что шифрование или дешифрование не выполняется, если символ ch в алфавите отсутствует. Оно основано на линейном поиске, в котором функция findfirst отыскивает индекс кортежа с интересующим ключом. Если row – это кортеж ('B', 'Q'), то first(row) – это 'B', а last(row) – 'Q'. При шифровании в качестве ключа поиска сначала используется first(row), а затем при дешифровании в качестве ключа поиска выступает last(row).

У вас может возникнуть вопрос: не намного ли линейный поиск в массиве медленнее, чем поиск по словарю? Нет. Линейный поиск в таком коротком массиве элементарных значений будет работать

быстрее. Прежде чем станет заметной разница в производительности поиска, потребуется как минимум 100 записей.

15.5 Разработка службы, не зависящей от алгоритма шифрования

Представьте, что у вас есть служба с использованием шифра и вы хотите упростить замену используемого шифра. Здесь я представлю игрушечный пример, чтобы передать саму концепцию использования службы хранения паролей. Она поддерживает словарь с логинами в качестве ключей и зашифрованными паролями в качестве значений.

Листинг 15.9 Служба хранения паролей

```
mutable struct Vault ①
    passwords::Dict{String, String}
    shift::Int64
end

function Vault(shift::Integer) ②
    emptydict = Dict{String, String}()
    Vault(emptydict, shift)
end

function addlogin!(vault::Vault,
                    login::AbstractString,
                    password::AbstractString) ③
    vault.passwords[login] = map(password) do ch
        encrypt(ch, vault.shift)
    end
end

function getpassword(vault::Vault, login::AbstractString) ④
    map(ch -> decrypt(ch, vault.shift), vault.passwords[login])
end
```

- ① Хранилище сделано мутируемым, так как необходима возможность добавлять пароли.
- ② Инициализировать пустой словарь паролей.
- ③ Добавить зашифрованный пароль в словарь паролей.
- ④ Найти пароль для логина.

Хотя исходный код работает, с этим подходом возникает несколько проблем:

- 1 он жестко запрограммирован под поддержку только одной схемы шифрования: с использованием шифра Цезаря. Должен быть выбор любого шифра;

- 2 служба подразумевает шифрование, а дешифрование выполняется по одной букве за раз. Шифрование следует обобщить для обработки целых строковых литералов, поскольку оно не обязательно реализуется как подстановка букв.

15.6 Разработка службы шифрования с использованием объектно-ориентированного программирования

В листинге 15.9 шифр жестко запрограммирован. Мы хотим иметь возможность менять шифр на другой. Давайте посмотрим, как это сделать.

Решением будет абстрактный интерфейс к шифрам, поэтому пользователям службы паролей не нужно знать какие-либо конкретные сведения о каждом типе шифра. В листинге 15.10 показан объектно-ориентированный подход к этой задаче. Сначала определяется шифр как абстрактный тип с несколькими функциями, которые он должен поддерживать. В каждую функцию добавляются методы, чтобы добавить поддержку конкретного шифра.

Листинг 15.10 Определение абстрактного интерфейса шифрования

```
abstract type Cipher end

function encrypt(cipher::Cipher, char::Char)
    error("Implement encrypt(::", typeof(cipher), ", char)")
end

function decrypt(cipher::Cipher, char::Char)
    error("Implement decrypt(::", typeof(cipher), ", char)")
end

function encrypt(cipher::Cipher, message::AbstractString)
    map(ch -> encrypt(cipher, ch), message)
end

function decrypt(cipher::Cipher, ciphertext::AbstractString)
    map(ch -> decrypt(cipher, ch), ciphertext)
end
```

В этот исходный код заложена опциональная поддержка функциями `encrypt` и `decrypt` *текста сообщения* и *шифротекста*. По умолчанию реализация предусматривает использование функций `encrypt` и `decrypt` на отдельных буквах. Однако если они не будут реализованы в вашем собственном исходном коде, то при попытке выполнить шифрование или дешифрование с помощью своего шифра будет выдано сообщение об ошибке.

Возможно, вы заметили, что здесь к определению интерфейсов был применен несколько иной подход, чем в главе 12. Обычный рекомендуемый подход заключается в определении функций и документировании методов, которые пользователь вашего API должен реализовывать. Здесь же в явной форме задается сообщение об ошибке с объяснением того, что нужно сделать. Полезно знать оба практических приема. В данном случае мне показалось, что это будет практичнее, поскольку для работы со строковыми литералами используются те же функции, что и для работы с отдельными буквами. Сначала реализуется интерфейс `Cipher` для шифра Цезаря.

Листинг 15.11 Шифр Цезаря, реализующий интерфейс `Cipher`

```
struct CaesarCipher <: Cipher
    shift::Int
end

const n = length('A':'Z')

function encrypt(cipher::CaesarCipher, ch::Char)
    if ch in 'A':'Z'
        'A' + mod((ch - 'A') + cipher.shift, n)
    else
        ch
    end
end

function decrypt(cipher::CaesarCipher, ch::Char)
    if ch in 'A':'Z'
        'A' + mod((ch - 'A') - cipher.shift, n)
    else
        ch
    end
end
```

Эта новая реализация шифра Цезаря почти идентична предыдущей реализации (листинг 15.6), за исключением того, что в приведенном выше примере сдвиг поступает из объекта-шифра, а не напрямую.

В листинге 15.12 показан объектно ориентированный подстановочный шифр. Он похож на первоначальную реализацию (листинг 15.8), за исключением того, что отображение сохраняется в объекте `SubstitutionCipher`, а для вызова функций `encrypt` и `decrypt` требуется передача объекта-шифра, а не отображения.

Листинг 15.12 Реализация подстановочного шифра в интерфейсе `Cipher`

```
using Random

struct SubstitutionCipher <: Cipher
```

```

mapping::Vector{Tuple{Char, Char}}                                ❶

function SubstitutionCipher(substitute)
    mapping = zip('A':'Z', collect(substitute))                  ❷
    new(collect(mapping))                                         ❸
end
end

function encrypt(cipher::SubstitutionCipher, ch::Char)
    i = findfirst(row->first(row) == ch, cipher.mapping)
    if isnothing(i)
        ch
    else
        cipher.mapping[i][2]
    end
end

function decrypt(cipher::SubstitutionCipher, ch::Char)
    i = findfirst(row->last(row) == ch, cipher.mapping)
    if isnothing(i)
        ch
    else
        cipher.mapping[i][1]
    end
end
end

```

- ❶ Используется для поиска способа шифрования буквы.
- ❷ Создать список пар букв.
- ❸ Создать экземпляр типа SubstitutionCipher.

Теперь можно видоизменить службу хранения паролей, чтобы она указывала на абстрактный шифр, а не на конкретный (листинг 15.13). За счет этого обеспечивается возможность замены шифра, используемого в любом конкретном шифре, реализующем интерфейс Cipher.

Листинг 15.13 Служба хранения паролей, не зависящая от алгоритма шифрования

```

mutable struct Vault
    passwords::Dict{String, String}
    cipher::Cipher
end

function Vault(cipher::Cipher)
    Vault{Dict{String, String}}{(), cipher}
end

function addlogin!(vault::Vault, login::AbstractString,
    ➡ password::AbstractString)
    vault.passwords[login] = encrypt(vault.cipher, password)
end

```

```
function getpassword(vault::Vault, login::AbstractString)
    decrypt(vault.cipher, vault.passwords[login])
end
```

Теперь можно попробовать обновленную службу хранения паролей с разными шифрами. Сначала будет показан пример с шифром Цезаря. Мы начнем с создания хранилища под размещение паролей. Хранилище инициализируется шифром, который будет использоваться для шифрования и дешифрования хранящихся в нем паролей.

Затем вызывается функция `addlogin!`, чтобы добавить пароли в хранилище. После этого используется функция `getpassword`, чтобы убедиться в получении тех же паролей, что и при вводе:

```
julia> vault = Vault(CaesarCipher(23))
Vault{Dict{String,String}{}, CaesarCipher(23)}

julia> addlogin!(vault, "google", "BING")
"YFKD"

julia> addlogin!(vault, "amazon", "SECRET")
"PBZOBQ"

julia> getpassword(vault, "google")
"BING"

julia> getpassword(vault, "amazon")
"SECRET"
```

Далее идет пример с подстановочным шифром. В данном случае подстановочный шифр инициализируется подстановочным алфавитом. Из примера видно, что буквы ABC будут заменены на буквы CQP:

```
julia> substitute = "CQPYXVFHRNZMWOITJSUBKLEGDA";

julia> cipher = SubstitutionCipher(substitute);

julia> vault = Vault(cipher);

julia> addlogin!(vault, "amazon", "SECRET")
"UXPSXB"

julia> addlogin!(vault, "apple", "JONAGOLD")
"NIOCFIMY"

julia> getpassword(vault, "amazon")
"SECRET"

julia> getpassword(vault, "apple")
"JONAGOLD"
```

15.7 Разработка службы шифрования с использованием функционального программирования

Смысл демонстрации исполнения абстракции с использованием объектно ориентированного подхода состоит в том, что с этим подходом программисты в основном уже знакомы. В данном случае *объектно ориентированный* означает, что задача решается, думая в терминах иерархии типов и объектов. Я представил шифр как объект и определил функции с методами, которые оперируют на этих объектах-шифрах.

В функциональном подходе мы будем стремиться решить задачу, думая в терминах функций: функций более высокого порядка и замыканий. Цель состоит в том, чтобы познакомиться с двумя разными взглядами на решение задач программирования. Для того чтобы считаться хорошим программистом на языке Julia, нужно понимать и тот, и другой.

15.7.1 Определение функционального шифра Цезаря

Шифр Цезаря будет определен с использованием техники частичного применения, впервые показанной в листинге 15.5. Указанный подход позволит расширить изначальное решение, разработанное в функциях `caesar_encrypt` и `caesar_decrypt` (листинг 15.6).

Обратите внимание, что в исходном коде листинга 15.14 больше нет типов-шифров. Нет объекта данных, представляющего шифр Цезаря. Вместо этого в `caesar_encrypt` и `caesar_decrypt` добавлены новые методы, которые обеспечивают возможность частичного применения, поэтому при предоставлении только аргумента сдвига (`shift`) будет возвращаться функция, принимающая букву, а не буква в шифротексте.

Листинг 15.14 Шифр Цезаря с функциональным колоритом

```
const n = length('A':'Z')

# Изначальные функции шифра
function caesar_encrypt(ch::Char, shift::Integer)
    if ch in 'A':'Z'
        'A' + mod((ch - 'A') + shift, n)
    else
        ch
    end
end
```

```

function caesar_decrypt(ch::Char, shift::Integer)
    caesar_encrypt(ch, -shift) ❶
end

# Реализовать функциональный интерфейс
# с использованием частичного применения
function caesar_encrypt(shift::Integer)
    msg -> map(msg) do ch ❷
        caesar_encrypt(ch, shift)
    end
end

function caesar_decrypt(shift::Integer)
    msg -> map(msg) do ch ❷
        caesar_decrypt(ch, shift)
    end
end

```

- ❶ Уловка, позволяющая избежать реализации почти идентичного кода.
- ❷ Вернуть замыкание с захваченным сдвигом.

Давайте посмотрим на применение этих функций. Начнем с вызова функции `caesar_encrypt` со значением сдвига 1. Она возвращает функцию, предназначенную для шифрования. Затем эта функция используется для шифрования строкового литерала "ABC". Аналогичный шаблон используется для создания и использования функции дешифрования:

```

julia> encrypt = caesar_encrypt(1)

julia> encrypt("ABC")
"BCD"

julia> decrypt = caesar_decrypt(1)

julia> decrypt("BCD")
"ABC"

julia> encrypt('A') ❶
0-dimensional Array{Char, 0}:
'A'

julia> decrypt('B') ❶
0-dimensional Array{Char, 0}:
'A'

```

- ❶ Это работает, потому что функция `map` может выполнять отображение отдельных букв.

Выгода от этого решения состоит в том, что с помощью конвейерного оператора `|>` результаты на выходе из функций можно легко выстраивать в цепочку:

```
julia> "HELLO" |> caesar_encrypt(2)
"JGNNQ"

julia> "HELLO" |> caesar_encrypt(2) |> caesar_decrypt(2)
"HELLO"
```

15.7.2 Определение подстановочного шифра с функциональным колоритом

Для создания подстановочного шифра будет расширен исходный код подстановочного шифра, приведенный в листинге 15.8. Здесь мы снова воспользуемся техникой частичного применения, добавив в существующие функции `substitution_encrypt` и `substitution_decrypt` два метода (листинг 15.15). На входе они принимают только отображение (`mapping`), но возвращают функции шифрования, которые будут шифровать либо дешифровать соответственно переданное им сообщение либо шифротекст.

Листинг 15.15 Подстановочный шифр с функциональным колоритом

```
function substitution_encrypt(mapping::Vector)
    msg -> map(msg) do ch
        substitution_encrypt(ch, mapping)
    end
end

function substitution_decrypt(mapping::Vector)
    msg -> map(msg) do ch
        substitution_decrypt(ch, mapping)
    end
end
```

Подстановочный шифр будет использоваться аналогично шифру Цезаря. Основное отличие состоит в том, что в качестве ключа шифрования будет использоваться отображение, а не сдвиг:

```
julia> alphabet = join('A':'Z');

julia> substitute = join(shuffle('A':'Z'));

julia> mapping = collect(zip(alphabet, substitute))

julia> "HELLO" |> substitution_encrypt(mapping)
"NEPPR"

julia> "HELLO" |> substitution_encrypt(mapping) |>
  ➡ substitution_decrypt(mapping)
"HELLO"
```

Теперь вы знаете все части, необходимые для создания хранителя паролей на основе принципов функционального программирования.

15.7.3 Реализация функциональной службы хранения паролей

Теперь давайте соберем все это вместе и создадим хранитель паролей, в котором для сохранения и извлечения логинов и паролей используются функции шифрования и дешифрования. Данную задачу можно решить несколькими способами. В листинге 15.16 я намеренно переборщу, чтобы создать сильный контраст с объектно ориентированным решением.

Листинг 15.16 Служба хранения паролей с чрезмерно функциональным стилем

```
function makevault(encrypt::Function, decrypt::Function)
    passwords = Dict{String, String}{}

    function addlogin(login::AbstractString, password::AbstractString)
        passwords[login] = encrypt(password)
    end

    function getpassword(login::AbstractString)
        decrypt(passwords[login])
    end

    addlogin, getpassword
end
```

Давайте рассмотрим пример использования этой реализации, чтобы определить хранителя паролей с использованием шифра Цезаря. Хранилище инициализируется двумя объектами-функциями, созданными вызовами соответственно функций `caesar_encrypt` и `caesar_decrypt`:

```
julia> addlogin, getpasswd = makevault(
                                caesar_encrypt(2),
                                caesar_decrypt(2));

julia> addlogin("google", "SECRET")
"UGETGV"

julia> addlogin("amazon", "QWERTY")
"SYGTVA"

julia> getpasswd("google")
"SECRET"

julia> getpasswd("amazon")
"QWERTY"
```

В подстановочном шифре требуется выполнить немного больше настроек, чтобы создать вектор отображений. Во всех остальных аспектах хранилище паролей настраивается так же, как и шифр Цезаря:

```
julia> using Random

julia> alphabet = join('A':'Z')
"ABCDEFGHIJKLMNOPQRSTUVWXYZ"

julia> substitute = join(shuffle('A':'Z'))
"TQRBVPMHNFUESGZOLIDXCWYJK"

julia> mapping = collect(zip(alphabet, substitute));

julia> addlogin, getpasswd = makevault(
    substitution_encrypt(mapping),
    substitution_decrypt(mapping));

julia> addlogin("google", "SECRET")
"DVRIVX"

julia> addlogin("amazon", "QWERTY")
"LWVIXJ"

julia> getpasswd("google")
"SECRET"

julia> getpasswd("amazon")
"QWERTY"
```

Теперь самое время отойти на несколько шагов назад и подумать о причине, по которой нужно конструировать замыкания именно так, как показано в примерах. Целевая задача остается той же, что и в объектно ориентированном случае: представить обобщенный интерфейс к шифрам, чтобы программист мог менять используемый шифр без изменения реализации хранителя паролей.

Это было сделано путем возвращения функций шифрования и дешифрования, которые не раскрывают никаких деталей реализации в своих сигнатурах. *Сигнатура функции* относится к тому, какие аргументы функция принимает, их порядку и типу. Шифр Цезаря и подстановочный шифр производят функции шифрования и дешифрования с одинаковыми сигнатурами. Именно поэтому они взаимозаменяемы.

Я попытался сделать каждое представленное выше решение как можно более различающимся, чтобы придать большую ясность сути разницы между функциональным и объектно ориентированным программированием. Иногда карикатура помогает донести мысль. Тем не менее в реальных программных решениях всегда следует стараться использовать хороший вкус и находить разумный баланс между объектно ориентированным и функциональным подходом.

Программное решение для хранения паролей на основе объектно ориентированного подхода, на мой взгляд, было лучше, тогда как преобразование змеиногo регистра в верблужий регистр очень хорошо работало с функциональным подходом. В чем разница? Функциональное программирование работает очень хорошо, когда за-

дачу можно свести к некоторому преобразованию данных. Вместе с тем, когда вы имеете дело с чем-то, что по своей сути имеет состояние, например со службой хранения паролей, объектно ориентированный подход является более естественным, поскольку объектно ориентированное программирование сводится к видоизменению состояния.

Резюме

- Функциональное и процедурное программирование часто считают взаимозаменяемыми, но это не одно и то же. В функциональном программировании функции – это первоклассные объекты, которые можно передавать и которыми можно манипулировать с помощью функций более высокого порядка.
- Функционально ориентированный исходный код легче анализировать и отлаживать, так как он формирует более четкий поток данных.
- Трудночитаемая вложенность вызовов функций решается посредством оператора выстраивания функций в цепочку `|>`.
- *Анонимные функции* – это функции без имен. Они помогают упрощать создание внутристрочных замыканий.
- Анонимные функции можно создавать с помощью оператора `->` либо формы `do-end`.
- *Замыкание* – это функция, которая захватывает состояние из окружающей ее области видимости; это функция с памятью. Замыкания можно использовать для имитации объектов с состоянием, упрощения частичного применения и управления получением и высвобождением ресурсов (открытие и закрытие файла).
- *Частичное применение* – это технический прием, при котором возвращается не результат, а функция, когда функции были переданы не все аргументы. Благодаря ему упрощается создание аргументов функций для функций более высокого порядка, таких как `map`, `filter` и `reduce`.
- С помощью функции `shuffle` из встроенного модуля `Random` элементы массива можно перемешивать в случайном порядке. Многие алгоритмы шифрования требуют случайного перемешивания входных данных. Указанная функция также неплохо справляется с созданием входных данных для тестирования функций сортировки.
- В целях достижения оптимальных результатов программирования на языке Julia следует комбинировать функциональные и объектно ориентированные технические приемы. Разные задачи требуют разных подходов. Благодаря практике вы разовьете более глубокое интуитивное понимание случаев, когда один подход будет работать лучше другого.

16

Организация и модуляризация исходного кода

Эта глава охватывает следующие ниже темы:

- введение концепции вычислительных сред в управление зависимостями;
- добавление и удаление зависимостей пакетов с помощью вычислительных сред;
- разработка исходного кода в разных вычислительных средах и переключение между ними;
- создание пакета Julia и добавление к нему зависимостей;
- организация исходного кода внутри пакета;
- добавление тестов в пакеты;
- обследование взаимосвязи между модулями и пакетами.

Как было показано в предыдущих главах, в случае крупномасштабной разработки программного обеспечения нельзя просто выгружать исходный код в файлы Julia в бессистемном порядке. Вместо этого исходный код должен быть модульным и организованным. Организованный исходный код более понятен, и в нем легче ориентироваться. В этой главе на примере геометрического исходного кода вы научитесь организовывать разработанный ранее исходный код в пакет Julia.

Пакеты удобны тем, что они дают возможность комплектовать взаимосвязанный исходный код, распространять и обеспечивать его версионный контроль. Пакеты могут зависеть от других пакетов, образуя сложные деревья зависимостей. Реальные существующие системы строятся путем комбинирования многочисленных пакетов в одну большую систему. Если бы вы работали в крупной организации над масштабным проектом, то разные коллективы разработчиков, скорее всего, создавали бы разные пакеты, которые затем комбинировались бы в единое целое для создания законченной системы.

В этой главе вы научитесь упорядочивать исходный код внутри пакета, добавлять и удалять зависимости от других пакетов и настраивать тесты для пакета. Не забудьте загрузить модули с помощью инструкций `using Statistics` и `using LinearAlgebra`. Кроме того, будут даны пояснения по поводу того, как понятие *модуля* соотносится с понятием пакета.

16.1 Настройка рабочей среды

При разработке программного обеспечения нередко возникает потребность использовать разные версии одних и тех же пакетов. При разработке исходного кода Julia на своем рабочем месте в коллективе вы, скорее всего, будете использовать наиболее стабильную и хорошо протестированную версию каждого пакета, но это бывает не так важно для любительских проектов, и тогда вы, возможно, будете предпочитать самые последние версии с последними интересными функциональными возможностями.

Вычислительные среды Julia позволяют переключаться между *проектной* и *любительской* средами с различными версиями пакетов. Я использую слово *проектный* (`job`) вместо *рабочий*, чтобы не путать его с рабочим каталогом, который относится к каталогу, в котором вы сейчас работаете. В этом разделе мы рассмотрим следующие темы:

- создание вычислительной среды;
- добавление в вычислительную среду и удаление из нее разных версий пакетов;
- активация и переключение между разными вычислительными средами;
- понимание взаимосвязи между модулем и пакетом.

Создание среды сводится к созданию каталога для хранения каждой вычислительной среды. В этом примере мы запускаем Julia и переходим в режим оболочки, нажав `;`, а затем выполняем команды оболочки Unix по созданию каталогов. В качестве альтернативы вполне допустимо использовать графический файловый менеджер, такой как Finder или File Explorer:

```
shell> mkdir job hobby ❶
```

```
shell> ls ❷
hobby    job
```

- ❶ Создать каталоги `job` и `hobby`.
- ❷ Вывести содержимое текущего каталога.

Напомню, что в интерактивной среде REPL языка Julia вход в пакетный режим осуществляется с помощью нажатия `]`. В пакетном режиме можно исполнять команды активации разных вычислительных сред и добавления в них пакетов. При переключении вычислительной среды приглашение изменится и будет показывать среду, в которой вы находитесь. Например, если вы находитесь в проектной (`job`) среде, то приглашение будет выглядеть так: (`job`) `pkg>`. В этом примере мы активируем проектную среду, поэтому все последующие команды будут видоизменять именно ее:

```
(@v1.7) pkg> activate job
```

```
(job) pkg>
```

Далее мы добавим в эту среду несколько пакетных зависимостей. В демонстрационных целях не важно, какие пакеты Julia добавляются, но я решил продемонстрировать на графопостроительном пакете `CairoMakie` (<https://makie.juliaplots.org>) и пакете `ElectronDisplay` (<https://github.com/queryverse/ElectronDisplay.jl>) для вывода графиков в окне. Если вы используете редактор Visual Studio Code, то пакет `ElectronDisplay` не потребуется, так как указанный редактор уже может выводить любой график Julia.

ПРИМЕЧАНИЕ Пакет `CairoMakie` является частью связанных графопостроительных пакетов Julia под названием `Makie`. Все пакеты `Makie` предоставляют пользователям одинаковые типы и функции, и единственная разница заключается в типе создаваемой графики. Пакет `CairoMakie` позволяет создавать высококачественные двумерные векторные графики, а пакет `GLMakie` – интерактивные трехмерные графики.

Допустим, в своих любительских проектах вы любите использовать последнюю версию пакета `CairoMakie`, но ваш работодатель немного консервативен, и на работе вместо нее вы должны использовать версию 0.5.10:

```
(job) pkg> add CairoMakie@0.5.10 ❶
Updating registry at `~/\.julia/registries/General.toml`
Resolving package versions...
[13f3f980] + CairoMakie v0.5.10
Updating `~/dev/job/Manifest.toml`
```

```
(job) pkg> add ElectronDisplay ②
  Resolving package versions...
  Updating `~/dev/job/Project.toml`
  [d872a56f] + ElectronDisplay v1.0.1
  Updating `~/dev/job/Manifest.toml`

(job) pkg> status ③
  Status `~/dev/job/Project.toml`
  [13f3f980] CairoMakie v0.5.10
  [d872a56f] ElectronDisplay v1.0.1
```

- ① Добавить пакет CairoMakie версии 0.5.10 в проектную среду.
- ② Добавить последнюю версию пакета ElectronDisplay.
- ③ Вывести список всех добавленных в эту среду пакетов.

При исполнении пакетных команд `add` вы увидите гораздо больше информации, чем показано здесь. Я убрал большую ее часть, потому что она заняла бы несколько страниц – были оставлены только самые важные фрагменты информации.

При вызове `add ElectronDisplay` выводится сообщение об изменении файла `~/dev/job/Project.toml`. Что это за файл? Проектная среда находится в каталоге `~/dev/job`. Если в текущей активной среде нет файла `Project.toml`, то Julia его создаст, чтобы хранить в нем информацию о добавленных в среду пакетах.

Строка `ElectronDisplay v1.0.1` сообщает об установленной версии пакета `ElectronDisplay`. В 2022 году, когда писалась данная книга, это была последняя версия.

Обратите внимание, что команда `add` для пакета `CairoMakie` немного отличалась. К имени пакета было добавлено `@0.5.10`, чтобы сообщить менеджеру пакетов Julia, что нужна версия 0.5.10 пакета `CairoMakie`, а не последняя текущая версия. При переключении на любительскую (`hobby`) среду мы будем использовать последнюю версию:

```
(work) pkg> activate hobby
  Activating new project at `~/dev/hobby`

(hobby) pkg> add CairoMakie ElectronDisplay
  Resolving package versions...
  Updating `~/dev/hobby/Project.toml`
  [13f3f980] + CairoMakie v0.7.5
  [d872a56f] + ElectronDisplay v1.0.1
  Updating `~/dev/hobby/Manifest.toml`

(hobby) pkg> status
  Status `~/dev/hobby/Project.toml`
  [13f3f980] CairoMakie v0.7.5
  [d872a56f] ElectronDisplay v1.0.1
```

В любительской среде команда `add` была применена немного по-другому. Были перечислены все пакеты, которые нужно было до-

бавить, чтобы сделать все за один раз. После этого Julia уведомляет о том, что пакет CairoMakie v0.7.5 был установлен.

Команда `status` применяется в любой ситуации, когда нужно получить сводку об установленных в текущей активной среде пакетах. Указанная команда читает информацию, хранящуюся в `Project.toml`. Эта информация используется для отыскания нужного пакета в локальном хранилище пакетов по адресу `~/julia/packages`:

```
shell> ls hobby
Manifest.toml      Project.toml

shell> cat hobby/Project.toml
[deps]
CairoMakie = "13f3f980-e62b-5c42-98c6-ff1f3baf88f0"
ElectronDisplay = "d872a56f-244b-5cc9-b574-2017b5b909a8"
```

Длинные цепочки букв и цифр, начинающиеся соответственно с `13f3f980` и `d872a56f`, являются универсальным уникальным идентификатором (UUID)¹ каждого пакета. Имена пакетов не обязательно будут уникальными. Несколько разработчиков уже могли создать пакет под названием `CairoMakie`; поэтому, чтобы иметь возможность однозначно идентифицировать конкретный пакет, каждому из них нужен уникальный UUID. Поскольку в настоящее время нет другого пакета с именем `CairoMakie`, конфликт имен пакетов отсутствует.

Что делать, если бы существовал еще один пакет `CairoMakie`? В этом случае пакет необходимо добавить с помощью следующей ниже команды:

```
(hobby) pkg> add CairoMakie=13f3f980-e62b-5c42-98c6-ff1f3baf88f0
```

Система пакетов поставляется со своей собственной справочной системой, поэтому можно написать `? add` и получить полную сводку всевозможных способов добавления пакета. Для удаления пакетов используется команда `rm`. Можно добавить пакет и взглянуть на изменения в файле `Project.toml` при удалении пакета. Ниже приведен пример добавления и удаления пакета `Dates`:

```
(hobby) pkg> add Dates
  Resolving package versions...
  Updating `~/dev/hobby/Project.toml`
  [ade2ca70] + Dates

(hobby) pkg> status
  Status `~/dev/hobby/Project.toml`
  [13f3f980] CairoMakie v0.7.5
  [d872a56f] ElectronDisplay v1.0.1
  [ade2ca70] Dates

(hobby) pkg> rm Dates
```

¹ Англ. Universally Unique IDentifier. – Прим. перев.

```
Updating `~/dev/hobby/Project.toml`
[ade2ca70] - Dates

(hobby) pkg> status
Status `~/dev/hobby/Project.toml`
[13f3f980] CairoMakie v0.7.5
[d872a56f] ElectronDisplay v1.0.1
```

16.1.1 Использование пакета в интерактивной среде REPL языка Julia

Добавление пакета в активную вычислительную среду не делает предоставляемые пакетом функции и типы доступными в интерактивной среде REPL или проекте Julia, который вы программируете. Они становятся доступными только после того, как будет исполнена инструкция `using` или `import`. Пакет Makie располагает большим числом разных функций для построения графиков. Я продемонстрирую применение двух функций: `lines` и `scatter`. Если вы не используете редактор VS Code, то прежде чем что-либо станет видимым, сперва нужно загрузить пакет ElectronDisplay.

Мы будем использовать Makie, чтобы построить график кривых синуса и косинуса. Для этого сгенерируем серию координат x , y , хранящихся соответственно в `xs`, `ys1` и `ys2`.

Листинг 16.1 Построение графика кривых синуса и косинуса с помощью Makie

```
using ElectronDisplay ❶
using CairoMakie      ❷

xs = 1:0.1:10         ❸
ys1 = map(sin, xs)
ys2 = map(cos, xs)

scatter(xs, ys1)
scatter!(xs, ys2)     ❹

current_figure()      ❺
```

- ❶ Создать окно для отображения данных графика.
- ❷ Сделать доступными функции `scatter`, `scatter!` и `current_figure`.
- ❸ Создать значения от 1 до 10 с шагом 0.1.
- ❹ Изменить текущий рисунок, добавив график косинуса.
- ❺ Отправляет текущий рисунок на электронный дисплей.

При вычислении инструкции `using CairoMakie` Julia будет искать пакет с именем `CairoMakie` в своей текущей среде. Если текущая среда является любительской (`hobby`), то она загрузит исходный код пакета Makie версии 0.7.5. Однако если этот исходный код вычисляется в проектной (`job`) среде, то вместо этого будет загружена

версия v0.5.10. Если мы исполним исходный код в интерактивной среде REPL, то получим рисунок с двумя пунктирными графиками (рис. 16.1).

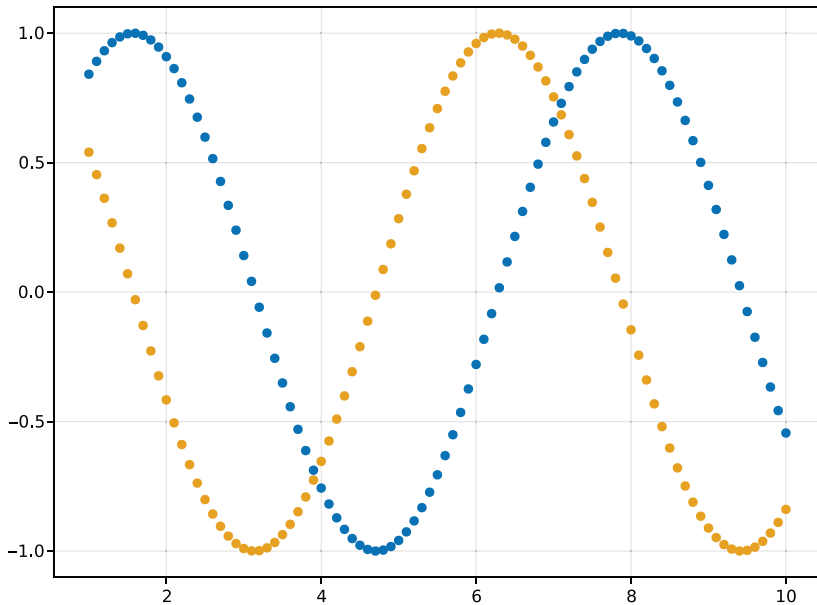


Рис. 16.1 Диаграмма рассеяния Makie синусной и косинусной функций

Эти графики называются диаграммами рассеяния. Каждая координата (x, y) генерирует цветную точку. Если нужны линии, соединяющие каждую координатную точку, то вместо этого следует использовать функции `lines` и `lines!`, как показано ниже.

Листинг 16.2 Построение графика кривых синуса и косинуса с плавными линиями

```
lines(xs, ys1, linewidth=5) ❶  
lines!(xs, ys2, linewidth=5) ❶  
  
current_figure()
```

❶ Начертить линии толщиной 5.

В результате вычисления исходного кода будет сгенерирован график, показанный на рис. 16.2. Для изменения внешнего вида каждого графика существует целый ряд именованных аргументов. Например, именованный аргумент `linewidth=5` делает линии толще.

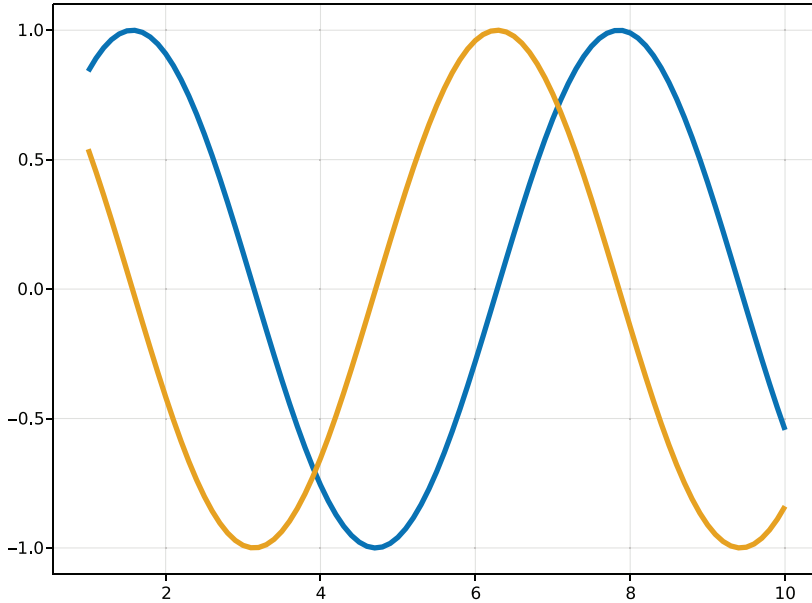


Рис. 16.2 Линейный график Makie синусной и косинусной функций

Подробнее о построении графиков в Julia с использованием пакета Makie можно узнать, посетив официальный веб-сайт Makie (<https://docs.makie.org/stable/>).

16.1.2 Как модули соотносятся с пакетами

Описание процесса загрузки пакетов было не совсем точным. В пакетах определяется модуль с тем же именем, что и пакет. Следовательно, при написании `using Dates` Julia просматривает пакет `Dates` и загружает определенный в нем модуль `Dates`. Различие станет намного яснее, когда мы определим свой собственный пакет и модуль.

Пакет Julia можно рассматривать как физический пакет исходного кода, ресурсов и метаданных, таких как версия, имя и зависимости. Эти компоненты важны для механизма загрузки пакетов Julia, но не являются языковыми конструкциями. Подобно ключевым словам `function` и `struct`, которые служат для определения функций и составных типов, в Julia есть ключевое слово `module`, предназначенное для определения модуля.

В отличие от функций, которые позволяют группировать фрагменты исходного кода, модули позволяют группировать взаимосвязанные функции и типы. Модули также создают *пространство имен*, как и функция. Это означает, что в разных функциях можно использовать переменные с одинаковым именем, и они не будут мешать друг другу, поскольку каждая функция образует отдельное пространство имен. Функции `volume` в следующем ниже листинге не

мешают друг другу, так как они находятся в двух отдельных модулях: `Cylinder` и `Cone`.

Листинг 16.3 Функции с одинаковыми именами в разных модулях

```
module Cylinder
    volume(r,h) =  $\pi * r^2 * h$ 
end

module Cone
    volume(r, h) =  $\pi * r^2 * h / 3$ 
end
```

Эти модули можно вычислить в интерактивной среде REPL и вызвать разные функции `volume` следующим образом:

```
julia> Cylinder.volume(2.5, 3)
58.90486225480862

julia> Cone.volume(2.5, 3)
19.634954084936208
```

Однако в данной главе вы сосредоточитесь на определении одного модуля на один пакет. В Julia такое программное решение является самым прагматичным.

16.2 Создание собственного пакета и модуля

На следующих страницах я покажу, как создавать свой собственный пакет и модуль, взяв функции объема и тригонометрические функции из глав 2 и 4 и организовав их в модуль `ToyGeometry`, хранящийся в пакете. Пакет можно создать с нуля вручную, но удобнее использовать диспетчер пакетов Julia, который сгенерирует каркас модуля.

После создания пакета и осмотра его структуры я покажу, как добавлять исходный код в пакет. Затем вы научитесь расширять пакет функциональностью графопостроения, чтобы лучше понять принцип работы с пакетными зависимостями.

16.2.1 Генерирование пакета

Мы начнем с каталога `~/dev`, в котором находятся каталоги любительской и проектной сред, соответственно `hobby` и `job`. Разумеется, его можно организовать как угодно. Мы используем режим оболочки, чтобы перейти в любительский каталог, в котором хотим создать свой пакет `ToyGeometry`. Команда `generate` применяется в пакетном режиме для создания пакета:

```
shell> cd hobby/
~/dev/hobby
```

```
(hobby) pkg> generate ToyGeometry
Generating project ToyGeometry:
  ToyGeometry/Project.toml
  ToyGeometry/src/ToyGeometry.jl
```

Более сложный способ создания пакетов состоит в использовании библиотеки `PkgTemplate`, но для начала команда `generate` подойдет лучше, так как она создает минималистичный пакет. Если вы посмотрите на содержимое пакета, то увидите, что он содержит только два файла: `Project.toml` и `src/ToyGeometry.jl`:

```
shell> tree ToyGeometry
ToyGeometry
├── Project.toml
└── src
    └── ToyGeometry.jl
```

Возможно, вы будете удивлены, увидев там файл `Project.toml`. Раз-ве он не используется для определения среды? Все верно! И пакет `Julia` – это, по сути, самостоятельная среда. Пакет `Julia`, будучи средой, может добавлять другие пакеты, от которых он зависит.

ПРИМЕЧАНИЕ Вычислительные среды могут быть вложенными, но это не имеет практического значения. Полезнее вкладывать именно модули, но вложение модулей в этой книге рассматриваться не будет.

На данный момент зависимостей нет, поэтому в файле `Project.toml` будут отображаться только данные о пакете, такие как его имя, идентифицирующий пакет уникальный `UUID`, автор пакета и текущая версия пакета:

```
shell> cat ToyGeometry/Project.toml
name = "ToyGeometry"
uuid = "bbcec4ee-a196-4f18-8a9a-486bb424b745"
authors = ["Erik Engheim <erik.engheim@mac.com>"]
version = "0.1.0"
```

Давайте добавим зависимый пакет в пакет `ToyGeometry`, чтобы посмотреть, как работает добавление зависимостей. Мы добавим два пакета, `Dates` и `Base64`, которые находятся в стандартной библиотеке, поставляемой в комплекте с `Julia` (скачивать из интернета не нужно). Так как мы хотим добавить эти зависимости не в любительскую среду, а в среду `ToyGeometry`, сначала нужно переключить активные среды:

```
(hobby) pkg> activate ToyGeometry/
Activating project at `~/dev/hobby/ToyGeometry`

(ToyGeometry) pkg> add Dates Base64
Resolving package versions...
Updating `~/dev/hobby/ToyGeometry/Project.toml`
```

```
[2a0f44e3] + Base64
[ade2ca70] + Dates
  Updating `~/dev/hobby/ToyGeometry/Manifest.toml`
[2a0f44e3] + Base64
[ade2ca70] + Dates
[de0858da] + Printf
[4ec0a83e] + Unicode

(ToyGeometry) pkg> status
  Project ToyGeometry v0.1.0
  Status `~/dev/hobby/ToyGeometry/Project.toml`
[2a0f44e3] Base64
[ade2ca70] Dates
```

Файл `Project.toml` в пакете `ToyGeometry` теперь будет обновлен и будет показывать зависимости пакета:

```
shell> cat ToyGeometry/Project.toml
name = "ToyGeometry"
uuid = "bbcec4ee-a196-4f18-8a9a-486bb424b745"
authors = ["Erik Engheim <erik.engheim@mac.com>"]
version = "0.1.0"

[deps]
Base64 = "2a0f44e3-6c83-55bd-87e4-b1978d98bd5f"
Dates = "ade2ca70-3891-5945-98fb-dc099432e06a"
```

Мы также получили новый файл с именем `Manifest.toml`, в котором хранится информация о пакетах, от которых зависят пакеты `Dates` и `Base64`. Например, `Dates` зависит от пакета `Printf`, предназначенного для форматирования строковых литералов дат. Кроме того, пакет `Printf` зависит от пакета `Unicode`:

```
shell> cat ToyGeometry/Manifest.toml
julia_version = "1.7.2"
manifest_format = "2.0"

[[deps.Base64]]
uuid = "2a0f44e3-6c83-55bd-87e4-b1978d98bd5f"

[[deps.Dates]]
deps = ["Printf"]
uuid = "ade2ca70-3891-5945-98fb-dc099432e06a"

[[deps.Printf]]
deps = ["Unicode"]
uuid = "de0858da-6303-5e67-8744-51eddeeeb8d7"

[[deps.Unicode]]
uuid = "4ec0a83e-493e-50e2-b9ac-8f72acf5a8f5"
```

Для пакетов, не входящих в стандартную библиотеку Julia, файловые записи будут более сложными. Ниже приведена запись из файла `Manifest.toml` в проектной (job) среде для пакета `Colors`:

```
[[deps.Colors]]
deps = ["ColorTypes", "FixedPointNumbers", "Reexport"]
git-tree-sha1 = "417b0ed7b8b838aa6ca0a87aadf1bb9eb111ce40"
uuid = "5ae59095-9a9b-59fe-a467-6f913c188581"
version = "0.12.8"
```

Вы видите, что в нем указан не только UUID пакета `Colors`, но и его версия. Вам вовсе не нужно знать значение строкового литерала `git-tree-sha1`, за исключением того, что он помогает Julia отыскивать для загрузки нужный пакет в локальном хранилище пакетов.

16.2.2 Добавление исходного кода в пакет

На данный момент пакет ничего не делает, поэтому давайте добавим исходный код, скопировав функции объема и тригонометрии из глав 2 и 4. Начнем с создания двух файлов, `volume.jl` и `trig.jl`, чтобы хранить в них этот исходный код:

```
shell> cd ToyGeometry/
~/dev/hobby/ToyGeometry

shell> touch src/volume.jl src/trig.jl

shell> tree .
.
├─ Manifest.toml
├─ Project.toml
└─ src
    ├─ ToyGeometry.jl
    ├─ trig.jl
    └─ volume.jl
```

Добавим исходный код, показанный в листинге 16.4, в файл `src/volume.jl`.

Листинг 16.4 ToyGeometry/src/volume.jl

```
"""
    sphere_volume(r)
Вычислить объем сферы с радиусом `r`
"""
function sphere_volume(r::Number)
    4π*r^3/3
end

"""
    cylinder_volume(r, h)
Вычислить объем цилиндра с радиусом `r`
и высотой `h`.
"""
function cylinder_volume(r::Number, h::Number)
    π*r^2*h
end
```

```

"""
    cone_value(r, h)
Вычислить объем конуса с радиусом `r`
и высотой `h`.
"""
function cone_value(r::Number, h::Number)
     $\pi * r^2 * h / 3$ 
end

```

Далее добавим функцию из главы 4, в которой для вычисления синуса использовался ряд Тейлора; позже мы добавим косинусную функцию. Этот исходный код войдет в файл `src/trig.jl` следующим образом.

Листинг 16.5 ToyGeometry/src/trig.jl

```

"""
    sine(x)
Вычислить синус угла `x` в радианах
"""
function sine(x::Number)
    n = 5
    total = 0
    for i in 0:n
        total += (-1)^i * x^(2i+1) / factorial(2i + 1)
    end
    total
end

```

В пакете `ToyGeometry` определяется модуль с именем `ToyGeometry`. Мы хотим, чтобы все написанные функции были частью модуля `ToyGeometry`. При выполнении функций `include` внутри определения модуля все определения функций вычисляются внутри модуля `ToyGeometry` и становятся его частью.

Листинг 16.6 ToyGeometry/src/ToyGeometry.jl

```

module ToyGeometry

include("volume.jl")
include("trig.jl")

end # модуль

```

Если вы мыслите как разработчик C/C++, то механизм работы `include`, возможно, будет для вас неочевидным, но на самом деле он довольно прост. Давайте проведем небольшой эксперимент, чтобы продемонстрировать.

Начнем с создания файла с именем `arithmetic.jl` и напомним в нем `3 + 4`. Функцию `include` можно вызывать где угодно, даже внутри определения функции:

```
julia> x = include("arithmetic.jl")
7

julia> function calc()
    include("arithmetic.jl")
end

julia> calc()
7
```

Вовсе не нужно разбивать исходный код на несколько файлов и добавлять их в модуль с помощью `include`. Модуль `ToyGeometry` вполне можно было бы определить, как показано ниже.

Листинг 16.7 ToyGeometry/src/ToyGeometry.jl

```
module ToyGeometry

sphere_volume(r)      = 4π*r^3/3
cylinder_volume(r, h) = π*r^2*h
cone_value(r, h)      = π*r^2*h/3

function sine(x)
    n = 5
    total = 0
    for i in 0:n
        total += (-1)^i*x^(2i+1)/factorial(2i + 1)
    end
    total
end

end # модуль
```

Тогда почему бы не определить его таким образом? Помещение всего исходного кода в файл определения модуля `ToyGeometry.jl` плохо масштабируется. По мере того как пакет будет становиться все больше, хранить весь исходный код в одном файле будет непрактично. Вот почему мы разделили исходный код пакета на несколько файлов и включили эти файлы в определение модуля.

16.3 Модификация и разработка пакета

Если следовать этому примеру, то получится правильная структура и определение пакета; следующий шаг касается понимания процесса разработки пакета. Это повторяющийся процесс, в ходе которого функциональность пакета тестируется снова и снова и добавляются новые функциональные возможности.

При разработке своего собственного пакета следует учитывать, что существует много пакетов Julia, которые можно использовать в качестве помощи в своих усилиях по разработке, но которые не

должны быть частью зависимостей вашего пакета. Ниже приведена пара таких пакетов, которые я люблю использовать для повышения производительности:

- OhMyREPL – обеспечивает подсветку синтаксиса и соответствие истории в интерактивной среде REPL;
- Revise – отслеживает изменения исходного кода в пакетах, загруженных в интерактивную среду REPL, и обновляет REPL с учетом этих изменений.

Возвращаясь к примеру, теперь давайте переключимся на любительскую среду, так как мы не хотим, чтобы эти пакеты добавлялись как зависимости пакета ToyGeometry:

```
shell> pwd ❶  
~/dev/hobby/ToyGeometry  
  
(ToyGeometry) pkg> activate .. ❷  
Activating project at `~/dev/hobby`  
  
(hobby) pkg> add OhMyREPL Revise  
Resolving package versions...  
Updating `~/dev/hobby/Project.toml`  
[5fb14364] + OhMyREPL v0.5.12  
[295af30f] + Revise v3.3.3  
Updating `~/dev/hobby/Manifest.toml`
```

- ❶ Просто проверить, где мы сейчас находимся.
- ❷ Любительская среда находится на один каталог выше.

Теперь можно загрузить пакеты OhMyREPL и Revise в интерактивную среду REPL, но попытка загрузить пакет ToyGeometry завершится ошибкой. Сможете догадаться, почему?

```
julia> using ToyGeometry  
ERROR: ArgumentError: Package ToyGeometry not found in current path:  
- Run `import Pkg; Pkg.add("ToyGeometry")` to install the ToyGeometry  
package.
```

Сообщение об ошибке в этом случае не очень-то информативно. Julia, например, никогда не ищет пакеты в текущем пути файловой системы. На самом деле проблема заключается в том, что мы не добавили пакет ToyGeometry в любительскую среду. Среда, в которой я сейчас работаю, не знает о моем конкретно-прикладном пакете.

О пакете можно было бы сообщить с помощью команды `add` в диспетчере пакетов. Однако проблема с использованием `add` заключается в том, что она захватывает последнюю версию пакета и замораживает ее. Обычно это хорошо, потому что мы не хотим, чтобы добавленные в рабочую среду сторонние пакеты внезапно менялись. Но во время активной разработки пакета потребности отличаются от тех, которые возникают при его использовании. Мы хотим, чтобы все изменения исходного кода были доступны в рабо-

чей среде без использования явной команды `update` для получения последних изменений. Таким образом, вместо команды `add` я покажу, как использовать пакетную команду `dev`. Команде `dev` нужен путь к каталогу пакета:

```
shell> pwd
~/dev/hobby

shell> ls
Manifest.toml   Project.toml   ToyGeometry
(hobby) pkg> dev ./ToyGeometry
  Resolving package versions...
  Updating `~/dev/hobby/Project.toml`
  [bbcec4ee] + ToyGeometry v0.1.0 `ToyGeometry`
  Updating `~/dev/hobby/Manifest.toml`
  [bbcec4ee] + ToyGeometry v0.1.0 `ToyGeometry`
```

По статусу любительской среды видно, что добавлены пакеты `OhMyREPL`, `Revise` и `ToyGeometry`:

```
(hobby) pkg> status
  Status `~/dev/hobby/Project.toml`
  [13f3f980] CairoMakie v0.7.5
  [d872a56f] ElectronDisplay v1.0.1
  [5fb14364] OhMyREPL v0.5.12
  [295af30f] Revise v3.3.3
  [bbcec4ee] ToyGeometry v0.1.0 `ToyGeometry`
```

Теперь текущая рабочая среда знает, где находится пакет `ToyGeometry`, поэтому она знает, какой исходный код загружать при написании `using ToyGeometry`. Напомню, что пакетов с именем `ToyGeometry` может быть много, поэтому среде нужно явно указывать, какой именно пакет следует загружать:

```
julia> using ToyGeometry
[ Info: Precompiling ToyGeometry [bbcec4ee-a196-4f18-8a9a-486bb424b745]

julia> ToyGeometry.sphere_volume(4)
268.082573106329

julia> ToyGeometry.sine(π/2)
0.999999943741051

julia> sin(π/2)
1.0

julia> sine(π/2)
ERROR: UndefVarError: sine not defined
```

- ❶ Вызвать свою конкретно-прикладную синусную функцию.
- ❷ Встроенная в Julia синусная функция.
- ❸ Попытка вызвать `sine` без указания имени модуля.

Писать `ToyGeometry.sine` всякий раз, когда требуется вызвать конкретно-прикладную синусную функцию, не совсем удобно, но, как показывает пример, Julia не будет знать, как ее вызывать, если не добавить к ней префикс с именем модуля. И все же, как вы видели в других пакетах, таких как `CaioMakie`, предварять функции построения графиков, такие как `scatter` и `lines`, префиксом с именем модуля вовсе не требуется. Как добиться того же самого?

Уловка заключается в использовании инструкции `export`, чтобы сообщить Julia, какие функции и типы следует экспортировать из модуля (делать общедоступными). Раз уж мы находимся здесь, давайте добавим косинусную функцию и тоже ее экспортируем:

```
export sine, cosine ❶

"""
    sine(x)
Вычислить синус угла `x` в радианах
"""
function sine(x::Number)
    n = 5
    total = 0
    for i in 0:n
        total += (-1)^i*x^(2i+1)/factorial(2i + 1)
    end
    total
end

function cosine(x::Number)
    n = 5
    mapreduce(+, 0:n) do i
        (-1)^i*x^(2i)/factorial(2i)
    end
end
```

❶ Сделать синусную и косинусную функции общедоступными.

Возможно, вы заметили, что в функции `cosine` неуклюжий цикл `for` был заменен более элегантной функцией `mapreduce`, которая рассматривалась в главе 4. Функция `mapreduce` – это комбинация функций более высокого порядка `map` и `reduce`.

Вы заметите, что теперь можно писать `sine` и `cosine` без префиксации имени модуля. Следующий ниже простой тест показывает, что конкретно-прикладные функции дают такой же результат, что и встроенные функции:

```
julia> sine(π/2)
0.999999943741051
```

```
julia> sin(π/2)
1.0
```

```
julia> cosine(π)
-1.0018291040136216

julia> cos(π)
-1.0
```

Как Julia вдруг смогла узнать `sine` и даже `cosine` без перезагрузки пакета? Это произошло благодаря магии пакета `Revise`. Поскольку пакет `Revise` был загружен перед `ToyGeometry`, он будет отслеживать изменения в модуле и автоматически включать их в интерактивную среду REPL.

Всегда ли следует использовать пакет `Revise`? Иногда в исходный код вносятся изменения, которые нет необходимости сразу же отражать в интерактивной среде REPL. Просто пользуйтесь здравым смыслом. Пакет `Revise` даже можно использовать в отношении отдельных файлов, вызывая `includet` вместо `include`.

При написании исходного кода вы, естественно, будете делать ошибки. Интерактивная среда REPL будет быстро проверять исходный код, помогая убедиться в получении правильных результатов. Визуализация данных является одним из лучших способов быстрого анализа большого объема данных и выявления проблем. Поэтому давайте построим график встроенной функции `sin` и конкретно-прикладной функции `sine`, чтобы убедиться в получении похожих графиков. Давайте вычислим следующий ниже исходный код в интерактивной среде REPL.

Листинг 16.8 Сравнение встроенной функции `sin` с конкретно-прикладной функцией `sine`

```
using ToyGeometry, CairoMakie, ElectronDisplay

xs = 0:0.1:2π
ys1 = map(sin, xs)
ys2 = map(sine, xs)

lines(xs, ys1, linewidth=5)
lines!(xs, ys2, linewidth=5)

current_figure()
```

Ой! Они разные. В значении по оси x , равном примерно 4.5, конкретно-прикладная функция `sine` явно дает сбой.

К счастью, благодаря пакету `Revise` можно видоизменить строку кода, в которой указано `n = 5`. Попробуйте другие значения `n`, чтобы увидеть изменения на графике. Указав `n = 8`, проблема будет решена. Волшебной причины, по которой работает именно 8, не существует, кроме того факта, что более высокие значения обеспечивают более высокую точность, в том смысле, что существует компромисс между производительностью и точностью.

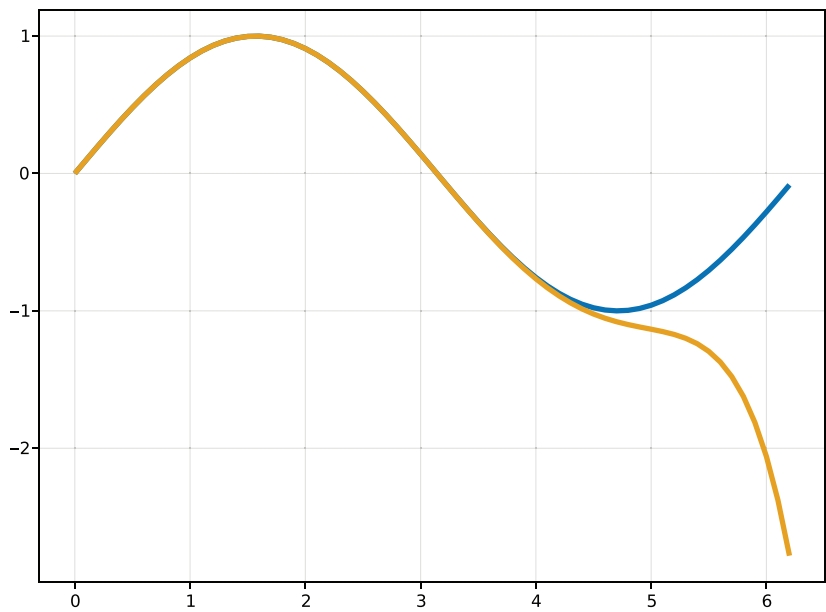


Рис. 16.3 Диаграмма рассеяния Makie функций sin и sine, $n = 5$

16.4 Устранение распространенных заблуждений о модулях

Если вы пришли из другого языка программирования, то, вероятно, многие концепции, относящиеся к вычислительным средам, пакетам и модулям, могут вас дезориентировать. Например, в мире Java модуль называется пакетом, а в Julia пакет ближе к JAR-файлу¹. В табл. 16.1 представлен краткий обзор различий в терминологии, используемой для описания модулей, пакетов и сред в разных языках программирования.

Таблица 16.1. Различия в использовании терминологии пакетов и модулей в популярных языках программирования

Julia	Java	C++	Python
Модуль	Пакет	Пространство имен	Модуль
Пакет	JAR	DLL	Пакет
Среда	Среда	Песочница или контейнер	Виртуальная среда

¹ Java-архив (JAR) – это пакетный файловый формат, который обычно используется для агрегирования файлов классов Java и связанных с ними метаданных и ресурсов (например, текста и изображений) в один файл с целью распространения.

Еще одна проблема заключается в том, что мейнстримные языки программирования имеют тенденцию помещать ключевые слова, такие как `public`, перед функциями, которые должны быть экспортированы или сделаны общедоступными, как показано в следующем ниже листинге.

Листинг 16.9 Сравнение Julia с экспортом функций в стиле Java

```
#### В стиле Julia ####
export sphere_volume

function sphere_volume(r::Number)
    4π*r^3/3
end

#### Экспорт в стиле Java ####
# ВАЖНО: это в Julia не работает!
public function sphere_volume(r::Number)
    4π*r^3/3
end
```

Хотя функция Julia `include` может показаться похожей на директиву C/C++ `#include`, следует принять во внимание, что в Julia `include` – это обычный вызов функции, которая возвращает результат. Директива `#include` в C/C++ – это вовсе не вызов функции, а механизм, который вставляет содержимое включаемых файлов в файл с директивами `include`.

16.5 Тестирование пакета

В процессе разработки пакета имеется возможность оперативно тестировать функции в интерактивной среде REPL. Для долгосрочной масштабной разработки этого недостаточно. Вы должны иметь возможность быстро верифицировать прежнюю работоспособность исходного кода, который был написан несколько недель или месяцев назад. В реальной разработке программного обеспечения изменения в исходный код будут вноситься в большое число разных файлов. Отслеживать все функции, затронутые правками исходного кода, по разным причинам бывает затруднительно; и поэтому возникает необходимость настроить масштабный комплект тестов, который сможет верифицировать работоспособность всего ранее написанного исходного кода.

Тема тестирования является очень обширной, поэтому в данной книге будут изложены только самые основы. Рекомендую прочитать следующие ниже статьи из моего подстека *Erik Explores*, чтобы обследовать данную тему поглубже:

- «Организация тестов в Julia» (<https://erikexplores.substack.com/p/julia-test-organizing>);

- «Тестирование на языке Julia: лучшие образцы практики» (<https://erikexplores.substack.com/p/julia-testing-best-practice>).

Тесты в Julia можно выполнять из менеджера пакетов. Он будет пытаться исполнить файл с именем `test/runtests.jl`. В примере у меня нет ни этого файла, ни каталога `test`, поэтому мне придется создать и то, и другое:

```
shell> pwd
~/dev

shell> cd hobby/ToyGeometry/
~/dev/hobby/ToyGeometry

shell> mkdir test

shell> touch test/runtests.jl test/trigtests.jl
```

В Julia используется понятие *вложенных комплектов тестов*, означающее, что комплект тестов считается успешно пройденным, если все содержащиеся в нем комплекты тестов были пройдены успешно. По общепринятому в Julia правилу все комплекты тестов помещаются в один комплект тестов верхнего уровня. Именно это и показано в файле `test/runtests.jl` (листинг 16.10); в любом случае я буду следовать той же стратегии, распределяя тесты по нескольким файлам и включая эти файлы в комплект тестов верхнего уровня.

Листинг 16.10 ToyGeometry/test/runtests.jl

```
using ToyGeometry
using Test

@testset "All tests" begin

include("trigtests.jl")

end
```

Тесты выполняются отдельно от модуля, поэтому тестируемый модуль нужно загрузить с помощью инструкции `using`. Для того чтобы получить доступ к макрокомандам тестирования, таким как `@testset` и `@test`, необходимо импортировать пакет `Test`. Этот пакет в среде `ToyGeometry` отсутствует. Добавлять `Test` в эту среду не имеет смысла, так как он будет использоваться только при тестировании.

Julia предлагает изящное решение этой проблемы. Каталог `test` можно трактовать как самостоятельную среду и добавлять `Test` только в эту среду:

```
shell> pwd
~/dev/hobby/ToyGeometry

(hobby) pkg> activate test
Activating project at `~/dev/hobby/ToyGeometry/test`
```

```
(test) pkg> add Test
Resolving package versions...
Updating `~/dev/hobby/ToyGeometry/test/Project.toml`
[8dfed614] + Test
Updating `~/dev/hobby/ToyGeometry/test/Manifest.toml`
```

❶ Показать каталог, в котором исполняются пакетные команды.

Прежде чем приступить к выполнению тестов, необходимо добавить их в исходный код. Я продемонстрирую это, добавив тесты в файл `test/trigtests.jl`, который содержит тесты, связанные с тригонометрическими функциями.

Каждый фактический тест указывается с помощью макрокоманды `@test`. Поскольку числа с плавающей точкой трудно сделать абсолютно одинаковыми, я буду сравнивать результаты функций, используя вместо оператора `==` оператор `≈`, который можно написать в интерактивной среде REPL, набрав `\approx` и нажав клавишу **Tab**.

Иногда стандартный допуск оператора `≈` слишком строг, и требуется более свободное определение равенства. В таких случаях для сравнения значений следует использовать функцию `isapprox`. Она принимает именованный аргумент `atol`, который можно использовать для задания величины разницы между двумя сравниваемыми результатами, которую вы считаете приемлемой, как показано ниже.

Листинг 16.11 ToyGeometry/test/trigtests.jl

```
@testset "cosine tests" begin
    @test cosine(π) ≈ -1.0
    @test cosine(0) ≈ 1.0

    for x in 0:0.1:2π
        @test isapprox(cos(x), cosine(x), atol=0.05)
    end
end

@testset "sine tests" begin
    @test sine(0) ≈ 0.0
    @test sine(π/2) ≈ 1.0

    for x in 0:0.1:2π
        @test isapprox(sin(x), sine(x), atol=0.05)
    end
end
```

❶ Выполнить тест на примерное равенство всех значений косинуса в диапазоне от 0 до 2π с шагом 0.1.

Если попытаться выполнить этот тест в тестовой (`test`) среде, то он, по иронии судьбы, не сработает, потому что среда `test` на самом деле не знает о пакете `ToyGeometry`. Следовательно, для того чтобы выполнить тесты, необходимо переключиться на любительскую среду `hobby` либо среду `ToyGeometry`:

```
(test) pkg> test ToyGeometry
ERROR: The following package names could not be resolved:
* ToyGeometry (not found in project or manifest)

(test) pkg> activate .
Activating project at `~/dev/hobby/ToyGeometry`

(ToyGeometry) pkg> test ToyGeometry
Testing ToyGeometry

Test Summary: | Pass  Fail  Total
All tests     | 108   22   130
cosine tests  |  43   22   65
sine tests    |  65           65
ERROR: LoadError: Some tests did not pass:
108 passed, 22 failed, 0 errored, 0 broken
```

❶ Невозможно протестировать из среды test.

В приведенном выше листинге из-за циклов `for` мы можем выполнить в общей сложности 130 тестов. Тест косинуса не проходит, потому что было задано $n = 5$, что не делает результаты достаточно точными. Поэтому для большей точности было задано $n = 9$. Обратите внимание, что выполнение тестов приводит к большому количеству выходных данных, которые я отфильтровал ради удобочитаемости.

Листинг 16.12 Изменение косинусной функции в файле `trig.jl`

```
function cosine(x::Number)
    n = 9 # modified
    mapreduce(+, 0:n) do i
        (-1)^i * x^(2i) / factorial(2i)
    end
end
```

При повторном выполнении тестов они должны пройти успешно:

```
(ToyGeometry) pkg> test
Testing ToyGeometry
Test Summary: | Pass  Total
All tests     | 130   130
Testing ToyGeometry tests passed
```

Тема тестирования и вычислительных сред намного шире, чем можно охватить в этой главе. Рассмотренный в ней материал дает прочную основу для дальнейшего изучения. Вот некоторые отправные точки:

- pkgdocs.julialang.org – веб-сайт модуля Pkg, предоставляющего функциональность менеджера пакетов;
- docs.julialang.org – веб-сайт официальной документации по Julia. Здесь находятся подробные описания модулей, пакетов и сред, включая тестирование.

Резюме

- Вычислительная среда похожа на рабочую область; в ней программист настраивает пакеты, с которыми хочет работать и которые хочет использовать.
- Julia позволяет поддерживать различные среды, сконфигурированные под использование разных пакетов и разных версий этих пакетов.
- CairoMakie – это пакет Julia для построения двумерной векторной графики. Он является частью набора графопостроительных пакетов под названием Makie.
- scatter и lines – это функции, используемые в библиотеке Makie для построения графиков функций.
- ElectronDisplay – это пакет, предоставляющий окно для отображения графики, например данных Makie. Это альтернатива графопостроению в редакторе VS Code.
- Переключение между разными вычислительными средами выполняется с помощью пакетной команды activate. Например, эта команда используется в ситуациях, когда требуется изменить зависимости другого разрабатываемого пакета.
- Для того чтобы сделать модуль доступным в исходном коде, его надо добавить с помощью пакетной команды add.
- Пакет следует удалять для модулей, которые больше не используются в программе; это делается с помощью пакетной команды remove.
- Проверка добавленных в среду пакетов выполняется с помощью команды status. Она также используется для проверки зависимостей у конкретно-прикладного пакета.
- В случаях, когда много пакетов имеют одинаковое имя, следует указывать UUID интересующего пакета при его добавлении в рабочую среду.
- Пакет создается с помощью пакетной команды generate.
- В файле Project.toml указаны прямые зависимости среды, а в Manifest.toml – косвенные зависимости.
- При добавлении разрабатываемого локального пакета следует использовать команду dev вместо add. За счет этого обеспечивается вставка последних изменений исходного кода при каждой загрузке модуля из этого пакета.
- Пакет Revise используется для отслеживания изменений исходного кода в пакетах, загруженных после Revise.
- Команда test <имя пакета> используется для выполнения тестов в отношении пакета. Например, команда test ToyGeometry выполнит тесты в отношении пакета ToyGeometry.
- Каталог test – это самостоятельная среда, которая позволяет добавлять пакет Test в качестве зависимости только в эту тестовую среду.

Часть V

Углубленное изучение

Теперь у вас есть основа для более глубокого изучения. В главе 5 тема ввода-вывода была охвачена лишь поверхностно, потому что система типов Julia еще не была рассмотрена. Глава 17 расширяет главу 5, концентрируя ваше внимание на иерархии типов ввода/вывода.

В главе 10 вы увидели, как параметрические типы используются для представления разных форм несуществования или отсутствия, а в главе 6 – как они помогают определять строго типизированные коллекции. В главе 18 будет дано объяснение тому, как параметрические типы способствуют обеспечению правильности типов, производительности и потребления памяти.

17

Ввод и вывод

Эта глава охватывает следующие ниже темы:

- понимание системы ввода-вывода языка Julia;
- использование наиболее распространенных функций чтения и записи файлов;
- чтение и запись в строковые литералы, сокеты и процессы;
- добавление исходного кода в пример с ракетой для загрузки ракетных двигателей из CSV-файлов.

Реальные программы должны иметь возможность читать данные, вводимые пользователями, и записывать результаты. В этой главе вы узнаете о системе ввода-вывода Julia (системе В/В). Она обеспечивает абстракцию для работы с файлами, сетевыми и межпроцессными взаимодействиями, а также для взаимодействия с консолью (клавиатурой и экраном).

Язык Julia очень популярен в науке о данных, в которой мы много работаем с входными данными в виде CSV-файлов (значениями, отделенными запятыми). Именно по этой причине главный пример исходного кода будет сосредоточен на синтаксическом разборе CSV-файла, содержащего данные о ракетных двигателях, а также на записи данных о ракетных двигателях в CSV-файл.

17.1 Знакомство с системой ввода-вывода языка Julia

Давайте взглянем на систему ввода-вывода языка Julia с высоты птичьего полета. Она центрирована на абстрактном типе `IO`. У него есть конкретные подтипы, такие как `IOStream`, `IOBuffer`, `Process` и `TCPSocket`. Каждый тип позволяет читать и писать данные из различных устройств ввода-вывода, таких как файлы, текстовые буферы, работающие процессы (запущенные программы) или сетевые соединения.

Из иерархии типов на рис. 17.1 видно, что такие функции, как `print`, `show`, `read`, `readline` и `write`, доступны для всех типов ввода-вывода. Некоторые функции, такие как `eof` и `position`, доступны не для всех типов ввода-вывода.

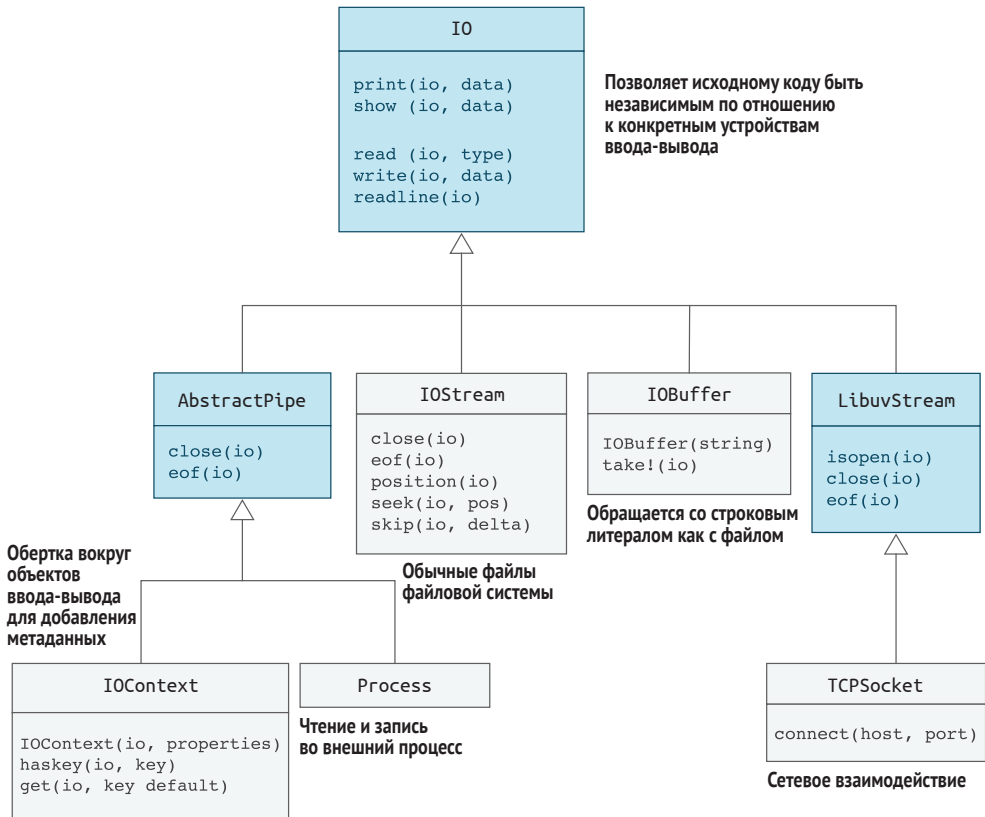


Рис. 17.1 Иерархия типов системы ввода-вывода, показывающая цели, для которых используются различные подтипы. Светло-серые прямоугольники представляют конкретные типы

Разные объекты ввода-вывода открываются по-разному, но после создания для всех них можно использовать одинаковые функции. Я продемонстрирую работу с похожими данными из файла, строкового литерала и процесса. Используемые данные будут взяты из CSV-файла с именем `rocket-engines.csv` со следующим содержимым:

```
name,company,mass,thrust,Isp
Curie,Rocket Lab,0.008,0.12,317
RS-25,Aerojet Rocketdyne,3.527,1860,366
Merlin 1D,SpaceX,0.47,845,282
Kestrel 2,SpaceX,0.052,31,317
RD-180,NPO Energomash,5.48,3830,311
Rutherford,Rocket Lab,0.035,25,311
```

Я открою файл, буду читать его до тех пор, пока не дойду до конца, и закрою его. В листинге 17.1 я читаю по одной строке с помощью функции `getline` и с помощью функции `eof` делаю проверку на достижение конца файла. Я разбиваю каждую файловую строку с помощью функции `split` на несколько слов, используя запятые в качестве разделителей. Каждая прочитанная строка выводится на консоль. Все эти функции есть в модуле `Base`, который всегда подгружается.

Листинг 17.1 Построчное чтение CSV-файла для демонстрации базовых функций ввода-вывода

```
io = open("rocket-engines.csv")
while !eof(io)
    line = readline(io)
    words = split(line, ',')
    println(join(words, '\t'))
end
close(io)
```

Данные из строкового литерала можно обрабатывать очень похожим образом. Я продемонстрирую пример, создав строковый литерал, содержащий первую строку файла `rocket-engines.csv` и попробовав ее обработать разными функциями ввода-вывода. Я буду использовать функцию `readuntil`, которая читает из объекта ввода-вывода до тех пор, пока она не достигнет определенного символа или строкового литерала. Я буду использовать функцию `position`, чтобы проверять, на сколько букв в `IOStream` я продвинулся, периодически применяя функцию `eof`, чтобы выполнять проверку на достижение конца объекта ввода-вывода:

```
julia> s = "name,company,mass,thrust,Isp";

julia> io = IOBuffer(s);

julia> readuntil(io, ',')
"name"
```

```
julia> position(io), eof(io)
(5, false)

julia> readuntil(io, ','), position(io), eof(io)
("company", 13, false)

julia> readuntil(io, ','), position(io), eof(io)
("mass", 18, false)

julia> readuntil(io, ','), position(io), eof(io)
("thrust", 25, false)

julia> readuntil(io, ','), position(io), eof(io)
("Isp", 28, true)
```

Вы можете поэкспериментировать с выполнением тех же операций на объекте ввода-вывода, полученным путем открытия файла `rocket-engines.csv`. По завершении не забывайте вызывать `close(io)`; в противном случае можно потерять ограниченные ресурсы ОС. Объект ввода-вывода очень важно закрывать особенно при записи во избежание утери данных.

17.2 Чтение данных из процесса

Скриптовые языки, такие как Python, Ruby и Perl, приобрели популярность отчасти благодаря тому, что они являются хорошими *связующими языками*. Связующий язык отлично подходит для соединения существующих программных компонентов, зачастую написанных на разных языках.

Мы кратко рассмотрим способность языка Julia работать как связующий язык. Давайте представим, что в Julia нет возможности выполнять поиск в текстовых файлах, и для этой цели мы хотим использовать инструмент Unix `grep`¹. Сначала мы должны перейти в режим оболочки, нажав точку с запятой, просто чтобы продемонстрировать, что наша команда `grep` будет делать: она находит строки с текстом "Rocket Lab". Нажав клавишу ← (**Backspace**), мы вернемся в режим Julia. Затем мы должны открыть соединение с запущенным (порожденным) нами процессом `grep`. Обратите внимание на использование обратных кавычек для обозначения команд командной оболочки, которые мы хотим исполнить:

```
shell> grep "Rocket Lab" rocket-engines.csv
Curie,Rocket Lab,0.008,0.12,317
Rutherford,Rocket Lab,0.035,25,311
```

¹ `grep` – это стандартная утилита командной строки в системах Unix, служащая для отыскания файловых строк, соответствующих заданным критериям поиска.

```
julia> io = open(`grep "Rocket Lab" rocket-engines.csv`);

julia> readuntil(io, ',')
"Curie"

julia> readuntil(io, ',')
"Rocket Lab"

julia> readuntil(io, ',')
"0.008"

julia> position(io)
ERROR: MethodError: no method matching position(::Base.Process)
julia> close(io)
```

В отличие от многих скриптовых языков, таких как Perl и Ruby, обратные кавычки в Julia не вызывают немедленного исполнения команды командной оболочки. Вместо этого они вызывают создание командного объекта типа `Cmd`. При вызове функции `open` на командном объекте команда выполняется фактически и порождает процесс. Возвращаемый объект ввода-вывода относится к типу `Process` и представляет соединение с выходными данными запущенного процесса. Таким образом, процесс можно читать почти так же, как если бы это был файл (рис. 17.2).



Рис. 17.2 Функция `open` порождает дочерний процесс. Оба процесса связаны через конвейер, представленный объектом ввода-вывода, возвращаемым функцией `open`

Почему при вызове функции `position` выдается сообщение об ошибке? Потому что к функции `position` не привязаны методы, которые оперируют на объектах `Process`. Понятие позиции в потоке имеют лишь объекты `IOStream`, оперирующие на файлах.

17.3 Чтение из сокета и запись в сокет

Сокет представляет сетевое соединение. Я приведу краткий пример, демонстрирующий сетевое соединение с помощью утилиты Unix `netcat`. `Netcat` – это простой инструмент, служащий для эксперимен-

тов с сетевыми взаимодействиями на основе TCP/IP-сокетов¹. Утилиту `netcat` можно выполнять как в качестве клиента, так и в качестве сервера.

ПРИМЕЧАНИЕ Утилита `netcat` уже установлена в Linux и macOS. Пользователи Windows в качестве замены могут скачать утилиту `ncat` с веб-сайта nmap.org/ncat. При каждом появлении команды `nc` в тексте вместо нее в Windows следует писать `ncat`.

Давайте выполним следующие действия: откроем два окна терминала. В первом окне запустим Julia, а во втором окне запустим `netcat` как сервер, прослушивающий соединения на порту 1234. При этом можно указать практически любой номер порта, при условии что он еще не занят. После запуска утилиты `netcat` напечатаем строку `"name,company,mass,thrust,Isp"` и нажмем **Enter**:

```
shell> nc -l 1234
name,company,mass,thrust,Isp
```

В окне Julia мы применим функцию `connect`, чтобы подсоединиться к локальному серверу, работающему на порту 1234. Функция `connect` вернет объект ввода-вывода типа `TCPSocket`:

```
julia> using Sockets

julia> sock = connect(1234)
TCPSocket{RawFD{23}} open, 0 bytes waiting

julia> readuntil(sock, ',')
"name"

julia> readuntil(sock, ','), isopen(sock)
("company", true)

julia> readline(sock)
"mass,thrust,Isp"
```

Сокет обычно является двухпутным, поэтому можно писать сообщения в сокет и видеть, как они появляются в окнах с запущенной утилитой `netcat`:

```
julia> println(sock, "hello netcat")

julia> close(sock)
```

¹ TCP/IP – это протокол, используемый в интернете для сетевого взаимодействия.

Вы должны увидеть, как во втором окне всплыла текстовая строка "hello netcat".

На этих простых примерах я продемонстрировал, что одинаковые функции, такие как `read`, `readuntil`, `readline` и `println`, можно использовать для любого типа объекта ввода-вывода, независимо от того, что он представляет: строковый литерал, файл или же сетевое соединение.

17.4 Синтаксический разбор CSV-файла

Давайте создадим более полный пример исходного кода. Мы усовершенствуем свой ракетный исходный код из повторяющегося примера, добавив возможность загружать определения ракетных двигателей из CSV-файлов. Для того чтобы попрактиковаться в создании пакетов Julia, давайте создадим пакет Julia под названием `ToyRockets`, содержащий исходный код ракеты. Я уже создал этот пакет и разместил его на GitHub по адресу github.com/ordovician/ToyRockets.jl, чтобы можно было сверяться.

Пакет `ToyRockets` создается командой `generate` в менеджере пакетов Julia. Затем создадим каталог `data`, служащий для хранения файла `rocket-engines.csv`, и добавим следующие ниже файлы в каталог `src`:

- `interfaces.jl` – содержит определения абстрактных типов, таких как `Engine`;
- `custom-engine.jl` – определение конкретного типа двигателя;
- `io.jl` – коллекция функций для загрузки и сохранения деталей ракет.

Теперь мы выполним необходимые команды, обращая внимание на приглашения в командной строке. Когда в приглашении написано `(@v1.7) pkg>`, это будет означать, что нужно сначала нажать `]`, чтобы войти в пакетный режим. Когда в приглашении написано `shell>`, это будет означать, что нужно нажать `;`, чтобы войти в режим оболочки:

```
(@v1.7) pkg> generate ToyRockets
Generating project ToyRockets:
ToyRockets/Project.toml
ToyRockets/src/ToyRockets.jl
```

```
shell> cd ToyRockets/
~/Development/ToyRockets
```

```
shell> mkdir data
```

```
shell> cd src
~/Development/ToyRockets/src
```

```
shell> touch interfaces.jl custom-engine.jl io.jl
```

Если мы правильно следовали инструкциям и поместили файл `rocket-engines.csv` в каталог `data/`, то пакет `ToyRockets` должен выглядеть так:

```
ToyRockets/  
├── Project.toml  
├── data  
│   └── rocket-engines.csv  
└── src  
    ├── ToyRockets.jl  
    ├── custom-engine.jl  
    ├── interfaces.jl  
    └── io.jl
```

При этом следует проверить, чтобы в файл `ToyRockets.jl`, в котором определяется модуль пакета, были включены все файлы исходного кода (листинг 17.2).

Листинг 17.2 Файл `src/ToyRockets.jl`

```
module ToyRockets  
  
include("interfaces.jl")  
include("custom-engine.jl")  
  
include("io.jl")  
  
end
```

Далее каждую строку в файле `rocket-engines.csv` нужно превратить в объект `CustomEngine`, поэтому сначала нужно определить типы `Engine`.

Листинг 17.3 Определение типов двигателей

```
# файл interfaces.jl  
export Engine  
abstract type Engine end  
  
# файл custom-engine.jl  
export CustomEngine  
  
struct CustomEngine <: Engine  
    mass::Float64  
    thrust::Float64  
    Isp::Float64  
end
```

В следующих двух разделах мы загрузим и сохраним данные ракетного двигателя.

17.4.1 Загрузка данных ракетного двигателя

Теперь, перед тем как я пройдуся по всем деталям и объясню принцип работы различных частей исходного кода, вы взглянете на окончательную его версию (листинг 17.4). Исходный код начинается с чтения всех строк CSV-файла, в котором каждая строка представляет ракетный двигатель. Мы прокручиваем каждую строку файла, выполняем ее синтаксический разбор и конвертируем в объект `CustomEngine`, который добавляется в словарь `rocket_engines`, содержащий все двигатели, загруженные из входного файла.

Листинг 17.4 Файл `io.jl`, содержащий исходный код загрузки объектов-двигателей в словарь

```
export load_engines

function load_engines(path::AbstractString)
    rocket_engines = Dict{String, Engine}()

    rows = readlines(path)
    for row in rows[2:end]
        cols = split(row, ',')

        if any(isempty, cols)
            continue
        end
        name, company = cols[1:2]
        mass, thrust, Isp = map(cols[3:end]) do col
            parse(Float64, col)
        end

        engine = CustomEngine(
            mass * 1000,      ①
            Isp,
            thrust * 1000)   ②
        rocket_engines[name] = engine
    end

    rocket_engines
end
```

① Из тонн в килограммы.

② Из килоньютон в ньютон.

Функция `load_engines` берет путь к CSV-файлу, содержащему данные ракетных двигателей, и выполняет их синтаксический разбор, чтобы создать словарь ракетных двигателей. Вот пример ее применения:

```
julia> using Revise, ToyRockets

julia> pwd()
"~/Development/ToyRockets"
```

```
julia> engines = load_engines("data/rocket-engines.csv")
Dict{String, Engine} with 6 entries:
  "RD-180"      => CustomEngine(5480.0, 311.0, 3.83e6)
  "Kestrel 2"   => CustomEngine(52.0, 317.0, 31000.0)
  "Curie"      => CustomEngine(8.0, 317.0, 120.0)
  "Merlin 1D"   => CustomEngine(470.0, 282.0, 845000.0)
  "RS-25"       => CustomEngine(3527.0, 366.0, 1.86e6)
  "Rutherford"  => CustomEngine(35.0, 311.0, 25000.0)

julia> engines["Curie"]
CustomEngine(8.0, 317.0, 120.0)
```

Функция `load_engines` следует довольно стандартному шаблону, который я использую при обработке данных, системно организованных в строках файла, таких как CSV-файлы (см. главу 5). Здесь мы используем функцию `readlines`, чтобы получить строки файла, и функцию `split`, чтобы получить каждый столбец каждой строки (рис. 17.3).

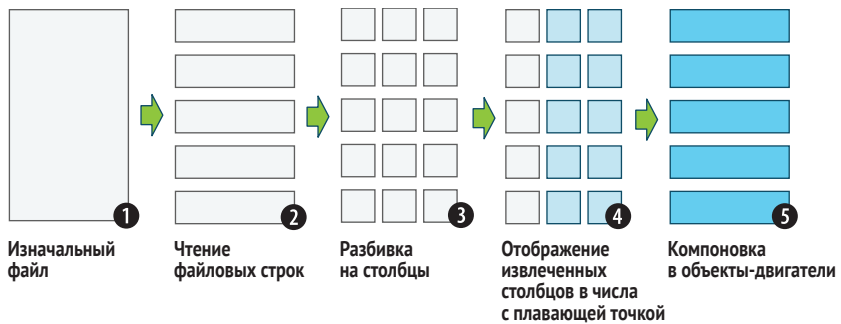


Рис. 17.3 Файл ракетных двигателей разбирается на части, преобразовывается и компоновается в ракетные двигатели в несколько этапов

В целях более глубокого понимания принципа работы исходного кода следует скопировать и вставить части строк исходного кода в интерактивную среду REPL, чтобы увидеть процесс обработки входных данных:

```
julia> path = "data/rocket-engines.csv"
"data/rocket-engines.csv"

julia> rows = readlines(path)
7-element Vector{String}:
 "name,company,mass,thrust,Isp"
 "Curie,Rocket Lab,0.008,0.12,317"
 "RS-25,Aerojet Rocketdyne,3.527,1860,366"
 "Merlin 1D,SpaceX,0.47,845,282"
 "Kestrel 2,SpaceX,0.052,31,317"
 "RD-180,NPO Energomash,5.48,3830,311"
 "Rutherford,Rocket Lab,0.035,25,311"
```

Далее следует выбрать произвольную строку и разбить ее на столбцы, чтобы убедиться, что синтаксический разбор строки работает надлежащим образом. Иногда данные могут отсутствовать, поэтому следует проверить, чтобы каждый столбец содержал данные. Этого можно добиться с помощью функции более высокого порядка `any(isempty, cols)`, которая применяет функцию-предикат `isempty` к каждому столбцу. Если какой-либо из столбцов будет пустым, то она будет возвращать `true`:

```
julia> row = rows[2]
"Curie,Rocket Lab,0.008,0.12,317"

julia> cols = split(row, ',')
5-element Vector{SubString{String}}:
 "Curie"
 "Rocket Lab"
 "0.008"
 "0.12"
 "317"

julia> any(isempty, cols)
false
```

Далее мы воспользуемся небольшой магией Julia, именуемой *деструктуризацией*, чтобы извлечь название двигателя и компанию, которая его производит. При деструктуризации несколько переменных помещаются в левую часть оператора присваивания `=`. С правой стороны нужно поместить итерируемую коллекцию, по крайней мере с таким же числом элементов, что и число переменных с левой стороны:

```
julia> name, company = cols[1:2]
2-element Vector{SubString{String}}:
 "Curie"
 "Rocket Lab"

julia> name
"Curie"

julia> company
"Rocket Lab"
```

Выражение `cols[1:2]` производит двухэлементный массив. Julia прокручивает этот массив в цикле и назначает элементам массива имя, а затем компанию. Кортеж или словарь сработал бы так же хорошо.

Следующая часть немного сложнее, так как мы берем три последних элемента, `cols[3:end]`, и с помощью функции `parse(Float64, col)` отображаем их в значения с плавающей точкой. Как следствие текстовые представления массы, тяги и удельного импульса (`Isp`) конвертируются в значения с плавающей точкой, которые можно передать конструктору `CustomEngine` для создания объекта-двигателя.

Последний шаг – сохранить этот двигатель в словаре под именем двигателя.

17.4.2 Сохранение данных ракетных двигателей

На этом этапе можно добавить в файл `io.jl` исходный код, позволяющий сохранять ракетные двигатели в файл. По умолчанию файл открыт для чтения. Если вы хотите в него писать, то в функцию `open` нужно передать аргумент `"w"`, обозначающий операцию *записи*. В указанном исходном коде есть еще ряд других новых концепций, которые необходимо рассмотреть подробнее.

Листинг 17.5 Файл `io.jl` с добавленным исходным кодом функции `save_engines`

```
function save_engines(path::AbstractString, engines)
    open(path, "w") do io
        println(io, "name,company,mass,thrust,Isp")
        for (name, egn) in engines
            row = [name, "", egn.mass, egn.thrust, egn.Isp]
            join(io, row, ',')
            println(io)
        end
    end
end
```

Заметили ли вы, как мы используем форму `do-end` с функцией `open`? Это означает, что она принимает функцию в качестве первого аргумента. В чем тут смысл? Изучите следующую ниже реализацию, чтобы постараться выдвинуть догадку.

Листинг 17.6 Реализация `open(f, args...)`

```
function open(f::Function, args...)
    io = open(args...)
    f(io)
    close(io)
end
```

Выгода от приведенного выше программного решения состоит в том, что по завершении работы с объектом ввода-вывода можно передавать ответственность за его закрытие языку Julia. Вы также заметите использование оператора многоточия `...`. Он применяется для передачи переменного числа параметров. Независимо от того, сколько аргументов будет передано для открытия, они будут собраны в кортеж `args`. При вызове `open(args...)`, используя оператор многоточия, этот кортеж снова расширяется в аргументы.

А что насчет функции `join`, принимающей объект ввода-вывода в качестве первого аргумента? Вместо того чтобы возвращать ре-

зультат соединения нескольких элементов с помощью разделителя, функция `join` запишет результат в предоставленный объект ввода-вывода. Вот демонстрация записи результата в стандартный вывод:

```
julia> join(stdout, [false, 3, "hi"], ':')  
false:3:hi
```

Теперь у вас должно быть общее представление о системе ввода-вывода языка Julia. Для того чтобы узнать больше, изучите документацию по описанным выше функциям и типам, используя встроенную справочную систему.

Резюме

- `IOStream`, `IOBuffer`, `Process` и `TCPSocket` – это объекты ввода-вывода для чтения и записи файлов, строковых литералов, порожденных процессов или сетевых соединений.
- Функции `readuntil`, `readline`, `readlines` и `read` используются для чтения данных из любого объекта ввода-вывода.
- Функции `print` и `println` используются для записи данных в объект ввода-вывода.
- `split` – удобная функция для конвертации строк в массивы объектов путем их разбивки с помощью разделителя.
- Деструктуризация назначает несколько элементов коллекции нескольким переменным, обеспечивая компактный и элегантный способ доступа к элементам.

18

Определение параметрических типов

Эта глава охватывает следующие ниже темы:

- работа с параметрическими методами и типами и их определение;
- использование параметрических типов для повышения типобезопасности и выявления дефектов;
- улучшение потребления памяти и производительности за счет использования параметрических типов.

В главе 10 были представлены параметрические типы, чтобы объяснить принцип работы типа-объединения. Мы также обсудили параметрические типы при работе с такими коллекциями, как массивы, словари и множества. С помощью параметров-типов можно ограничивать элементы, которые могут использоваться в коллекциях, и, таким образом, повышать типобезопасность. Однако все предыдущие случаи применения параметрических типов были показаны с точки зрения использования параметрических типов, определенных другими.

В этой главе будет показано, как создавать свои собственные параметрические типы, и рассказано о некоторых распространенных заблуждениях и ловушках при работе с параметрическими типами. Но зачем нужно создавать свои собственные параметрические

типы? В языке Julia параметрические типы имеют две ключевые выгоды, которые мы подробно рассмотрим в этой главе:

- 1 обеспечение более безопасного исходного кода. Julia может обнаруживать дефекты на ранних этапах выполнения программы, например попытки помещать строковый литерал в массив чисел;
- 2 повышение производительности при работе с большими наборами данных.

Для простоты мы проведем разведывательный анализ этих тем с помощью геометрического исходного кода в 2D-пространстве. Я попытаюсь мотивировать вас их возможным использованием в примере проекта с ракетой: для того чтобы выражать положение ракеты в космосе, нужен тип `Point`, который будет представлять положение в пространстве, и тип `Vec2D`, который будет представлять силу, ускорение и скорость в разных направлениях.

18.1 Определение параметрических методов

Я начну с представления методов, которые просто принимают параметры-типы, и разведывательного анализа того, как параметры-типы делают методы более типобезопасными и уменьшают стереотипность. Прежде чем приступить к делу, стоит освежить в памяти концепцию *параметров-типов*. Ранее я использовал аналогию с вызовом функции $y = f(x)$, где функция f принимает значение x и возвращает новое значение y (рис. 18.1). Точно так же можно думать и о выражении-типе $S = P\{T\}$, которое можно трактовать как параметрический тип P , принимающий параметр-тип T и возвращающий новый конкретный тип S . И T , и S – это конкретные типы, тогда как P – просто шаблон для создания типов.

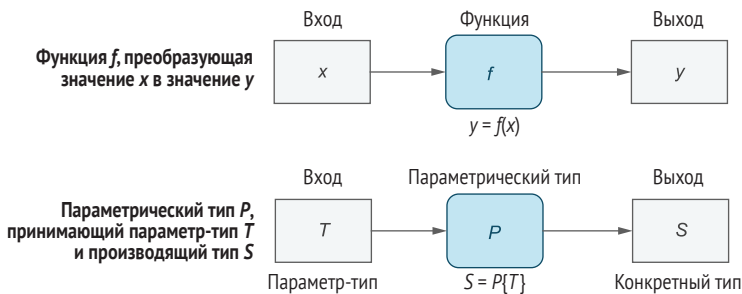


Рис. 18.1 Аналогия между функцией и параметризованным типом. Первая производит значения, а второй – типы

Определенная в листинге 18.1 функция `linearsearch` выполняет линейный поиск в массиве `haystack` в поисках элемента `needle`.

ПРИМЕЧАНИЕ Линейный поиск не делает ничего умного. Поиск начинается с первого элемента и далее последовательно просматривает каждый элемент. При отыскании искомого элемента возвращается индекс этого элемента.

`linearsearch` является параметрическим методом, потому что он принимает параметр-тип `T`. Именно предикатное условие `where T` и определяет `T` как параметр-тип.

Листинг 18.1 Линейный поиск `linearsearch` элемента `needle` в коллекции `haystack`

```
function linearsearch(haystack::AbstractVector{T}, needle::T) where T
    for (i, x) in enumerate(haystack)
        if needle == x
            return i
        end
    end
    nothing
end
```

Какие преимущества дает использование параметра-типа в данном случае? Нельзя ли вместо этого проаннотировать `haystack` типом `AbstractVector` и назначить `needle` аннотацию типа `Any`? Нет. Это не даст такой же строгой проверки типов во время выполнения программы. Линейный поиск был определен так, чтобы элемент `needle` имел тот же тип, что и все элементы коллекции `haystack`. Вот пример в интерактивной среде REPL:

```
julia> linearsearch([4, 5, 8], 5)
2

julia> linearsearch([4, 5, 8], "five")
ERROR: MethodError: no method matching linearsearch(::Vector{Int64}, ::String)
```

Сообщение об ошибке говорит об отсутствии метода, принимающего вектор с элементами типа `Int64` и поисковый объект типа `String`. Я определил линейный поиск таким образом, что каким бы ни был тип `T`, он должен быть одинаковым для элементов коллекции `haystack` и объекта `needle`.

Параметрические типы не только повышают типобезопасность, но и предоставляют возможности для сокращения стереотипного исходного кода. Предположим, вы хотите реализовать собственную версию функции `Julia typeof`. При наивном подходе было бы написано что-то вроде следующего ниже.

Листинг 18.2 Наивная реализация функции в стиле `typeof`

```
kindof(x::Int64) = Int64
kindof(x::Float64) = Float64
kindof(x::String) = String
```

Функцию `kindof` можно попробовать в интерактивной среде REPL и убедиться, что она работает с 64-битовыми целыми значениями, значениями с плавающей точкой и строковыми литералами. Однако на этом все. Глупо пытаться добавлять метод для каждого типа Julia. Как вы, возможно, уже догадались, определение функции `kindof` как параметрического метода элегантно решает проблему, как показано ниже.

Листинг 18.3 Реализация функции `kindof` с использованием параметра типа

```
function kindof(x::T) where T
    return T
end
```

Хотя такие имена, как `T`, `S`, `T1` и `T2`, пользуются популярностью при обозначении параметров-типов, можно использовать любое имя; имя тут не принципиально. Именно предикатное условие `where` превращает имя в параметр-тип. Давайте закрепим этот момент в своей памяти, перезапустив Julia и определив параметр-тип с именем `Int64`:

```
julia> kindof(x::Int64) where Int64 = Int64
kindof (generic function with 1 method)

julia> kindof(3)
Int64

julia> kindof('C')
Char

julia> kindof(4.2)
Float64

julia> kindof(3 => 8)
Pair{Int64, Int64}
```

Тот факт, что `Int64` является фактическим типом, здесь не имеет никакого значения. Предикатное условие `where` превращает `Int64` в параметр-тип и предотвращает его интерпретацию в качестве конкретного типа. Разумеется, следует избегать использования известных типов в качестве имен параметров-типов, так как это сильно запутывает читателей исходного кода.

18.2 Определение параметрических типов

На протяжении всей книги мы использовали параметрические типы, такие как массивы, словари и кортежи, но сами не определяли такие типы. Давайте попробуем это сделать.

Мы определим типы `Point` и `Vec2D`. На рис. 18.2 показано соотношение между точками и векторами в координатном пространстве. Точки обычно рисуются кружками, а векторы рисуются стрелками. Вектор представляет смещение вдоль каждой оси в системе координат.

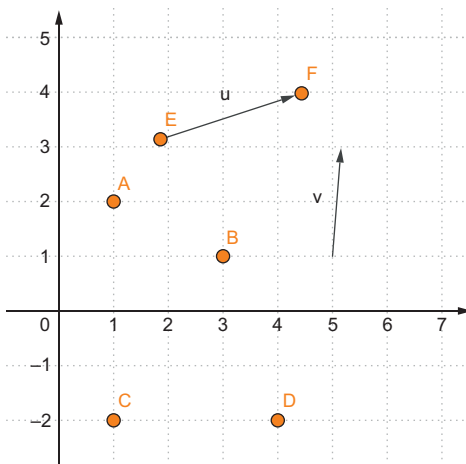


Рис. 18.2 Геометрическое соотношение между точками и векторами

С математической точки зрения точки и векторы связаны посредством различных операций. Если вычесть точку E из точки F, то получится вектор **u**. Эту операцию можно перевернуть и прибавить вектор **u** к точке E, чтобы получить точку F. Позже указанные детали будут рассмотрены подробнее. Сперва я хотел бы подробно рассмотреть определение параметрического типа (листинг 18.4). Мы пока не будем их определять в интерактивной среде REPL, потому что по ходу дела будем видоизменять определения.

Листинг 18.4 Определение параметрических типов `Point` и `Vec2D`

```
"Точка в координате (x, y)"
struct Point{T}
    x::T
    y::T
end
```

```
"Вектор со смещением (Δx, Δy)"
struct Vec2D{T}
    Δx::T
    Δy::T
end

"Вычислить модуль вектора (его длину)"
norm(v::Vec2D) = sqrt(v.Δx^2 + v.Δy^2)
```

Для получения символа Δ надо набрать \Delta. В математике этот символ обычно используется для обозначения разниц или смещений.

Point и Vec2D следует рассматривать не как типы, а как шаблоны для создания фактических типов. В целях создания фактического типа нужно указать конкретный тип для параметра-типа T. Без параметрических типов пришлось бы определять множество конкретных типов, чтобы работать с разными числами. Пришлось бы определять свой метод для каждого типа, что привело бы к раздуванию исходного кода. Например, с помощью параметрического типа метод norm можно определить всего один раз. Как показано в следующем ниже листинге, без него пришлось бы определять его для каждого конкретного типа 2D-вектора.

Листинг 18.5 Раздувание исходного кода при отсутствии параметрических типов

```
struct IntVec2D
    x::Int
    y::Int
end

struct FloatVec2D
    Δx::Float64
    Δy::Float64
end

norm(v::IntVec2D) = sqrt(v.Δx^2 + v.Δy^2)
norm(v::FloatVec2D) = sqrt(v.Δx^2 + v.Δy^2)
```

Однако разработчик на Python, Ruby или JavaScript против такого подхода возразит и скажет, что совершенно необязательно определять несколько конкретных типов. Если в типе для Δx и Δy требуется гибкость, надо просто не указывать аннотацию типа, как показано в следующем ниже листинге.

Листинг 18.6 Тип Vec2 без аннотаций типов (любой тип)

```
struct Vec2D
    Δx
    Δy
end
```

Ничто не мешает определить 2D-вектор таким образом. Так почему бы не сделать это именно в подобном ключе и избежать всех дополнительных сложностей, связанных с параметрическими типами? По многим причинам удаление аннотаций имеет плохие последствия:

- 1 во время выполнения программы устраняется ценная проверка типов, выполняемая JIT-компилятором языка Julia;
- 2 увеличивается потребление памяти и снижается производительность.

Ниже я расскажу о каждом из этих пунктов более подробно.

18.3 Выгоды от параметрических типов для типобезопасности

Я буду сравнивать 2D-векторы, в которых есть аннотации типов и в которых их нет, чтобы показать выгоды от типобезопасности при использовании параметрических типов. И начну я с создания двух временных типов, `GVec2D` и `PVec2D`, только для этого сравнения. Я не буду разрабатывать эти типы дальше. Обратите внимание, что определение типа можно задавать в одной строке – надо просто отделять инструкции точкой с запятой. `GVec2D` – это слабо типизированный вариант, а `PVec2D` – строго типизированный вариант:

```
julia> struct GVec2D Δx; Δy end ❶

julia> struct PVec2D{T} Δx::T; Δy::T end ❷

julia> v = GVec2D(2, 3.0) ❸
GVec2D(2, 3.0)

julia> u = PVec2D(2, 3.0) ❹
ERROR: MethodError: no method matching PVec2D{::Int64, ::Float64}

julia> u = PVec2D(2, 3) ❺
PVec2D{Int64}(2, 3)

julia> u = PVec2D{Int}(2, 3.0) ❻
PVec2D{Int64}(2, 3)
```

- ❶ Создать составной тип данных без аннотаций типов.
- ❷ Добавить аннотации типов в поля нового типа данных.
- ❸ Ограничения на типы аргументов отсутствуют.
- ❹ Аннотации типов не допускают ввод неправильных типов.
- ❺ Вычислить параметр-тип из аргументов.
- ❻ Задать для параметра-типа значение `Int` в явной форме.

В `GVec2D` аннотации типов не используются, поэтому при создании точки `q` Julia не жалуется на использование двух разных типов чисел для представления дельт `x` и `y`. Если бы вы попробовали это сделать

с `PVec2D`, в котором есть аннотации типов, то JIT-компилятор Julia пожаловался бы, потому что вы пытаетесь использовать два разных типа Δx и Δy . Из-за этой проблемы язык Julia не сможет вычислить параметр-тип `T`, и ему придется выдать исключение. Такое поведение помогает выявлять случаи, когда вы не обращаете внимания на типы чисел, которые вы передаете в своем исходном коде.

Данная проблема решается двумя способами: либо обеспечивать, чтобы все аргументы имели одинаковый тип, либо использовать фигурные скобки `{}`, чтобы указывать параметр-тип в явной форме, а не просить Julia вычислять его. Тогда Julia будет знать тип каждого поля и будет выполнять автоматическое преобразование в этот числовой тип. Оба варианта вполне допустимы.

Типобезопасность значительно улучшается за счет параметрических типов. Вспомните оператор подтипа `<:`. Указанный оператор использовался в различных случаях, включая проверку того, является ли один тип подтипом другого, а также при указании на то, что составной тип является подтипом абстрактного типа. Этот оператор также можно использовать для наложения ограничений на параметр-тип `T`. В настоящее время `T` может быть любым типом, включая нечисловые типы. Это делать нежелательно, так как координаты представлены числами. Окончательный тип `Vec2D`, показанный в следующем ниже листинге, будет ограничивать `T` числом.

Листинг 18.7 `Point` и `Vec2D` определены таким образом, чтобы параметр-тип был типом числа

```
import Base: +, -

struct Point{T<:Number}
    x::T
    y::T
end

struct Vec2D{T<:Number}
    Δx::T
    Δy::T
end

"Прибавление вектора `v` к точке `p` создает новую точку"
function +(p::Point{T}, v::Vec2D{T}) where T
    Point(p.x + v.Δx, p.y + v.Δy)
end

"Вычитание двух точек дает вектор"
function -(p::Point{T}, q::Point{T}) where T
    Vec2D(p.x - q.x, p.y - q.y)
end
```

Как можно убедиться, нет никаких проблем с созданием `PVec2D`, состоящим из символов, потому что параметр-тип `T` не имеет ника-

ких ограничений. С другой стороны, `Vec2D` не принимает символы в качестве аргументов. Попробуйте поэкспериментировать с разными значениями самостоятельно, чтобы убедиться, что ограничения на применяемый тип действительно работают:

```
julia> v = PVec2D('A', 'B')
PVec2D{Char}{'A', 'B'}

julia> u = Vec2D('A', 'B')
ERROR: MethodError: no method matching Vec2D{::Char, ::Char}

julia> u = Vec2D(8, 4)
Vec2D{Int64}(8, 4)
```

До сих пор во всех выражениях использовался только один параметр-тип. Но из словарей, кортежей и пар мы знаем, что параметров-типов может быть много. Для оператора вычитания точек в листинге 18.7 я указал, что каждая точка p и q должна иметь одинаковый тип числа, но это вовсе не обязательно. В листинге 18.8 показан оператор вычитания, реализованный с разными типами чисел для точек p и q .

Листинг 18.8 Вычитание определено так, что аргументы p и q не обязательно должны иметь одинаковый тип

```
function -(p::Point{T}, q::Point{S}) where {T, S}
    Vec2D(p.x - q.x, p.y - q.y)
end

function -(p::Point, q::Point) ❶
    Vec2D(p.x - q.x, p.y - q.y)
end
```

❶ Сокращенная версия без явных параметров-типов.

Поскольку в этом примере параметры-типы T и S не используются для каких-либо ограничений, их можно полностью опустить. В Julia запись `Point` эквивалентна записи `Point{T}`, при условии что T не имеет ограничений (рис. 18.3 иллюстрирует эти отношения типов). Например, если дана функция `sum`, которая принимает вектор в качестве аргумента, и типы элементов не интересуют, то можно написать `sum(xs::Vector)`, что идентично написанию `sum(xs::Vector{T})` where T .

В параметрических методах ключевая причина использования именованного параметра-типа, такого как T , заключается в желании указать, что два или более аргумента используют одинаковый параметр-тип. В других случаях можно обойтись без применения такого строгого требования. Вместо этого нужно, чтобы параметры-типы просто имели одинаковый тип. Скажем, вы хотите, чтобы вычитание между точками применялось только к целым числам (см. следующий ниже листинг).

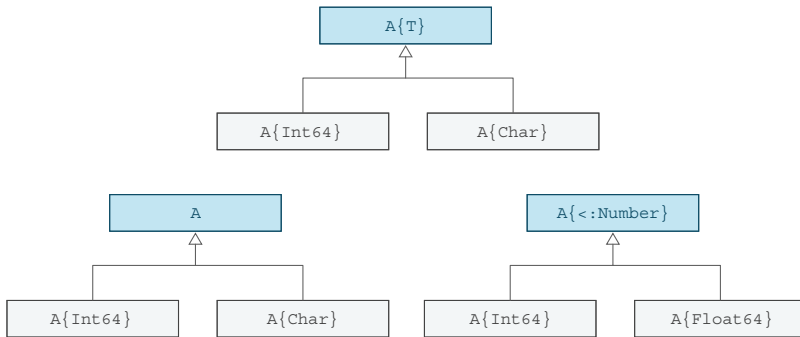


Рис. 18.3 Отношения подтипов между параметрическими типами. Например, функции, принимающие параметрический тип `A`, будут принимать значения типа `A{Int64}` и `A{Char}`

Листинг 18.9 Аргументы `p` и `q` должны иметь целочисленные параметры-типы

```
function -(p::Point{<:Integer}, q::Point{<:Integer})
    Vec2D(p.x - q.x, p.y - q.y)
end
```

В данном случае `p` может иметь поля `UInt8`, а `q` – поля `Int16`. Пример, конечно, надуманный, так как конкретно это ограничение не имеет смысла. Так почему тогда я не ограничил `p` и `q` типом `Number`? Разве это не было бы реалистичнее? Нет, и причина в невозможности создать объекты `Point`, которые не содержат чисел. Напомню, что здесь речь идет об ограничении на параметр-тип для самого типа `Point` и связанного с ним конструктора.

18.4 Выгоды в производительности за счет параметрических типов

Julia – это высокопроизводительный динамический язык. Без параметрических типов это было бы невозможно. Давайте обсудим влияние параметрических типов на производительность. Выгода от их понимания заключается в возможности легче предвидеть проблемы с производительностью в своем исходном коде.

Одна из ключевых причин медленной работы динамически типизированных языков кроется в так называемом *упаковывании*¹. Название связано с тем, что большинство значений должны помещаться в специальные контейнеры, которые содержат информацию о содержимом в них значении, включая его тип и сведения для

¹ Англ. *boxing*; син. *обертывание*; размещение примитивного типа в объекте. – Прим. перев.

сборки мусора, которые могут быть меткой или счетчиком ссылок (см. рис. 18.4). Специфика выполнения сборки мусора (т. е. высвобождения памяти) не является для этого аргумента существенной. Ключевым моментом является то, что эти универсальные контейнеры содержат кучу служебных данных.

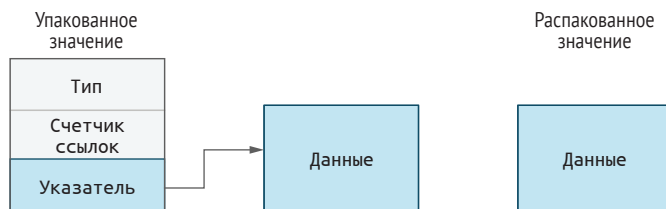


Рис. 18.4 Разница между упакованным и распакованным значениями

Какой смысл в этих служебных данных? Это именно то, что позволяет оперировать произвольными значениями во время выполнения программы. Представьте себе простую функцию `multiply`, служащую для скалирования вектора на скалярную константу k (см. следующий листинг).

Листинг 18.10 Vec2D, определенный с полями абстрактного типа для имитации обычных динамических языков

```
struct Vec2D
    Δx::Number
    Δy::Number
end

function multiply(u::Vec2D, k::Number)
    Vec2D(k * u.Δx, k * u.Δy)
end
```

❶ Для объяснения, не для выполнения.

❷ Требуется упаковка.

Это выглядит просто, верно? Но в динамическом языке для выполнения этой операции необходимо выполнить много работы. В следующем ниже листинге будет продемонстрирован вариант псевдокода на Julia, в котором показано то, что происходит на самом деле.

Листинг 18.11 Псевдокод механизма работы функции `multiply` в обычном динамическом языке

```
function multiply(u::Vec2D, k::Number)
    ux = getfield(u, :Δx)
    if !isa(ux, Float64)
        error("x must be a float")
    end
```

```

uy = getfield(u, :Δy)
if !isa(uy, Float64)
    error("y must be a float")
end

c = convert(Float64, k)
Vec2D(floatmul(c, ux), floatmul(c, uy))
end

```

Приведенный выше исходный код не показывает в точности то, что происходит. Думайте о нем больше как о форме псевдокода, помогающего развить приблизительное представление о том, как динамические языки работают «под капотом».

В большинстве динамических языков компилятор не знает, какие поля есть у составного типа, до момента выполнения программы. Это означает, что он не может генерировать машинный код, обращающийся к полям напрямую, поэтому в листинге 18.11 вы видите строку `getfield(u, :Δx)`. Он должен убедиться, что каждое поле действительно присутствует и имеет ожидаемый тип.

В языке Julia этой проблемы нет, потому что он накладывает на свои типы ряд ограничений, которые не распространены в других динамических языках:

- поля фиксируются по типу. Экземпляры типа не могут добавлять или удалять поля во время выполнения программы;
- нельзя хранить значения, не соответствующие типам полей составного объекта.

Эти ограничения значительно упрощают работу JIT-компилятора Julia, когда он пытается сгенерировать оптимизированный машинный код. Для того чтобы освежить информацию о вызовах методов Julia, обратитесь к разделу 7.4. Ключевой вывод, который следует вынести из этого обсуждения, состоит в том, что когда Julia генерирует машинный код метода, он точно знает тип каждого входного аргумента. Поскольку типы в Julia не изменяются, JIT-компилятор также будет точно знать имеющиеся у аргументов поля и их типы. За счет этого Julia имеет возможность генерировать высокооптимизированный машинный код.

18.5 *Выгоды от параметрических типов для памяти*

Выгоды от параметрических типов не ограничиваются предоставлением компилятору возможности создавать более оптимизированный машинный код и включают в себя более оптимизированное распределение данных в памяти. Если определяется массив типа `Vector{Point{Int32}}` с количеством элементов N , то JIT-компилятор Julia может точно определять число байтов, которое потребуется для

хранения всех этих элементов. За счет этого устраняется фрагментация памяти, которая снижает объем доступной памяти и производительность приложений. Подытоживая, параметрические типы обеспечивают более высокую типобезопасность, производительность и потребление памяти.

Резюме

- Параметрические типы повышают типобезопасность во время выполнения программы.
- Julia идентифицирует тип как параметр-тип в определении функции, глядя на предикатное условие `where`.
- Параметры-типы могут носить какое угодно имя, включая имена фактических типов, такие как `Int8` и `Char`. Однако, для того чтобы не дезориентировать разработчиков, читающих ваш исходный код, старайтесь использовать такие имена, как `T`, `T1`, `T2` и `S`.
- Для составных типов следует использовать `{T1, T2, T3}`, чтобы указывать тип с тремя разными параметрами-типами.
- Оператор подтипа `T <: S` следует использовать для ограничения параметрического типа `T` подтипом другого типа `S`.
- Упаковывание означает хранение значений вместе со служебной информацией, помогающей определять тип хранящихся данных во время выполнения программы.
- Упаковывание радикально снижает производительность динамических языков. Параметрические типы в сочетании с немутуируемыми типами сводят использование упаковывания в Julia к минимуму и, таким образом, позволяют компилятору генерировать оптимизированный машинный код.
- Параметрические типы обеспечивают более оптимальное распределение памяти для коллекций и отдельных объектов, уменьшая потребление памяти и повышая производительность.

Дополнение А

Установка и конфигурирование среды Julia

В этом дополнении рассказывается о скачивании и установке Julia в Linux, macOS и Windows. Я также расскажу о том, как конфигурировать среду Julia с целью повышения эффективности вашей работы.

Инструкции в этом дополнении основаны на знании таких понятий, как путь к командным утилитам, а также на умении их устанавливать; эти инструкции будут различаться в зависимости от операционной системы. Для Unix-подобных систем, таких как macOS и Linux, путь поиска конфигурируется путем редактирования конфигурационных файлов, таких как `.zshrc`, `.profile` или `.config/fish/config.fish`. В этих файлах можно задавать средовые переменные, такие как `PATH`, `EDITOR` и `JULIA_EDITOR`. Если вы незнакомы с командной строкой Unix, то ниже приведено несколько ресурсов из моего подстека Erik Explores:

- «Ускоренный курс по командной строке Unix» (<https://erikexplores.substack.com/p/unix-crash-course>);
- «Команды, конвейеры и процессы Unix» (<https://erikexplores.substack.com/p/unix-pipes>);
- «Оболочки и терминалы Unix» (<https://erikexplores.substack.com/p/unix-shells-and-terminals>).

A.1 Скачивание языка Julia

- 1 Перейдите на веб-страницу скачивания языка Julia (рис. А.1): <https://julialang.org/downloads>.
- 2 Выберите правильную версию языка Julia, соответствующую вашей операционной системе. Рекомендую установить Julia 1.7 или выше.

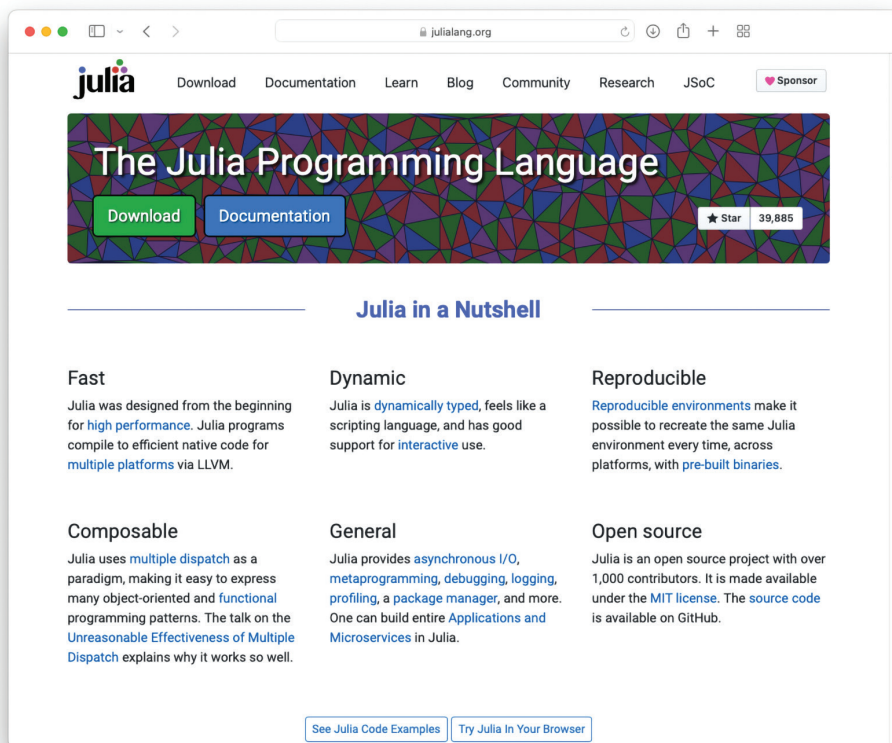


Рис. А.1 Домашняя страница языка Julia

А.2 Установка языка Julia

Прочтите инструкции по установке в используемой вами операционной системе.

ПРИМЕЧАНИЕ Некоторые команды оболочки имеют префикс `sudo`, который дает исполняемой команде привилегии суперпользователя. Эти привилегии необходимы для модификации файлов или каталогов, не принадлежащих вошедшему в систему пользователю.

А.3.1 Установка в Linux

Существует много разных дистрибутивов Linux, но в приведенных здесь инструкциях мы будем исходить из наличия каталога `Downloads`, в котором хранятся скачанные файлы. Скорректируйте данную команду в зависимости от места, где в Linux-машине хранятся скачанные файлы.

- 1 Распакуйте файл `.tar.gz` с именем `julia-1.7.3-linux-x86_64.tar.gz` или что-то вроде этого.
- 2 Переместите распакованный каталог в каталог `/opt`. Если каталог `/opt` не существует, то создайте его. Также создайте каталог `/opt/bin`, так как он понадобится позже:

```
$ sudo mkdir -p /opt/bin
$ cd $HOME/Downloads
$ sudo mv julia-1.7.3 /opt
```

Далее, для того чтобы легко запускать Julia из терминала, создайте символическую ссылку:

```
$ sudo rm /opt/bin/julia
$ sudo ln -s /opt/julia-1.7.3/bin/julia /opt/bin/julia
```

❶ Удалить любую старую ссылку.

A.3.2 Установка в macOS

- 1 Откройте скачанный файл `.dmg` с именем `julia-1.7.3-mac64.dmg` или что-то вроде этого.
- 2 Перетащите пакет приложений Julia в папку `/Applications`.

На этом установка завершена. Все очень просто!

Следующий шаг необязателен, но удобен. Выполните следующие ниже шаги, если хотите запускать Julia, просто набирая `julia` в терминале, а не кликая по значку приложения Julia.

- 1 Откройте консольное приложение `Terminal.app`.
- 2 Создайте символическую ссылку из места установки на каталог в вашем пути, например `/usr/local/bin`.

```
$ ln -s /Applications/Julia-1.7.app/Contents/Resources/julia/bin/
julia/usr/local/bin/julia
```

A.3.3 Установка в Windows

Скачайте файл `.exe`, который является автономным установщиком языка Julia. Дважды кликните по нему и следуйте инструкциям по установке Julia. Процесс установки аналогичен установке большинства других программ для Windows.

A.4 Конфигурирование языка Julia

Давайте сконфигурируем язык Julia так, чтобы им было удобнее пользоваться. Linux и macOS имеют очень похожие конфигурации, поскольку обе они являются Unix-подобными операционными системами.

A.4.1 Конфигурирование в Linux и macOS

В целях упрощения запуска Julia из терминала полезно сконфигурировать среду оболочки под Julia. Если вашей оболочкой является оболочка Z (zsh), то необходимо отредактировать файл .zshrc в своем домашнем каталоге. Если вы используете bash (Bourne Again SHell), то вместо него нужно отредактировать файл .profile. Оболочка Z в настоящее время является стандартом для macOS.

Вот пример конфигурирования Julia под использование с оболочкой bash в Linux, где Sublime Text (см. <https://www.sublimetext.com>), subl, используется в качестве текстового редактора исходного кода Julia:

```
# ~/.zshrc file
export JULIA_EDITOR=subl
export PATH=/opt/bin:$PATH
```

Следует отметить, что можно применять свою собственную оболочку. Например, я использую современную, удобную для пользователя оболочку для всех Unix-подобных систем под названием fish. В этом случае необходимо отредактировать файл .config/fish/config.fish в своем домашнем каталоге. В следующем ниже исходном коде я конфигурирую свой Mac под использование редактора VS Code, который запускается с помощью команды code:

```
# ~/.config/fish/config.fish file
set -x JULIA_EDITOR code
set -x PATH /usr/local/bin $PATH
```

A.4.2 Конфигурирование в Windows

В Windows средовые переменные конфигурируются посредством графического интерфейса, показанного на рис. A.2. Действия по открытию этого диалогового окна зависят от версии Windows. В Windows 8 и более поздних версиях выполните следующие действия:

- 1 в поле Поиск найдите и выберите «Система» (Панель управления);
- 2 кликните по ссылке «Дополнительные параметры системы».

В Windows Vista и Windows 7 выполните следующие действия:

- 1 на рабочем столе кликните правой кнопкой мыши на значке «Компьютер»;
- 2 в контекстном меню выберите «Свойства»;
- 3 кликните по ссылке «Дополнительные параметры системы».

В Windows устанавливать средовую переменную JULIA_EDITOR не нужно, так как операционная система откроет диалоговое окно и спросит вас, какой редактор использовать при необходимости. После этого Windows свяжет приложение с файлами .jl.

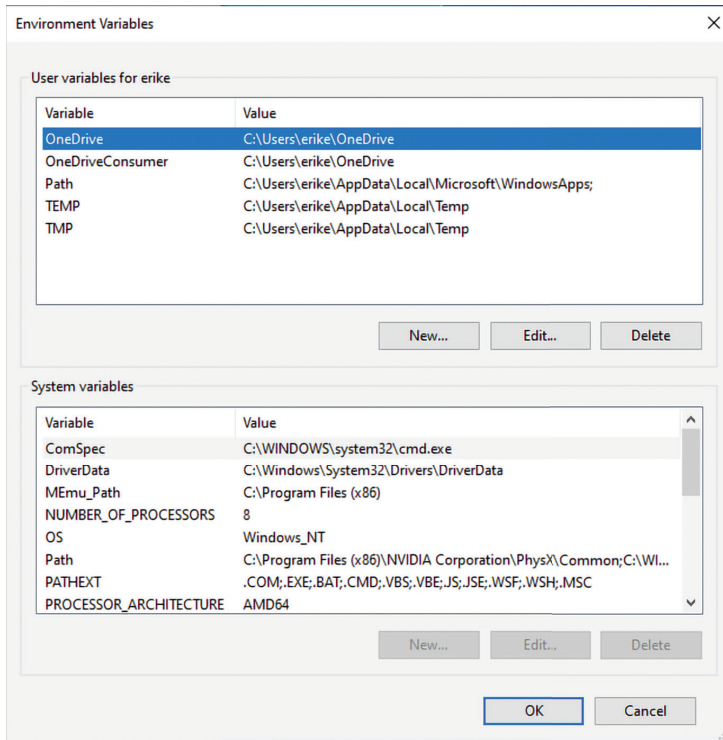


Рис. А.2 Диалоговое окно для конфигурирования пути двоичного поиска Windows

Конфигурировать среду оболочки, вероятно, менее необходимо в Windows, чем в Linux/macOS, потому что интерфейсы командной строки используются разработчиками Windows не так часто. Разработчики, интересующиеся командной строкой в Windows, возможно, предпочтут использовать Windows-подсистему для Linux (WSL; <https://docs.microsoft.com/en-us/windows/wsl/about>). Если вы применяете WSL, то следуйте инструкциям по установке и конфигурированию Linux.

А.5 Запуск языка Julia

Теперь, когда язык Julia установлен и сконфигурирован, можно попробовать его запустить. Кликните по значку приложения Julia либо откройте окно терминала, введите `julia` и нажмите **Enter**.

Когда программа Julia запускается, она входит в так называемую интерактивную среду под названием REPL (цикл чтения–оценивания–печати). REPL – это программа, которая принимает исходный код Julia, вычисляет его, а затем выводит результат вычисления этого исходного кода:

```
$ julia
```

```

      _       _       _
     ( )     | ( ) ( ) | Documentation: https://docs.julialang.org
      _ _   _ | _ _ _ _ |
     | | | | | | | / _ | | Type "?" for help, "]"? for Pkg help.
     | | | | | | | ( _ | | Version 1.7.2 (2022-02-06)
    _/ | \_ ' _ | | \_ ' _ | Official https://julialang.org/ release
   |_/

```

```
julia> print("hello world")
hello world
```

```
julia> 3 + 2
5
```

```
julia> reverse("ABCD")
"DCBA"
```

Каждая строка исходного кода обычно называется *выражением*. (Во многих других языках проводится различие между инструкциями и выражениями.) После нажатия **Enter** Julia вычислит выражение и покажет значение, полученное в результате его вычисления.

КАК ВЫХОДИТЬ ИЗ JULIA? Все, что делается в Julia, можно прервать, удерживая клавишу **Ctrl** и нажимая **C**. Данное сочетание клавиш записывается как **Ctrl+C**. Для выхода из Julia удерживайте **Ctrl+D** либо наберите `exit()`.

Ctrl+C обычно используется для остановки выполнения зависшего исходного кода Julia. Например, необходимость остановить исполнение может возникнуть в результате попадания в бесконечный цикл.

A.6 Режимы интерактивной среды REPL языка Julia

Интерактивная среда REPL может находиться в разных режимах, на которые указывает отображаемое в данный момент приглашение.

Зеленое приглашение с именем `julia>` означает, что вы находитесь в стандартном режиме Julia.

Вот другие режимы, которые вы увидите в примерах исходного кода:

- `help?>` – получить справку о функции или типе. Вход в справочный режим осуществляется путем нажатия `?`;
- `pkg>` – пакетный режим предназначен для установки и удаления пакетов. Вход в пакетный режим осуществляется путем нажатия `]`;

- `shell>` – режим оболочки позволяет выполнять команды оболочки Unix, такие как `ls`, `cp` и `cat`. Вход в режим оболочки осуществляется путем нажатия `;`.

Выйти из режима можно, перейдя в начало строки и нажав клавишу `←` (**Backspace**), в результате чего вы вернетесь в режим Julia.

Читая эту книгу, смотрите на приглашение командной строки, чтобы определять текущий режим. Если не перевести интерактивную среду REPL в правильный режим, то вводимые команды не будут работать. Ниже приведен пример справочного режима. Приглашение будет желтым:

```
help?> 4 / 2
/(x, y)
```

Right division operator: multiplication of x by the inverse of y on the right.

Gives floating-point results for integer arguments.

Режимы интерактивной среды REPL более подробно рассматриваются в главах 5, 16 и 17.

A.7 Установка сторонних пакетов

Существует несколько сторонних пакетов, которые, хотя и не являются необходимыми, улучшают ваш рабочий процесс. Пакеты более подробно рассматриваются в главе 16.

Войдите в пакетный режим, чтобы установить пакеты. Ниже мы установим пакеты `OhMyREPL`, `Revise` и `Debugger`:

```
(@v1.7) pkg> add OhMyREPL, Revise, Debugger
Resolving package versions...
```

Загрузите пакет в среду Julia с помощью ключевого слова `using`. При этом можно загрузить несколько пакетов сразу, отделив их запятой.

```
julia> using OhMyREPL, Revise, Debugger
```

Пакет `OhMyREPL` обеспечивает подсветку синтаксиса и улучшенную историю поиска в REPL. Пакет `Debugger` позволяет выполнять исходный код с помощью макрокоманды `@enter`. Например, следующий ниже исходный код перейдет к выполнению функции `titlecase`. Перейдите к следующему шагу отладки, нажав `N`, и выйдите, нажав `Ctrl+D`:

```
julia> @enter titlecase("hello")
```

Самым интересным и полезным пакетом является пакет `Revise`, позволяющий отслеживать изменения исходного кода. Обычно

в интерактивную среду REPL загружают исходный код одного файла с помощью функции `include`. Если вместо этого применяется функция `includet` из пакета `Revise`, то исходный код файла будет отслеживаться. Допустим, вы создаете файл с именем `hello.jl` со следующим ниже исходным кодом:

```
greet() = "hello world"
```

Этот файл можно загрузить в интерактивную среду REPL, используя пакет `Revise`:

```
julia> includet("hello.jl")
```

```
julia> greet()  
"hello world"
```

Файл исходного кода можно изменить, поменяв приветствие на `"hello Mars"`, и это изменение будет принято без необходимости перезагрузки файла в явной форме:

```
julia> greet()  
"hello Mars"
```

Дополнение В

Числа

В этом дополнении рассматриваются некоторые распространенные проблемы и вопросы, связанные с числами в программировании. Эти темы не совсем уникальны для Julia; например, мы затронем ситуацию, когда результатом целочисленной арифметической операции является большее или меньшее число, чем может быть представлено целочисленным типом. Я также коснусь причины неточности чисел с плавающей точкой, в отличие от целых.

В.1 *Различные типы чисел и их длина в битах*

Число – это не просто число как таковое. Существуют разные *типы* чисел, такие как целые числа, действительные числа, иррациональные числа и т. д. Например, целые числа – это арифметические числа, такие как 2, 7, 43, 820, –52, 0, 6 и –4, а действительные числа содержат числа с десятичной точкой, такие как 3.45, 0.042, 1331.0, 78.6.

Однако взгляды математика и программиста на числа принципиально различаются. В компьютере нам важны такие детали, как длина числа в битах и является ли число со знаком или без знака¹. Если вы использовали языки программирования, к примеру такие, как Java, C#, C или C++, то вы уже с этим хорошо знакомы.

Если вы занимаетесь программированием на другом динамическом языке, таком как Python, Ruby, JavaScript или Lua, то эти концепции могут быть вам незнакомы. Хотя Julia тоже является динамическим языком, числа в нем играют более важную роль. Например, в Python и JavaScript не требуется уделять много внимания различным типам чисел. В Julia же они играют более важную роль, так как числа были тщательно продуманы, чтобы сделать Julia пригодным для высокопроизводительных вычислений.

¹ Числа со знаком могут быть отрицательными, а числа без знака – только положительными.

Я познакомлю вас с основами. Вы, вероятно, знаете об ограничениях на количество вводимых цифр при заполнении формы. С компьютерами – все то же самое. Если вы храните свои числа в виде четырех цифр, то наибольшее число, которое можно использовать в любом расчете, равно 9999. Ключевое отличие состоит в том, что в современном цифровом компьютере все числа хранятся в памяти как двоичные числа, а не десятичные. Таким образом, типы чисел, из которых вы выбираете, не являются 4-значными числами, 8-значными числами и т. д. Отнюдь! Вы выбираете среди таких типов чисел, как, например, 8-битовые, 16-битовые или 64-битовые числа. Как это сказывается на практике?

Числа выражаются единицами и нулями. 8-битовое число – это число, состоящее из 8 двоичных бит. Самое большое число, которое можно представить 8 битами, записанное в двоичной форме, равно 0b11111111. Префикс 0b в этом числе предназначен для уточнения того, что это не десятичное число. В переводе на десятичную запись оно будет равно 255. Следовательно, если попытаться сохранить 256 как 8-битовое число, то сделать это не получится. (Обратите внимание, что для ясности сообщение об ошибке было опущено.)

```
julia> UInt8(256)
ERROR: InexactError: trunc(UInt8, 256)

julia> UInt8(255)
0xff

julia> Int(ans)
255
```

Но почему нужно ограничивать размер чисел, с которыми мы работаем? Почему бы каждый раз просто не использовать максимально возможное число? Потому что в компьютере нет бесконечного объема памяти. Если вы используете только несколько чисел, то их величина не имеет значения. Однако если вы обрабатываете миллионы чисел, то длина в битах начинает сказываться. Во-вторых, для выполнения вычислений с большими числами нередко требуется больше времени, чем с малыми числами. В Julia по умолчанию используются 64-битовые числа, потому что с ними очень удобно работать. 64-битовое целое число со знаком имеет максимальное значение 9 223 372 036 854 775 807. Вы вряд ли будете работать с более крупными числами.

Но как узнать наибольшее и наименьшее значения, которые тип чисел может содержать? К счастью, Julia предоставляет функции `typemax` и `typemin`, которые позволяют выяснять это самостоятельно. Однако на данный момент вам просто нужно принять принцип работы этих функций за чистую монету. Вы даете им имя числового типа Julia, например `Int8`, `Int16` или `UInt8`, и эти функции возвращают наименьшее и наибольшее числовые значения, которые можно

представить этим типом чисел. Например, 8-битовое целое число со знаком `Int8` не может представлять значения больше 127:

```
julia> typemax{Int8}
127
```

```
julia> typemin{Int8}
-128
```

```
julia> typemax{Int16}
32767
```

```
julia> typemax{Int64}
9223372036854775807
```

```
julia> typemin{Int32}
-2147483648
```

Вызов `typemin{Int8}` возвращает значение `-128`, так как 8-битовое целое число со знаком не может представлять числовые значения меньше `-128`.

Хотя все эти типы чисел могут показаться сложными, вам редко придется думать о них на практике. В большинстве случаев наилучшим выбором будет использование типов, применяемых по умолчанию, таких как `Int64`. Учитывать размеры других целых чисел придется, только если вы обрабатываете много чисел и сталкиваетесь с проблемами производительности или памяти. Более крупное число может также понадобиться из-за того, что вы работаете с очень большими значениями. В этом случае вы можете рассмотреть `Int128` или даже `BigInt`.

РАЗЛИЧИЯ МЕЖДУ УСТАНОВКОЙ 64-БИТОВОЙ И 32-БИТОВОЙ ВЕРСИЙ JULIA

Если вы скачали и установили 32-битовую версию Julia, то по умолчанию будет использоваться целочисленный тип `Int32`, а не `Int64`. Примеры исходного кода в этой книге основаны на допущении, что вы используете 64-битовую версию Julia.

Вы пробовали найти максимальное значение типа `BigInt`, но не смогли заставить его работать? Читайте дальше, чтобы узнать причину.

Для того чтобы узнать тип, используемый для представления определенного числового литерала, используется функция `typeof`. Надо лишь передать ей число, и она вернет тип числа, используемый для представления этого числа. На самом деле ее можно использовать для любого типа. Если это не имеет большого смысла, не волнуйтесь, так как чуть позже функция `typeof` будет рассмотрена более подробно:

```
julia> typeof(797298432432432432)
Int64
```

```
julia> typeof(797298432432432432709090)
Int128

julia> typeof(797298432432432432709090697343)
Int128

julia> typeof(7972984324324324327090906973430912321321)
BigInt
```

`BigInt` – это особый тип чисел, так как он не имеет предопределенного количества цифр. Он просто продолжает расти, чтобы вместить все цифры, так что память компьютера является его единственным реальным ограничением; вот почему для `BigInt` не существует четко определенного максимального значения.

Почему бы не использовать `BigInt` постоянно? Тогда не придется думать о количестве необходимых битов. Ответ очевиден: это будет способствовать плохой производительности. Вследствие этого следует стараться ограничивать использование `BigInt` разделами своего исходного кода, которые от этого выигрывают и не оказывают критически важного влияния на производительность.

В.2 Переполнение и числа со знаком и без знака

Давайте соединим вместе все, что мы узнали о числовых форматах и длинах в битах в главе 2, чтобы охватить несколько важных тем. Во-первых, это концепция переполнения. Подумайте о механических счетчиках, подобных показанному на рис. В.1. Он состоит из четырех цифр. И что произойдет, когда он дойдет до 9999? Он начнет отсчет сначала, и вы снова получите 0000.



Рис. В.1 Механический счетчик.
Он увеличивается на единицу при каждом нажатии на металлическую кнопку

Числа в компьютерах работают точно так же. Поскольку каждый тип чисел может содержать максимальное значение, в итоге может

возникнуть ситуация, когда будут выполняться арифметические операции, результатами которых будут значения, превышающие те, которые могут храниться в переменных. Вот несколько примеров тому на практике:

```
julia> UInt8(255) + UInt8(1)
0x00
```

```
julia> UInt8(255) + UInt8(2)
0x01
```

```
julia> 0xff + 0x01
0x00
```

```
julia> 0xff + 0x05
0x04
```

Поскольку UInt8 может содержать значения только до 255, при добавлении дополнительных значений вы получаете циклический возврат в начало, или, говоря более точным языком, вы получите переполнение. В этом случае легче понять концепцию, работая с шестнадцатеричными числами. 8-битовое число может содержать не более двух шестнадцатеричных цифр, и последняя цифра в шестнадцатеричном числе – F. Для получения переполнения 16-битовых чисел нужны гораздо более высокие значения:

```
julia> UInt16(65535) + UInt16(1)
0x0000
```

```
julia> UInt16(65535) + UInt16(3)
0x0002
```

Механизм работы переполнения отличается у чисел со знаком и без знака. Посмотрите на следующее ниже поведение чисел со знаком и без знака и постарайтесь его понять:

```
julia> UInt8(127) + UInt8(1)
0x80
```

```
julia> Int(ans)
128
```

```
julia> Int8(127) + Int8(1)
-128
```

```
julia> Int8(127) + Int8(2)
-127
```

```
julia> Int8(127) + Int8(127)
-2
```

Результат выглядит странно, не правда ли? Вы складываете положительные числа и получаете отрицательные. Как это возможно? Это

связано с тем, что память компьютера может хранить только числа. Отрицательный знак нигде не хранится. Вместо этого для имитации отрицательных чисел используется поведение циклического перехода в начало. Возвращаясь к примеру с механическим счетчиком на рис. В.1, хранящему четырехзначные десятичные числа, $4 + 9998$ в конечном итоге будет равно 2. Вообразите, что вы четыре раза щелкаете по механическому счетчику, начиная с 9998. Счетчик вернется по кругу к началу и в итоге будет равен 2.

Это означает альтернативный способ думать о числе 9998 – представить его как число -2 . $4 + (-2) = 2$. Таким образом, 9999 становится -1 , 9995 становится -5 и т. д. Следуя этой логике, 1 можно интерпретировать как -9999 . Однако не следует заходить так далеко; в противном случае невозможно будет представлять положительные числа четырьмя цифрами.

В современных компьютерах используется схема, которая состоит в том, чтобы делить каждый диапазон чисел примерно пополам, поэтому половина значений используется для представления отрицательных чисел, а другая половина – для представления положительных чисел. 8-битовое целое число без знака может представлять числа от 0 до 255; однако 8-битовое целое число со знаком представляет значения от -128 до 127, которые вы уже видели ранее:

```
julia> typemin(Int8)
-128
```

```
julia> typemax(Int8)
127
```

```
julia> typemin(Int16)
-32768
```

```
julia> typemax(Int16)
32767
```

Таким образом, то, что хранится в памяти, на самом деле ничем не отличается. Единственная разница будет в том, как *интерпретировать* то, что там хранится, при выполнении различных вычислений. При использовании чисел без знака подразумевается, что все числа являются положительными. При использовании чисел со знаком подразумевается, что половина значений является отрицательными. На самом деле с помощью функции `reinterpret` Julia может показать вам, как одни и те же биты в памяти интерпретируются по-разному:

```
julia> reinterpret(Int8, UInt8(253)) ❶
-3
```

```
julia> reinterpret(UInt8, Int8(-1)) ❷
0xff
```

```
julia> Int(ans)  
255
```

- ❶ Переинтерпретировать 8-битовое число 253 без знака как 8-битовое число со знаком. Число 253, интерпретируемое как число со знаком, равно -3 .
- ❷ Переинтерпретировать биты в памяти для числа -1 со знаком как число без знака. Число -1 без знака соответствует 255 для 8-битовых чисел.

В.3 Числа с плавающей точкой

Целые числа не могут представлять числа с десятичной точкой, к примеру такие, как 4.25, 80.3 или 0.233; существуют разные способы представления таких чисел. Исторически использовались числа с *фиксированной точкой*, но в современных компьютерах мы склонны использовать так называемые *числа с плавающей точкой*. Числа с фиксированной точкой используются для таких вещей, как валютные расчеты. В этом типе представления чисел количество десятичных знаков после точки фиксировано: всегда имеется только два десятичных знака.

Компьютеры не могут хранить знаки или десятичные точки в памяти. Память компьютера хранит только числа и только в двоичном формате. Это не так уж и отличается, скажем, от счетов. Хранить отрицательные числа или представлять десятичную точку на счетах в явной форме просто невозможно; однако можно сформулировать правила.

Например, можно просто решить, что две последние цифры чисел, с которыми вы работаете, должны находиться после точки. Это означает, что если вы хотите ввести 1, то вам на самом деле нужно вводить 100. Точно так же 23 становится 2300. Числа с фиксированной точкой имитируются аналогичным образом, используя только целые числа. Можно взять целые числа, к примеру такие, как 4250 и 850, и представить, что перед двумя последними цифрами стоит десятичная точка. Следовательно, эти числа можно будет интерпретировать как 42.50 и 8.50. Такой подход отлично работает в расчетах:

```
julia> 4250 + 850  
5100
```

```
julia> 42.50 + 8.50  
51.0
```

Для расчетов с деньгами это подходящий выбор; поскольку такие расчеты обычно включают округление до ближайших двух десятичных знаков. Но для научных расчетов с большим количеством знаков после точки это слишком непрактично. Вот почему существуют числа с плавающей точкой, которые основаны на идее, что таким способом можно представить любое число (рис. В.2).

$$12.34 = 1234 \times 10^{-2}$$

$$1.234 = 1234 \times 10^{-3}$$

$$0.1234 = 1234 \times 10^{-4}$$

Рис. В.2 Представление любого десятичного числа, используя целые числа, умноженные на число с основанием 10 и положительный или отрицательный показатель степени

Рассмотрим первую строку: число 1234 называется *значащей частью числа*¹. Вторая часть указывает *основание числа*. В данном случае *основание* равно 10, а показатель степени равен -2 . В числе с плавающей точкой значащая часть и показатель степени внутренне хранятся отдельно. Такая компоновка позволяет десятичной точке *плавать*. Типы чисел с плавающей точкой могут представлять гораздо более крупные числа, чем целые, но с тем недостатком, что они не совсем точны. Почему? Это слишком обширная тема, чтобы рассматривать ее в данной книге для начинающих, но я могу дать несколько советов по этому поводу. Рассмотрим число, к примеру, такое как $\frac{2}{3}$. Мы записываем его десятичными числами как 0.6666666666666666. Цифры все растут и растут. В десятичных числах цифры после точки представляют дроби, знаменатель которых кратен 10. Таким образом, мы делаем аппроксимацию дроби, как показано на рис. В.3.

$$\frac{2}{3} \approx \frac{6}{10} + \frac{6}{100} + \frac{6}{1000} + \dots$$

Рис. В.3 Аппроксимация дроби десятичными числами

Аппроксимация никогда не будет абсолютно точной. В примерах, которые я использую в этой книге, это, как правило, не будет проблемой, но об этом стоит знать, если вы начнете серьезно работать с числами с плавающей точкой. Однако не думайте, что каждое число с плавающей точкой должно быть неточным. Многие числа можно выражать точно, например 0.5 или 42.0. Очевидно, что для компьютеров числа с плавающей точкой представляются не в виде дробей с основанием 10, а в виде дробей с основанием 2.

¹ Син. мантисса. – Прим. перев.

Предметный указатель

Символы

$\sqrt{}$, функция, 56
_collect, функция, 291
@edit, макрокоманда, 208, 210
@enter, макрокоманда, 420
@printf, макрокоманда, 256
@sprintf, макрокоманда, 256
@test, макрокоманда, 383
@testset, макрокоманда, 383
@view, макрокоманда, 329
ф (golden), идентификатор, 45

A

AbstractArray, тип, 275
add, команда, 365, 377
all, функция, 283
Angle, абстрактный тип, 217
Angle, базовый класс, 225
ans, переменная, 49
Any, тип, 112, 165, 168, 169
any(isempty, cols), функция, 396, 398
АОТ-компиляция (ahead-of-time), 27
args, кортеж, 399
Array, тип данных, 94
attack!, методы функции, 182
attack!, функция, 170, 175
average, функция, 98

B

Base64, пакет, 372
Base, модуль, 218, 344, 390
battle!, функция, 170
battle!, method, 183
begin, ключевое слово, 102, 327

BigFloat, объект, 262
BigInt (большое целое число), 52, 262, 269

C

caesar_decrypt, функция, 357
caesar_encrypt, функция, 357
CairoMakie, пакет, 365, 366
calc, функция, 236
camel_case, функция, 249, 340
cat, команда, 137
cat, функция, 331
catch, инструкция, 84
ceil, функция, 61
ch, буквенный символ, 347
Char, тип, 93, 231
Cipher, интерфейс, 353
clipboard(), функция, 249
Cluster, абстрактный тип, 282
Cluster, конструктор, 283
Cluster, методы, 282
Cluster, подтипы, 201, 292
Cluster, тип, 201, 274, 282, 288
Cmd, тип, 392
code, команда, 417
collect, функция, 111, 151, 289
color, именованный аргумент, 127
Colors, пакет, 373
Component, атрибут, 319
Cone, модуль, 371
connect, функция, 393
const, ключевое слово, 45
constructor, функция, 192
convert, определения методов, 221
convert, функция, 213, 214, 215, 216, 224, 292

cos, функция, 56, 217, 221
CSV-файл (файл значений, отделенных запятой), 134
 синтаксический разбор, 394
 загрузка данных ракетных двигателей, 396
 сохранение данных ракетных двигателей, 399
CSV-файл (файл значений, отделенных запятыми)
 запись в файл, 136
 чтение из файла, 137
CustomEngine, конструктор, 398
CustomEngine, объект, 395
custom-engine.jl, 394
Cylinder, модуль, 371

D

DataFrames, пакет, 320
DateFormat, объект, 262
DateFormat, строковые литералы, 262
Dates, пакет, 367, 370, 372
Debugger, пакет, 420
deg2rad, функция, 73, 105
dev, пакетная команда, 378
Dict, конструктор, 350
Dict, тип, 147
Dict{String, Float64}, тип, 153
div, функция, 55
dlm, разделитель, 340
DMS (градусы, минуты, секунды), 216
DMS, тип, 216
DMS, тип угла, 216
DMS-числа, 217
DomainError, тип исключения, 192
DriveStyle, конструктор, 315
DriveStyle, тип, 314
drymass, функция, 281
dump, функция, 148

E

e, идентификатор, 45
eachindex, функция, 248
EDITOR средовая переменная, 414
ElectronDisplay, пакет, 365, 368
eltype (тип элемента), функция, 105
encrypt, функция, 85

end, ключевое слово, 69, 90, 102, 327
endswith, функция, 269, 344
Engine, объект, 185
Engine, типы, 394
enumerate, функция, 150
eof, функция, 389
Erik Explores, подстек, 382, 414
export, инструкция, 379

F

fib, функция, 86, 88
fill, функция, 103
filter, функция, 92, 273, 281, 301, 313
find, функции, 251
findfirst, функция, 227, 252
first, функция доступа, 147
Float32, тип, 54
Float64, тип, 54, 103, 213
floor, функция, 61
function, ключевое слово, 76, 370

G

generate, команда, 371, 394
get(), функция, 155
getindex, функция, 293, 326
GetPropellant, метод, 190
GLMakie, пакет, 365
golden (φ), идентификатор, 45
grep, инструмент, 391
grep, команда, 391
grep, процесс, 391

H

HasLength, подтип, 292
hcat, функция, 331
HeadType, конкретно-прикладной тип, 314
help?>, режим, 419
hobby, среда, 364, 365, 366, 368, 371

I

IDE (Интегрированная среда разработки), 281
import, инструкция, 368
Inf (infinity, бесконечность), 235

instances, функция, 316
 Int8(4), целое число, 169
 Int8, целочисленный тип, 51
 Int8, числовой тип Julia, 423
 Int16, целочисленный тип, 51
 Int16, числовой тип Julia, 423
 Int32(5), целое число, 169
 Int32, целочисленный тип, 51
 Int64, тип, 213, 324, 404
 Int64, целочисленный тип, 51
 Int128, целочисленный тип, 51
 Integer, тип, 169
 intersect, операция, 309, 313
 IntRange, переменная, 231
 io, объект, 136
 IO, объект, 136, 137, 220, 390, 392
 IO, тип и его конкретные подтипы, 389
 IOBuffer, конкретный тип, 389
 IOStream, конкретный тип, 389
 IOStream, объекты, 392
 isapprox, функция, 384
 isdigit, функция, 269
 isletter, функция, 269
 Isp, метод, 284
 Isp (удельный импульс), 196
 isuppercase, функция, 252, 269
 iterate, методы, 284
 iterate, функция, 281, 285
 iteration, интерфейс, 285
 iteration, методы, 288
 IteratorEltype, черта, 291
 IteratorSize, метод, 290
 IteratorSize, черта, 291

J

JAR-файл, 381
 JIT-компиляция (just-in-time), 27
 job, среда, 364, 365, 366, 371
 join, функция, 399
 Julia
 главная тема в книге, 39
 динамически типизированные
 языки в сравнении со статически
 типизированными, 27
 запуск, 418
 как универсальное решение, 34
 конфигурирование, 416
 в Linux и macOS, 417

 в Windows, 417
 общая информация, 26
 ограничения, 37
 применения, 36
 научные, 36
 ненаучные применения, 37
 проблема двух языков, 29
 установка
 в Linux, 415
 режимы REPL, 419
 скачивание, 414
 сторонние пакеты, 420
 элегантность, продуктивность
 и производительность, 28
 Julia для анализа данных (книга), 320,
 334
 JULIA_EDITOR, средовая
 переменная, 210, 414, 417

K

KeyError, исключение, 154

L

last, функция доступа, 147
 lastindex, функция, 246
 launch!, функция, 186, 204
 length, метод, 289
 length, функция, 241
 linearsearch, параметрический
 метод, 403
 linearsearch, функция, 402
 lines!, функции, 380
 lines, функция, 368
 Linux
 конфигурирование Julia, 417
 установка Julia, 415
 в macOS, 416
 в Windows, 416
 load_engines, функция, 396
 lpad, функция, 122, 130

M

macOS
 конфигурирование Julia, 417
 установка Julia, 416
 Main, модуль, 218

Makie, пакеты, 365
map, функция, 105, 281, 289, 301, 343, 348
 массивы, 118
 математическая таблица синусов, 119
map, функция более высокого
 порядка, 120, 273, 292
mapper, функция, 340
mapreduce, функция, 118
mapreduce, функция более высокого
 порядка, 119
mass, метод, 284
Material, конкретно-прикладной
 тип, 314
mean, функция, 114
median, функция, 114
Merlin, двигатель, 300
methods, функция, 176
MetricDegree, тип угла, 223
MIME-тип, 270
missing, объект, 234
Missing, тип, 227, 234
MixedCluster, конкретный подтип, 282
mod, функция, 354, 357
module, ключевое слово, 370
mount!, функция, 170, 172
multiply, функция, 411
mutable, ключевое слово, 172

N

NaN (не число), 235
nc, команда, 393
netcat, утилита, 392
new, функция, 194
nextind, функция, 246
nil, ключевое слово, 228
nmap, утилита, 393
norm, функция, 326
nothing, объект, 295
 в структурах данных, 229
 параметрический тип, 230
 применение типа-объединения
 для завершения товарного
 поезда, 232
 краткий обзор, 228
Nothing, тип, 69, 227
null, ключевое слово, 228
NumPair, переменная, 231

O

OhMyREPL, пакет, 377, 420
ones, функция, 103
open, функция, 123, 399

P

parse, функция, 123, 139, 253
parse(Float64, col), функция, 398
parse_roman, функция, 149, 159
PATH средовая переменная, 414
pgk>, режим, 419
pi (π), идентификатор, 45
Pkg, модуль, 385
PkgTemplate, библиотека, 372
Point, объекты, 408
Point, составной тип, 214
Point, тип, 405
pop!, функция массива, 309
popfirst!, метод, 295
popfirst!, функция, 300
position, функция, 389, 390
practice, функция, 141
print, функция, 124
Printf, модуль, 257
Printf, пакет, 373
printf, функция, 256
println, инструкция, 132
println, функция, 124, 394
printstyled, функция, 123, 124, 127, 130, 132
Priority, атрибут, 319
Process, конкретный подтип, 389
Process, объекты, 392
prod, функция, 79
Project, атрибут, 319
promote, метод, 223
promote, функция, 211
promote_rule, функция, 224, 292
promotion_rule, функция, 217
propellant!, метод-сеттер, 190
propellant!, функция, 190
propellant, функция доступа, 187, 188
push, функция, 108
push!, функция, 108
push!, функция массива, 309
pushfirst!, метод, 294
PVec2D, тип, 407

R

Radian, тип, 216
 rand, функция, 44, 103
 Random, модуль, 349
 read, функция, 389, 394
 readline, функция, 123, 389, 394
 reduce, функция, 105, 301
 массивы, 118
 математическая таблица синусов, 119
 refill!, методы, 188
 RegexMatch, объекты, 267
 reinterpret, функция, 427
 rem, функция, 55
 REPL, интерактивная среда
 использование пакета
 в интерактивной среде REPL, 368
 установка, 418
 цикл чтения–вычисления–
 печати, 418
 REPL, программа командной строки
 краткий обзор, 43
 работа с функциями, 58
 REPL (цикл прочитать–вычислить–
 напечатать), 39
 replace, функция, 265
 reshape, функция, 329
 resupply!, функция, 172
 return, инструкция, 90
 Revise, пакет, 377, 420
 rm, команда, 367
 round, функция, 61
 round(), функция, 142
 rpad, функция, 122, 130
 Rutherford, составной тип, 221

S

sametype, функция, 283
 scatter, функция, 368
 Screw, объект, 315
 searchsortedfirst, функция, 157
 Set, конструктор, 313
 setdiff, операция, 309
 setindex!, функция, 293, 326
 Severity, атрибут, 319
 shell>, режим, 419
 shoot!, функция, 170, 172, 176

show, объект, 221
 show, функция, 216, 220, 389
 shuffle, функция, 349, 359
 sin, функция, 56, 77, 119, 217, 221
 sine, функция, 78, 119, 380
 sinus, функция, 119
 SizeUnknown, подтип, 292
 skipmissing, функция, 235
 sort!, функция, 97
 splat, трехточечный оператор, 211
 split, функция, 123, 138, 249, 269, 344, 390
 splitter, функция, 340
 sprintf, функция, 257
 SQL (Язык структурированных
 запросов), 313
 stackoverflow, сообщение
 об ошибке, 80
 startswith, функция, 269
 Statistics, модуль, 98, 218
 status, команда, 365, 372
 stdin, клавиатура, 138
 stdout (стандартный вывод), 136
 String, тип, 122
 string(), функция, 129, 255
 struct, 171, 193, 194, 237
 struct, ключевое слово, 370
 SubstitutionCipher, объект, 354
 subtypes, функция, 166
 sum, функция, 114, 120, 286, 292, 301
 supertype, функция, 166

T

T, параметр типа, 406
 tail, функция, 297
 Tank, интерфейс, 278
 Tank, объект, 185
 Tank, тип, 187, 190, 277, 279
 taylor, функция, 119
 TCPSocket, тип, 389, 393
 Test, пакет, 383
 test, среда, 384
 throw, функция, 84
 thrust, метод, 284
 time(), функция, 142
 titlecase, функция, 420
 totalmass, функция, 280

ToyGeometry, модуль, 371, 376
ToyGeometry, пакет, 371, 384
ToyGeometry, проектная среда пакета, 372
ToyRockets, пакет, 394
Tuple, тип данных, 93, 276
typemax, функция, 51, 423
typemin, функция, 51, 423
typeof, функция, 51, 403

U

UInt8, числовой тип Julia, 423
Unicode, пакет, 373
UniformCluster, подтип, 282
union, операция, 309
Union, определение типа, 232
update, команда, 378
update!, функция, 202
using, инструкция, 368
using, ключевое слово, 420
UTF-8, 241
UUID (универсальный уникальный идентификатор), 372

V

vals, массив, 159
vcat, функция, 331
Vec2D, тип, 405

W

Wagon, конструктор, 230
while, ключевое слово, 69
Windows
 конфигурирование Julia, 417
 установка Julia, 416
write, функция, 389

X

xclip, команда, 249
xsel, команда, 249

Z

zeros, функция, 103
zip, функция, 154, 350

A

Адрес возврата, 83
Алгебра
 линейная, 323
 реляционная, 320
Алгоритм линейный, 290
Аннотация типа, 165
Аппроксимация, 65
Атом, 163

Б

База данных реляционная, 320
Бесконечность (Inf), 235
Блок-схема, 71

В

Вектор, 95
 в математике, 323
 комбинирование векторов и матриц, 331
 конструирование матрицы из строк и столбцов, 323
 нарезка массива, 326
 размер, длина и норма массива, 325
 создание матриц, 334
Вектор-столбец, 95
Вектор-строка, 94
Вектор цен (prices), 96
Взаимодействие с пользователем, 140
Взаимосвязь линейная, 306
Видоизменение пакета, 376
Винт
 поиск, 316
 поиск с использованием словарей, 318
 размещение объектов-винтов в множества, 317
Включение в массив, 149
Выдача исключений, 83
Выражение, 70, 419
Выражение булево
 краткий обзор, 66
 составные инструкции, 68
Выражение включения
 в массив, 149
Выражение регулярное, 241, 265

Г

Гипотенуза, 326
Градус, 219

Д

Данные неопределенные, 237
Дерево поиска двоичное, 157
Деструктуризация, 398
Дешифрование (форма do-end), 349
Диаграмма рассеяния, 369
Диспетчеризация множественная, 27, 39
 вызов методов, 175
 отличие от перегрузки функций, 183
 сравнение с объектно ориентированными языками, 181
Диспетчеризация одиночная, 181
Длина в битах, 422
Длина массива, 325
 понятие, 325
Добавление линий, 131
Доступ произвольный, 308
Дубликат, 307

Е

Единица измерения углов, 215
 арифметические операции на углах, 218
 конвертации типов, 221
 конструкторы углов, 217
 красивые литералы углов, 222
 отображение DMS-углов на экране, 220
 приведения типов, 223
 функции доступа для извлечения градусов, минут и секунд, 219
Единица кодовая, 242

З

Заголовок, 135
Загрузка данных ракетных двигателей, 396
Замыкание, 339, 342
Запись в сокет, 392
Запрос, 316

Запуск Julia, 418
Значение
 отображение в массивах, 105
 перечисление значений, 150
 присваивание и привязывание переменным, 48
Значение возвращаемое, 78
Значение неизвестное
 неопределенные данные, 237
 объект nothing, 228
 отсутствующие значения, 234
 применение объекта nothing в структурах данных, 229
 параметрический тип, 230
 применение объединения для завершения товарного поезда, 232
NaN (не число), 235
Значение отсутствующее, 234

И

Идентификатор, 44
Иерархия типов, 166
Импульс удельный, 196
Инвариант, 191
Индекс
 перечисление, 150
 элемента, 94
Инструкция, 70
Инструкция составная, 68
Инструкция if, 80
 инструкции if-else, 81
 подчиненный компонент elseif, 82
Инструкция if-else, 81
Интерполяция строковых литералов, 254, 258
Интерфейс
 краткий обзор, 274
 по общепринятому соглашению, 281
 пример интерфейса топливного бака, 277
Интерфейс итерируемый, 284
Итерации по сравнению с рекурсией, 89
Итерация
 итеративная обработка кластера двигателей, 282
 реализация итеративной обработки ступеней ракеты, 287

К

Камински, Богумил, 320, 334
Квантификатор, 267
Класс, 166
Класс буквенных символов, 267
Кластер, 282
Код исходный
 генерирование на основе
 интерполяции строкового
 литерала, 258
организация и модуляризация
 видеоизменение и разработка
 пакета, 376
 заблуждения о модулях, 381
 настраивание рабочей среды, 364
 создание своего собственного
 пакета и модуля, 371
 тестирование пакета, 382
поддержание инвариантов, 191
применение множеств, 312
Коллекция
 интерфейсы, 274
 интерфейсы по общепринятому
 соглашению, 281
 полезность конкретно-прикладных
 типов, 300
 пример интерфейса топливного
 бака, 277
 реализация итеративной обработки
 кластера двигателей, 282
 реализация итеративной обработки
 ступеней ракеты, 287
 сравнение связанных списков
 и массивов, 292
Коллекция итерируемая, 273
Компилятор, 184
Компиляция
 AOT (ahead-of-time), 27
 JIT (just-in-time), 27, 28
Комплект тестов вложенный, 383
Компонент подчиненный `elseif`, 82
Конвертация
 единицы измерения углов, 215
 арифметические операции на
 углах, 218
 конвертации типов, 221
 конструкторы углов, 217
 красивые литералы углов, 222

 отображение DMS-углов на
 экране, 220
 приведения типов, 223
 функции доступа для извлечения
 градусов, минут и секунд, 219
 конвертация чисел, 212
 система приведения типов чисел, 209
Конкатенация массивов, 322
Конкатенация строковых
 литералов, 254
Константа
 краткий обзор, 44
 литеральные коэффициенты, 50
Конструктор внешний, 194
Конструктор внутренний, 194
Конструктор углов, 217
Конфигурирование Julia, 416
 в Linux и macOS, 417
 в Windows, 417
Корень квадратный, функция, 44
Кортеж
 хранение данных, 112
 именованный в качестве
 словаря, 158
Коэффициент литеральный, 50

Л

Литерал строковый, 109
 в верблюжьем регистре, 248
 в змеином регистре, 248
 нестандартные строковые
 литералы, 261
 неформатированные строковые
 литералы, 265
 применение регулярных
 выражений для отыскания
 совпадений в тексте, 265
 создание больших целых чисел
 с помощью `BigInt`, 269
 строковые литералы
 `DateFormat`, 262
 MIME-типы, 270
нестандартный
 неформатированные строковые
 литералы, 265
 применение регулярных
 выражений для отыскания
 совпадений в тексте, 265

создание больших целых чисел
с помощью BigInt, 269
строковые литералы
DateFormat, 262
MIME-типы, 270
неформатированный, 265
операции, 248
 конвертирование из верблюжьего
 регистра в змеиный регистр, 248
 конвертирование между числами
 и строковыми литералами, 253
функции
 интерполяция и конкатенация
 строковых литералов, 254
 применение интерполяции
 строкового литерала для
 генерирования исходного кода, 258
 форматирование с использованием
 функции sprintf, 256
 UTF-8 и Юникод, 241
Литера угла, 222
Лямбда-функция, 339

М

Мантисса. См. *Часть числа значащая*
Масса, 187
 общая, 187
 ракетного топлива, 187
 сухая, 187
Массив, 292
 булевых значений, 120
 векторы и матрицы
 вертикальная и горизонтальная
 нарезки массива, 326
 в математике, 323
 комбинирование, 331
 конструирование матрицы из строк
 и столбцов, 323
 размер, длина и норма массива, 325
 создание матриц, 334
 выполнение операций, 96
 добавление и удаление
 элементов, 294
 отображение значений, 105
 отображение и редукция, 118
 создание, 103
 сравнение свойств массивов
 и множеств, 304

 дубликаты, 307
 проверки на принадлежность
 объектов, 307
 произвольный доступ
 и упорядочивание элементов, 308
 сортированный массив, 306
 типы, 94
Массив ассоциативный, тип
данных, 144
Массив продаж (sales), 114
Массив сортированный, 306
Математика, 323
Матрица, 95
 вертикальная и горизонтальная
 нарезки массива, 326
 в математике, 323
 комбинирование векторов
 и матриц, 331
 конструирование из строк
 и столбцов, 323
 размер, длина и норма массива, 325
 создание, 334
Метод, 172, 179
 в чем множественная
 диспетчеризация отличается
 от перегрузки функций, 183
 сравнение множественной
 диспетчеризации Julia с объектно
 ориентированными языками, 181
Минута, 219
Множество
 вид задач, решаемых с помощью
 множеств, 303
 операции
 как применять, 309
 поиск товаров, 313
 поиск в инструменте отслеживания
 дефектов, 319
 применение в исходном коде, 312
 реляционные базы данных, 320
 сравнение свойств массивов и
 множеств, 304
 дубликаты, 307
 проверки на принадлежность
 объектов, 307
 произвольный доступ
 и упорядочивание элементов, 308
 сортированный массив, 306
Множество, тип-коллекция, 318

Модуль, 381
 заблуждения о модулях, 381
 модификация и разработка пакета, 376
 настраивание рабочей среды, 364
 использование пакета
 в интерактивной среде REPL, 368
 как модули соотносятся с пакетами, 370
 создание, 371
 генерирование пакета, 371
 добавление исходного кода в пакет, 374
 тестирование пакетов, 382

Н

Навигация, 64
Нагрузка полезная, 186, 195
Нарезка массива, 326
Наследование, 164
Наука
 потребность в динамически типизированных языках, 30
 применения языка Julia, 36
Не число (NaN), 235
Норма массива, 325
 понятие, 325

О

Обработка
 итеративная кластера двигателей, 282
 ошибок, 83
Объединение множеств, 309
Объект, 192
 диапазон, 74
 итерируемый, 76
 мутируемый, 109, 113, 172
 первоклассный, 339
 поисковый типа String, 403
Оператор
 конвейерный, 343
 меньше или равно, 67
 трехточечный, 211
Операция
 арифметическая на углах, 218
 логическая ИЛИ (OR), 68
 логическая И (AND), 68

 на массиве, 93
 на множестве
 как применять, 309
 перечисления, 315
 поиск винтов, 316
 поиск винтов с использованием словарей, 318
 поиск товаров, 313
 размещение объектов-винтов в множествах, 317
 создание тестовых данных для выполнения запросов, 316
 на строковых литералах, 248
 интерполяция и конкатенация строковых литералов, 254
 конвертирование из верблужьего регистра в змеиный регистр, 248
 конвертирование между числами и строковыми литералами, 253
 форматирование с использованием функции `sprintf`, 256
 постоянная, 294
Основание числа, 429

П

Пакет
 заблуждения о модулях, 381
 модификация и разработка пакета, 376
 настраивание рабочей среды, 364
 использование пакета
 в интерактивной среде REPL, 368
 как модули соотносятся с пакетами, 370
 создание, 371
 генерирование пакета, 371
 добавление исходного кода в пакет, 374
 сторонний, 420
 тестирование, 382
Память, 412
Параметр типа, 402
Перегрузка функций, 175
Переменная, 44
 литеральные коэффициенты, 50
 переменная `ans`, 49
 присваивание и привязывание значений, 48

- Переполнение, 425
- Пересечение множеств, 309
- Перечисление, 315
- Печать
 - многочисленных элементов, 128
 - простые таблицы, 129
 - тригонометрические таблицы, 132
 - функция `print`, 124
 - функция `println`, 124
 - функция `printstyled`, 124
- Поведение. См. *Черта*
- Поезд товарный, 232
- Поиск
 - в инструменте отслеживания дефектов, 319
 - двоичный, 306
- Поток данных по сравнению с потоком управления, 85
- Поток управления
 - булево выражение
 - краткий обзор, 66
 - составные инструкции, 68
 - инструкции `if`, 80
 - инструкции `if-else`, 81
 - подчиненный компонент `elseif`, 82
 - многострочные функции
 - краткий обзор, 76
 - тригонометрическая функция `sine`, 77
 - навигация и тригонометрия, 64
 - обработка ошибок посредством выдачи исключения, 83
 - по сравнению с потоком данных, 85
 - пример роста популяции кроликов, 86
 - базовый случай, 89
 - возвращать или не возвращать, 90
 - итерации по сравнению с рекурсией, 89
 - реализация факториала, 79
 - с использованием рекурсии, 80
 - формирование циклов, 69
 - блок-схемы, 71
 - математические таблицы синусов, 72
 - объекты-диапазоны, 74
 - циклы `for`, 75
- Предикат
 - комбинирование функций более высокого порядка, 117
 - фильтрация на основе предикатов, 116
- Представление, 328
- Приведение типов
 - единицы измерения углов, 215
 - арифметические операции на углах, 218
 - конвертации типов, 221
 - конструкторы углов, 217
 - красивые литералы углов, 222
 - отображение DMS-углов на экране, 220
 - приведения типов, 223
 - функции доступа для извлечения градусов, минут и секунд, 219
 - конвертация чисел, 212
 - система приведения типов чисел, 209
- Привязывание, 48
- Применение
 - функции частичное, 344
 - частичное, 344
 - чисел на практике, 62
- Пример
 - анализа продаж пиццы, 93
 - фильтрация пицц на основе предикатов, 116
 - хранение данных о пиццах в кортежах, 112
 - интерфейса топливного бака, 277
 - роста популяции кроликов, 86
 - базовый случай, 89
 - инструкции `return`, 90
 - итерации по сравнению с рекурсией, 89
 - таблицы продажи пицц, 129, 134
 - выравнивание текста, 130
 - добавление линий, 131
 - запись в CSV-файлы, 136
 - распечатка многочисленных элементов, 128
 - распечатка простых таблиц, 129
 - функция `print`, 124
 - функция `println`, 124
 - функция `printstyled`, 124
 - чтение из CSV-файлов, 137
- Проблема двух языков, 29
 - потребность в динамически типизированных языках со стороны разработчиков, 32

потребность в динамически
типизированных языках со стороны
ученых, 30
Julia, 33
Проверка на принадлежность, 307
Программирование
интерактивное, 30
объектно ориентированное
отличия от функционального
программирования, 338
разработка службы
шифрования с использованием
объектно ориентированного
программирования, 353
сравнение с множественной
диспетчеризацией, 181
процедурное, 339
функциональное
анонимные функции и
замыкание, 341
генерирование новых функций
с помощью частичного
применения, 344
избегание глубокой вложенности
вызовов с помощью выстраивания
функций в цепочку, 340
как и зачем думать
функционально, 339
объектно ориентированное
программирование, 39, 338
отличия от функционального
программирования, 338
применение конвейерного
оператора $|>$, 343
программирование объектно
ориентированное, разработка
службы шифрования, 353
разработка службы
шифрования, 357
определение функционального
подстановочного шифра, 359
определение функционального
шифра Цезаря, 357
функциональная служба
хранения паролей, 360
создание службы, не зависящей от
алгоритма шифрования, 352
шифр Цезаря и подстановочный
шифр, 345

реализация подстановочного
шифра, 349
реализация шифра Цезаря, 347
Производительность, 28, 33, 410
Прокручивание в цикле буквенных
символов, 149
Пространство имен, 370
Пуск ракеты, 204

Р

Разбор
римских цифр, 145
синтаксический CSV-файла, 394
загрузка данных ракетных
двигателей, 396
сохранение данных ракетных
двигателей, 399
Размер массива, 325
Разность множеств, 309
Разработка пакета, 376
Ракета многоступенчатая, 199
Редукция, 118
Рекурсия
по сравнению с итерациями, 89
факториал, 80
Ряд Тейлора, 65

С

Сборка ракеты
запуск в космическое
пространство, 204
моделирование ракетных двигателей
и полезных нагрузок, 195
отличия между внешним
и внутренним конструкторами, 194
поддержание инвариантов
в исходном коде, 191
реализация итеративной обработки
ступеней ракеты, 287
сборка простой ракеты, 198
синтаксический разбор
CSV-файла, 394
загрузка данных ракетных
двигателей, 396
сохранение данных ракетных
двигателей, 399

- создание объектов с помощью функций-конструкторов, 192
 - создание с несколькими ступенями и двигателями, 199
 - строительство простой ракеты, 186
 - Секунда, 219
 - Сигнатура функции, 361
 - Символ, 128
 - Символ буквенный, 109
 - Симулятор сражений, 170
 - добавление поведения к воинам, 172
 - персонажи, 170
 - применение множественной диспетчеризации для вызова методов, 175
 - Система ввода-вывода (В/В)
 - краткий обзор, 389
 - синтаксический разбор
 - CSV-файла, 394
 - загрузка данных ракетных двигателей, 394
 - сохранение данных ракетных двигателей, 399
 - чтение данных из процесса, 391
 - чтение из сокета и запись в сокет, 392
 - Скаляр, 323
 - Скачивание Julia, 414
 - Словарь
 - доступ к элементам, 154
 - именованные кортежи, 158
 - использование, 152
 - перечисление значений и индексов, 150
 - поиск винтов, 318
 - причина использования, 155
 - прокручивание буквенных символов в цикле, 149
 - процесс конвертации, 151
 - создание, 152
 - тип Dict, 147
 - Словарь, тип данных, 144, 147
 - Словарь, тип-коллекция, 318
 - Служба, не зависящая от алгоритма шифрования, 352
 - Служба хранения паролей, 360
 - Служба шифрования, 357
 - определение функционального подстановочного шифра, 359
 - определение функционального шифра Цезаря, 357
 - разработка с использованием объектно ориентированного программирования, 353
 - функциональная служба хранения паролей, 360
 - Случай базовый, 89
 - Соединение внутреннее, 320
 - Сокет, 392
 - Сообщение, 345
 - Сообщение (строковые литералы), 353
 - Сохранение данных ракетных двигателей, 399
 - Список связный, 229, 292
 - Среда, 364
 - использование пакета в интерактивной среде REPL, 368
 - как модули соотносятся с пакетами, 370
 - любительская. См. *hobby*, *среда*
 - проектная. См. *job*, *среда*
 - разработки интегрированная (IDE), 281
 - Стек вызовов, 83
 - Столбец, 323
 - Строка, 323
 - Структура данных, 180, 229
 - двоичное дерево поиска, 157
 - параметрический тип, 230
 - применение типа-объединения для завершения товарного поезда, 232
- ## Т
-
- Таблица
 - математическая синусов с использованием функций `map` и `reduce`, 119
 - формирование циклов, 72
 - печать
 - простые таблицы, 129
 - тригонометрические таблицы, 132
 - пример таблицы продажи пицц, 129, 134
 - Текст
 - взаимодействие с пользователем, 140
 - выравнивание, 130
 - добавление линий, 131

печать
 многочисленных элементов, 128
 функция print, 124
 функция println, 124
 функция printstyled, 124
CSV-файлы, 134
 запись в CSV-файлы, 136
 чтение из CSV-файлов, 137
Телетайп, 126
Теорема Пифагора, 326
Тестирование пакетов, 382
Тип
 выгоды для типобезопасности от
 параметрических типов, 407
 иерархия, 166
 как Julia выбирает метод, который
 следует вызывать, 179
 в чем множественная
 диспетчеризация отличается от
 перегрузки функций, 183
 сравнение множественной
 диспетчеризации Julia с объектно
 ориентированными языками, 181
 конвертации, 221
 полезность конкретно-прикладных
 типов, 300
 приведения типов, 223
 примитивный, 163
 создание симулятора сражений, 170
 добавление поведения к
 воинам, 172
 определение типов воинов, 170
 применение множественной
 диспетчеризации для вызова
 методов, 175
 создание составных типов из
 примитивных, 163
 составной, 163, 172
Тип-объединение, 232
Тип параметрический, 230
 выгоды для памяти, 412
 выгоды для производительности, 410
 выгоды для типобезопасности, 407
 определение, 405
 параметрические методы, 405
Тип числа
 конвертация, 212
 конвертирование в строковый
 литерал, 253
 система приведения типов, 209

Тип числовой
 длина в битах, 51, 422
 написание чисел с использованием
 разных числовых форматов, 52
 со знаком и без знака, 425
 числа с плавающей точкой, 428
Тип null для статистиков, 237
Точка кодовая, 242

у

Упаковывание, 410
Упорядочивание элементов, 308
Упрощение, 90
Установка Julia
 в Linux, 415
 в macOS, 416
 в Windows, 416
 режимы REPL, 419
 скачивание, 414
 сторонние пакеты, 420

Ф

Факториал
 реализация, 79
 с использованием рекурсии, 80
Форма инфиксная, 59
Форма префиксная, 59
Форматирование с использованием
функции sprintf, 256
функции
 комбинирование функций более
 высокого порядка, 117
Функциональность наподобие
электронной таблицы
 буквенные символы, 109
 доступ к элементам, 100
массивы
 булевых значений, 120
 выполнение операций, 96
 отображение значений, 105
 отображение и редукция, 118
 создание, 103
 типы, 94
модуль Statistics, 98
предикаты
 комбинирование функций более
 высокого порядка, 117

фильтрация на основе предикатов, 116
 пример анализа продаж пиццы, 93
 строковые литералы, 109
 хранение данных в кортежах, 112
Функциональность, похожая на калькулятор, 62
 командная строка, 43
 константы и переменные, 44
 литеральные коэффициенты, 50
 переменная `ans`, 49
 присваивание и привязывание значений переменным, 48
функции
 важная особенность, 59
 определение, 56
 полезные, 60
 работа в интерактивной среде REPL, 58
 сохранение определений в файлах, 57
числа с плавающей точкой, 54
числовые типы
 длина в битах, 51
 написание чисел с использованием разных числовых форматов, 52
Функция
 анонимная, 341
 более высокого порядка, 106, 339
 выстраивание функций в цепочку, 340
 анонимные функции и замыкание, 341
 генерирование новых функций с помощью частичного применения, 344
 применение конвейерного оператора `|>`, 343
 двоичная, 118
 доступа, 115, 129, 219
 многострочная
 краткий обзор, 76
 тригонометрическая функция `sine`, 77
 перегрузка, 183
 рекурсивная, 168

Х

Хеш-таблица, 157

Ц

Цикл
 итерационный, 86
 рекурсивный, 86
 формирование, 69
 блок-схемы, 71
 математические таблицы синусов, 72
 объекты-диапазоны, 74
 циклы `for`, 75
Цикл `for`, 75
Цифра римская, синтаксический разбор, 145

Ч

Часть числа значащая, 429
Черта, 291
Число
 без знака, 425
 действительное, 422
 иррациональное, 44
 со знаком, 425
 с плавающей точкой, 428
 выполнение операций на них, 55
 краткий обзор, 54
 Фибоначчи, 86
 целое, 51, 422
 выполнение операций на них, 55
 создание больших целых чисел с помощью `BigInt`, 269
 целое без знака, 51, 52
Чтение
 данных из процесса, 391
 из сокета, 392

Ш

Шаблон Холи по приданию типам черт, 290
Шифрование симметричное, 345
Шифротекст, 345, 353, 357
Шифр
 подстановочный, 345
 реализация, 349
 служба шифрования, 359
Цезаря, 345
 реализация, 347
 служба шифрования, 357

Э

Экземпляр, 164

Элемент

добавление и удаление, 294

доступ

словари, 154

функциональность наподобие

электронной таблицы, 100

печать многочисленных

элементов, 128

упорядочивание, 308

Этап понижения, 293

Ю

Юникод, 241

Я

Язык

динамически типизированный

за и против, 27

потребность со стороны

разработчиков, 32

потребность со стороны ученых, 30

связующий, 391

статически типизированный, за

и против, 27

структурированных запросов

(SQL), 313

Книги издательства «ДМК ПРЕСС»
можно купить оптом и в розницу
в книготорговой компании «Галактика»
(представляет интересы издательств
«ДМК ПРЕСС», «СОЛОН ПРЕСС», «КТК Галактика»).

Адрес: г. Москва, пр. Андропова, 38, оф. 10;

тел.: **(499) 782-38-89**, электронная почта: **books@aliens-kniga.ru**.

При оформлении заказа следует указать адрес (полностью),
по которому должны быть высланы книги;
фамилию, имя и отчество получателя.

Желательно также указать свой телефон и электронный адрес.

Эти книги вы можете заказать и в интернет-магазине: **<http://www.galaktika-dmk.com/>**.

Эрик Энгхейм

Julia в качестве второго языка

Главный редактор *Мовчан Д. А.*
dmkpress@gmail.com

Зам. главного редактора *Сенченкова Е. А.*

Перевод *Логунов А. В.*

Корректор *Синяева Г. И.*

Верстка *Чаннова А. А.*

Дизайн обложки *Мовчан А. Г.*

Гарнитура PT Serif. Печать цифровая.

Усл. печ. л. 36,24. Тираж 200 экз.

Веб-сайт издательства: **www.dmkpress.com**